

Claude Code 신입 개발자 온보딩 — 첫 기능부터 첫 PR까지

목차

1장. 첫날: Claude Code 설치·인증과 첫 세션 [입문]

- 도입: 팀에 합류한 첫날, 페어 프로그래머를 들이다
- 정의: Claude Code란 무엇인가(터미널·IDE·웹)
- Windows 11 설치: PowerShell 네이티브 설치 관리자와 npm
- Git for Windows(Bash 도구)와 작업 디렉터리 준비
- 동작 원리: 인증 흐름(구독 vs API 키)
- worked example: 설치→로그인→/status로 인증 확인
- 대화형 REPL과 1회성 실행(-p) 구분
- worked example: 빈 폴더에서 첫 파일 생성·실행
- 세션 관리: 중단·재개(--continue/--resume)·종료
- 즉시 연습(2종 1세트): 첫 세션 점검 체크 채우기
- 흔한 오류와 오개념: API 키 과금·자동완성 오해·붙여넣기 습관

2장. 에이전트형 코딩의 작동 방식: 관찰→계획→도구→확인 [입문]

- 도입: 자동완성과 무엇이 다른가(선수 상기)
- 정의: 에이전트형 코딩이란
- 동작 원리: 관찰→계획→도구 호출→결과 확인 루프
- 읽기 전용 도구(Read·Glob·Grep)와 부수효과 도구(Edit·Write·Bash)
- worked example: 같은 작업을 자동완성식 vs 에이전트식으로 관찰
- diff로 변경을 제시하고 검토하기
- worked example: 도구 호출 순서(Grep→Read→Edit→Bash) 따라 읽기
- 루프가 끊기는 지점: 모호함·잘못된 가정
- 즉시 연습(2종 1세트): 루프 단계 분해 채우기
- 흔한 오류와 오개념: 한 번에 정답 기대·검증 생략

3장. 효과적인 작업 지시: 목표·맥락·제약·완료 조건 [입문]

- 도입: 결과는 지시의 명확성에 달렸다(선수 상기)
- 좋은 지시의 네 요소(목표·맥락·제약·완료 조건)
- 동작 원리: 모호한 지시가 잘못된 가정으로 이어지는 흐름
- worked example: 모호한 지시 vs 구체 지시 비교
- 작업 분해와 단계적 지시(한 번에 하나)
- @파일 멘션·참조로 맥락 주입하기
- worked example: @main.py 참조로 같은 스타일 엔드포인트 추가
- 반복 교정(iterate): 검토→피드백→재시도
- 즉시 연습(2종 1세트): 지시문 4요소 채우기
- 흔한 오류와 오개념: 과대 요청·모호한 형용사·검증 누락

4장. 낯선 팀 코드 파악하기: 에이전트로 코드베이스 탐색 [기초]

- 도입: 신입의 첫 난관은 낯선 코드다
- 에이전트 검색: 구조·의존성·흐름을 자동 파악
- 동작 원리: 탐색 질문이 코드 이해로 이어지는 흐름
- worked example: FastAPI 저장소 구조·엔드포인트 요약시키기
- 온보딩 질문법: 시니어에게 문득 질문하기
- worked example: tasks 데이터 흐름·검증 경로 추적
- 특정 함수·엔드포인트의 책임과 예외 흐름 묻기
- 읽기 전용 탐색과 결과 메모로 정리하기
- 즉시 연습(2종 1세트): 구조 지도 빈칸 채우기
- 흔한 오류와 오개념: 다 읽고 시작·요약 맹신·탐색 중 변경

5장. 안전하게 변경하기 (1): 권한과 승인 [기초]

- 도입: 손대기 전에 안전장치부터(선수 상기)
- 부수효과 도구와 읽기 전용 도구의 권한 차이
- 동작 원리: allow·deny·ask 판정 흐름
- worked example: pytest만 allow·위험 삭제는 deny하는 settings.json
- 도구·명령 패턴 매칭(Bash(pytest):·Edit(app/*))
- 권한 범위: 사용자·프로젝트 공유·로컬 개인
- worked example: /permissions로 안전 명령 미리 허용·체크인
- 전체 자동 승인의 위험과 최소 권한 원칙
- 즉시 연습(2종 1세트): allow·deny 규칙 채우기
- 흔한 오류와 오개념: 전체 bypass·이름만 허용·로컬 오남용

6장. 안전하게 변경하기 (2): 플랜 모드로 먼저 계획 [기초]

- 도입: 바꾸기 전에 계획을 먼저 본다(선수 상기)
- 플랜 모드는: 변경 전 읽기 전용 계획 단계
- 동작 원리: 계획→승인→실행 분리 흐름(Shift+Tab)
- 플랜 모드에서 편집·쓰기 도구가 차단되는 이유
- worked example: 엔드포인트 추가를 플랜 모드로 계획받아 실행
- 계획 교정: 단계 추가·삭제·재정렬
- worked example: 잘못된 가정을 교정 지시로 바로잡기
- 언제 쓰는가: 복잡·고위험·다중 파일·첫 기능 추가
- 즉시 연습(2종 1세트): 계획 검토·교정 채우기
- 흔한 오류와 오개념: 단순작업 과용·무비판 실행·읽기전용 오해
- 단원 미니 프로젝트: 안전 장비 갖추고 변경 계획 받기(2종 1세트)

7장. 실행·테스트로 검증하기 [중급]

- 도입: 검증 전까지는 끝난 게 아니다(선수 상기)
- 결과 확인: 실행·테스트가 곧 완료의 증거
- 동작 원리: 변경→실행→테스트→통과 확인 루프
- worked example: 엔드포인트를 pytest(TestClient)로 검증
- FastAPI 검증: 서버 실행·요청으로 엔드포인트 확인
- 테스트 주도 개발(TDD): 테스트 먼저 → 구현
- worked example: 실패 메시지를 근거로 구현 교정
- '검증 전까지는 끝난 게 아니다' 습관화
- 즉시 연습(2종 1세트): 테스트 케이스 빈칸 채우기
- 흔한 오류와 오개념: '됐다' 신뢰·테스트 미룸·근거 없는 동작 주장

8장. 팀 규약 주입: CLAUDE.md로 컨벤션 명문화 [중급]

- 도입: 세션마다 팀 규칙을 다시 설명하지 않으려면
- CLAUDE.md의 역할: 세션마다 자동으로 읽히는 헌법
- 동작 원리: 컨텍스트 계층(프로젝트·사용자·하위)과 추가 로딩
- worked example: /init로 빌드·테스트·컨벤션 담은 CLAUDE.md 생성
- 무엇을 담는가: 명령·컨벤션·아키텍처·금지사항
- /memory 편집과 @참조로 모듈화
- worked example: 팀 규칙(프로젝트)과 개인 취향(~/.claude) 분리
- 좋은 CLAUDE.md의 원칙: 간결·구체·최신 유지
- 즉시 연습(2종 1세트): CLAUDE.md 골격 채우기
- 흔한 오류와 오개념: 방치된 낡은 규칙·과적·개인에 둔 팀 규칙

9장. 첫 커밋과 첫 PR: git 워크플로 [실무]

- 도입: 신입의 통과의례, 첫 PR(선수 상기)
- git 상태·diff 확인과 변경 맥락 파악
- 동작 원리: 브랜치→커밋→푸시→PR 흐름
- worked example: 작업 브랜치에서 feat: 메시지로 커밋
- 작은 논리 단위 커밋과 Conventional Commits
- PR 초안: 변경 요약·근거·테스트 결과(gh:/commit-push-pr)
- worked example: 좋은 PR 본문 작성으로 리뷰어 돕기
- 커밋·PR 권한(Bash(git commit:*)·gh) 설계
- 즉시 연습(2종 1세트): 커밋 단위·PR 본문 채우기
- 흔한 오류와 오개념: 통제 없는 커밋·몰아 커밋·빈약한 PR 설명

10장. 코드 리뷰 받기·대응하기 [실무]

- 도입: PR을 올리면 리뷰가 따라온다(선수 상기)
- /code-review: 터미널에서 올리기 전 자가 점검
- 동작 원리: 자동 리뷰가 PR에 인라인 코멘트를 다는 흐름
- worked example: /code-review로 점검해 지적 미리 반영
- 리뷰 트리거: PR 열림·매 푸시·요청('@claude review')
- worked example: 인라인 코멘트를 읽고 해당 줄 수정
- /security-review로 취약점 사전 점검
- 피드백 대응: 반영·재요청·근거 있는 반론
- 즉시 연습(2종 1세트): 리뷰 답글 채우기
- 흔한 오류와 오개념: 방어적 수용·자동 리뷰 맹신·무비판 반영

11장. 에이전트와 함께 디버깅하기 [실무]

- 도입: 막힘을 푸는 능력이 곧 생산성이다(선수 상기)
- 에러 메시지·스택 트레이스를 통째로 전달하기
- 동작 원리: 재현→가설→최소 수정→회귀 테스트 루프
- worked example: 실패한 pytest 메시지로 원인 진단·수정
- 증상·기대·실제 동작을 구체적으로 진술하기
- worked example: FastAPI 422 검증 에러를 최소 수정
- 최소 수정 원칙: 한 번에 한 가설
- 회귀 테스트로 수정 검증·재발 방지
- 즉시 연습(2종 1세트): 디버깅 진술 채우기
- 흔한 오류와 오개념: '고쳐 줘' 막연함·메시지 자르기·회귀 미확인

12장. 더 나아가기(맛보기): 서브에이전트와 MCP [실무]

- 도입: 온보딩 이후, 어디로 더 갈 수 있나
- 서브에이전트란: 독립 컨텍스트·프롬프트·도구
- 동작 원리: 위임·컨텍스트 격리·병렬 처리
- worked example: 테스트 실행 서브에이전트(.claude/agents) 정의·호출
- 정의 파일: YAML 프런트매터(name·description·tools)
- MCP란: 외부 통합을 위한 표준 프로토콜
- worked example: 문서 MCP 서버를 .mcp.json에 등록·참조
- 권한·비밀 관리: 동일 권한 체계·토큰은 환경으로
- 즉시 연습(2종 1세트): 서브에이전트·mcp.json 채우기
- 흔한 오류와 오개념: 맥락 공유 가정·플러그인 난립 오해·토큰 노출

13장. 종합 프로젝트: 첫 기능·첫 PR — 분석(요구사항·범위) [실무]

- 드라이빙 퀘스천과 POV 다시 보기
- 기존 FastAPI 저장소 파악(에이전트 탐색)
- 기능 요구사항(FR): 완료 처리·상태 필터 엔드포인트
- 비기능 요구사항(NFR): 테스트·안전·팀 규약
- 사용자 스토리와 유스케이스
- 범위 In/Out 확정(작은 첫 기능으로)
- 추적성: 요구사항 ↔ 단위 개념 매핑
- 작성형 연습(2종 1세트): 요구사항 표 채우기·자가체크

14장. 종합 프로젝트: 첫 기능·첫 PR — 설계(데이터·모듈·ADR) [실무]

- 설계 개요: 기존 구조에 최소 변경으로 끼우기
- 데이터 구조 설계: 요청·응답 모델과 상태 필드
- 모듈·함수 변경 분해(라우터·모델·저장소·테스트)
- 설계 결정 기록: ADR 5건(플랜 모드로 도출)
- 요청 처리 순서도: 입력→검증→상태 변경→응답
- 작성형 연습(2종 1세트): ADR 작성·자가체크

15장. 종합 프로젝트: 첫 기능·첫 PR — 구현(마일스톤 증분) [실무]

- 구현 전략과 마일스톤 개요
- M1 CLAUDE.md·권한·플랜 모드로 셋업·계획(체크포인트)
- M2 테스트 먼저(TDD): 완료 처리·필터 테스트 작성
- M3 엔드포인트 구현과 실행·테스트 검증
- M4 논리 단위 커밋과 첫 PR 초안 작성

- M5 /code-review로 리뷰 받고 디버깅·반영
- 통합 실행과 결과 확인(테스트 통과·PR 본문)
- 작성형 연습(2종 1세트): 빈칸 코드·테스트 채우기·자가체크

16장. 종합 프로젝트: 첫 기능·첫 PR — 발표·평가·성찰 [실무]

- 산출물 시연: 동작하는 기능·통과한 테스트·올린 PR
 - 평가 루브릭
 - 동료 피드백(모의 리뷰)
 - 성찰 회고: 12개 개념의 적용
 - 다음 단계: 서브에이전트·MCP 심화로 가는 길
 - 작성형 연습(2종 1세트): 성찰 템플릿·자가체크
-

이 책에 대하여 (머리말)

이 책의 목적

이 책의 목적은 Python·웹 백엔드 기초가 있는 주니어 개발자가 Claude Code를 처음 익혀, 기존 팀의 FastAPI 저장소에 작은 기능 하나를 안전하게 추가하고 테스트와 함께 리뷰어가 신뢰할 수 있는 '첫 PR'까지 올리는 온보딩 여정을 끝까지 해내게 하는 것입니다. Claude Code의 기능을 백과사전처럼 나열하지 않고, 신입의 실제 첫 업무 흐름(설치 → 코드베이스 파악 → 안전하게 변경 → 검증 → 팀 규약 → 커밋·PR → 리뷰 대응 → 디버깅)에 맞춰 필요한 만큼만 배웁니다.

이 책을 마치면 할 수 있는 것

- Claude Code를 설치·인증하고 대화형 세션에서 에이전트와 일하는 흐름을 운용할 수 있습니다
- 낯선 팀 코드베이스를 에이전트 탐색으로 빠르게 파악하고, 권한·플랜 모드로 안전하게 변경할 수 있습니다
- 변경을 pytest 실행·테스트로 검증하고, CLAUDE.md로 팀 규약을 주입할 수 있습니다
- 작은 논리 단위로 커밋해 첫 PR을 올리고, 코드 리뷰에 근거 있게 대응하며, 에이전트와 함께 디버깅할 수 있습니다
- 종합 프로젝트로 기존 FastAPI 저장소에 '완료 처리·상태 필터' 기능을 추가해 테스트와 함께 첫 PR을 완성할 수 있습니다

대상 독자

Python 기초 문법과 웹 백엔드(HTTP·REST)의 기본 개념을 갖춘 주니어 개발자로, Claude Code(에이전트형 코딩 도구)는 처음 쓰는 분. 컴퓨터공학 전공 여부는 무관하지만, 아래 선수 지식은 가정합니다(전공이 아니라 '아래를 아는가'가 기준입니다).

선수 지식

- Python 기초 문법: 함수·조건문·리스트/딕셔너리·모듈 import를 읽고 쓸 수 있음(코드를 처음 보는 수준이면 먼저 Python 입문을 권합니다)
- 터미널·명령줄 기본: 디렉터리 이동(cd)·명령 실행·환경 변수 개념
- git 기본: add·commit·branch·PR(Pull Request)이 무엇인지 아는 정도(능숙할 필요는 없음 — 9장에서 Claude Code와 함께 익힙니다)
- HTTP·REST 기초: GET/POST 같은 메서드와 상태 코드(200·404 등)의 의미

- FastAPI는 몰라도 됩니다: 실습 저장소(team-tasks-api)의 코드를 '읽을 수 있으면' 충분하며, 필요한 부분은 본문에서 설명합니다

실습 환경

- 운영체제: Windows 11 기준(명령은 macOS·Linux에도 대응; 차이는 그때그때 표기)
 - Python 3.10 이상과 Git for Windows(Git Bash)
 - Claude 구독(Pro/Max) 또는 API 키 — 1장에서 인증 방법을 안내
 - 공용 실습 저장소: team-tasks-api(이 책의 assets/에 포함; 1장에서 git init·pip·pytest로 셋업)
-

첫날: Claude Code 설치·인증과 첫 세션

이 장의 학습 목표

- 주어진 Windows 11 작업 머신에서 네이티브 설치 관리자 또는 npm으로 Claude Code를 설치하고, `claude --version` 과 Git Bash 실행으로 설치 완료를 확인할 수 있습니다.
- 구독(Pro/Max)과 API 키 두 인증 방식의 과금 차이를 한 가지 이상 들어 구분하고, 의도한 방식으로 로그인한 뒤 `/status` 로 그 방식이 맞는지 확인할 수 있습니다.
- 주어진 실습 폴더에서 대화형 REPL(`claude`)을 열어 첫 작업을 지시해 도구 호출 1사이클을 완료하고, 1회성(`-p`) 실행과 언제 무엇을 쓰는지 구분해 설명할 수 있습니다.
- 진행 중 세션을 중단(Esc)·재개(`--continue` / `--resume`)해 직전 맥락이 이어지는지 확인하고, `/status` 로 인증·세션 상태를 점검할 수 있습니다.

이 책은 기존 팀의 FastAPI 저장소에 작은 기능 하나를 추가해 첫 PR(Pull Request, 풀 리퀘스트)까지 올리는 신입 개발자의 여정을 따라갑니다. 그 여정의 출발점은 단 하나입니다. 옆자리에 앉아 함께 코드를 읽고 고치고 실행해 줄 페어 프로그래머를 내 자리에 들이는 것, 곧 **Claude Code**를 설치하고 인증해 첫 세션을 여는 것입니다.

이 장의 선수 지식: 이 장은 책의 첫 단원(u1)이며, 앞서 배운 *이 책의* 개념을 전제하지 않습니다. 다만 책 전체의 선수 지식(Python 기초·터미널·git·HTTP 기초 — 자세한 내용은 머리말 「이 책에 대하여」 참고)은 가정합니다. 설치·인증·첫 세션 자체는 Claude Code가 처음이어도 따라올 수 있도록 모든 동작을 단계로 풀어 설명합니다.

이 장에서 가르치는 개념은 하나입니다.

- **개념 c01 — 설치·인증과 첫 세션(installation, authentication, and first session):**
Windows 11에 Claude Code를 설치하고, 내 의도에 맞는 방식으로 인증하고, 대화형 REPL과 1회성 실행을 구분해 첫 작업을 지시하고, 세션을 중단·재개하는 법.

1. 도입: 팀에 합류한 첫날, 페어 프로그래머를 들이다

입사 첫날의 풍경은 대개 비슷합니다. 노트북을 받고, 사번으로 로그인하고, 팀의 코드 저장소 주소를 전달받습니다. 그리고 곧바로 막막함이 찾아옵니다. 수백 개의 파일, 한 번도 본 적 없는 폴더 구조, "이게 어디서 호출되는 거지?"라는 질문이 끝없이 이어집니다. 신입에게 가장 비싼 자원은 시간이 아니라 **물어볼 수 있는 사람**입니다. 시니어 개발자는 바쁘고, 같은 질문을 두 번 묻기는 눈치가 보입니다.

Claude Code는 바로 이 자리를 메우는 도구입니다. 지치지 않고, 같은 질문을 백 번 물어도 괜찮으며, 저장소 전체를 몇 초 만에 훑어 "이 API는 무엇을 하고 엔드포인트가 무엇인지" 설명해주는 페어 프로그래머입니다. 단순히 설명만 하는 것이 아니라, 파일을 직접 열어 읽고, 코드를 고치고, 테스트를 돌려 결과를 보여 줍니다.

그런데 이 페어 프로그래머를 제대로 쓰려면 첫날에 세 가지를 정확히 해 두어야 합니다. **설치**(내 컴퓨터에 도구를 올리기), **인증**(내 계정·요금제로 연결하기), 그리고 **첫 세션**(대화를 시작해 도구가 일하는 방식을 직접 보기)입니다. 이 세 가지를 대충 넘기면, 뒤에서 "왜 과금이 되지?", "왜 명령이 안 먹히지?", "어제 하던 작업이 사라졌네?" 같은 문제로 되돌아오게 됩니다. 이 장은 그 세 가지를 한 번에, 정확히 끝내는 것을 목표로 합니다.

한눈에 보는 이 장의 지도

- 페어 프로그래머를 자리에 앉히기 → **설치**(PowerShell 네이티브 설치 관리자 또는 npm)
- 도구가 Bash 명령을 쓸 수 있게 하기 → **Git for Windows(Git Bash) 준비**
- 내 요금제로 연결하기 → **인증**(구독 Pro/Max 브라우저 로그인 vs API 키)
- 대화를 시작하기 → **대화형 REPL**(`claude`)
- 한 번만 묻고 답만 받기 → **1회성 실행**(`claude -p`)
- 자리를 비웠다 돌아오기 → **세션 재개**(`--continue` · `--resume`)

2. 정의: Claude Code란 무엇인가(터미널·IDE·웹)

정의

이 장의 핵심 개념을 한 문장으로 정의하겠습니다.

Claude Code(클로드 코드)는 터미널·IDE·웹에서 동작하는 Anthropic 공식 **에이전트형 코딩 CLI(agent coding CLI)**로, 단순히 다음 코드를 제안하는 자동완성이 아니라 파일을 직접 읽고 편집하고 명령을 실행하며 커밋까지 돕는 AI 페어 프로그래머입니다. 그리고 **첫 세션(first session)**이란 설치·인증을 마친 뒤 대화형 REPL을 처음

열어, 자연어 지시에 에이전트가 도구를 호출해 변경을 만들어 내는 과정을 처음으로 관찰하는 출발점입니다.

용어를 하나씩 정리하겠습니다.

- **CLI(Command-Line Interface, 명령줄 인터페이스)**: 마우스 클릭이 아니라 터미널에 명령어를 입력해 쓰는 프로그램입니다. Claude Code의 기본 진입점은 터미널에 `claude` 라고 입력하는 것입니다.
- **에이전트(agent)**: 목표를 받아 스스로 도구를 골라 쓰며 여러 단계를 거쳐 작업을 완수하는 프로그램입니다. "다음 한 줄"을 제안하는 데서 그치지 않고, 읽고·계획하고·실행하고·확인하기를 반복합니다.
- **REPL(Read-Eval-Print Loop, 읽고-실행하고-출력하는 반복 루프)**: 입력을 받아 처리하고 결과를 보여 주기를 반복하는 대화형 화면입니다. `claude` 를 실행하면 나타나는 대화 화면이 바로 REPL입니다.

같은 엔진, 여러 표면

Claude Code는 하나의 엔진을 여러 **표면(surface)** 에서 쓸 수 있게 제공합니다. 어디서 쓰든 "에이전트가 도구를 호출해 일한다"는 핵심은 같습니다.

표면	진입 방법	잘 맞는 상황
터미널 (CLI)	셸에서 <code>claude</code> 실행	대부분의 개발 작업, 스크립트·CI 연동 (이 책의 기본 무대)
IDE 확장	VS Code·JetBrains 확장에서 호출	편집기 안에서 diff를 보며 작업할 때
웹	브라우저에서 클라우드 작업공간 실행	로컬 설치 없이 원격에서 작업할 때

이 책은 가장 이식성이 좋고 자동화에 직결되는 **터미널(CLI)** 을 기본 무대로 삼습니다. 터미널에서 익힌 개념은 IDE·웹에 그대로 옮겨집니다.

자동완성과의 한 줄 차이 자동완성 도구는 "지금 타이핑하는 커서 주변"만 보고 다음 몇 줄을 제안합니다. Claude Code는 "이 저장소에서 `tasks` 라우터를 찾아 새 엔드포인트를 추가하고 테스트까지 통과시켜 줘" 같은 **완결된 작업** 을 받아 스스로 파일을 탐색·편집·실행합니다. 이 차이는 2장에서 에이전트 루프로 자세히 다룹니다.

3. Windows 11 설치: PowerShell 네이티브 설치 관리자 와 npm

이제 페어 프로그래머를 자리에 앉혀 보겠습니다. 이 책의 대상 환경은 **Windows 11**입니다. 설치 방법은 두 갈래이며 **둘 중 하나만** 고르면 됩니다. 공식이 권장하는 방법은 네이티브 설치 관리자입니다.

방법 1: PowerShell 네이티브 설치 관리자(권장)

Windows 11의 **PowerShell**(시작 메뉴에서 "PowerShell" 검색)을 열고 다음 한 줄을 입력합니다.

```
irm https://claude.ai/install.ps1 | iex
```

이 명령은 두 부분으로 이루어져 있습니다. `irm`은 `Invoke-RestMethod`의 별칭으로 주어진 주소에서 설치 스크립트를 내려받고, `iex`는 `Invoke-Expression`의 별칭으로 그 스크립트를 실행합니다. 설치가 끝나면 바이너리가 사용자 폴더(`C:\Users\사용자이름\local\bin`)에 놓이므로 **관리자 권한이 필요 없습니다**. 네이티브 설치본은 백그라운드에서 자동으로 최신 버전으로 갱신됩니다.

`irm`이 인식되지 않는다는 오류가 나면 '`irm`' 용어가 `cmdlet...` 이름으로 인식되지 않습니다 라는 메시지가 보이면 지금 연 창이 **PowerShell이 아니라 명령 프롬프트(CMD)**입니다. `irm`은 PowerShell 전용 별칭이므로, 시작 메뉴에서 정확히 "PowerShell"을 다시 검색해 실행한 뒤 명령을 입력합니다.

방법 2: npm 전역 설치(대안)

이미 Node.js가 설치돼 있다면 npm으로 설치할 수도 있습니다. PowerShell이나 Git Bash에서 다음을 입력합니다.

```
npm install -g @anthropic-ai/claude-code
```

`-g`는 전역(global) 설치라는 뜻으로, 어느 폴더에서나 `claude` 명령을 쓸 수 있게 합니다. 패키지 이름은 정확히 `@anthropic-ai/claude-code`입니다.

설치 확인

어느 방법을 썼든 새 터미널 창을 열고 버전을 확인합니다. (설치 직후에는 PATH가 갱신되도록 터미널을 새로 여는 것이 안전합니다.)

```
claude --version
```

```
2.0.x (Claude Code)
```

버전 번호가 출력되면 설치는 성공입니다. 숫자는 시점에 따라 다릅니다.

4. Git for Windows(Bash 도구)와 작업 디렉터리 준비

Claude Code는 명령을 실행할 때 **셸(shell)** 을 빌려 씁니다. Windows에는 기본으로 PowerShell 과 CMD가 있지만, Claude Code의 **Bash 도구**는 이름 그대로 Bash 셸을 기대합니다. 그래서 Windows에서는 **Git for Windows**(Git Bash 포함)를 함께 설치하는 것을 권장합니다. Git for Windows가 있으면 Claude Code가 그 Bash를 도구로 사용하고, 없으면 일부 Bash 기반 명령이 어색하게 동작할 수 있습니다. 또한 9장에서 다룰 git 커밋·PR 워크플로에도 Git이 반드시 필요합니다.

git-scm.com에서 Git for Windows 설치 관리자를 내려받아 설치한 뒤, 설치를 확인합니다.

```
git --version
```

```
git version 2.45.0.windows.1
```

작업 디렉터리 준비

Claude Code는 "지금 터미널이 위치한 폴더"를 작업 대상으로 삼습니다. 그래서 도구를 열기 전에 **작업할 폴더로 먼저 이동**하는 습관이 중요합니다. 연습용 빈 폴더를 하나 만들어 보겠습니다.

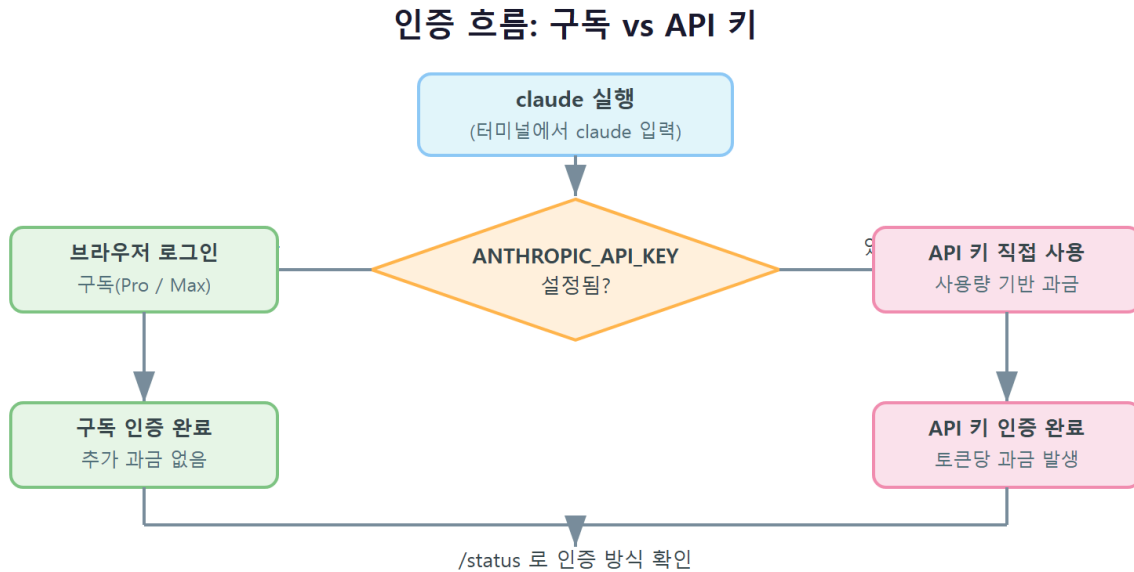
```
mkdir onboarding-practice
cd onboarding-practice
```

`mkdir` 은 폴더를 만들고, `cd` 는 그 폴더로 이동합니다. 이제 이 폴더 안에서 `claude` 를 실행하면, Claude Code는 이 폴더를 작업 무대로 인식합니다.

작업 폴더를 먼저 정하는 이유 Claude Code는 실행한 위치의 폴더와 그 하위만 기본 작업 범위로 봅니다. 엉뚱한 상위 폴더(예: `C:\Users\사용자`)에서 도구를 열면 수많은 파일을 작업 대상으로 오해할 수 있습니다. "작업할 폴더로 `cd` 한 뒤 `claude` "를 첫날부터 몸에 익히면 사고를 줄입니다.

5. 동작 원리: 인증 흐름(구독 vs API 키)

설치를 마쳤다면, 이제 Claude Code를 **내 계정·요금제**에 연결해야 합니다. 이 단계에서 신입이 가장 자주 혼동하는 지점이 바로 인증 방식입니다. 인증에는 두 갈래가 있고, 둘은 **요금이 청구되는 방식이 다릅니다**.



두 가지 인증 방식

방식	누가 쓰나	어떻게 청구되나
구독(Pro/Max)	Claude 구독 요금제를 쓰는 개인·팀원	월 구독료 안에서 사용량 소진
API 키 (ANTHROPIC_API_KEY)	Anthropic 플랫폼 API 사용자	토큰 사용량만큼 종량 과금

구독 방식은 Claude Pro 또는 Max 요금제를 결제한 사람이 쓰는 방법입니다. `claude`를 처음 실행하면 브라우저가 열리고, Claude 계정으로 로그인하면 구독 사용량으로 Claude Code가 동작합니다. 신입 온보딩에서는 대개 이 방식을 씁니다.

API 키 방식은 환경 변수 `ANTHROPIC_API_KEY`에 발급받은 키를 넣어 두는 방법입니다. 이 경우 Anthropic 플랫폼에서 토큰 사용량만큼 **종량 과금**이 발생합니다.

핵심: 두 방식은 같은 자리를 두고 경쟁한다

여기서 가장 중요한 동작 원리는 이것입니다. `ANTHROPIC_API_KEY` 환경 변수가 설정돼 있으면, **Claude Code는 구독 대신 그 API 키를 우선해 사용합니다.** 즉 구독 요금제를 쓰려고 했는데 예전에 실습하느라 설정해 둔 API 키가 환경 변수에 남아 있으면, 본인도 모르게 종량 과금이 발생할 수 있습니다.

따라서 인증 흐름은 다음 분기로 정리됩니다.

1. `claude` 를 실행한다.
2. Claude Code가 `ANTHROPIC_API_KEY` 환경 변수를 확인한다.
3. 변수가 있으면 → 그 API 키로 동작(종량 과금).
4. 변수가 없으면 → 브라우저 로그인으로 구독(Pro/Max) 사용량을 쓴다.
5. 어느 쪽으로 인증됐는지 `/status` 로 확인한다.

그래서 **구독을 쓰려면 `ANTHROPIC_API_KEY` 를 설정하지 않아야 합니다.** 이미 설정돼 있다면 비워두어야 구독 로그인으로 넘어갑니다.

Windows에서 환경 변수 확인·제거 PowerShell에서 현재 세션의 API 키가 설정돼 있는지 확인하려면 다음을 입력합니다.

```
echo $env:ANTHROPIC_API_KEY
```

아무것도 출력되지 않으면 설정돼 있지 않은 것입니다. 값이 보인다면(그리고 구독을 쓰고 싶다면) 현재 PowerShell 세션에서 임시로 비울 수 있습니다.

```
Remove-Item Env:\ANTHROPIC_API_KEY
```

영구적으로 설정해 둔 경우라면 Windows의 "시스템 환경 변수 편집"에서 사용자 변수 목록의 `ANTHROPIC_API_KEY` 를 제거해야 합니다.

6. worked example: 설치 → 로그인 → /status로 인증 확인

지금까지 배운 설치·인증 절차를 처음부터 끝까지 한 번 따라가 보겠습니다. 이것은 첫날 누구나 한 번은 거치는 정확한 절차입니다.

1단계: 설치하고 버전 확인

PowerShell에서 설치하고 새 터미널에서 버전을 확인합니다.

```
irm https://claude.ai/install.ps1 | iex
```

```
Claude Code installed to C:\Users\jihoon\.local\bin
```

새 터미널을 열어 확인합니다.

```
claude --version
```

```
2.0.x (Claude Code)
```

2단계: 작업 폴더로 이동해 첫 실행

```
cd onboarding-practice
claude
```

처음 `claude` 를 실행하면 테마 선택 등 초기 설정을 묻고, 인증이 안 된 상태라면 브라우저가 열립니다. 브라우저에서 Claude 계정으로 로그인하고 권한을 승인하면 터미널로 돌아와 다음과 같은 대화형 REPL 화면이 나타납니다.

```

  * Welcome to Claude Code
  /help 로 도움말, /status 로 상태 확인

```

```
>
```

> 프롬프트가 보이면 이제 자연어로 지시할 준비가 된 것입니다.

3단계: /status로 인증 방식 확인

REPL 안에서 슬래시 명령 `/status` 를 입력합니다. 슬래시 명령은 자연어 지시가 아니라 Claude Code에 직접 내리는 제어 명령입니다.

```
> /status
```

```
Account: jihoon@team.com
Login method: Claude Max subscription
Model: claude-opus-4-x
Working directory: C:\Users\jihoon\onboarding-practice
```

여기서 `Login method`가 핵심입니다. `Claude Max subscription`(또는 `Pro`)이라고 나오면 구독으로 인증된 것이고, `API key`라고 나오면 종량 과금 방식으로 연결된 것입니다. 의도와 다르면 5장에서 본 대로 `ANTHROPIC_API_KEY`를 확인·제거하고 다시 로그인합니다.

이 절차에서 짚을 것

- **설치 → 버전 확인**: 버전 번호가 나와야 PATH에 정상 등록된 것입니다.
- **cd 후 claude**: 작업 폴더 안에서 실행해야 그 폴더가 작업 무대가 됩니다.
- **브라우저 로그인**: `ANTHROPIC_API_KEY`가 없을 때만 구독 로그인으로 넘어갑니다.
- **/status**: "내가 의도한 방식으로 인증됐는가"를 눈으로 확인하는 마지막 관문입니다. 첫 날 이것을 확인하지 않으면 과금 방식을 모른 채 쓰게 됩니다.

7. 대화형 REPL과 1회성 실행(-p) 구분

Claude Code를 쓰는 방식은 크게 두 가지입니다. 둘의 차이를 처음부터 분명히 해 두면, 상황에 맞는 방식을 골라 쓸 수 있습니다.

대화형 REPL: `claude`

`claude`만 입력하면 앞에서 본 대화 화면(REPL)이 열립니다. 여기서는 사람과 대화하듯 여러 차례 지시를 주고받습니다. 에이전트가 파일을 고치면 검토하고, 추가 지시를 주고, 테스트를 돌리게 하고, 다시 고치게 합니다. **맥락이 누적되는** 작업, 즉 한 번의 대화로 끝나지 않는 대부분의 개발 작업에 적합합니다.

1회성 실행(print 모드): `claude -p "..."`

질문 하나에 답 하나만 받고 끝내고 싶을 때는 `-p` (print) 플래그를 씁니다. REPL을 열지 않고, 명령 한 줄에 결과를 출력하고 곧바로 종료합니다.

```
claude -p "이 폴더에 뭐가 있는지 한 줄로 요약해 줘"
```

이 폴더에는 아직 파일이 없는 빈 작업 디렉터리입니다.

`-p` 모드는 대화를 이어가지 않고 한 번 답하고 끝나므로, 셸 스크립트나 자동화 파이프라인에 끼워 넣기 좋습니다. "지금 빠르게 한 가지만 묻고 싶다"면 `-p`, "함께 작업을 진행하겠다"면 REPL입니다.

구분	대화형 REPL (<code>claude</code>)	1회성 실행 (<code>claude -p "..."</code>)
화면	대화 화면이 열림	답만 출력하고 종료
맥락	여러 턴에 걸쳐 누적	한 번의 응답으로 끝
적합한 상황	함께 코드를 만들고 고치는 작업	빠른 질의, 스크립트·자동화
종료	<code>/exit</code> 또는 <code>Ctrl+D</code>	자동으로 종료

언제 무엇을 쓰나 지금 단계에서는 거의 항상 **대화형 REPL**을 씁니다. `-p`는 "폴더 요약 한 줄만 받기"처럼 한 번에 끝나는 단순 질의나, 나중에 자동화에 끼워 넣을 때 쓰는 도구라고 기억해 두면 됩니다.

8. worked example: 빈 폴더에서 첫 파일 생성·실행

이제 에이전트가 실제로 **파일을 만들고 실행하는** 모습을 직접 보겠습니다. 자동완성과의 차이를 눈으로 확인하는 순간입니다. 앞서 만든 `onboarding-practice` 빈 폴더에서 `claude`를 실행한 상태라고 가정합니다.

REPL에 다음과 같이 지시합니다.

```
> hello.py 파일을 만들어 "Hello, Claude Code!"를 출력하게 하고,
python으로 실행해서 결과를 보여 줘.
```

에이전트는 먼저 `hello.py`를 만들겠다는 계획을 보여 주고, 파일을 생성한 뒤(부수효과가 있는 작업이므로 권한을 확인합니다 — 5장에서 상세히 다룹니다), 실행 명령을 돌립니다. 만들어진 파일은 다음과 같습니다.

```
print("Hello, Claude Code!")
# 출력: Hello, Claude Code!
```

그리고 에이전트가 `python hello.py`를 실행한 결과를 화면에 보여 줍니다.

```
> python hello.py
Hello, Claude Code!
```

여기서 자동완성과의 결정적 차이가 드러납니다. 자동완성이었다면 `print(...)` 한 줄을 제안하는 데서 끝나고, 파일 생성과 실행은 사람이 직접 해야 합니다. 에이전트는 **파일을 만들고(쓰기), 명령을 실행하고(Bash), 결과를 확인**하는 데까지 스스로 갑니다.

조금 더 실무에 가까운 예: 작은 함수

이 책의 실습 스택은 웹 백엔드(FastAPI)입니다. 본격적인 엔드포인트는 뒤에서 다루지만, 맛보기로 작은 인사 함수를 만들어 실행해 보겠습니다.

```
> greet.py를 만들어. 이름을 받아 "안녕하세요, {이름}님"을 돌려주는
greet 함수와, greet("지훈")을 출력하는 코드를 넣고 실행해 줘.
```

에이전트가 만든 파일은 다음과 같습니다.

```
def greet(name: str) -> str:
    return f"안녕하세요, {name}님"

print(greet("지훈"))
# 출력: 안녕하세요, 지훈님
```

코드 줄별 해설

- `def greet(name: str) -> str:` — 문자열 `name` 을 받아 문자열을 돌려주는 함수를 정의합니다. `: str`, `-> str` 은 타입 힌트로, 이 책의 코딩 컨벤션을 따릅니다.
- `return f"안녕하세요, {name}님"` — f-string으로 받은 이름을 문장에 끼워 돌려줍니다.
- `print(greet("지훈"))` — 함수를 호출한 결과를 화면에 출력합니다.
- `# 출력: 안녕하세요, 지훈님` — 실제 `python greet.py` 실행 결과와 정확히 일치합니다.

실행 결과를 직접 확인합니다.

```
> python greet.py
안녕하세요, 지훈님
```

이렇게 "지시 → 에이전트가 파일 생성 → 실행 → 결과 확인"까지가 한 흐름으로 이어지는 것이 에이전트형 코딩의 기본 모습입니다.

9. 세션 관리: 중단·재개(--continue/--resume)·종료

신입이 처음 당황하는 순간 중 하나가 "어제 하던 대화가 사라진 것 같다"입니다. Claude Code는 세션(대화)을 저장하므로, 닫았다가도 이어서 작업할 수 있습니다. 이 동작을 정확히 알아 두면 작업 맥락을 잃지 않습니다.

작업 중단: Esc

에이전트가 엉뚱한 방향으로 일하고 있을 때, **Esc** 키를 누르면 진행 중인 작업을 멈춥니다. 잘못된 지시를 줬다고 끝까지 기다릴 필요가 없습니다. 멈춘 뒤 지시를 다시 다듬어 주면 됩니다.

세션 종료: /exit 또는 Ctrl+D

대화를 끝내고 터미널로 나가려면 REPL에서 `/exit`를 입력하거나 Ctrl+D를 누릅니다.

```
> /exit
```

직전 세션 이어가기: --continue

터미널을 닫았다가 다시 같은 작업을 이어가고 싶으면, 작업 폴더에서 `--continue` (또는 짧게 `-c`)로 가장 최근 대화를 곧바로 복원합니다.

```
claude --continue
```

이렇게 하면 직전 세션의 대화 내용과 맥락이 그대로 살아난 채 REPL이 열립니다. 에이전트는 어제 무엇을 하고 있었는지 기억합니다.

여러 세션 중 골라 재개: --resume

같은 폴더에서 여러 갈래의 작업을 했다면, `--resume` (또는 `-r`)으로 저장된 세션 목록을 보고 골라서 재개합니다.

```
claude --resume
```

저장된 세션을 선택하세요:

1. hello.py 만들고 실행 (3분 전)
2. greet 함수 추가 (1시간 전)
3. 폴더 구조 질문 (어제)

원하는 세션을 고르면 그 시점의 맥락으로 돌아갑니다.

명령	하는 일
Esc	진행 중인 작업 즉시 중단
/exit 또는 Ctrl+D	현재 세션 종료(터미널로 나감)
<code>claude --continue (-c)</code>	가장 최근 세션을 바로 이어감
<code>claude --resume (-r)</code>	저장된 세션 목록에서 골라 재개

--continue 와 --resume 의 차이 `--continue` 는 "문지 않고 가장 최근 것"을 바로 엽니다. 빠르게 어제 작업을 잇고 싶을 때 씁니다. `--resume` 은 "어느 세션이었지?"를 고를 수 있게 목록을 보여줍니다. 같은 폴더에서 여러 작업을 병행했을 때 유용합니다.

10. 즉시 연습(2종 1세트): team-tasks-api에서 첫 세션을 직접 열어 본다

지금까지 읽은 설치·인증·세션 흐름을, **종이에 채우지 말고 실습 저장소 team-tasks-api 에서 실제로 돌려 봅니다.** 미리 정한 답을 빈칸에 적는 것이 아니라, **본인 컴퓨터 화면에 실제로 나타난 출력을 그대로 옮겨 적는 것이 핵심**입니다. 한 번 손으로 열어 봐야 "REPL이 어떻게 뜨는지, `/status` 가 무엇을 보여 주는지, `--continue` 가 정말 맥락을 잇는지"가 눈에 들어옵니다.

모범: team-tasks-api 폴더에서 첫 세션을 연 실제 기록(예)

아래는 한 신입이 `team-tasks-api` 폴더에서 실제로 명령을 실행하며 화면 출력을 옮겨 적은 예입니다. 값(버전 번호·로그인 방식·경로)은 사람마다 다르므로, 본인 화면의 값으로 바꿔 적어야 합니다.

```
[설치 확인] team-tasks-api 폴더에서
- claude --version → 2.0.x (성공)
- git --version → 2.45.x (Bash 도구 준비됨)

[세션 시작] 같은 폴더에서 claude 실행 → REPL 프롬프트(>)가 떴다
- /status → Login method: Claude Max subscription / Model: Sonnet
  (의도한 구독 방식과 일치)

[작은 작업] REPL에 지시:
> 이 폴더에 NOTES.md를 만들고 "team-tasks-api 온보딩 1일차" 한 줄을 적어 줘.
→ 에이전트가 Write 도구로 NOTES.md 생성(권한 확인이 떴고 허용함)
→ 탐색기에서 NOTES.md가 실제로 생긴 것을 눈으로 확인

[세션 재개] /exit로 닫은 뒤 같은 폴더에서 claude --continue
→ 직전 대화(NOTES.md 작업)가 그대로 이어져 떴다
```

직접 해보기 (team-tasks-api에서 실제로 실행)

연습 1 — team-tasks-api 폴더에서 첫 세션을 실제로 열고, 화면 출력을 옮겨 적으세요.

1장에서 받아 둔 `team-tasks-api` 폴더로 이동해(터미널에서 그 폴더를 연 상태로) 아래를 차례로 실행하고, **본인 화면에 실제로 찍힌 값**을 적으세요. 미리 외운 답이 아니라, 방금 본 출력을 그대로 옮기는 것입니다.

```
[설치 확인] team-tasks-api 폴더에서 실행한 결과
- claude --version 의 실제 출력:
- git --version 의 실제 출력:

[세션 시작] 그 폴더에서 claude 를 실행했을 때
- REPL 프롬프트(>)가 떴는가(예/아니오):
- /status 의 실제 Login method:
- 그 방식이 내가 의도한 방식과 같은가:
```

연습 2 — 에이전트로 team-tasks-api에 작은 파일을 실제로 만들고 눈으로 확인하세요.

연습 1에서 연 세션의 REPL에 다음 지시를 그대로 주세요(이 작업은 코드를 바꾸지 않고 메모 파일 하나만 만듭니다).

```
> 이 폴더에 NOTES.md를 만들고 "team-tasks-api 온보딩 1일차" 한 줄을 적어 줘.
```

실행되는 동안 **실제로 일어난 일**을 적으세요(추측이 아니라 화면에 나타난 것).

```
- 에이전트가 사용한 도구(화면에 표시된 이름, 예: Write):
- 권한 확인(승인 요청)이 떴는가:
- 파일 탐색기 또는 dir 명령으로 NOTES.md가 실제로 생긴 것을 확인했는가:
```

연습 3 — `--continue` 로 세션이 정말 이어지는지 실제로 확인하세요.

REPL에서 `/exit` 로 세션을 닫은 뒤, 같은 `team-tasks-api` 폴더에서 `claude --continue` 를 실행하세요. 그리고 직전 대화(NOTES.md 작업)가 이어지는지 확인합니다.

- `claude --continue` 실행 후 직전 대화가 이어져 났는가(예/아니오):
- 이어진 화면에서 직전에 만든 NOTES.md 작업 내용이 보였는가:
- (선택) `claude --resume` 으로는 무엇이 달랐는가(세션 목록이 났는가):

자가체크 — 머리로가 아니라 손으로 `team-tasks-api`에서 했는지 확인하세요.

- [] `team-tasks-api` 폴더에서 `claude --version · git --version` 을 직접 실행해 출력을 봤다.
- [] 그 폴더에서 `claude` 를 실행해 REPL 프롬프트가 뜨는 것을 봤고, `/status` 로 내 `Login method` 를 직접 확인했다.
- [] 에이전트에 지시해 `NOTES.md` 를 **실제로** 만들고, 파일 탐색기나 `dir` 로 생긴 것을 눈으로 확인했다.
- [] 부수효과 도구(Write) 호출 때 권한 확인이 뜬 것을 직접 봤다.
- [] `/exit` 후 `claude --continue` 로 직전 대화가 이어지는 것을 화면으로 확인했다.
- [] `claude (REPL) · claude -p (1회성) · --continue · --resume` 의 차이를 내가 겪은 경험으로 설명할 수 있다.

단원 미니 프로젝트: 내 자리 만들기

이 단원(u1)에는 개념 챕터가 이 장 하나뿐이므로, 단원 미니 프로젝트도 여기서 마무리합니다.

미니 프로젝트 — "내 자리 만들기: 설치·인증하고 첫 세션 기록" 목표: Windows 11에 Claude Code를 설치·인증하고, 첫 세션에서 작은 파일을 만들어 실행하며 온보딩 환경을 점검합니다.

마일스톤(증분):

1. **M1 설치 확인:** `claude --version · git --version` 이 버전을 출력한다.
2. **M2 인증 확정:** `/status` 의 `Login method` 가 의도한 방식(구독 또는 API 키)과 일치한다.
3. **M3 첫 산출물:** 빈 폴더에서 `greet.py` (이름을 받아 인사말을 돌려주는 함수)를 에이전트로 만들고 실행해 출력을 확인한다.
4. **M4 세션 재개:** 세션을 `/exit` 로 닫은 뒤 `claude --continue` 로 다시 열어 맥락이 이어지는지 확인한다.

산출물: 위 4단계의 명령과 결과를 적은 "첫 세션 기록" 메모(위 점검표 형식).

미니 프로젝트 자가체크

- [] M1~M4를 모두 실제로 실행해 결과를 확인했다.
- [] greet.py의 출력이 코드의 # 출력: 주석과 정확히 일치했다.
- [] --continue로 연 세션이 직전 작업을 기억하고 있었다.
- [] 인증 방식을 추측이 아니라 /status 출력으로 확정했다.

11. 이 책의 실습 저장소 준비 (이후 모든 장의 무대)

지금까지는 빈 폴더에서 hello.py 같은 작은 파일을 직접 만들어 보았습니다. 그러나 2장부터 다루는 일들 — 낯선 코드 파악(4장), 안전한 변경(5~6장), 테스트 검증(7장), CLAUDE.md 작성(8장), 첫 커밋·PR(9~10장), 그리고 종합 프로젝트(13~16장) — 은 모두 "팀에서 받은 기존 저장소" 하나를 무대로 합니다. 이 책은 그 저장소를 team-tasks-api 라는 작은 FastAPI 서비스로 함께 제공합니다. 신입이 첫 업무로 받는 "할 일(task) 관리 API"라고 생각하면 됩니다.

이 저장소는 책과 함께 제공되는 자료의 assets/team-tasks-api/ 폴더에 들어 있습니다. 지금 여러분의 작업 폴더로 복사해 두고, 이후 모든 장의 미니 프로젝트를 이 한 저장소에 누적해 나갑니다.

무엇이 들어 있나

```
team-tasks-api/
  main.py          # FastAPI 앱 진입점 + GET /health
  models.py        # 모델(Task, TaskCreate)과 status 열거형(todo, done)
  store.py         # 인메모리 저장소(create, get_all, get)
  routers/tasks.py # GET /tasks 목록, POST /tasks 생성, GET /tasks/{id}
  tests/test_tasks.py # 기본 기능 pytest(TestClient)
  requirements.txt # fastapi, pytest, httpx
```

지금은 완료 처리·상태 필터가 들어가기 전의 기본 버전입니다. 그 두 기능을 직접 더하는 것이 13~16장 종합 프로젝트의 과제입니다.

핵심 두 파일만 미리 살펴봅니다. 데이터 모델과 라우터입니다.

```
# models.py
from enum import Enum
from pydantic import BaseModel

class TaskStatus(str, Enum):
    todo = "todo"
    done = "done"

class Task(BaseModel):
    id: int
    title: str
    status: TaskStatus = TaskStatus.todo
```

```
# routers/tasks.py (발췌)
@router.get("", response_model=list[Task])
def list_tasks() -> list[Task]:
    return store.get_all_tasks()

@router.get("/{task_id}", response_model=Task)
def get_task(task_id: int) -> Task:
    task = store.get_task(task_id)
    if task is None:
        raise HTTPException(status_code=404, detail="Task not found")
    return task
```

worked example: 실습 저장소 셋업과 동작 확인

작업 폴더에 `team-tasks-api` 를 복사한 뒤, 그 폴더에서 다음을 차례로 실행합니다(Windows 11, PowerShell + Git Bash).

```
git init
git add .
git commit -m "chore: team-tasks-api 초기 상태"
```

```
python -m venv .venv
.venv\Scripts\Activate.ps1
pip install -r requirements.txt
```

테스트가 모두 통과해야 준비가 끝난 것입니다. **"검증 전까지는 끝난 게 아니다"**(7장)를 여기서부터 습관으로 들입니다.

```
> python -m pytest -q
..... [100%]
7 passed
```

서버를 띄워 브라우저로도 확인할 수 있습니다. `http://127.0.0.1:8000/docs` (Swagger UI)에서 `GET /tasks` 를 눌러 보면 됩니다.

```
> python -m uvicorn main:app --reload
INFO:      Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
```

GitHub에 올려 원격(origin) 만들기

지금까지는 **내 컴퓨터 안의 로컬 저장소**만 만들었습니다. 그런데 9장에서 변경을 `git push` 로 올리고 `gh` 로 첫 PR을 만들려면, 이 저장소가 **GitHub에도 있어야** 합니다. 실무에서는 팀의 저장소를 `git clone` 으로 받으면 원격(origin)이 자동으로 잡히지만, 이 책은 `team-tasks-api` 를 실습용 폴더로 받았으므로 **내 GitHub 계정에 올려** 원격을 직접 만들어 둡니다. 한 번만 해 두면 됩니다.

먼저 GitHub CLI(`gh`, 깃허브 명령줄 도구)를 설치합니다(Windows 11은 `winget`, 또는 cli.github.com에서 설치 관리자).

```
winget install --id GitHub.cli
```

설치한 새 터미널에서 GitHub 계정으로 한 번 로그인합니다. 브라우저가 열리고 안내대로 인증하면 됩니다.

```
gh auth login
```

이제 이 폴더를 내 GitHub 계정의 새 저장소로 올리고 원격 이름 `origin` 으로 연결합니다. 아래 한 명령이 저장소 생성·`origin` 연결·첫 푸시까지 한 번에 처리합니다.

```
gh repo create team-tasks-api --private --source=. --remote=origin --push
```

`--source=.` 는 지금 폴더를 올린다는 뜻, `--remote=origin` 은 원격 이름을 `origin` 으로 둔다는 뜻, `--push` 는 첫 푸시까지 한 번에 한다는 뜻입니다. 끝나면 원격이 연결됐는지 확인합니다.

```
git remote -v
```

```
origin https://github.com/<내-계정>/team-tasks-api.git (fetch)
origin https://github.com/<내-계정>/team-tasks-api.git (push)
```

`origin` 이 보이면 준비가 끝났습니다. 9장의 `git push -u origin ...` 과 `gh pr create` 가 이제 동작합니다.

왜 로컬 git 저장소 + GitHub 원격을 둘 다 준비하나 9~10장에서 변경을 브랜치·커밋(로컬)으로 쌓고, `push`·PR(원격)로 올리려면 둘 다 필요합니다. 지금 `git init` (로컬 저장소)과 `gh repo create` (GitHub 원격 `origin`)를 해 두면, 이후 각 장에서 만든 변경을 깔끔한 단위로 커밋해 첫 PR까지 올릴 수 있습니다. 앞으로 **장마다 새 작업 브랜치를 만들어** 실습하고, 끝나면 `python -m pytest -q` 로 검증한 뒤 진행하세요.

12. 흔한 오류와 오개념

오개념 1: "Claude Code는 똑똑한 자동완성이다." 아닙니다. 자동완성은 커서 주변의 다음 몇 줄을 제안하고 멈추며, 파일을 직접 만들거나 명령을 실행하지 못합니다. Claude Code는 **파일을 직접 생성·편집하고 명령을 실행하는 에이전트**입니다. 8장 worked example에서 보았듯 `hello.py` 를 만들고 `python hello.py` 까지 스스로 실행해 결과를 보여 줍니다. "다음 줄 제안기"가 아니라 "옆자리 페어 프로그래머"로 이해해야 도구의 쓰임이 보입니다.

오개념 2: "구독을 결제했으니 어떻게 로그인하든 구독으로 쓰이겠지." 틀립니다.

`ANTHROPIC_API_KEY` 환경 변수가 설정돼 있으면 Claude Code는 구독이 아니라 그 API 키를 우선 사용해 **종량 과금이 발생합니다**. 예전에 API 실습을 하며 키를 영구 환경 변수로 박아 둔 신입이 자기도 모르게 종량 과금을 쓰는 일이 흔합니다. 구독을 쓰려면 `echo $env:ANTHROPIC_API_KEY` 로 키가 비어 있는지 확인하고, `/status` 의 `Login method` 가 `subscription` 인지 **눈으로 확인**해야 합니다.

오개념 3: "터미널을 닫으면 대화가 사라진다." 아닙니다. Claude Code는 세션을 저장합니다. 같은 작업 폴더에서 `claude --continue` 를 실행하면 **가장 최근 대화의 맥락이 그대로 복원**되고, `claude --resume` 은 저장된 여러 세션 중 골라 재개합니다. 닫혔다고 처음부터 다시 설명할 필요가 없습니다. 단, 세션은 작업 폴더 기준으로 관리되므로, 재개하려면 **그 작업을 하던 폴더에서** 명령을 실행해야 합니다.

오개념 4: "claude 든 claude -p 든 그게 그거다." 다릅니다. `claude` 는 대화가 누적되는 REPL을 열어 여러 턴에 걸쳐 함께 작업합니다. `claude -p` "질문" 은 답 한 번을 출력하고 **곧바로 종료**하며 맥락을 이어가지 않습니다. 함께 코드를 만들고 고치는 작업에 `-p` 를 쓰면 매번 맥락이 끊겨

비효율적입니다. 반대로 "폴더 요약 한 줄"처럼 한 번에 끝날 질의에 REPL을 여는 것도 번거롭습니다. 작업 성격에 맞게 고릅니다.

자주 막히는 설치·인증 문제

- `irm` 이 인식되지 않는다 → PowerShell이 아니라 CMD를 열었습니다. PowerShell을 다시 엽니다.
- `claude` 가 명령으로 인식되지 않는다 → 설치 직후 PATH가 갱신되지 않았습니다. **터미널을 새로 엽니다.**
- 브라우저 로그인에 열리지 않는다 → `ANTHROPIC_API_KEY` 가 이미 설정돼 API 키 방식으로 넘어갔을 수 있습니다. 변수를 확인합니다.
- Bash 명령이 어색하게 동작한다 → Git for Windows(Git Bash)를 설치하지 않았습니다.

이 장 요약

- **Claude Code**는 터미널·IDE·웹에서 동작하는 에이전트형 코딩 CLI로, 파일을 직접 읽고 편집하고 실행하는 페어 프로그래머입니다.
- Windows 11 설치: **PowerShell 네이티브 설치 관리자(irm**
`https://claude.ai/install.ps1 | iex`, 권장) 또는 **npm(@anthropic-ai/claude-code)** 입니다. Bash 도구를 위해 **Git for Windows**를 함께 둡니다.
- 인증은 **구독(Pro/Max)** 과 **API 키** 두 갈래이며, `ANTHROPIC_API_KEY` 가 있으면 **API 키가 우선**됩니다. 구독을 쓰려면 그 변수를 비워 두고, `/status` 로 방식을 확인합니다.
- `claude` (대화형 REPL)와 `claude -p` (1회성 실행)를 작업 성격에 맞게 고릅니다.
- 세션은 저장되며 `--continue` (최근 바로 잇기), `--resume` (목록에서 고르기)으로 맥락을 이어갑니다.

다음 장에서는 이 페어 프로그래머가 **어떻게** 일하는지, 즉 "관찰 → 계획 → 도구 호출 → 결과 확인"의 에이전트 루프를 자동완성과 대비해 자세히 들여다봅니다.

참고 출처

- [Set up Claude Code — Claude Docs](#)
- [Advanced setup — Claude Code Docs](#)
- [Your first day in Claude Code — Claude Help Center](#)
- [Troubleshoot Claude Code installation and authentication — Claude Help Center](#)

에이전트형 코딩의 작동 방식: 관찰 → 계획 → 도구 → 확인

이 장의 학습 목표

- 같은 작업 하나를 두고 자동완성과 에이전트형 코딩의 차이를 구체적 예 1개 이상으로 설명할 수 있습니다.
- 에이전트가 수행한 한 작업의 실행 기록에서 관찰·계획·도구 호출·결과 확인 네 단계를 각각 짚어 식별할 수 있습니다.
- 주어진 도구 호출 순서에서 읽기 전용 도구(Read·Glob·Grep)와 부수효과 도구(Edit·Write·Bash)를 구분해 각각 언제 쓰였는지 읽어 낼 수 있습니다.
- 에이전트 루프의 어느 단계에서 모호한 지시가 잘못된 가정으로 이어지는지 짚어, 명확한 지시와 검토가 왜 필요한지 근거를 들어 설명할 수 있습니다.

선수 지식 상기(c01): 앞 장에서 Claude Code를 설치·인증하고 첫 세션을 열어, 빈 폴더에서 `hello.py` 를 만들어 실행시켜 보았습니다. 그때 "지시 → 파일 생성 → 실행 → 결과 확인"이 한 흐름으로 이어지는 것을 보았는데, 이 장은 바로 **그 흐름의 정체**를 자세히 들여다봅니다. 1장에서 도구를 켜는 법을 배웠다면, 2장에서는 그 도구가 **어떻게 일하는지**를 이해합니다.

이 장에서 가르치는 개념은 하나입니다.

- **개념 c02 — 에이전트 루프와 자동완성과의 차이(the agent loop versus autocomplete):** 에이전트형 코딩이 자동완성과 근본적으로 다른 이유를 "관찰 → 계획 → 도구 호출 → 결과 확인"의 루프로 이해하고, 어떤 도구가 언제 쓰이는지 읽어 내며, 왜 명확한 지시와 검토가 핵심인지 설명하는 법.

1. 도입: 자동완성과 무엇이 다른가

신입이 면접이나 학교에서 써 본 AI 코딩 도구는 대개 **자동완성(autocomplete)** 이었을 것입니다. 편집기에서 코드를 타이핑하면 회색 글씨로 다음 몇 줄이 떠오르고, Tab을 누르면 받아들여

지는 방식입니다. 빠르고 편리하지만 한계가 분명합니다. 자동완성은 **"지금 커서가 놓인 한 지점"**의 문맥만 봅니다. 옆 파일을 열어 보지도, 테스트를 돌리지도, 여러 파일에 걸친 변경을 스스로 적용하지도 못합니다. 결정과 실행은 전부 사람의 손에 남습니다.

1장에서 본 Claude Code는 달랐습니다. "hello.py를 만들어 실행해 줘"라는 한 문장에, 도구는 **파일을 만들고(쓰기) 명령을 실행하고(Bash) 결과를 확인**하는 데까지 스스로 갔습니다. 사람은 한 줄 한 줄을 받아 적은 것이 아니라, 목표를 주고 결과를 검토했습니다.

이 차이가 어디서 오는지를 정확히 이해하는 것이 이 장의 목표입니다. 왜냐하면 이 작동 방식을 알아야 **"왜 모호하게 지시하면 엉뚱한 결과가 나오는지"**, **"왜 에이전트가 만든 변경을 반드시 검토해야 하는지"**가 자연스럽게 이해되기 때문입니다. 도구의 작동 원리를 모르면 도구를 탓하게 되지만, 알면 더 잘 부릴 수 있습니다.

한눈에 보는 이 장의 지도

- 자동완성과 에이전트의 근본 차이 → **"다음 토큰 제안" vs "목표를 받아 루프를 돌"**
- 에이전트가 일하는 방식 → **관찰 → 계획 → 도구 호출 → 결과 확인의 4단계 루프**
- 에이전트가 쓰는 두 종류 도구 → **읽기 전용(Read·Glob·Grep) vs 부수효과(Edit·Write·Bash)**
- 변경을 안전하게 보는 법 → **diff로 제시하고 사람이 검토**
- 루프가 끊기는 지점 → **모호한 지시·잘못된 가정**

2. 정의: 에이전트형 코딩이란

정의

이 장의 핵심 개념을 한 문장으로 정의하겠습니다.

에이전트형 코딩(agent coding)은 모델이 커서 위치에서 다음 코드 한 토막을 제안하는 자동완성과 달리, **목표를 받아 "관찰(코드·파일을 읽고 검색) → 계획(무엇을 할지 정함) → 도구 호출(Read·Glob·Grep·Edit·Write·Bash 등 실제 도구 실행) → 결과 확인(실행·테스트로 검증)"의 루프를 스스로 반복하는 작동 방식**입니다. Claude Code는 이 루프를 돌며 코드베이스를 직접 탐색하고 변경을 만들어 diff로 보여 주므로, 사용자는 코드를 통째로 붙여넣을 필요 없이 에이전트가 직접 파일을 읽게 합니다.

용어를 정리하겠습니다.

- **토큰(token)**: 모델이 다루는 텍스트의 작은 조각(대략 단어·기호 단위)입니다. 자동완성은 본질적으로 "다음 토큰"을 예측해 이어 붙이는 일입니다.
- **도구(tool)**: 에이전트가 실제 세계에 작용하기 위해 호출하는 기능입니다. 파일을 읽는 Read, 검색하는 Grep, 편집하는 Edit, 명령을 실행하는 Bash 등이 있습니다.
- **루프(loop)**: 한 번 하고 끝나는 것이 아니라, 결과를 보고 다시 관찰·계획·실행으로 돌아오는 반복 구조입니다.

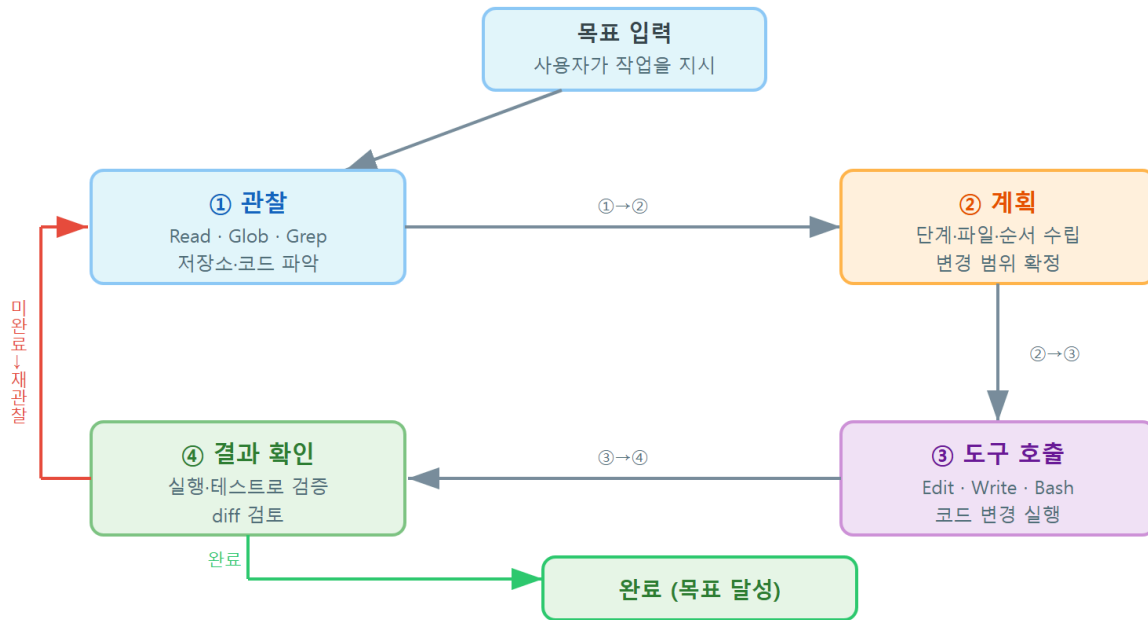
자동완성과의 대비

구분	자동완성(autocomplete)	에이전트형 코딩(Claude Code)
보는 범위	커서 주변 한 지점	프로젝트 전체(스스로 탐색)
받는 입력	지금까지 타이핑한 코드	자연어로 된 목표
하는 일	다음 몇 토큰 제안	읽기·검색·편집·실행까지
누가 실행하나	사람이 직접 타이핑·실행	에이전트가 도구로 실행
작업 단위	한 줄~한 블록	완결된 작업(여러 파일·여러 단계)
검증	사람이 따로	에이전트가 실행해 확인, 사람은 검토

한 문장 비유 자동완성이 "받아쓰기를 도와주는 손"이라면, 에이전트형 코딩은 "**일을 통째로 맡길 수 있는 동료**"입니다. 손에게는 "이 단어 다음"을 맡기지만, 동료에게는 "이 기능을 추가해 테스트까지 통과시켜 줘"를 맡길 수 있습니다. 그래서 동료에게는 **명확한 지시**가 필요합니다.

3. 동작 원리: 관찰 → 계획 → 도구 호출 → 결과 확인 루프

에이전트 루프: 관찰→계획→도구 호출→결과 확인



에이전트형 코딩이 자동완성과 근본적으로 다른 이유는 **루프로 동작하기** 때문입니다. 사람이 한 번 지시하면, 에이전트는 다음 네 단계를 한 번 또는 여러 번 돕니다.

1. **관찰(observe)**: 작업에 필요한 코드를 스스로 읽습니다. 폴더 구조를 훑고(Glob), 키워드를 검색하고(Grep), 관련 파일을 엽니다(Read). 사람이 코드를 붙여넣어 줄 필요가 없습니다.
2. **계획(plan)**: 관찰한 내용을 바탕으로 "어떤 파일의 어디를 어떻게 바꿀지" 순서를 세웁니다. 복잡한 작업이면 사람에게 계획을 먼저 보여 주기도 합니다(플랜 모드, 6장).
3. **도구 호출(act)**: 계획대로 부수효과가 있는 도구를 씁니다. 파일을 외과적으로 고치고(Edit), 새 파일을 만들고(Write), 명령을 실행합니다(Bash). 부수효과 도구는 권한 확인을 거칩니다(5장).
4. **결과 확인(verify)**: 변경을 적용한 뒤 테스트나 실행으로 결과를 확인합니다. 기대와 다르면 1번으로 돌아가 다시 관찰·교정합니다.

이 네 단계가 **사람의 피드백을 받아 반복**되는 것이 핵심입니다. 자동완성은 1회성 제안이지만, 에이전트는 "고쳤다 → 실행했다 → 실패했다 → 원인을 찾아 다시 고쳤다"를 스스로 돌 수 있습니다.

한 작업을 루프로 따라가 보기

예를 들어 "tasks 목록을 돌려주는 엔드포인트가 빈 리스트를 반환하는 문제를 고쳐 줘"라는 지시를 받으면, 에이전트의 머릿속은 대략 이렇게 흐릅니다.

단계	에이전트가 하는 일	쓰는 도구
관찰	"tasks"가 어디서 정의·사용되는지 검색하고 관련 파일을 읽음	Grep, Read
계획	"라우터의 반환 부분이 빈 리스트를 주고 있으니 저장소에서 읽어오도록 고친다"	(도구 없음, 추론)
도구 호출	해당 줄을 수정	Edit
결과 확인	테스트를 돌려 통과를 확인	Bash(pytest)

만약 결과 확인에서 테스트가 실패하면, 에이전트는 실패 메시지를 **새로운 관찰**로 삼아 다시 루프를 돕니다. 이 반복이 에이전트형 코딩의 본질입니다.

기억할 한 가지 사람의 역할이 "코드를 받아 적는 사람"에서 **"목표를 주고 결과를 검토·교정하는 사람"**으로 바뀝니다. 그래서 다음 장(3장)이 곧바로 "어떻게 지시할 것인가(프롬프팅)"입니다. 좋은 지시가 좋은 루프를 만듭니다.

4. 읽기 전용 도구(Read·Glob·Grep)와 부수효과 도구(Edit·Write·Bash)

에이전트가 루프를 돌며 쓰는 도구는 성격에 따라 두 부류로 나뉩니다. 이 구분은 단순 분류가 아니라, **5장의 권한·승인 전체를 떠받치는 토대**이므로 지금 정확히 익혀 둡니다.

읽기 전용 도구: 보기만 한다

도구	하는 일	비유
Read	파일 하나를 열어 내용을 읽음	책을 펼쳐 읽기
Glob	이름 패턴으로 파일 목록을 찾음(예: <code>**/*.py</code>)	서가에서 제목으로 책 찾기
Grep	파일 안의 텍스트를 검색(예: <code>def create_task</code>)	책 안에서 단어 찾기

읽기 전용 도구는 파일을 **바꾸지 않습니다**. 관찰(루프 1단계)에서 주로 쓰이며, 무엇이 어디 있는지 파악하는 데 쓰입니다. 아무것도 망가뜨리지 않으므로 안심하고 마음껏 쓸 수 있습니다.

부수효과 도구: 실제로 바꾼다

도구	하는 일	비유
Edit	기존 파일의 일부를 외과적으로 고침	문서에서 한 문장 고쳐 쓰기
Write	파일을 새로 만들거나 통째로 덮어쓰	새 문서 작성·기존 문서 갈아엎기
Bash	터미널 명령을 실행(테스트·실행·삭제 등)	실제로 손을 써서 작동시키기

부수효과 도구는 파일을 바꾸거나 명령을 실행해 **세계에 영향을 줍니다**. 도구 호출(루프 3단계)과 결과 확인(4단계, 예: `pytest` 실행)에서 쓰입니다. 영향이 있으므로 Claude Code는 이런 도구를 호출할 때 **권한을 확인**합니다(5장에서 상세히). "보기만 하는 도구는 자유롭게, 바꾸는 도구는 확인을 거쳐"가 핵심입니다.

왜 이 구분이 중요한가 이 책의 흐름이 "탐색(4장) → 안전장치(5·6장) → 변경·검증(7장)"인 이유가 여기 있습니다. 읽기 전용 도구만 쓰는 **탐색**은 위험이 없어 먼저 익히고, 부수효과 도구를 쓰는 **변경**은 권한·플랜 모드라는 안전장치를 갖춘 뒤에 합니다. 신입이 "이 작업이 읽기만 하는가, 바꾸는가"를 구분할 줄 알면 사고를 크게 줄입니다.

5. worked example: 같은 작업을 자동완성식 vs 에이전트 식으로 관찰

같은 목표 "인사 함수를 추가한다"를 두 방식으로 처리하면 어떻게 다른지 직접 비교해 보겠습니다.

자동완성식: 사람이 다 한다

자동완성 도구를 쓰면 사람이 직접 파일을 열고, 함수 이름을 타이핑하기 시작하면 본문을 제안 받아 Tab으로 받아들입니다.

```
def greet(name: str) -> str:
    return f"안녕하세요, {name}님"
```

여기서 끝입니다. 이 함수가 **정말 동작하는지** 확인하려면 사람이 직접 호출 코드를 쓰고, 직접 터미널을 열어 실행해야 합니다.

```
print(greet("지훈"))
# 출력: 안녕하세요, 지훈님
```

파일 저장도, 실행도, 결과 확인도 전부 사람의 몫입니다.

에이전트식: 목표만 주면 루프가 돈다

같은 작업을 Claude Code에 맡기면 한 문장이면 됩니다.

```
> greet.py에 이름을 받아 "안녕하세요, {이름}님"을 돌려주는 greet 함수를
  추가하고, greet("지훈")을 실행해 결과를 보여 줘.
```

에이전트는 루프를 돕니다.

- **관찰:** `greet.py`가 있는지 확인합니다(Glob/Read).
- **계획:** 함수를 추가하고 호출해 실행한다는 순서를 세웁니다.
- **도구 호출:** 파일을 만들거나 고칩니다(Write/Edit).
- **결과 확인:** `python greet.py`를 실행해 출력을 보여 줍니다(Bash).

만들어진 파일과 실행 결과는 다음과 같습니다.

```
def greet(name: str) -> str:
    return f"안녕하세요, {name}님"

print(greet("지훈"))
# 출력: 안녕하세요, 지훈님
```

```
> python greet.py
안녕하세요, 지훈님
```

두 방식의 결정적 차이

- 자동완성은 "**본문 제안**"에서 멈춥니다. 파일 저장·실행·확인도 사람이 합니다.
- 에이전트는 **목표 한 문장**을 받아 관찰·편집·실행·확인까지 한 흐름으로 처리합니다.
- 결과물(코드)은 같아 보여도, **누가 일을 했는가**가 다릅니다. 에이전트식에서는 사람이 "검토자"가 됩니다.
- 그래서 사람의 핵심 능력이 "타이핑"에서 "**명확한 지시 + 결과 검토**"로 옮겨갑니다.

6. diff로 변경을 제시하고 검토하기

에이전트가 부수효과 도구로 파일을 바꿀 때, Claude Code는 무엇을 어떻게 바꿨는지를 **diff(차이)**로 보여 줍니다. diff는 변경 전후를 한눈에 비교해 주는 표준 형식으로, 신입이 **에이전트의 변경을 검토하는 가장 중요한 화면**입니다.

예를 들어 위 `greet.py`에 인사말 끝에 느낌표를 더하라고 지시하면, 다음과 같은 diff가 제시됩니다.

```
def greet(name: str) -> str:
-     return f"안녕하세요, {name}님"
+     return f"안녕하세요, {name}님!"
```

읽는 법은 간단합니다.

- `-`로 시작하는 줄(빨강): **삭제되는 줄**(변경 전).
- `+`로 시작하는 줄(초록): **추가되는 줄**(변경 후).
- 기호 없는 줄: 바뀌지 않고 맥락으로 보여 주는 줄.

이 diff를 보면 "함수 본문의 반환 문자열 끝에 **!**가 붙는다"는 변경을 정확히 파악할 수 있습니다. 다른 줄은 건드리지 않았다는 것도 확인됩니다.

한 변경이 여러 곳에 걸칠 때

변경이 한 줄에 그치지 않고 여러 곳에 흩어지면, diff는 변경된 묶음을 각각 보여 줍니다. 예를 들어 `greet` 함수에 인사말 끝의 느낌표를 더하면서 호출부의 이름도 바꾸면 두 묶음이 나옵니다.

```
def greet(name: str) -> str:
-     return f"안녕하세요, {name}님"
+     return f"안녕하세요, {name}님!"

- print(greet("지훈"))
+ print(greet("수민"))
```

이때 검토 포인트는 "두 변경이 **내가 지시한 것과 같은가**"입니다. 느낌표 추가만 지시했는데 호출부의 이름까지 바뀌었다면, 그것은 **요청하지 않은 변경**이므로 되돌리도록 지시해야 합니다.

diff는 이런 "은근슬쩍 섞인 변경"을 잡아내는 그물입니다. 변경 묶음이 많을수록 한 번에 통과시키기 쉬운데, 묶음이 늘어날수록 오히려 더 천천히 한 묶음씩 확인하는 습관이 사고를 줄입니다.

diff를 검토하는 신입의 눈 변경을 무심코 통과시키지 말고 세 가지를 봅니다.

1. **의도한 곳만 바뀌었나** — 엉뚱한 파일·엉뚱한 줄이 바뀌지 않았는지.
2. **빠진 것은 없나** — 지시한 변경이 빠짐없이 반영됐는지.
3. **부작용은 없나** — 다른 동작을 깨뜨릴 변경이 섞이지 않았는지.

이 검토 습관이 7장의 "실행·테스트로 검증"과 짝을 이뤄, 에이전트가 만든 코드를 신뢰할 수 있게 만듭니다.

7. worked example: 도구 호출 순서(Grep → Read → Edit → Bash) 따라 읽기

이번에는 조금 더 실무에 가까운 상황에서, 에이전트가 **어떤 도구를 어떤 순서로** 쓰는지 따라 읽어 보겠습니다. 무대는 1장에서 셋업한 실습 저장소 `team-tasks-api` 입니다. 이 저장소의 `routers/tasks.py` 에는 다음 목록 엔드포인트가 있습니다.

```
# routers/tasks.py (team-tasks-api 발췌)
@router.get("", response_model=list[Task])
def list_tasks() -> list[Task]:
    return store.get_all_tasks()
```

여기에 "할 일 개수를 돌려주는 `GET /tasks/count` 엔드포인트를 추가해 줘"라고 지시했을 때, 에이전트가 도는 루프를 도구 호출 순서로 보겠습니다.

1) Grep — 어디서 정의되나 검색(관찰)

```
> Grep: "tasks" in routers/
→ routers/tasks.py:11 @router.get("", response_model=list[Task])
routers/tasks.py:12 def list_tasks() -> list[Task]:
```

`/tasks` 관련 엔드포인트가 `routers/tasks.py` 에 모여 있다는 것을 찾아냈습니다.

2) Read — 해당 파일을 읽어 맥락 파악(관찰)

```
> Read: routers/tasks.py (30 lines)
```

기존 엔드포인트가 `@router.get(...)` 데코레이터와 함수로 정의되고, 데이터는 `store` 에서 읽어 오는 스타일을 파악합니다. 같은 스타일로 추가해야 하기 때문입니다.

3) Edit — 새 엔드포인트를 같은 스타일로 추가(도구 호출)

에이전트가 제시하는 diff는 다음과 같습니다. 새 `/count` 경로는 `/task_id` 보다 위에 두어야 합니다. 그렇지 않으면 `count` 가 `task_id` 값으로 잡혀 버립니다.

```
return store.create_task(data)

+ @router.get("/count")
+ def count_tasks() -> dict:
+     return {"count": len(store.get_all_tasks())}
+
+
+ @router.get("/{task_id}", response_model=Task)
+ def get_task(task_id: int) -> Task:
```

4) Bash — 테스트로 결과 확인(결과 확인)

마지막으로 동작을 검증합니다. `team-tasks-api` 의 `tests/test_tasks.py` 에 테스트를 더해 저장소 전체를 돌립니다.

```
# tests/test_tasks.py (추가)
def test_count_tasks() -> None:
    client.post("/tasks", json={"title": "빨래"})
    client.post("/tasks", json={"title": "설거지"})
    response = client.get("/tasks/count")
    assert response.status_code == 200
    assert response.json() == {"count": 2}
```

```
> python -m pytest -q
.....
8 passed [100%]
```

도구 호출 순서를 읽어 내는 법

- **Grep → Read**(관찰): "무엇이 어디 있나"를 먼저 찾고, 그 파일을 읽어 기존 스타일을 파악합니다. 그래서 추가된 엔드포인트가 기존과 같은 모양(`@router.get`, `store` 사용)입니다.

- **Edit**(도구 호출): 파일 전체를 갈아엎지 않고, 필요한 부분만 외과적으로 더합니다(diff의 + 줄).
- **Bash: pytest**(결과 확인): 기존 7개에 새 1개가 더해져 8 passed 가 나옵니다. 이는 `/tasks/count` 가 빨래.설거지 2개에 대해 `{"count": 2}` 를 정확히 돌려준다는 증거입니다. 이 마지막 단계가 없으면 "동작한다"는 주장은 근거가 없습니다(7장).
- 신입은 이 순서를 화면에서 따라 읽으며 "에이전트가 지금 관찰 중인가, 바꾸는 중인가, 검증 중인가"를 가늠할 수 있어야 합니다.

8. 루프가 끊기는 지점: 모호함.잘못된 가정

에이전트 루프는 강력하지만 만능이 아닙니다. 루프가 헛돌거나 엉뚱한 방향으로 가는 데는 거의 항상 같은 원인이 있습니다. 바로 **지시의 모호함**과 그로 인한 **잘못된 가정**입니다.

에이전트는 관찰한 것과 지시받은 것을 합쳐 계획을 세웁니다. 그런데 지시가 모호하면 빈틈을 **가정으로 메웁니다**. 그 가정이 사용자의 의도와 다르면, 그 뒤의 도구 호출.결과 확인이 전부 어긋난 방향으로 진행됩니다.

예: 모호한 지시가 부르는 어긋남

```
> 엔드포인트 좀 고쳐 줘.
```

이 지시에는 **어느 엔드포인트인지, 무엇이 문제인지, 어떻게 되어야 하는지**가 없습니다. 에이전트는 가정할 수밖에 없습니다. "아마 `/tasks` 겠지", "아마 반환 형식을 바꾸라는 거겠지" 하고 추측해 엉뚱한 곳을 고칩니다. 결과는 사용자가 원하는 것과 다릅니다.

반면 같은 의도를 구체적으로 주면 가정의 여지가 사라집니다.

```
> team-tasks-api의 routers/tasks.py에서 목록 엔드포인트 옆에
  GET /tasks/count를 추가해, store.get_all_tasks()의 개수를
  {"count": n}으로 반환하도록 고치고, pytest로 개수가 맞는지 확인해 줘.
```

이제 에이전트는 추측 없이 정확한 파일.문제.기대 결과.완료 조건을 알고 루프를 돕니다.

지시	에이전트의 가정	결과
"엔드포인트 좀 고쳐 줘"	대상·문제·기대를 전부 추측	어긋날 가능성 큼
"routers/tasks.py에 GET /tasks/count를 추가해 store 개수를 반환하도록 고치고 pytest로 확인"	추측할 여지 거의 없음	의도대로 진행

이것이 다음 장(3장 효과적 프롬프팅)이 필요한 이유입니다. 루프를 이해하면, 좋은 지시가 곧 좋은 결과의 전제임이 분명해집니다.

루프가 끊기는 또 다른 지점: 검증 생략 지시가 좋아도 결과 확인(4단계)을 건너뛰면 루프가 미완성으로 끝납니다. 에이전트가 "고쳤습니다"라고 해도 실행·테스트로 확인하기 전까지는 추정일 뿐입니다. 7장에서 이 습관을 본격적으로 다룹니다.

9. 즉시 연습: team-tasks-api에서 직접 돌려 보기

지금까지 읽은 루프를, 이번에는 **종이에 채우지 말고 실제로 돌려 봅니다**. 1장에서 준비한 `team-tasks-api` 저장소를 Claude Code로 열고 아래 과제를 직접 수행한 뒤, **실제로 일어난 일**을 기록하세요. 미리 정한 답을 빈칸에 적는 것이 아니라, **화면에 나타난 도구 호출과 테스트 결과를 그대로 옮기는 것**이 핵심입니다. 손으로 한 번 돌려 봐야 "에이전트가 지금 관찰 중인가, 바꾸는 중인가, 검증 중인가"가 눈에 들어옵니다.

모범: 한 작업을 4단계로 분해(실제 실행 예)

작업 지시: " /tasks/count 엔드포인트를 추가하고 테스트로 확인해 줘."

```
[관찰] count 대상이 어디 있나? → Grep "tasks", Read routers/tasks.py
      → routers/tasks.py에 목록 엔드포인트(list_tasks)와 store 사용이 있음(기존 스타일 파악)
[계획] @router.get("/count") 함수를 추가, len(store.get_all_tasks())를 count로 반환,
      /{task_id}보다 위에 둔다
[도구 호출] Edit로 routers/tasks.py에 엔드포인트 추가(diff의 + 줄)
[결과 확인] Bash: python -m pytest -q → 8 passed (count == 2 확인)
사용한 도구: Grep, Read(읽기 전용) / Edit, Bash(부수효과)
```

직접 해보기 (team-tasks-api에서 실제로 실행)

연습 1 — 새 엔드포인트를 추가하며 루프를 관찰하라.

team-tasks-api 폴더에서 `claude` 를 실행하고, 다음 지시를 그대로 주세요.

```
> team-tasks-api의 routers/tasks.py에 할 일 제목만 모아 반환하는
  GET /tasks/titles 엔드포인트를 추가하고, python -m pytest -q로 확인해 줘.
```

수행되는 동안 에이전트가 실제로 호출한 도구를 화면에서 보고, 아래에 *방금 본 것*을 적으세요 (미리 정한 답이 아니라 화면에 나타난 도구 이름·diff·테스트 출력을 그대로 옮깁니다).

```
[관찰] 실제로 부른 읽기 도구와 알아낸 것:
[계획] 에이전트가 제시한 계획(또는 diff 직전 설명):
[도구 호출] 실제로 바꾼 파일과 추가된 + 줄:
[결과 확인] python -m pytest -q 실제 출력:
```

끝나면 서버를 띄워(`python -m uvicorn main:app --reload`) `http://127.0.0.1:8000/docs` 에서 `GET /tasks/titles` 를 직접 눌러 응답을 눈으로 확인하세요.

연습 2 — 방금 세션에서 실제로 쓰인 도구를 분류하라.

연습 1에서 에이전트가 호출한 도구들을 떠올려 읽기 전용과 부수효과로 나누세요. 책에서 외운 목록이 아니라, 이번 세션 화면에서 실제로 본 도구만 적습니다(부수효과 도구에는 권한 확인이 났을 것입니다).

```
이번 세션에서 본 읽기 전용 도구:
이번 세션에서 본 부수효과 도구(권한 확인이 났던 것):
```

연습 3 — 모호한 지시를 구체화해 실제로 줘 보라.

"이 API 좀 빠르게 만들어 줘"처럼 모호한 지시는 에이전트를 헤매게 합니다. 이를 team-tasks-api에 대고 *실행 가능한* 지시로 다시 쓰고(대상 파일·문제·기대·완료 조건 포함), **실제로 에이전트에 줘 보세요.** 결과가 의도대로인지 확인합니다.

```
다시 쓴 구체 지시:
실제로 준 뒤 결과(의도대로였나? pytest 통과했나? 빗나갔다면 어디서?):
```

자가체크 — 머리로가 아니라 손으로 했는지 확인하세요.

- [] 연습 1을 team-tasks-api에서 실제로 실행했고, `python -m pytest -q` 가 통과하는 것을 화면 출력으로 봤다.
- [] [도구 호출]에 적은 것이 추측이 아니라 화면에 실제로 나타난 diff·도구 이름이다.

- [] `GET /tasks/titles`의 응답을 Swagger UI 또는 테스트로 눈으로 확인했다.
- [] 연습 3에서 구체 지시를 실제로 쥬 보고 결과를 확인했다.
- [] 모호한 지시와 구체 지시가 에이전트의 동작을 어떻게 다르게 만드는지 직접 겪어 설명할 수 있다.

단원 미니 프로젝트: 에이전트 루프 관찰 일지

이 단원(u2)에는 개념 챕터가 이 장 하나뿐이므로, 단원 미니 프로젝트도 여기서 마무리합니다.

미니 프로젝트 — "에이전트 루프 관찰 일지" 목표: 작은 작업 하나를 에이전트에 맡기고, 관찰 → 계획 → 도구 호출 → 결과 확인의 각 단계를 직접 기록해 자동완성과의 차이를 정리합니다.

마일스톤(증분):

1. **M1 작업 선정·브랜치:** `team-tasks-api`에서 새 작업 브랜치를 만든다(`git checkout -b loop-journal`). 작업은 "`routers/tasks.py`에 `GET /tasks/count` 추가"처럼 이 저장소의 실제 파일을 대상으로 정한다.
2. **M2 지시·관찰:** 구체적으로 지시하고, 에이전트가 어떤 도구로 무엇을 읽는지(Grep/Read) 화면을 보며 적는다.
3. **M3 변경·검토:** 제시된 diff에서 무엇이 바뀌는지(+/- 줄)를 적고, 의도한 곳만 바뀌었는지 검토한다.
4. **M4 검증·정리:** 테스트를 실행(`python -m pytest -q`)한 결과를 적고, "자동완성이었다면 내가 직접 해야 했을 일"을 한 문단으로 정리한다.

실행으로 검증: `python -m pytest -q`가 통과해야(8 passed) 끝입니다. 통과를 확인한 뒤 브랜치를 커밋합니다.

산출물: 위 4단계를 채운 "관찰 일지"(연습 1의 분해 형식 + 자동완성 대비 소감 한 문단) + 통과한 pytest 출력.

미니 프로젝트 자가체크

- [] `team-tasks-api`에서 새 작업 브랜치를 만들어 작업했다.
- [] 네 단계 각각에 실제 사용된 도구 이름을 적었다.
- [] diff를 보고 "의도한 곳만 바뀌었는가"를 직접 검토했다.
- [] `python -m pytest -q`가 통과(8 passed)하는 것을 화면 출력으로 확인했다.
- [] 자동완성과 에이전트의 차이를 내 작업 경험으로 한 문단 설명했다.

10. 흔한 오류와 오개념

오개념 1: "에이전트는 한 번에 완성된 정답을 내놓는다." 아닙니다. 에이전트는 루프를 돌며 점진적으로 만듭니다. 관찰 → 계획 → 도구 호출 → 결과 확인을 돌고, 결과 확인에서 문제가 보이면 그것을 새 관찰로 삼아 다시 돕니다. 7장 worked example의 pytest 실패처럼, 실패는 루프의 끝이 아니라 다음 반복의 입력입니다. "한 방에 완벽"을 기대하면 에이전트를 잘못 쓰게 됩니다. 검토와 교정 반복이 정석입니다.

오개념 2: "도구 호출은 일종의 마법이다." 아닙니다. 도구 호출은 읽기(Read)·검색(Grep·Glob)·편집(Edit·Write)·실행(Bash)의 평범한 조합일 뿐입니다. 7장에서 본 것처럼 "Grep으로 찾고 → Read로 읽고 → Edit로 고치고 → Bash로 테스트"가 전부입니다. 이 점을 알면 에이전트가 지금 무엇을 하는지 화면에서 따라 읽을 수 있고, 엉뚱한 도구를 쓰면 바로 알아챌 수 있습니다.

오개념 3: "에이전트가 '됐다'고 하면 된 것이다." 틀립니다. 결과 확인(실행·테스트) 없이 끝난 작업은 검증된 것이 아닙니다. 에이전트가 "엔드포인트를 추가했습니다"라고 해도, pytest가 통과했다는 출력이 없으면 그것은 추정입니다. 루프의 4번째 단계인 결과 확인을 사람이 함께 챙겨야 합니다(7장에서 본격적으로). diff 검토(6장)와 테스트 확인은 신입이 절대 건너뛰면 안 되는 두 관문입니다.

오개념 4: "코드를 채팅창에 붙여넣어 줘야 에이전트가 본다." 아닙니다. 에이전트는 직접 파일을 읽습니다(Read·Grep). 코드를 붙여넣으면 오히려 실제 파일과 어긋난 옛 버전을 보게 될 수 있습니다. 정확한 작업을 원하면 붙여넣기 대신 "routers/tasks.py를 읽고..."처럼 실제 파일을 가리키는 것이 맞습니다(다음 장의 @파일 멘션으로 더 정교해집니다).

자주 묻는 질문

Q. 에이전트가 도구를 쓰는 게 화면에 어떻게 보이나요? A. REPL에서 작업을 지시하면, 에이전트가 도구를 호출할 때마다 화면에 어떤 도구로 무엇을 했는지가 줄 단위로 표시됩니다(예: Read routers/tasks.py, Edit routers/tasks.py, Bash: python -m pytest -q). 이 줄들을 따라 읽으면 지금 에이전트가 관찰 중인지, 바꾸는 중인지, 검증 중인지 알 수 있습니다. 7장 worked example의 1)~4)가 바로 그 화면을 글로 옮긴 것입니다.

Q. 에이전트가 엉뚱한 방향으로 가면 어떻게 멈추나요? A. 1장에서 배운 대로 Esc로 진행 중인 작업을 중단합니다. 그런 뒤 지시를 더 구체적으로 다듬어 다시 줍니다. 루프가 어긋나는 원인은 대개 모호한 지시(8장)이므로, 멈추고 지시를 고치는 것이 가장 빠른 교정입니다.

Q. 읽기 전용 도구만 쓰는 작업은 권한 확인이 없나요? A. 읽기 전용 도구(Read·Glob·Grep)는 파일을 바꾸지 않으므로 자유롭게 쓰입니다. 권한 확인이 붙는 것은 부수효과 도구(Edit·Write·Bash)입니다. 자세한 권한 규칙은 5장에서 다룹니다. 지금은 "보기만 하는 도구와 바꾸는 도구가 나뉘고, 안전장치는 바꾸는 쪽에 붙는다"만 기억하면 됩니다.

Q. 결과 확인은 항상 테스트로 해야 하나요? A. 테스트(pytest)가 가장 분명한 증거이지만, 작업에 따라 명령 실행 결과·서버 응답·출력 확인으로도 합니다. 핵심은 **"에이전트의 말"이 아니라 "실제 실행 결과"로 확인**한다는 점입니다. 7장에서 실행·테스트 검증을 본격적으로 익힙니다.

이 장 요약

- 에이전트형 코딩은 자동완성("다음 토큰 제안")과 달리 **목표를 받아 루프를 도는** 작동 방식입니다.
- 루프는 **관찰(Read·Grep·Glob) → 계획 → 도구 호출(Edit·Write·Bash) → 결과 확인(실행·테스트)**의 4단계이며, 미완이면 다시 관찰로 돌아옵니다.
- 도구는 **읽기 전용(보기만 함)**과 **부수효과(실제로 바꿈)**로 나뉘고, 부수효과 도구는 권한 확인을 거칩니다(5장).
- 변경은 **diff**로 제시되며, 신입은 "의도한 곳만·빠짐없이·부작용 없이" 바뀌었는지 검토합니다.
- 루프가 끊기는 주범은 **모호한 지시 → 잘못된 가정과 검증 생략**입니다. 그래서 다음 장은 **명확한 지시(프롬프팅)**입니다.

다음 장에서는 루프를 제대로 굴리는 출발점, 곧 **목표·맥락·제약·완료 조건**을 갖춘 좋은 작업 지시를 작성하는 법을 배웁니다.

3장 효과적인 작업 지시: 목표·맥락·제약·완료 조건

이 장은 단원 3(u3)의 유일한 챕터이며, 여기서 배우는 개념은 **효과적 프롬프팅과 작업 지시(effective prompting and task instruction, c03)** 한 가지입니다. 앞 단원에서 익힌 에이전트 루프(c02) 위에, 그 루프를 *원하는 방향으로 움직이게 만드는 말*하기를 다룹니다. 신입에게 이 장의 기술은 시니어에게 일을 부탁하듯 명확하게 요청하는 법과 같습니다.

이 장을 마치면 할 수 있는 것

- 주어진 모호한 한 줄 요청을 목표·맥락·제약·완료 조건 네 요소를 모두 갖춘 작업 지시로 다시 작성할 수 있습니다.
- 하나의 큰 작업을 각각 검증 가능한 3단계 이상으로 쪼개, 한 번에 한 단계씩 순차 지시문으로 작성할 수 있습니다.
- 기존 코드 파일을 @파일 멘션으로 참조시켜, 그 파일의 스타일과 일치하는 결과를 내도록 지시문을 작성할 수 있습니다.
- 에이전트가 내놓은 계획·중간 산출을 검토해 어긋난 점 1개 이상을 찾아 교정 지시로 작성할 수 있습니다.

3.1 도입: 결과는 지시의 명확성에 달렸다

앞 단원에서 우리는 Claude Code가 **관찰 → 계획 → 도구 호출 → 결과 확인**의 루프를 돌며 일한다는 것을 보았습니다(c02). 그런데 이 루프는 출발점이 하나 있습니다. 바로 **여러분이 준 작업 지시(프롬프트)**입니다. 루프 전체는 그 한 문장에서 시작해 그 문장이 가리키는 목표를 향해 굴러갑니다. 그래서 지시가 흐릿하면 루프 전체가 흐릿한 방향으로 굴러가고, 지시가 또렷하면 루프가 또렷한 결과로 수렴합니다.

신입이 가장 먼저 겪는 오해가 여기서 생깁니다. "AI니까 알아서 잘하겠지"라고 생각하고 "엔드포인트 좀 고쳐 줘" 같은 한 줄을 던지는 것입니다. 그러면 에이전트는 *어떤* 엔드포인트인지, 무

엇이 잘못됐는지, *어떻게* 고쳐야 끝인지를 스스로 추측합니다. 추측이 맞으면 운이 좋은 것이고, 빗나가면 엉뚱한 파일을 고치거나, 멀쩡한 코드를 바꾸거나, 여러분이 원하지 않은 라이브러리를 새로 끌어옵니다. 이걸 에이전트가 멍청해서가 아니라, **줄 정보가 부족하면 빈칸을 가정으로 메우기 때문**입니다.

왜 이 개념이 지금 필요한가

지금까지(1~2장) 우리는 "에이전트가 어떻게 일하는가"를 배웠습니다. 이 장부터는 "에이전트를 어떻게 부리는가"로 넘어갑니다. 코드베이스 탐색(4장)·안전한 변경(5~6장)·검증(7장)·커밋과 PR(9장)까지, 앞으로 하는 모든 작업은 결국 **에이전트에게 무언가를 시키는 일**입니다. 그 모든 작업의 품질이 이 장에서 배우는 "지시하는 법"에 달려 있습니다. 그래서 온보딩 여정의 이른 자리에 이 개념을 둡니다.

선수 개념 한 줄 상기(c02). 에이전트는 한 번에 정답을 내는 기계가 아니라, 관찰·계획·도구 호출·결과 확인을 반복하는 루프였습니다. 이 점을 기억해야 이 장의 핵심 두 가지가 자연스럽게 이해됩니다. 첫째, **계획 단계가 정확하려면 지시가 구체적이어야** 합니다(빈칸이 적어야 가정이 줄어듭니다). 둘째, 루프는 반복이므로 **한 번에 완벽을 기대하기보다 검토하고 다시 지시하는 교정이 정석**입니다.

3.2 정의: 좋은 지시의 네 요소(목표·맥락·제약·완료 조건)

먼저 개념을 한 문장으로 못 박겠습니다.

효과적 프롬프팅과 작업 지시(c03)란, 에이전트에게 일을 시킬 때 **목표(무엇을)**, **맥락(어떤 코드·상황에서)**, **제약(무엇은 하지 말 것)**, **완료 조건(무엇이 되면 끝인지)** 네 가지를 구체적으로 담아, 큰 작업은 검증 가능한 단계로 쪼개고, 검토→피드백→재시도로 다듬어 가는 작업 요청 방식을 말합니다.

이 정의에서 가장 중요한 알맹이는 **네 요소**입니다. 하나씩 풀어 봅니다.

요소	무엇인가	빠지면 생기는 일	예
목표(goal)	무엇을 만들거나 바꾸려는가	엉뚱한 일 포함	"할 일 1건을 ID로 삭제하는 DELETE 엔드포인트를 추가"
맥락(context)	어느 코드·상황에서 하는가	엉뚱한 파일을 해맴	" routers/tasks.py 의 기존 /tasks 엔드포인트와 같은 스타일로"
제약(constraint)	무엇은 하지 말 것인가	원하지 않은 변경·의존성	"새 라이브러리는 추가하지 말고, 기존 store 를 통해 데이터를 다룬다"
완료 조건(done criteria)	무엇이 되면 끝인가	"됐다"의 기준이 없음	"없는 ID면 404를 반환하고, 이 동작을 확인하는 pytest 테스트가 통과하면 끝"

이 네 가지는 사실 **여러분이 시니어 동료에게 일을 부탁할 때 자연스럽게 말하는 것과 똑같습니다**. "이거(목표) 우리 routers/tasks.py 스타일대로(맥락) 새 라이브러리 쓰지 말고(제약) 테스트 통과하게(완료 조건) 해 줄 수 있나요?"라고 부탁하지, "이거 좀 해 줘"라고만 던지지는 않습니다. 사람에게 일을 잘 맡기는 사람은 에이전트에게도 일을 잘 맡깁니다.

프롬프트는 길수록 좋은 게 아닙니다

흔히 "길고 정중하게 쓰면 더 잘하겠지"라고 생각하지만, 길이가 아니라 **밀도**가 중요합니다. "정말 정말 꼼꼼하게 최고의 코드로 부탁드립니다" 같은 수식어는 네 요소 중 어디에도 해당하지 않아 정보량이 0입니다. 같은 글자 수라면 "없는 ID면 404"라는 한 줄(완료 조건)이 훨씬 값집니다. 짧아도 네 요소가 차 있으면 좋은 지시입니다.

3.3 동작 원리: 모호한 지시가 잘못된 가정으로 이어지는 흐름

왜 모호한 지시가 위험한지를 루프의 관점에서 들여다봅니다.

모호한 지시 vs 명확한 지시: 결과 비교



에이전트는 지시를 받으면 곧바로 **계획**을 세웁니다. 계획을 세우려면 "무엇을, 어디서, 어떻게, 어디까지"가 정해져야 하는데, 지시에 그 정보가 없으면 에이전트는 빈칸을 **가장 그럴듯한 추측**으로 메웁니다. 이 추측이 곧 "잘못된 가정"의 씨앗입니다.

두 경로를 비교하면 흐름이 또렷해집니다.

경로 A — 모호한 지시:

1. 지시: "엔드포인트 좀 고쳐 줘."
2. 빈칸 발생: *어떤* 엔드포인트? *무엇이* 문제? *어떻게* 고쳐야 끝?
3. 가정으로 메움: "아마 가장 최근에 본 `/tasks` 겠지", "아마 에러 처리가 없는 게 문제겠지", "예외를 던지게 바꾸면 되겠지".
4. 계획: 그 가정 위에 세워짐(이미 어긋났을 수 있음).
5. 도구 호출: 어긋난 계획대로 파일을 바꿈.
6. 결과: 여러분이 원한 것과 다름 → 처음부터 다시 설명.

경로 B — 네 요소를 갖춘 지시:

1. 지시: "(목표) `DELETE /tasks/{id}` 엔드포인트 추가, (맥락) `routers/tasks.py`의 기존 스타일로, (제약) 새 의존성 없이 기존 `store`를 통해 데이터 처리, (완료 조건) 없는 ID면 404 + `pytest` 통과."
2. 빈칸 거의 없음.
3. 가정 최소화.
4. 계획: 여러분의 의도와 거의 일치.
5. 도구 호출: 의도대로 변경.
6. 결과: 한 번에 기대에 근접 → 작은 교정만 남음.

핵심은 **빈칸의 개수**입니다. 지시에 빈칸이 많을수록 에이전트가 메워야 할 가정이 많아지고, 가정이 많을수록 어긋날 확률이 커집니다. 네 요소는 바로 그 빈칸을 미리 채워 주는 체크리스트입니다.

"알아서 해 줘"가 통할 때와 안 통할 때

탐색·요약처럼 *읽기만 하고 결과가 되돌릴 수 있는* 작업은 다소 모호해도 괜찮습니다("이 폴더 뭐 하는지 요약해 줘"). 하지만 *파일을 바꾸는* 작업은 모호하면 비용이 큼니다. 잘못 바꾼 것을 되돌려야 하기 때문입니다. 변경 작업일수록 네 요소를 또렷이 채우는 습관이 중요합니다.

3.4 worked example: 모호한 지시 vs 구체 지시 비교

말로만 보면 와닿지 않으니, 같은 일을 두 가지 지시로 시켜 결과 차이를 봅니다. 무대는 1장에서 셋업한 실습 저장소 `team-tasks-api` 이고, 핵심 파일 `routers/tasks.py` 의 기존 코드는 다음과 같습니다.

```
# routers/tasks.py (team-tasks-api 기존 코드)
from fastapi import APIRouter, HTTPException
from models import Task, TaskCreate
import store

router = APIRouter(prefix="/tasks", tags=["tasks"])

@router.get("", response_model=list[Task])
def list_tasks() -> list[Task]:
    return store.get_all_tasks()

@router.post("", response_model=Task, status_code=201)
def create_task(data: TaskCreate) -> Task:
    return store.create_task(data)
```

목표는 "할 일 1건을 ID로 삭제하는 엔드포인트 추가"입니다(이 저장소에는 아직 삭제 기능이 없습니다). 같은 목표를 두 가지로 지시해 봅니다. 아래는 Claude Code 대화형 세션(REPL)에서의 입력과, 그에 따른 에이전트의 진행을 요약한 것입니다.

① 모호한 지시:

```
> task 삭제 기능 좀 추가해 줘
```

이 지시에는 맥락(어느 파일·어떤 스타일)도, 제약(무엇은 하지 말 것)도, 완료 조건(무엇이 되면 끝)도 없습니다. 에이전트는 빈칸을 가정으로 메우며, 흔히 다음과 같은 어긋남이 생깁니다.

- 단건 삭제가 아니라 일괄 삭제·필터 삭제까지 끼워 넣음(목표를 과하게 추측).
- 없는 ID 처리(404)를 빠뜨림(완료 조건이 없으므로).
- `store` 를 거치지 않고 데이터 저장 방식을 멋대로 바꿈(제약이 없으므로).

결과적으로 변경 범위가 커지고, 여러분은 "이게 아닌데"를 다시 설명해야 합니다.

② 네 요소를 갖춘 구체 지시:

```
> routers/tasks.py에 할 일 1건을 ID로 삭제하는 DELETE /tasks/{task_id}
엔드포인트를 추가해 줘. 기존 list_tasks.create_task와 같은 스타일을 따르고,
데이터는 store를 통해 다루되 store.py에 delete_task 함수를 더해 써. 없는 ID면
HTTPException으로 404를 반환해야 하고, 변경 후 이 동작을 확인하는 방법을 알려 줘.
```

이번에는 목표(DELETE /tasks/{task_id}), 맥락(routers/tasks.py · 기존 스타일), 제약(store 를 통해서만 데이터 처리), 완료 조건(없는 ID면 404)이 모두 들어 있습니다. 에이전트는 빈칸이 거의 없어, 다음과 같이 좁고 정확한 변경을 제시합니다.

```
# store.py 에 추가된 함수
def delete_task(task_id: int) -> bool:
    """ID로 할 일을 삭제. 있었으면 True, 없었으면 False."""
    return _db.pop(task_id, None) is not None
```

```
# routers/tasks.py 에 추가된 엔드포인트
@router.delete("/{task_id}", status_code=204)
def delete_task(task_id: int) -> None:
    if not store.delete_task(task_id):
        raise HTTPException(status_code=404, detail="Task not found")
```

이 변경이 의도대로인지 확인해 봅시다. 아래는 핵심 로직만 떼어내 직접 실행해 본 것입니다.

```
# 동작 확인(핵심 로직만 발췌 실행)
_db = {1: {"id": 1, "title": "배포 문서 읽기", "status": "todo"}}

def delete_task(task_id):
    return _db.pop(task_id, None) is not None

print(delete_task(1)) # 있던 것 삭제
print(delete_task(99)) # 없는 것
# 출력: True
# 출력: False
```

존재하는 ID(1)는 삭제 후 `True` (라우터에서 204), 없는 ID(99)는 `False` (라우터에서 404)를 끌어내 완료 조건을 정확히 만족합니다.

무엇이 결과를 갈랐나

- 지시 ②는 **목표를 단건 삭제로 좁혀** 일괄 삭제 같은 군더더기를 막았습니다.
- "기존 스타일을 따르라"는 **맥락**이 응답 모델·함수 형태를 기존 코드에 정렬시켰습니다.
- "store 를 통해서만 데이터를 다루라"는 **제약**이 저장 방식을 멋대로 바꾸는 일을 차단했습니다.
- "없는 ID면 404"라는 **완료 조건**이 에러 처리를 명시해, 빠지기 쉬운 엣지 케이스를 끌어냈습니다.
- 같은 에이전트, 같은 목표인데 **지시의 밀도만으로** 결과 품질이 갈렸습니다.

3.5 작업 분해와 단계적 지시(한 번에 하나)

네 요소를 갖췄어도, 한 번에 너무 큰 일을 시키면 루프가 길어지고 검토가 어려워집니다. 그래서 두 번째 기법이 **작업 분해(task decomposition)**입니다. 큰 작업을 **검증 가능한 작은 단계**로 쪼개 한 번에 하나씩 지시하는 것입니다.

"검증 가능한"이 핵심입니다. 각 단계는 끝났을 때 **눈으로 또는 테스트로 확인할 수 있어야** 합니다. 예를 들어 "할 일 완료 처리 기능 추가"라는 작업은 다음처럼 쪼갤 수 있습니다.

단계	지시	어떻게 검증하나(완료 조건)
1	"완료 처리 동작을 검증할 pytest 테스트를 먼저 작성해 줘(아직 구현 없이 실패하는 상태로)."	테스트가 실행되고, 예상대로 실패 하면 끝
2	"이제 그 테스트를 통과하도록 <code>PATCH /tasks/{id}/done</code> 엔드포인트를 구현해 줘."	1단계 테스트가 통과 하면 끝
3	"기존 테스트가 모두 통과하는지 한 번에 돌려 확인해 줘."	전체 테스트 초록색이면 끝

이렇게 쪼개면 좋은 점이 셋입니다. 첫째, **각 단계가 작아 검토가 쉽습니다**(diff가 짧습니다). 둘째, **어디서 어긋났는지 바로 보입니다**(3단계 중 2단계에서 틀리면 거기만 고치면 됩니다). 셋째,

중간 검증으로 가정이 일찍 교정됩니다(1단계에서 테스트 형태가 마음에 안 들면 구현 전에 바로잡습니다).

반대로, "완료 처리 기능을 테스트까지 다 만들어서 한 번에 끝내 줘"라고 한 덩어리로 시키면, 결과가 100줄짜리 diff로 돌아와 어디가 맞고 틀린지 검토하기 버겁고, 한 곳만 틀려도 전체를 의심하게 됩니다.

이제 이 분해를 **종이에 적지 말고 team-tasks-api에서 실제로 단계별로 지시해** 봅니다. 한 덩어리로 던지는 것과, 단계로 쪼개 한 번에 하나씩 주는 것의 차이를 직접 겪어야 분해의 이득이 손에 잡힙니다.

모범: status가 todo인 항목만 거르는 필터를 3단계로 실제 실행(예)

작업: "team-tasks-api의 routers/tasks.py 에 status가 todo 인 항목만 돌려주는 필터를 추가한다." 아래는 이를 단계로 쪼개 team-tasks-api에서 실제로 돌리며 화면 출력을 옮겨 적은 예입니다.

```
[단계 1] > status가 todo인 항목만 반환하는 동작을 검증할 pytest 테스트를 먼저,
          아직 구현 없이 실패하는 상태로 tests/test_tasks.py에 추가해 줘.
          → Edit로 test_list_todo_only 추가. python -m pytest -q 실행 → 1 failed (의도대로 실패)
[단계 2] > 이제 그 테스트를 통과하도록 routers/tasks.py의 list_tasks에
          status 쿼리 파라미터(todo면 todo만)를 더해 줘. 기존 스타일을 따른다.
          → diff: list_tasks에 status 파라미터와 store.get_all_tasks() 필터 추가
          → python -m pytest -q → 직전 실패 테스트가 통과
[단계 3] > 기존 테스트가 모두 통과하는지 한 번에 돌려 확인해 줘.
          → python -m pytest -q → 9 passed (기존 동작도 안 깨짐)
```

직접 해보기 (team-tasks-api에서 실제로 실행)

즉시 연습 A — 큰 작업을 쪼개 team-tasks-api에서 단계별로 실제 실행하라 (2종 1세트 · 1/2)

team-tasks-api에서 claude 를 실행하고, 위 모범처럼 **테스트 먼저 → 구현 → 전체 확인** 순서로 한 번에 한 단계씩 지시하세요(작업: status가 todo 인 항목만 거르는 필터 추가). 각 단계에서 **화면에 실제로 나타난 diff와 python -m pytest -q** 출력을 그대로 옮겨 적으세요.

```
[단계 1: 테스트 먼저] 내가 준 지시 / pytest 실제 출력(실패했는가):
[단계 2: 구현]       에이전트가 바꾼 파일과 추가된 + 줄 / pytest 실제 출력:
[단계 3: 전체 확인]  python -m pytest -q 의 최종 출력(N passed):
```

비교를 위해, **새 세션을 열어** 같은 작업을 한 덩어리("필터 기능을 테스트까지 다 만들어 한 번에 끝내 줘")로 던져 보고, 돌아온 diff의 길이와 검토 난이도가 어떻게 달랐는지 한 줄로 적으세요.

한 덩어리로 쫓을 때 diff/검토는 어떻게 달랠나:

즉시 연습 A 자가체크 — 손으로 했는지 확인하세요.

- [] 세 단계를 team-tasks-api에서 **실제로** 한 번에 하나씩 지시했다.
- [] 단계 1에서 테스트가 의도대로 **실패**하는 것을, 단계 2에서 **통과**로 바뀌는 것을 화면 출력으로 봤다.
- [] 단계 3에서 `python -m pytest -q`가 전체 통과(N passed)하는 것을 확인했다.
- [] 한 덩어리 지시와 단계 분해 지시의 diff 길이·검토 난이도 차이를 직접 겪어 적었다.

3.6 @파일 멘션·참조로 맥락 주입하기

네 요소 중 **맥락**을 가장 정확하게 주는 도구가 @파일 멘션입니다. Claude Code 대화형 세션에서 @를 입력하면 저장소의 파일·폴더 경로를 자동완성으로 고를 수 있고, 그렇게 멘션한 파일의 내용이 **그 지시의 맥락으로 직접 주입**됩니다.

> @routers/tasks.py 의 list_tasks와 같은 스타일로 status가 todo인 항목만 돌려주는 GET /tasks/pending 엔드포인트를 추가해 줘.

여기서 @routers/tasks.py 는 "이 파일을 보고, 이 파일의 스타일을 따르라"는 맥락을 한 토큰으로 전달합니다. 폴더도 멘션할 수 있어, @routers/ 처럼 디렉토리를 가리키면 그 안의 구조를 참고하게 할 수 있습니다.

@멘션이 중요한 이유는 **붙여넣기보다 정확하고 안전하게 하기 때문**입니다. 신입이 흔히 하는 실수가 채팅창에 코드를 통째로 복사·붙여넣기 하는 것인데, 이러면 세 가지 문제가 생깁니다.

- **버전 불일치**: 붙여넣은 코드가 디스크의 **진짜 현재 파일**과 다를 수 있습니다(이미 누가 고쳤을 수도). @멘션은 항상 디스크의 실제 파일을 읽습니다.
- **맥락 누락**: 함수 하나만 붙여넣으면 그 함수가 의존하는 import·모델이 빠집니다. @로 파일을 가리키면 에이전트가 필요한 부분을 알아서 같이 읽습니다.
- **수고**: 긴 파일을 복사하는 것보다 @경로 한 번이 빠릅니다.

"같은 스타일로"를 가능하게 하는 게 @멘션입니다

신입이 가장 자주 듣는 리뷰 지적이 "우리 컨벤션과 다르다"입니다. @routers/tasks.py 로 기존 파일을 참조시키고 "이 스타일을 따르라"고 하면, 에이전트는 함수 명명·타입 힌트·에러 처리 방

식을 그 파일에서 직접 학습해 따라 합니다. 8장에서 배울 `CLAUDE.md` 가 팀 규칙을 영속적으로 주입하는 장치라면, `@` 멘션은 그 작업 한정으로 맥락을 주입하는 즉석 도구입니다.

3.7 worked example: @main.py 참조로 같은 스타일 엔드포인트 추가

이제 네 요소 + 작업 분해 + `@` 멘션을 한데 묶어, 실제 작업 하나를 끝까지 해 봅니다. 목표는 "끝나지 않은 할 일만 돌려주는 `GET /tasks/pending` 추가"입니다.

1단계 — @ 멘션으로 맥락을 주며 테스트부터 지시:

```
> @routers/tasks.py 를 참고해서, status가 todo인 할 일만 돌려주는
  GET /tasks/pending 엔드포인트의 동작을 검증할 pytest 테스트를 먼저 작성해 줘.
  TestClient를 쓰고, 아직 엔드포인트 구현은 하지 마. 테스트는 일부러 실패해야 해.
```

에이전트는 `routers/tasks.py` 와 `models.py` 를 읽어 `Task`·`TaskStatus` 모델과 `store` 사용, 기존 엔드포인트 형태를 파악한 뒤, 같은 스타일의 테스트를 만듭니다.

```
# tests/test_pending.py
from fastapi.testclient import TestClient

from main import app
import store
from models import TaskStatus

client = TestClient(app)

def test_pending_returns_only_unfinished():
    store.reset()
    client.post("/tasks", json={"title": "문서 읽기"}) # id=1, todo
    client.post("/tasks", json={"title": "환경 설정"}) # id=2, todo
    store.get_task(2).status = TaskStatus.done # 2번을 완료 상태로

    response = client.get("/tasks/pending")

    assert response.status_code == 200
    titles = [t["title"] for t in response.json()]
    assert titles == ["문서 읽기"]
```

아직 `/tasks/pending` 이 없으므로 이 테스트는 실패합니다(404가 와서 `assert response.status_code == 200` 에서 멈춥니다). 완료 조건("일부러 실패")을 만족합니다.

2단계 — 테스트를 통과하도록 구현 지시:

> 이제 그 테스트가 통과하도록 GET /tasks/pending 엔드포인트를 routers/tasks.py에 구현해 줘. list_tasks와 같은 스타일로, store를 통해 데이터를 읽고 status로 걸러 줘. 단, /{task_id}보다 위에 돼야 해.

```
# routers/tasks.py 에 추가된 엔드포인트
@router.get("/pending", response_model=list[Task])
def list_pending_tasks() -> list[Task]:
    return [t for t in store.get_all_tasks() if t.status == TaskStatus.todo]
```

이 로직이 맞는지 핵심만 떼어 실행해 확인합니다.

```
# 동작 확인(핵심 로직만 발췌 실행)
class Task:
    def __init__(self, title, status):
        self.title = title
        self.status = status

all_tasks = [
    Task("문서 읽기", "todo"),
    Task("환경 설정", "done"),
]

pending = [t.title for t in all_tasks if t.status == "todo"]
print(pending)
# 출력: ['문서 읽기']
```

status 가 done 인 "환경 설정"은 빠지고 "문서 읽기"만 남아, 1단계 테스트의 기대(titles == ["문서 읽기"])와 일치합니다.

이 worked example에서 쓴 세 기법

- **@ 멘션(맥락):** @routers/tasks.py 로 기존 스타일을 직접 참조시켜, 새로 만든 코드가 기존 함수와 같은 모양(타입 힌트.반환형.명명)을 따르게 했습니다.
- **작업 분해(한 번에 하나):** 테스트(1단계)와 구현(2단계)을 나눠, 구현 전에 "무엇이 통과하면 끝인지"를 코드로 못 박았습니다. 이것이 7장에서 깊게 다룰 TDD의 씨앗입니다.
- **완료 조건:** 1단계는 "일부러 실패", 2단계는 "그 테스트 통과"라는 검증 가능한 조건을 줘서, "됐다"를 느낌이 아니라 테스트 결과로 판정했습니다.

3.8 반복 교정(iterate): 검토 → 피드백 → 재시도

마지막 기법은 **반복 교정(iterate)**입니다. 1~2장에서 에이전트는 한 번에 정답을 내는 기계가 아니라 루프를 돈다고 했습니다. 이 성질을 적극적으로 활용하는 것이 반복 교정입니다. 즉, 첫 결과를 완성품으로 보지 않고 **초안**으로 보고, 검토해서 구체적 피드백을 주고 다시 시키는 것입니다.

흔한 신입의 두 가지 잘못된 반응이 있습니다.

- **무비판 수용**: 에이전트가 준 코드를 검토 없이 그대로 받아들입니다 → 잘못된 가정이 그대로 묻어 들어갑니다.
- **통째로 포기**: 결과가 마음에 안 들면 "역시 AI는 못 쓰겠네" 하고 직접 처음부터 합니다 → 교정 한 번이면 될 일을 버립니다.

정석은 **구체적 피드백으로 한 단계 다듬기**입니다. 피드백도 네 요소처럼 구체적일수록 좋습니다.

나쁜 피드백	좋은 피드백
"그거 아닌데 다시 해 줘"	"404 대신 빈 리스트를 반환하게 바꿔 줘. 없는 게 에러는 아니야."
"코드가 별로야"	" list_pending_tasks 안의 조건을 한 줄 컴프리헨션 대신 명시적 for 루프로 풀어 줘. 팀 컨벤션이야."
"테스트 더 해 줘"	"모든 할 일의 status 가 done 일 때 빈 리스트가 오는지 확인하는 테스트 케이스를 하나 더 추가해 줘."

예를 들어 위 worked example 뒤에 이렇게 교정할 수 있습니다.

> 좋아. 그런데 모든 항목의 status가 done일 때 빈 리스트가 오는지 확인하는 테스트 케이스를 test_pending.py에 하나 더 추가해 줘.

이렇게 **검토 → 구체적 피드백 → 재시도**를 한두 번 돌리면, 첫 결과가 70점이어도 곧 95점으로 올라갑니다. 이게 에이전트를 잘 쓰는 사람과 못 쓰는 사람을 가르는 가장 큰 차이입니다.

교정은 새 세션이 아니라 같은 대화에서

교정 피드백은 **방금 그 작업을 한 같은 세션**에서 이어 주는 게 좋습니다. 에이전트가 직전에 무엇을 왜 그렇게 했는지의 맥락을 그대로 갖고 있어, "그것 말고 이렇게"가 정확히 통합니다. 세션을 달았다면 1장에서 배운 `--continue` 로 직전 대화를 이어 교정하면 됩니다.

3.9 즉시 연습(2종 1세트): 모호한 지시 vs 4요소 지시를 team-tasks-api에 실제로 줘 보기

이번에는 종이에 4요소를 채우는 것이 아니라, **모호한 한 줄과 네 요소를 갖춘 지시를 team-tasks-api에 실제로 줘 보고 결과 차이를 직접 기록합니다.** 같은 의도라도 지시의 구체성에 따라 에이전트의 동작이 얼마나 달라지는지 손으로 겪는 것이 목적입니다.

모범: 같은 의도를 모호하게 vs 4요소로 줬을 때(실제 실행 예)

상황: team-tasks-api의 `POST /tasks` 가 빈 제목(`title=""`)도 그대로 만들어 줍니다. "빈 제목이면 거부하게" 바꾸고 싶습니다.

모호한 지시를 주면 에이전트가 추측으로 메웁니다.

```
> 빈 제목 좀 막아 줘.
→ 어디서(라우터? 모델?) 어떻게(400? 422? 무시?) 막을지 추측. 테스트도 안 함.
```

같은 의도를 네 요소로 주면 추측의 여지가 사라집니다.

```
[목표] POST /tasks에서 title이 비어 있으면 생성하지 말고 거부한다.
[맥락] routers/tasks.py의 create_task, models.py의 TaskCreate. 기존 스타일을 따른다.
[제약] 새 라이브러리 추가 금지. 기존 store·모델만 사용(pydantic 검증 활용).
[완료] 빈 title이면 422(또는 400)를 반환하고, 이 동작을 확인하는
        pytest 테스트가 통과하면 끝. 기존 테스트도 모두 통과해야 한다.
```

대화형 세션에서는 한 문단으로 풀어 줍니다.

```
> @routers/tasks.py 의 create_task와 @models.py 의 TaskCreate를 같은 스타일로 고쳐서,
   title이 비어 있으면 만들지 말고 422를 반환하게 해 줘. 새 라이브러리는 쓰지 말고,
   이 동작을 검증하는 pytest 테스트를 추가한 뒤 전체 테스트가 통과하는지 확인해 줘.
```

직접 해보기 (team-tasks-api에서 실제로 실행)

즉시 연습 B — 모호한 지시와 4요소 지시를 실제로 줘 보고 차이를 기록하라 (2종 1세트)

team-tasks-api에서 `claude` 를 실행합니다. 먼저 **모호한 한 줄**을 주고(예: "빈 제목 좀 막아 줘."), 에이전트가 어디를 어떻게 고치려 했는지 화면에서 보세요. 그런 다음 **새 세션을 열어** 위 모범의 네 요소 지시(한 문단 버전)를 그대로 주고, 두 경우의 결과를 비교해 적으세요. 추측이 아니라 화면에 나타난 것을 옮깁니다.

[모호한 지시] "빈 제목 좀 막아 줘."를 썼을 때

- 에이전트가 고치려 한 파일/방식(추측으로 메운 부분이 보였는가):
- 테스트를 스스로 했는가:

[4요소 지시] 네 요소 한 문단을 썼을 때

- 실제로 바꾼 파일과 추가된 + 줄:
- `python -m pytest -q` 실제 출력(빈 title이 422로 거부되고 전체 통과했는가):

확인: 서버를 띄워(`python -m uvicorn main:app --reload`) `http://127.0.0.1:8000/docs`에서 `POST /tasks`에 `title`을 빈 값으로 보내, 실제로 422가 떨어지는지 눈으로 확인하세요.

Swagger에서 빈 `title`로 `POST`했을 때 실제 응답 코드:

즉시 연습 B 자가체크 — 머리로가 아니라 손으로 했는지 확인하세요.

- [] 모호한 지시와 4요소 지시를 `team-tasks-api`에서 **각각 실제로** 쥬 봤다.
- [] 모호한 지시에서 에이전트가 추측으로 메우는(또는 검증을 건너뛰는) 모습을 직접 봤다.
- [] 4요소 지시 뒤 `python -m pytest -q`가 통과하는 것을 화면 출력으로 확인했다.
- [] Swagger UI에서 빈 `title` `POST`가 실제로 422(또는 400)로 거부되는 것을 눈으로 봤다.
- [] 같은 의도라도 지시의 구체성이 결과를 어떻게 바꾸는지 내 경험으로 설명할 수 있다.

3.10 흔한 오류와 오개념: 과대 요청·모호한 형용사·검증 누락

오개념 1 — "한 번에 다 시키는 게 효율적이다"

신입이 "할 일 완료 처리, 필터, 검색, 정렬을 다 만들고 테스트도 붙여서 한 번에 끝내 줘"처럼 큰 덩어리를 던지는 경우가 많습니다. 이러면 100줄짜리 변경이 한꺼번에 돌아와 *어디가 맞고 틀린지 검토가 불가능*해지고, 한 곳만 어긋나도 전체를 의심하게 됩니다.

교정: 작업을 검증 가능한 단계로 쪼개 한 번에 하나씩 시킵니다(3.5). "완료 처리 테스트 작성 → 구현 → 전체 테스트 확인"처럼 각 단계가 끝났는지 확인 가능해야 합니다. 큰 작업일수록 분해의 이득이 큼니다.

오개념 2 — "정중하고 길게 쓰면 더 잘한다(모호한 형용사의 함정)"

"아주 깔끔하고 효율적이고 안전하게, 베스트 프랙티스로 부탁드립니다"는 길지만 정보량이 거의 0입니다. "깔끔하게"가 무엇을 뜻하는지(함수 분리? 주석? 줄 길이?)가 지시에 없으면 에이전트는 또 추측합니다. 모호한 형용사는 빈칸을 늘릴 뿐입니다.

교정: 형용사를 검증 가능한 제약·완료 조건으로 바꿉니다. "깔끔하게" → "한 함수는 20줄을 넘기지 말고", "안전하게" → "없는 id면 404, 잘못된 입력이면 422를 반환". 추상적 칭찬어 대신 구체적 기준을 줍니다.

오개념 3 — "에이전트가 됐다고 하면 된 것이다(검증 누락)"

지시에 완료 조건을 넣지 않으면, 에이전트는 코드를 작성한 것만으로 "완료했습니다"라고 말합니다. 하지만 그 코드가 실제로 도는지, 엣지 케이스를 처리하는지는 *확인되지 않은 상태*입니다. "됐다"는 말은 검증이 아닙니다.

교정: 모든 변경 지시에 "이 동작을 확인하는 pytest 테스트가 통과하면 끝"처럼 *검증을 완료 조건에 포함*시킵니다. 이 습관은 7장(실행·테스트로 검증)에서 본격화되지만, 지시 단계에서부터 완료 조건에 검증을 박아 두는 것이 출발점입니다.

3.11 단원 미니 프로젝트: 지시문 4요소로 엔드포인트 추가하기

이 단원에서 배운 것(네 요소·작업 분해·@ 멘션·반복 교정)을 한 번에 적용해, 작은 기능 하나를 *지시만으로* 끝까지 끌고 가 봅니다. 이것이 u3의 미니 프로젝트입니다.

미니 프로젝트 — "모호한 한 줄 → 네 요소 지시 → 단계 실행" (2종 1세트)

목표 산출물: 1장에서 셋업한 `team-tasks-api` 저장소에서 **새 작업 브랜치를 만들고**, "특정 할 일을 완료 처리하는 `PATCH /tasks/{task_id}/done` 엔드포인트"를 테스트와 함께 `routers/tasks.py`에 추가합니다. 이것은 13~16장 종합 프로젝트의 "완료 처리" 기능을 미리 맛보는 것이기도 합니다.

① 모범(진행 절차 예시):

마일스톤 0 — *작업 브랜치 만들기*

```
git checkout -b feat/mark-done
```

마일스톤 1 — *모호한 요청을 네 요소로 변환*

(받은 모호한 요청) "할 일 완료되게 좀 해 줘"

↓ 네 요소로 변환

[목표] PATCH /tasks/{task_id}/done 추가: 해당 할 일의 status를 done으로 바꾸고 그 항목을 반환.

[맥락] @routers/tasks.py, 기존 create_task/list_tasks 스타일. 데이터는 store를 통해.

[제약] 새 라이브러리·새 모델 금지. 기존 store.models(TaskStatus)만 사용.

[완료] 없는 id면 404. 동작 검증 pytest 통과 + 기존 7개 테스트 전부 통과.

마일스톤 2 — 단계로 쪼개 한 번에 하나 지시

- > (1) @routers/tasks.py 참고해서 PATCH /tasks/{task_id}/done 동작을 검증할 pytest 테스트를 tests/에 TestClient로 먼저 작성해 줘. 구현은 아직 하지 마.
- > (2) 이제 그 테스트를 통과하도록 엔드포인트를 같은 스타일로 구현해 줘. store를 통해 status를 done으로 바꾸고, 없는 id면 404를 반환해야 해.
- > (3) python -m pytest -q 를 한 번에 돌려 모두 통과하는지 확인해 줘.

마일스톤 3 — 검토 후 반복 교정(한 번)

- > 좋아. 이미 status가 done인 항목에 다시 요청하면 어떻게 되는지 확인하는 테스트를 하나 더 추가해 줘. 동작은 그대로 200이면 돼.

참고로, 구현될 핵심 로직은 다음과 같은 모양입니다(직접 발췌 실행으로 확인).

```
# 동작 확인(핵심 로직만 발췌 실행)
_db = {1: {"id": 1, "title": "문서 읽기", "status": "todo"}}

def mark_done(task_id):
    if task_id not in _db:
        return (404, "Task not found")
    _db[task_id]["status"] = "done"
    return (200, _db[task_id])

print(mark_done(1))
print(mark_done(99))
# 출력: (200, {'id': 1, 'title': '문서 읽기', 'status': 'done'})
# 출력: (404, 'Task not found')
```

실행으로 검증: python -m pytest -q 가 통과(8 passed: 기존 7 + 새 1)해야 끝입니다. 통과를 확인한 뒤 브랜치를 커밋합니다.

② 직접 작성·실행(team-tasks-api에서 실제로 돌려 보세요):

받은 모호한 요청: "할 일에 마감일도 좀 넣어 줘". 이를 네 요소 지시로 직접 써서 team-tasks-api 에 실제로 주고, 화면에 나타난 도구 호출·diff·테스트 출력을 그대로 기록하세요. 미리 정한 답을 적는 것이 아니라, *방금 화면에서 본 것을* 옮깁니다.

먼저 내가 쓴 네 요소 지시(각 라벨 옆에 한 줄로 적습니다):

[목표] (무엇을 만들지 한 문장 — 예: 마감일 필드/엔드포인트 추가):
 [맥락] (참조할 실제 파일 @멘션과 따라야 할 기존 스타일):
 [제약] (금지·준수 사항 — 새 라이브러리 금지 등):
 [완료] (검증 가능한 완료 조건 — pytest 통과 등):

(단계로 쪼개 실제로 준 지시 — 한 번에 하나씩)

> (1) 내가 실제로 준 1단계 지시:
 > (2) 내가 실제로 준 2단계 지시:
 > (3) 내가 실제로 준 3단계 지시:

(반복 교정 1회 — 첫 결과를 검토하고 실제로 준 피드백)
 > 내가 실제로 준 교정 지시:

그다음, 위 지시를 `team-tasks-api`에 썼을 때 *화면에 나타난 실제*를 기록합니다.

[관찰] 에이전트가 실제로 부른 읽기 도구(Grep/Read)와 알아낸 것:
 [도구 호출] 제시된 diff에서 실제로 바뀐 파일과 추가된 + 줄:
 [결과 확인] `python -m pytest -q` 실제 출력:

자가체크(미니 프로젝트 완료 기준):

- [] `team-tasks-api`에서 새 작업 브랜치를 만들어 작업했는가.
- [] 모호한 요청을 네 요소(목표·맥락·제약·완료 조건)로 빠짐없이 변환했는가.
- [] 맥락에 `@routers/tasks.py` 같은 `@` 멘션으로 실제 파일을 참조시켰는가.
- [] 작업을 "테스트 먼저 → 구현 → 전체 확인"의 검증 가능한 단계로 쪼갰는가.
- [] `python -m pytest -q`가 통과(8 passed)하는 것을 화면 출력으로 확인했는가.
- [] 첫 결과를 무비판 수용하지 않고, 구체적 피드백으로 최소 한 번 교정했는가.

3.12 마무리: 이 장에서 무엇을 가져가나

이 장에서 우리는 에이전트를 부르는 기본기를 배웠습니다. 핵심은 네 가지입니다.

1. **네 요소** — 목표·맥락·제약·완료 조건을 채우면 빈칸(가정)이 줄어 결과가 의도에 수렴합니다.
2. **작업 분해** — 큰 작업을 검증 가능한 단계로 쪼개 한 번에 하나씩 시키면 검토와 교정이 쉬워집니다.
3. **@ 멘션** — 붙여넣기 대신 실제 파일을 참조시켜 맥락을 정확·안전하게 주입합니다.

4. 반복 교정 — 첫 결과를 초안으로 보고 구체적 피드백으로 다듬는 것이 에이전트 활용의 정석입니다.

다음 장(4장)에서는 이 지시 기술을 *낯선 팀 코드를 파악하는 일*에 적용합니다. 잘 묻는 것도 잘 지시하는 것의 한 갈래이며, 무엇을 바꿀지 정하기 전에 코드를 이해하는 안전한 탐색을 배웁니다.

4장 낯선 팀 코드 파악하기: 에이전트로 코드베이스 탐색

이 장은 단원 4(u4)의 유일한 챕터이며, 여기서 배우는 개념은 **기존 코드베이스 탐색·이해시키기(exploring and understanding an existing codebase, c04)** 한 가지입니다. 앞 단원에서 익힌 작업 지시 기술(c03)을 **낯선 팀 코드를 이해하는 일**에 적용합니다. 무엇을 바꿀지 정하기 전에, 코드를 안전하게 파악하는 법을 다룹니다.

이 장을 마치면 할 수 있는 것

- 처음 받은 FastAPI 저장소에서 에이전트에게 구조와 핵심 흐름을 요약시켜, 엔드포인트·핵심 모듈을 담은 구조 지도 1장을 작성할 수 있습니다.
- 낯선 코드에 대해 시니어 엔지니어에게 문듯 구체적 질문 3개 이상을 던져 동작·책임을 파악할 수 있습니다.
- 특정 라우터·함수가 어디서 정의·사용되는지 에이전트에게 추적시켜 그 위치(파일·줄)를 특정할 수 있습니다.
- 주어진 작업이 변경 없이 이해만 하는 읽기 전용 탐색인지 변경 작업인지 구분해, 탐색 단계에서 파일을 바꾸지 않도록 판단할 수 있습니다.

4.1 도입: 신입의 첫 난관은 낯선 코드다

팀에 합류한 신입이 가장 먼저 마주하는 벽은 *기능이 어렵다*가 아니라 *코드가 낯설다*입니다. 처음 받은 저장소를 열면 폴더가 수십 개, 파일이 수백 개이고, 어디서부터 봐야 할지, 이 함수가 어디서 불리는지, 데이터가 어디를 거쳐 흐르는지 막막합니다. 예전 같으면 머칠을 들여 파일을 하나씩 열어 보거나, 바쁜 시니어를 붙잡고 물어보아야 했습니다.

Claude Code는 이 벽을 낮춰 줍니다. 에이전트가 *직접 저장소를 훑어* 구조·의존성·흐름을 몇 초 만에 설명할 수 있기 때문입니다. 신입은 시니어 동료에게 문듯 "이 프로젝트는 무엇을 하나요?", "할 일은 어디에 저장되나요?", "이 함수가 던지는 예외는 뭐죠?"라고 물으면 됩니다. 다른

점은, 이 시니어는 24시간 옆에 있고, 같은 질문을 백 번 물어도 귀찮아하지 않으며, 답의 근거가 된 실제 파일·줄까지 짚어 준다는 것입니다.

왜 이 개념이 지금 필요한가

종합 프로젝트에서 여러분은 *기존 팀 FastAPI 저장소에 첫 기능을 추가*하게 됩니다. 이 책이 제공하는 `team-tasks-api` (1장에서 셋업한 그 저장소)가 바로 그 "기존 팀 저장소"입니다. 그런데 낯선 코드를 이해하지 못한 채로 바꾸면, 기존 동작을 깨뜨리거나 팀 스타일과 어긋나기 쉽습니다. 그래서 "안전하게 변경하기"(5~6장)보다 **먼저** "안전하게 이해하기"를 배웁니다. 탐색은 변경의 선행 단계이고, 잘 묻는 능력은 잘 바꾸는 능력의 토대입니다.

선수 개념 한 줄 상기(c03). 앞 장에서 좋은 작업 지시는 목표·맥락·제약·완료 조건을 담는다고 배웠습니다. 탐색 질문에도 같은 원칙이 적용됩니다. "코드 좀 설명해 줘"(모호)보다 "`app/` 아래 구조를 폴더별 역할로 요약하고, 엔드포인트 목록을 표로 정리해 줘"(목표·맥락·완료 조건이 있음)가 훨씬 쓸모 있는 답을 끌어냅니다. 잘 묻는 것은 잘 지시하는 것의 한 갈래입니다.

4.2 정의: 에이전트 검색으로 구조·의존성·흐름을 자동 파악

개념을 한 문장으로 못 박습니다.

기존 코드베이스 탐색·이해시키기(c04)란, Claude Code의 **에이전트 검색(agent search)**을 이용해 저장소의 구조·의존성·핵심 흐름을 스스로 훑게 하고, 시니어에게 문듯 구체적 질문을 던져, 코드를 *바꾸지 않고* 이해만 하는 읽기 전용(read-only) 활동을 말합니다.

여기서 **에이전트 검색**이 무슨 뜻인지 풀어 봅니다. 2장에서 에이전트는 관찰 단계에서 **Glob(파일 이름 찾기)·Grep(내용 검색)·Read(파일 읽기)** 같은 읽기 전용 도구를 쓴다고 배웠습니다. 에이전트 검색이란 바로 이 도구들을 *에이전트가 스스로 조합해* 코드베이스를 훑는 것입니다. 여러분이 "이 프로젝트 뭐 하는 거야?"라고 물으면, 에이전트는 다음과 같이 알아서 움직입니다.

1. **Glob**으로 어떤 파일들이 있는지 목록을 봅니다(`**/*.py`, `README*` 등).
2. **Read**로 진입점(`main.py`)·설정(`requirements.txt`·`README`)을 읽습니다.
3. **Grep**으로 핵심 키워드(`FastAPI`, `APIRouter`, `include_router` 등)가 어디 있는지 찾습니다.
4. 모든 정보를 종합해 **구조·역할·흐름**을 설명합니다.

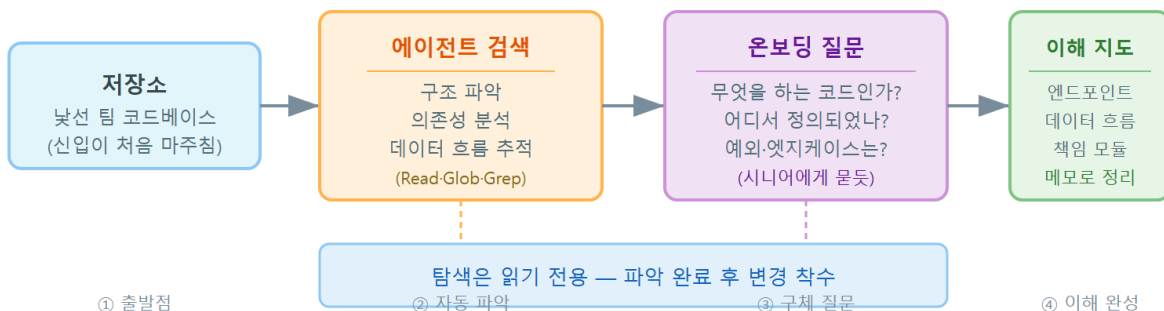
핵심은, **여러분이 컨텍스트 파일을 일일이 고를 필요가 없다는** 점입니다. 어떤 파일을 봐야 답할 수 있는지 에이전트가 스스로 판단해 찾아 읽습니다. 신입이 "어느 파일을 봐야 할지 모르겠다"는 바로 그 막막함을, 에이전트 검색이 대신 풀어 줍니다.

탐색은 "읽기 전용"이 기본입니다

이 장의 활동은 전부 **읽고 답만 받는** 것입니다. 파일을 바꾸지 않으므로 위험이 없고, 따라서 마음껏 물어봐도 됩니다. 관찰에 쓰이는 Glob-Grep-Read는 부수효과가 없어 권한 확인 없이 자유롭게 실행됩니다(권한은 5장에서 다룹니다). "이해가 먼저, 변경은 나중"이라는 순서를 몸에 익히는 것이 이 장의 큰 목표입니다.

4.3 동작 원리: 탐색 질문이 코드 이해로 이어지는 흐름

탐색 흐름: 탐색 질문 → 코드 이해



낯선 저장소를 이해하는 데에는 효과적인 **질문의 순서**가 있습니다. 넓게 시작해 점점 좁혀 들어가는 깔때기(funnel) 모양입니다.

단계	질문의 성격	예	얻는 것
1. 전체 조망	"이 프로젝트는 무엇을 하나?"	"이 API가 무슨 일을 하고, 폴더 구조는 어떻게 나뉘나요?"	큰 그림·진입점
2. 표면 목록	"무엇이 노출돼 있나?"	"엔드포인트 목록을 메서드·경로·역할 표로 정리해 주세요."	기능의 목록
3. 흐름 추적	"데이터가 어디를 거치나?"	"할 일을 만들면 요청이 어느 함수들을 거쳐 저장되나요?"	모듈 간 연결
4. 국소 심화	"이 한 곳은 어떻게 동작하나?"	"create_task 가 처리하는 입력과 던지는 예외는 무엇인가요?"	변경할 지점의 세부

이 순서가 중요한 이유는, 위에서 아래로 좁혀야 길을 잃지 않기 때문입니다. 처음부터 한 함수의 세부(4단계)로 뛰어들면 그 함수가 전체에서 어떤 역할인지 몰라 맥락이 붕 뜹니다. 반대로 1~2단계로 큰 지도를 먼저 그리면, 이후의 세부 질문이 그 지도 위에 정확히 꽂힙니다.

각 단계의 답은 다음 질문을 낳습니다. 2단계에서 "할 일을 만드는 `POST /tasks` 가 있구나"를 알면, 3단계에서 "그럼 만든 할 일은 어디에 저장되지?"가 자연스럽게 나오고, 그 답이 `store.py` 를 가리키면 4단계에서 "`store.create_task` 는 정확히 무엇을 하지?"로 이어집니다. 이렇게 **질문 → 답 → 다음 질문**이 사슬처럼 연결되며 이해가 쌓입니다. 이것이 탐색 질문이 코드 이해로 이어지는 흐름입니다.

4.4 worked example: FastAPI 저장소 구조·엔드포인트 요약시키기

이제 실제로 해 봅니다. 무대는 팀에서 처음 받은 FastAPI 저장소 — 바로 1장에서 셋업한 실습 저장소 `team-tasks-api` — 이고, 구조는 다음과 같습니다(이 저장소는 이 장 전체에서 계속 씁니다).

```

team-tasks-api/
├── main.py           # FastAPI 앱 생성, 라우터 등록, GET /health
├── models.py         # TaskStatus, TaskCreate, Task (pydantic 모델)
├── store.py          # 메모리 저장소 함수
├── routers/
│   ├── __init__.py
│   └── tasks.py      # /tasks 엔드포인트들
├── tests/
│   └── test_tasks.py
└── requirements.txt

```

각 파일의 실제 내용은 다음과 같습니다. 신입의 눈으로는 처음엔 이게 어떻게 연결되는지 한눈에 안 보입니다.

```

# main.py
from fastapi import FastAPI
from routers import tasks

app = FastAPI(title="Tasks API")
app.include_router(tasks.router)

@app.get("/health")
def health_check() -> dict[str, str]:
    return {"status": "ok"}

```

```

# models.py
from enum import Enum
from pydantic import BaseModel

class TaskStatus(str, Enum):
    todo = "todo"
    done = "done"

class TaskCreate(BaseModel):
    title: str

class Task(BaseModel):
    id: int
    title: str
    status: TaskStatus = TaskStatus.todo

```

```
# store.py
from typing import Optional
from models import Task, TaskCreate, TaskStatus

_db: dict[int, Task] = {}
_next_id: int = 1

def create_task(data: TaskCreate) -> Task:
    global _next_id
    task = Task(id=_next_id, title=data.title, status=TaskStatus.todo)
    _db[_next_id] = task
    _next_id += 1
    return task

def get_all_tasks() -> list[Task]:
    return list(_db.values())

def get_task(task_id: int) -> Optional[Task]:
    return _db.get(task_id)
```

```
# routers/tasks.py
from fastapi import APIRouter, HTTPException
from models import Task, TaskCreate
import store

router = APIRouter(prefix="/tasks", tags=["tasks"])

@router.get("", response_model=list[Task])
def list_tasks() -> list[Task]:
    return store.get_all_tasks()

@router.post("", response_model=Task, status_code=201)
def create_task(data: TaskCreate) -> Task:
    return store.create_task(data)

@router.get("/{task_id}", response_model=Task)
def get_task(task_id: int) -> Task:
    task = store.get_task(task_id)
    if task is None:
        raise HTTPException(status_code=404, detail="Task not found")
    return task
```

이 저장소를 열고, 대화형 세션에서 **전체 조망(1단계)**과 **표면 목록(2단계)**을 한 번에 물어봅니다.

> 이 저장소가 무슨 일을 하는 API인지, 루트의 폴더·파일 구조를 역할별로 요약해 줘.
그리고 노출된 엔드포인트를 메서드·경로·하는 일 표로 정리해 줘.

에이전트는 에이전트 검색으로 `main.py` · `routers/tasks.py` · `models.py` · `store.py` 를 찾아 읽고, 다음과 같이 답합니다.

이 프로젝트는 "할 일(task)"을 만들고 조회하는 작은 FastAPI 기반 REST API입니다.

구조:

- `main.py` : FastAPI 앱을 만들고 `tasks` 라우터를 등록하는 진입점(+ `/health`)
- `models.py` : 요청·응답 데이터 구조(`TaskCreate`, `Task`)와 상태 열거형(`TaskStatus`)
- `store.py` : 메모리(dict)에 할 일을 보관·조회하는 저장소 함수
- `routers/tasks.py` : `/tasks` 경로의 엔드포인트들

엔드포인트:

메서드	경로	하는 일
GET	<code>/health</code>	서버 상태 확인(<code>{"status": "ok"}</code>)
GET	<code>/tasks</code>	모든 할 일 목록 반환
POST	<code>/tasks</code>	새 할 일 생성(201), status는 todo로 시작
GET	<code>/tasks/{task_id}</code>	단건 조회, 없으면 404

이 한 번의 질문으로, 며칠 걸릴 "구조 파악"이 몇 초로 줄었습니다. 이제 큰 지도가 손에 들어왔습니다.

이 질문이 좋은 답을 끌어낸 이유

- **넓게 시작(1~2단계):** 세부 함수가 아니라 "무엇을 하는 API인가 + 구조 + 엔드포인트 목록"이라는 큰 그림을 먼저 물었습니다.
- **완료 조건 명시:** "역할별로 요약", "메서드·경로·하는 일 표"처럼 *답의 형태*를 지정해, 산만한 설명 대신 정리된 표를 받았습니다(c03의 완료 조건이 탐색 질문에도 그대로 적용됩니다).
- **읽기 전용:** 이 과정에서 파일은 한 글자도 바뀌지 않았습니다. 에이전트는 Read·Grep으로 읽기만 했습니다.

4.5 온보딩 질문법: 시니어에게 문듯 질문하기

큰 지도를 그렸으면, 이제 *시니어에게 문듯* 구체적으로 파고들 차례입니다. 좋은 온보딩 질문에는 공통된 형태가 있습니다. 신입이 막연하게 "이거 어떻게 되나요?"라고 묻는 대신, 다음 네 종류의 질문을 의식적으로 던지면 이해가 빠릅니다.

질문 유형	무엇을 알아내 나	예
무엇을(what)	책임·역할	"store.py 는 무슨 책임을 지나요?"
어디서(when)	정의·사용 위치	"Task 모델은 어디서 정의되고 어디서 쓰이나요?"
어떻게(how)	흐름·연결	"할 일을 만들면 요청이 어느 함수들을 차례로 거치나요?"
만약(what if)	예외·엣지 케이스	"빈 제목으로 만들면 어떻게 되나요? 없는 id를 조회하나요?"

이 네 가지는 사실 *실제 시니어에게 온보딩 받을 때 묻는 질문*과 똑같습니다. 다른 점은, 에이전트는 답하면서 **근거가 된 파일과 줄을 함께 짚어 준다**는 것입니다. 그래서 답을 받은 뒤 "그 부분을 직접 열어 확인"하기가 쉽습니다.

질문에 "근거를 보여 줘"를 붙이세요

탐색 질문 끝에 "그 근거가 되는 파일·함수도 같이 알려 줘"를 붙이면, 에이전트가 추측이 아니라 *실제 코드*를 짚어 답하게 유도할 수 있습니다. 예: "할 일이 어디 저장되는지, 그 근거 함수까지 알려 줘." 이러면 "store.py 의 create_task 가 _db 디렉터리에 넣습니다"처럼 검증 가능한 답이 옵니다. 요약은 곧이곧대로 믿지 않고 *근거로 확인*하는 습관이 신입에게 특히 중요합니다 (4.10 참고).

이제 이 네 유형 질문을 **종이에 만들지 말고 team-tasks-api 에이전트에게 실제로 물어** 봅니다. 이 장의 모든 연습은 *읽기 전용 탐색*이므로 코드를 바꾸지 않습니다.

모범: 네 유형 질문을 실제로 물어 근거까지 받은 예

team-tasks-api에서 `claude` 를 실행하고 네 유형 질문을 던지면, 에이전트는 답과 함께 근거 파일·함수를 짚어 줍니다(아래는 화면 답을 요약해 옮긴 예).

[무엇을] "이 저장소는 무엇을 하나요? 한 줄로." → 할 일을 만들고 조회하는 FastAPI REST API
 [어디서] "할 일이 실제로 어디에 저장되나요? 근거 함수까지." → store.py의 create_task가 _db(dict)
 [어떻게] "할 일을 만들면 요청이 어느 함수를 차례로 거치나요?" → create_task(routers) → store.c
 [만약] "빈 제목으로 만들면 어떻게 되나요?" → models.py TaskCreate.title 검증, 없으면 422

직접 해보기 (team-tasks-api 에이전트에게 실제로 물어보기)

즉시 연습 A — 네 유형 질문을 실제로 물어 답과 근거를 기록하라 (2종 1세트 · 1/2)

team-tasks-api에서 `claude` 를 실행하고, 네 유형(무엇을·어디서·어떻게·만약) 질문을 각각 하나씩 **실제로 물어보세요**(4.4의 예와 겹치지 않게). 각 질문 끝에 "그 근거가 되는 파일·함수도 같이 알려 줘"를 붙입니다. 그리고 **에이전트가 실제로 답한 내용과 짚어 준 근거**를 옮겨 적으세요.

[무엇을] 내가 물은 질문 / 에이전트 답 / 짚어 준 파일:
 [어디서] 내가 물은 질문 / 에이전트 답 / 짚어 준 파일·함수:
 [어떻게] 내가 물은 질문 / 에이전트 답(거치는 함수 순서):
 [만약] 내가 물은 질문 / 에이전트 답 / 검증이 일어나는 위치:

확인: 위 답 중 하나를 골라, 에이전트가 짚어 준 파일을 직접 열어(Read 또는 에디터로) 답이 실제 코드와 맞는지 한 번 대조하세요.

직접 열어 대조한 파일과, 답이 실제 코드와 맞았는지:

즉시 연습 A 자가체크 — 손으로 물어봤는지 확인하세요.

- [] 네 유형 질문을 team-tasks-api 에이전트에게 **실제로** 물었다.
- [] 각 답에 에이전트가 짚어 준 근거 파일·함수를 함께 기록했다.
- [] 답 하나 이상을 실제 파일을 열어 대조해, 요약을 맹신하지 않고 근거로 확인했다.
- [] 이 연습 동안 코드를 한 줄도 바꾸지 않았다(읽기 전용 유지).

4.6 worked example: tasks 데이터 흐름·검증 경로 추적

이번에는 **흐름 추적(3단계)**을 해 봅니다. "할 일을 만들 때 요청이 어디를 거쳐 저장되고, 어디서 검증되는가"를 추적시킵니다.

> POST /tasks로 할 일을 만들면, 요청이 어느 함수들을 차례로 거쳐 저장되는지 흐름을 단계로 정리해 줘. 제목 검증은 어디서 일어나는지도 근거 코드와 함께 알려 줘.

에이전트는 `routers/tasks.py` → `store.py` → `models.py` 를 따라가며 다음과 같이 답합니다.

POST /tasks 요청의 흐름:

1. routers/tasks.py 의 create_task(data: TaskCreate)가 요청을 받습니다.
- FastAPI가 요청 본문을 TaskCreate로 만들면서 *검증*이 먼저 일어납니다.
2. 검증 통과 시, store.create_task(data)를 호출합니다.
3. store.create_task는 새 id를 매겨 Task(id=..., title=..., status=todo)를 만들고 _db 디렉터리에 저장한 뒤 그 Task를 반환합니다.
4. 반환된 Task가 201 상태로 응답됩니다.

검증 경로:

- models.py 의 TaskCreate.title 은 str 타입으로 선언돼 있습니다. title이 빠지거나 문자열이 아니면 FastAPI가 422를 자동 반환합니다.
(지금은 길이·빈 문자열 제한이 없습니다 — 그 제한을 더하는 것이 이후 과제입니다.)

이 설명이 맞는지 검증 규칙만 떼어 직접 돌려 확인해 봅니다.

```
# 검증 규칙 동작 확인(pydantic 발췌 실행)
from pydantic import BaseModel, ValidationError

class TaskCreate(BaseModel):
    title: str

print(TaskCreate(title="문서 읽기").title)

try:
    TaskCreate() # title 누락
except ValidationError:
    print("title 누락 거부됨")
# 출력: 문서 읽기
# 출력: title 누락 거부됨
```

정상 요청은 통과하고, title이 빠진 요청은 검증에서 막힙니다. 에이전트가 설명한 "검증은 TaskCreate 에서 일어난다"가 실제와 일치함을 직접 확인했습니다.

흐름 추적에서 본 것

- 데이터는 라우터(입구) → 저장소(보관) → 모델(검증·형태) 의 세 모듈을 거칩니다. 한 파일만 보서는 안 보이던 연결이 드러났습니다.
- 검증은 핸들러 안의 if 문이 아니라 모델(TaskCreate 의 타입 선언) 에서 선언적으로 일어납니다. 이걸 알면, 나중에 검증 규칙(예: 빈 제목 거부)을 더할 때 어느 파일을 고쳐야 하는지가 분명해집니다.
- 추적 결과를 직접 실행으로 검증했습니다 — 요약을 믿기만 하지 않고 근거로 확인하는 태도입니다.

4.7 특정 함수·엔드포인트의 책임과 예외 흐름 묻기

마지막 깊이는 **국소 심화(4단계)**입니다. 변경할 후보 지점을 정했다면, 그 함수 하나의 **책임**과 **예외 흐름**을 정밀하게 묻습니다.

> routers/tasks.py 의 get_task 함수가 정확히 무슨 책임을 지고, 어떤 경우에 어떤 예외(상태 코드)를 내는지 알려 줘.

get_task(task_id: int)의 책임:

- 주어진 task_id로 단건 할 일을 조회해 반환합니다.

분기·예외 흐름:

- store.get_task(task_id)가 None을 돌려주면(=해당 id 없음) HTTPException(status_code=404, detail="Task not found")를 던집니다.
- 존재하면 그 Task를 그대로 반환합니다(200).
- 참고: task_id는 경로 파라미터가 int로 선언돼, 숫자가 아닌 값이 오면 FastAPI가 핸들러에 들어오기 전에 422로 막습니다.

이 "분기·예외 흐름"이 정확히 변경 작업의 출발점입니다. 예를 들어 "없는 id면 404 대신 빈 응답을 주자"는 요구가 생기면, *어디를 고쳐야 하는지*(이 `if task is None` 분기)가 이미 명확합니다. 탐색이 변경의 좌표를 찍어 준 셈입니다.

분기 동작을 핵심만 떼어 확인합니다.

```
# get_task 분기 로직 확인(발췌 실행)
_db = {1: {"id": 1, "title": "문서 읽기", "status": "todo"}}

def get_task(task_id):
    task = _db.get(task_id)
    if task is None:
        return (404, "Task not found")
    return (200, task)

print(get_task(1))
print(get_task(2))
# 출력: (200, {'id': 1, 'title': '문서 읽기', 'status': 'todo'})
# 출력: (404, 'Task not found')
```

존재하는 id는 200을, 없는 id는 404를 돌려주어, 에이전트가 설명한 예외 흐름과 정확히 맞습니다.

4.8 읽기 전용 탐색과 결과 메모로 정리하기

탐색의 마지막 습관은 두 가지입니다. **읽기 전용을 지키는 것과 알아낸 것을 메모로 남기는 것.**

읽기 전용을 지키다. 탐색 단계에서는 코드를 바꾸지 않습니다. 목적이 "이해"이기 때문입니다. 그런데 탐색하다 보면 "여기 이거 잘못된 것 같은데 김에 고칠까?" 하는 충동이 듭니다. 이건 위험합니다. 아직 전체 맥락을 다 파악하지 못한 상태라, 멀쩡한 것을 망가뜨리거나 다른 작업과 섞여 리뷰가 어려워집니다. 이해와 변경을 *분리*하는 것이 안전의 기본입니다.

순수하게 읽기만 하고 싶을 때 쓸 수 있는 두 가지 방법이 있습니다.

- **1회성 질의(claude -p):** 한 번 묻고 답만 받습니다. 세션을 길게 열지 않고 빠르게 확인할 때 좋습니다.

```
$ claude -p "이 폴더의 엔드포인트를 메서드·경로 표로 요약해 줘"
```

- **플랜 모드:** 에이전트를 *읽기 전용*으로 묶어 두는 모드입니다. 편집·쓰기 도구가 아예 차단되므로, 탐색·분석에 안성맞춤입니다. (플랜 모드는 6장에서 본격적으로 배웁니다 — 지금은 "탐색을 읽기 전용으로 강제하는 장치가 있다" 정도만 알아 둡니다.)

알아낸 것을 메모로 남긴다. 탐색으로 그린 지도는 머릿속에만 두면 곧 흐려집니다. 구조·엔드포인트·데이터 흐름·주의점을 짚은 메모(예: 저장소 안의 `NOTES.md` 나 개인 노트)로 정리해 두면, 다음 작업에서 같은 탐색을 반복하지 않아도 됩니다. 에이전트에게 정리까지 시킬 수도 있습니다.

```
> 지금까지 파악한 구조·엔드포인트·데이터 흐름·검증 위치를 NOTES.md 초안으로 정리해 줘. (이건 새 파일 작성이니, 내용만 보여 주고 내가 확인한 뒤 저장할게.)
```

메모 작성은 "변경"이라는 점에 주의

구조 *이해*는 읽기 전용이지만, `NOTES.md` 를 *디스크에 만드는 것*은 파일을 쓰는 변경 작업입니다. 그래서 위 예처럼 "내용만 보여 주고 내가 확인한 뒤 저장"으로 분리하거나, 권한 설정(5장)으로 통제하는 것이 좋습니다. 순수 탐색 중이라면 메모는 따로 모아 두었다가 탐색이 끝난 뒤 정리합니다.

4.9 즉시 연습(2종 1세트): team-tasks-api를 에이전트에 물어 구조 지도를 실제로 작성

탐색의 산출물은 *머릿속 지도*입니다. 이 지도를 종이 빈칸으로 추측해 채우는 게 아니라, **team-tasks-api 에이전트에게 실제로 물어 받은 답으로 한 장의 구조 지도를 작성**합니다. 모든 작업은 읽기 전용이라 코드를 바꾸지 않습니다.

모범: 에이전트에 물어 정리한 구조 지도(예)

아래는 4.4~4.7의 질문을 team-tasks-api에 실제로 던져 받은 답을 한 장으로 정리한 예입니다.

```
[프로젝트] 할 일(task)을 만들고 조회하는 작은 FastAPI REST API
[진입점] main.py (FastAPI 앱 생성 + tasks 라우터 등록 + /health)

[모듈 역할]
- routers/tasks.py : /tasks 엔드포인트(입구)
- store.py          : 메모리(dict) 저장·조회
- models.py         : TaskCreate/Task 데이터 구조와 TaskStatus 열거형

[엔드포인트]
- GET /tasks        → 전체 목록
- POST /tasks       → 생성(201), status는 todo로 시작
- GET /tasks/{task_id} → 단건, 없으면 404

[데이터 흐름(생성)] 요청 → create_task → store.create_task → _db에 저장
[검증 위치] models.py TaskCreate.title (str 타입, 누락/타입 오류 → 422)
[주의점] 저장에 메모리라 서버 재시작 시 사라짐
```

직접 해보기 (team-tasks-api 에이전트에게 실제로 물어 지도 작성)

즉시 연습 B — 에이전트에 실제로 물어 구조 지도를 채워라 (2종 1세트)

team-tasks-api에서 `claude` 를 실행하고, 4.5의 네 유형 질문(근거 파일까지 요청)을 던져 아래 지도를 **에이전트가 실제로 답한 내용**으로 채우세요. 빈칸에 추측을 적지 말고, 받은 답을 옮긴 뒤 **각 항목을 실제 파일을 열어 한 번씩 대조**합니다.

```
[프로젝트] (에이전트 답):
[진입점] (main.py를 열어 대조):
[모듈 역할]
- routers/tasks.py : (역할 / 대조한 줄):
- store.py         : (역할 / 대조한 함수):
- models.py        : (역할 / 대조한 클래스):
[엔드포인트] (실제 routers/tasks.py에서 확인한 메서드·경로·하는 일):
-
-
-
[데이터 흐름(생성)] (거치는 함수 순서, store.py까지 열어 확인):
[검증 위치] (models.py에서 title 검증 확인):
[주의점] (저장이 메모리라는 점 등, store.py로 확인):
```

핵심은 "에이전트의 답"으로 1차 작성 후, `main.py` · `routers/tasks.py` · `store.py` · `models.py` 를 **직접 열어** 답과 실제 코드가 맞는지 대조하는 것입니다.

즉시 연습 B 자가체크 — 묻고, 직접 대조했는지 확인하세요.

- [] 지도의 각 항목을 추측이 *아니라* 에이전트에게 실제로 물어 받은 답으로 채웠다.
- [] 모듈 역할·엔드포인트·데이터 흐름·검증 위치를 각각 실제 파일을 열어 대조했다.
- [] 엔드포인트 목록에 메서드·경로·하는 일이 모두 있다.
- [] 데이터 흐름을 *모듈을 거치는* 순서로 적었다(한 파일에 머물지 않았다).
- [] 이 지도를 그리는 동안 코드를 한 줄도 바꾸지 않았다(읽기 전용 유지).

4.10 흔한 오류와 오개념: 다 읽고 시작·요약 맹신·탐색 중 변경

오개념 1 — "낯선 코드는 전부 읽고 나서 시작해야 한다"

신입이 흔히 첫 주를 "저장소의 모든 파일을 처음부터 끝까지 정독"하는 데 씁니다. 실제 업무 저장소는 수만 줄이라 이 방식은 비효율적이고, 다 읽어도 *무엇이 중요한지*는 여전히 안 보입니다.

교정: 다 읽지 말고 **에이전트에게 물어 핵심부터** 파악합니다(4.3의 갈때기: 전체 조망 → 엔드포인트 목록 → 흐름 → 국소 심화). 지금 하려는 작업에 관련된 부분만 깊이 보고, 나머지는 필요할 때 다시 묻습니다. "이해는 작업에 필요한 만큼, 깊이는 그때그때"가 원칙입니다.

오개념 2 — "에이전트가 요약해 줬으니 그대로 믿으면 된다"

에이전트의 요약은 빠르고 대개 정확하지만, *항상* 정확하지는 않습니다. 특히 비슷한 이름의 함수가 여러이거나, 동적으로 호출되는 코드에서는 요약이 미묘하게 틀릴 수 있습니다. 요약만 믿고 변경에 들어가면, 틀린 가정 위에 코드를 쌓게 됩니다.

교정: **변경의 근거가 되는** 부분은 요약을 받은 뒤 **직접 파일을 열어 확인**합니다(4.5의 "근거를 보여 줘", 4.6~4.7의 발체 실행처럼). 요약은 *어디를 볼지*를 빠르게 좁혀 주는 안내일 뿐, 최종 근거는 실제 코드입니다. 신입일수록 "요약 → 근거 확인"의 두 박자를 지킵니다.

오개념 3 — "탐색하다 보이는 문제를 김에 같이 고친다"

코드를 읽다 보면 오타나 어색한 부분이 눈에 띄어 "이왕 보는 김에" 고치고 싶어집니다. 하지만 아직 전체 맥락을 다 파악하지 못한 상태의 즉흥 수정은, 의도치 않게 기존 동작을 깨거나 지금

하러던 작업과 뒤섞여 *리뷰를 어렵게* 만듭니다.

교정: 이해(읽기 전용)와 변경을 분리합니다(4.8). 탐색 중 발견한 개선점은 메모에 적어 두었다가, *별도의 작업·별도의 커밋*으로 다룹니다. 한 번에 한 가지 의도만 변경에 담는 것이 9장(작은 논리 단위 커밋)에서 배울 좋은 PR의 출발점입니다.

4.11 단원 미니 프로젝트: 팀 저장소 구조 지도 만들기

이 단원에서 배운 것(에이전트 검색·갈때기 질문·온보딩 질문법·읽기 전용)을 모아, 1장에서 셋업한 실습 저장소 `team-tasks-api` 를 *한 장의 구조 지도*로 정리합니다. 이것이 u4의 미니 프로젝트이며, 종합 프로젝트에서 "기존 저장소 파악" 단계의 예행연습입니다.

미니 프로젝트 — "team-tasks-api 구조 지도" (2종 1세트)

목표 산출물: 실습 저장소 `team-tasks-api` 를, 읽기 전용 탐색만으로 *엔드포인트·데이터 흐름·핵심 모듈·주의점*을 담은 한 장의 구조 지도로 정리합니다. 이 단원은 코드를 바꾸지 않으므로 작업 브랜치 없이 `main` 에서 읽기만 합니다.

① 모범(진행 절차 예시):

마일스톤 1 — *전체 조망 + 엔드포인트 목록(갈때기 1~2단계)*

> 이 저장소가 무엇을 하는 API인지, 폴더·파일을 역할별로 요약하고 엔드포인트를 메서드·경로·하는 일 표로 정리해 줘. 근거 파일도 같이.

마일스톤 2 — *핵심 흐름 추적(3단계)*

> 가장 중요한 쓰기 작업(예: 생성) 요청이 어느 모듈·함수를 차례로 거치는지, 검증은 어디서 일어나는지 근거 코드와 함께 정리해 줘.

마일스톤 3 — *국소 심화 + 메모 정리(4단계 + 4.8)*

> 앞으로 바꿀 가능성이 큰 핵심 함수 한두 개의 책임과 예외 흐름을 알려 줘. 그리고 지금까지 파악한 내용을 구조 지도 초안으로 정리해 줘(저장은 내가 확인 후).

체크포인트(이 미니 프로젝트의 검증): 4.9의 구조 지도 항목(프로젝트·진입점·모듈 역할·엔드포인트·데이터 흐름·검증 위치·주의점)이 *모두* 채워졌고, 각 항목을 `team-tasks-api` 의 실제 파일(`main.py` · `routers/tasks.py` · `store.py` · `models.py`)을 직접 열어 한 번씩 대조했으면 통과입니다.

② 직접 작성·실행(team-tasks-api에서 실제로 탐색하세요):

위 세 마일스톤의 질문을 직접 써서 team-tasks-api 에 실제로 묻고, 에이전트가 화면에서 부른 읽기 도구와 답의 근거 파일을 그대로 기록하세요. 미리 정한 답이 아니라, *방금 화면에서 본 것*을 옮깁니다.

(마일스톤 1 질문 — 내가 실제로 준 깔때기 1~2단계 질문)

> 내가 실제로 준 질문:

[관찰] 에이전트가 부른 읽기 도구(Glob/Grep/Read)와 알아낸 엔드포인트 목록:

(마일스톤 2 질문 — 내가 실제로 준 흐름 추적 질문)

> 내가 실제로 준 질문:

[관찰] 답이 근거로 든 실제 파일·함수(예: routers/tasks.py의 create_task):

(마일스톤 3 질문 — 내가 실제로 준 국소 심화 질문)

> 내가 실제로 준 질문:

[관찰] 핵심 함수의 책임·예외 흐름과 그 근거 줄:

(완성한 구조 지도 — 4.9의 빈 지도 양식을 채우되, 각 칸을 실제 파일을 열어 한 번씩 대조한 결과로

자가체크(미니 프로젝트 완료 기준):

- [] 큰 그림 → 엔드포인트 → 흐름 → 국소 심화의 순서로 좁혀 가며 물었는가.
- [] 각 질문에 *답의 형태나 근거*를 요구했는가(표로·근거 파일까지).
- [] 구조 지도의 모든 칸을 채웠고, *근거를 직접 확인*했는가(요약 맹신 금지).
- [] 탐색 내내 *코드를 바꾸지 않았*는가(읽기 전용 유지, 메모는 확인 후 저장).
- [] 발견한 개선점이 있다면 *변경에 섞지 않고* 메모로만 분리해 두었는가.

4.12 마무리: 이 장에서 무엇을 가져가나

이 장에서 우리는 낯선 팀 코드를 *안전하게 이해*하는 법을 배웠습니다. 핵심은 넷입니다.

1. **에이전트 검색** — 어느 파일을 봐야 할지 에이전트가 스스로 찾아 읽으므로, 신입은 컨텍스트 파일을 고를 필요 없이 질문만 하면 됩니다.
2. **깔때기 질문** — 전체 조망 → 엔드포인트 목록 → 흐름 추적 → 국소 심화로 좁혀 가면 길을 잃지 않습니다.
3. **온보딩 질문법** — 무엇을·어디서·어떻게·만약, 그리고 "근거를 보여 줘"로 검증 가능한 답을 끌어냅니다.
4. **읽기 전용 분리** — 이해와 변경을 나누고, 요약은 근거로 확인하며, 발견한 개선점은 변경에 섞지 않습니다.

다음 장(5장)에서는 드디어 *코드에 손대기 전의 안전장치* — 권한과 승인 — 를 배웁니다. 무엇을 바꿀지 정했으니, 이제 그 변경을 *안전하게* 통제하는 법으로 넘어갑니다.

안전하게 변경하기 (1): 권한과 승인

이 챕터에서 배우는 것

- 어떤 도구 호출에서 권한 확인이 발생하는지 부수효과 도구와 읽기 전용 도구를 구분해 설명할 수 있습니다.
- 주어진 작업에 맞춰 `settings.json` 에 `allow·deny·ask` 규칙을 작성해, 안전한 명령은 허용하고 위험한 명령은 차단하도록 도구를 통제할 수 있습니다.
- 도구·명령 패턴(예: `Bash(pytest:*) · Edit(routers/**)`)으로 허용 범위를 좁힌 최소 권한 규칙을 작성할 수 있습니다.
- 사용자·프로젝트 공유·로컬 세 범위를 구분해, 팀 공유 규칙과 개인 설정을 각각 올바른 `settings` 파일에 배치할 수 있습니다.
- 전체 자동 승인의 위험을 근거로 들어, 최소 권한 원칙에 맞는 규칙으로 첫 변경 환경을 안전하게 준비할 수 있습니다.

이번 챕터에서 가르치는 개념은 **권한과 승인(permissions and approval)** 한 가지입니다. 선수 개념은 ch_04에서 익힌 **기존 코드베이스 탐색·이해시키기(exploring and understanding an existing codebase)**입니다. ch_04에서는 코드를 **바꾸지 않고 읽기만 하는** 안전한 탐색으로 팀 저장소를 파악했습니다. 이제 드디어 코드에 손을 댈 차례인데, 그 전에 한 가지를 먼저 갖춰야 합니다. 바로 "에이전트가 무엇을 자동으로 해도 되고, 무엇을 반드시 물어봐야 하는가"를 정하는 안전장치입니다. 탐색은 읽기라 위험이 없었지만, 변경은 파일을 바꾸고 명령을 실행하므로 통제가 필요합니다. 이 챕터는 그 통제 장치인 권한 시스템을 다룹니다.

5.1 도입: 손대기 전에 안전장치부터

신입이 입사 첫 주에 가장 듣기 싫은 말이 있습니다. "그거 네가 지웠어?" 낯선 코드베이스에서 명령 하나를 잘못 실행해 데이터베이스를 날리거나, 엉뚱한 파일을 덮어쓰는 일은 실제로 자주 일어납니다. 사람도 실수하는데, 파일을 직접 편집하고 터미널 명령을 실행하는 에이전트라면 더 조심해야 합니다.

ch_02에서 우리는 Claude Code가 **부수효과 도구(편집·실행)**를 써서 실제로 파일을 바꾸고 명령을 돌린다는 것을 배웠습니다. 이것이 자동완성과의 결정적 차이였습니다. 그런데 바로 그 힘 때문에, 통제 없이 모든 것을 자동으로 맡기면 위험합니다. 예를 들어 에이전트가 "테스트 데이터

를 정리하겠습니다" 하면서 `rm -rf` 명령을 실행하려 한다면, 그게 정말 의도한 것인지 한 번은 확인하고 싶을 것입니다.

Claude Code는 이 문제를 **권한(permission)** 시스템으로 풀니다. 에이전트가 부수효과 도구를 호출할 때마다 "이 작업을 해도 되니까?"를 사용자에게 확인하고, 사용자는 그때그때 승인하거나, 미리 규칙을 정해 자주 쓰는 안전한 작업은 자동으로 허용하고 위험한 작업은 아예 막을 수 있습니다. 이 규칙을 잘 설계하면, 신입도 **위험 없이 빠르게** 일할 수 있습니다. 마찰(매번 묻는 번거로움)은 줄이되, 사고(돌이킬 수 없는 실수)는 막는 것이 목표입니다.

한눈에 보는 권한과 승인

Claude Code는 파일 편집·명령 실행처럼 **부수효과가 있는 도구**를 호출할 때 권한을 확인합니다. 규칙은 `settings.json`에 **allow(자동 허용)·deny(차단)·ask(물어보기)**로 선언하며, 도구·명령 패턴(예: `Bash(pytest:*)`)으로 범위를 좁힙니다. 규칙은 **사용자·프로젝트 공유·로컬 개인** 세 범위에 나눠 두고, 신입에게는 전체 자동 승인보다 **최소 권한(least privilege)**이 안전의 기본입니다.

5.2 정의: 부수효과 도구와 읽기 전용 도구의 권한 차이

권한과 승인(permissions and approval)이란, Claude Code가 부수효과를 일으키는 도구를 실행하기 전에 사용자의 허락을 확인하고, 그 허락을 규칙으로 미리 정해 두는 통제 체계입니다.

여기서 핵심은 **모든 도구가 권한 확인 대상이 아니라는 점**입니다. 도구는 두 종류로 나뉩니다.

- **읽기 전용 도구(read-only tools):** 파일을 읽거나(Read) 패턴으로 파일을 찾거나(Glob) 내용을 검색합니다(Grep). 아무것도 바꾸지 않으므로 위험이 없고, 기본적으로 **권한을 묻지 않고** 자유롭게 실행됩니다.
- **부수효과 도구(side-effect tools):** 파일을 고치거나(Edit) 새로 쓰거나(Write) 터미널 명령을 실행합니다(Bash). 시스템 상태를 바꾸므로, 기본적으로 **실행 전에 권한을 확인**합니다.

ch_04의 탐색이 안전했던 이유가 바로 이것입니다. 코드베이스를 요약시키고 함수를 추적시키는 작업은 전부 읽기 전용 도구로 이뤄지므로 권한 확인이 거의 없었습니다. 하지만 이번 챕터부터 다룰 "변경"은 부수효과 도구를 쓰므로, 매번 권한 확인이 따라붙습니다.

구분	읽기 전용 도구	부수효과 도구
예	Read(읽기), Glob(파일 찾기), Grep(검색)	Edit(편집), Write(쓰기), Bash(명령 실행)
하는 일	정보 수집(변경 없음)	파일·시스템 상태 변경
위험도	낮음(되돌릴 것이 없음)	높음(잘못 실행 시 손상 가능)
기본 동작	권한 확인 없이 실행	실행 전 권한 확인
통제 필요성	거의 없음	allow·deny·ask로 통제

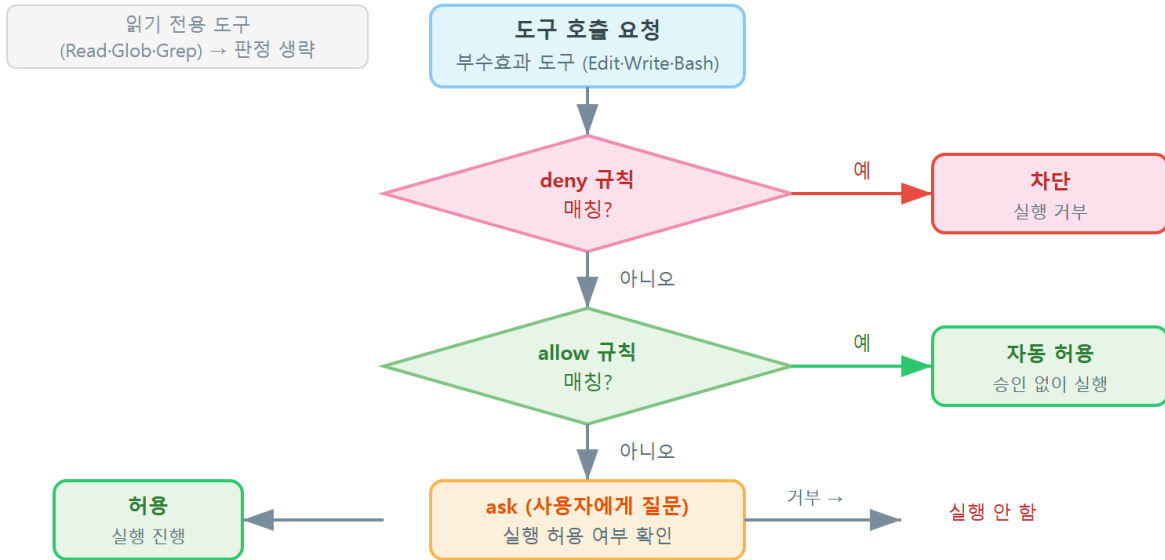
이 구분이 권한 시스템 전체의 출발점입니다. "왜 어떤 작업은 묻고 어떤 작업은 안 묻나?"라는 질문의 답이 바로 "그 도구가 무언가를 바꾸는가"이기 때문입니다.

용어 정리: 부수효과(side effect)

부수효과란 어떤 작업이 정보를 돌려주는 것 외에 **시스템의 상태를 바꾸는 것**을 말합니다. 파일을 저장하면 디스크가 바뀌고, `git push`를 하면 원격 저장소가 바뀌고, `rm`을 하면 파일이 사라집니다. 이렇게 "실행하고 나면 세상이 달라지는" 작업이 부수효과 도구이고, 권한 시스템이 통제하는 대상입니다.

5.3 동작 원리: allow·deny·ask 판정 흐름

권한 판정 흐름: allow · deny · ask



에이전트가 부수효과 도구를 호출하면, Claude Code는 그 도구 호출을 `settings.json`의 권한 규칙과 대조해 어떻게 처리할지 판정합니다. 규칙은 세 종류이고, **정해진 순서대로** 검사합니다.

1. **deny(차단)**: 이 패턴에 맞으면 도구를 **실행하지 못하게 막습니다**. 사용자에게 묻지도 않습니다. 위험한 명령(예: 파일 삭제)을 여기에 둡니다.
2. **ask(물어보기)**: 이 패턴에 맞으면 **매번 사용자에게 확인**합니다. 승인하면 그 한 번만 실행됩니다. 신중을 기하고 싶은 작업(예: `git push`)을 여기에 둡니다.
3. **allow(자동 허용)**: 이 패턴에 맞으면 **묻지 않고 자동으로 실행**합니다. 자주 쓰고 안전한 작업(예: 테스트 실행)을 여기에 둡니다.

가장 중요한 규칙은 **검사 순서가 deny → ask → allow**이고, **먼저 맞는 규칙이 결과를 정한다**는 것입니다. 규칙이 더 구체적이라고 우선되는 것이 아니라, **순서가 우선**을 정합니다. 그래서 어떤 명령이 deny에 걸리면, 설령 allow에도 똑같이 적혀 있어도 **차단**됩니다. deny가 가장 강합니다. 이 점이 안전 설계의 핵심입니다. "절대 하면 안 되는 것"을 deny에 두면, 다른 규칙이 아무리 느슨해도 그것만은 막힙니다.

세 규칙 중 **어디에도 맞지 않으면** Claude Code는 기본 동작으로 돌아가 **사용자에게 한 번 물어봅니다**. 즉 규칙을 안 정해 두면 부수효과 도구는 그때그때 확인을 거칩니다. 규칙은 이 "매번 묻기"를 줄이거나(allow) 더 엄격하게 막기(deny) 위한 장치입니다.

판정	의미	사용자 경험	적합한 작업
deny	차단(실행 불가)	막혔다는 안내, 실행 안 됨	<code>rm</code> , 비밀 파일 읽기 등 위험 작업
ask	매번 확인	승인/거부를 그때그때 선택	<code>git push</code> , 배포처럼 신중할 작업
allow	자동 허용	문지 않고 바로 실행	<code>pytest</code> , 코드 포맷처럼 안전·반복 작업
(해당 없음)	기본값	한 번 물어봄	규칙을 아직 안 정한 작업

이 흐름을 머릿속에 그려 두면, "왜 이 명령은 막히고 저 명령은 그냥 실행되지?"라는 의문이 풀립니다. 항상 deny부터 차례로 검사한다는 것만 기억하면 됩니다.

순서가 곧 안전이다

deny가 ask·allow보다 먼저 검사되므로, **위험한 작업을 deny에 두면 어떤 allow 규칙도 그것을 뚫지 못합니다.** 반대로 deny에 안 두고 allow만 넓게 잡으면, 위험한 명령이 자동 실행될 수 있습니다. "막고 싶은 것은 반드시 deny에 명시한다"가 첫 번째 안전 습관입니다.

5.4 worked example: pytest만 allow·위험 삭제는 deny하는 settings.json

이제 실제 규칙을 작성해 보겠습니다. 상황은 이렇습니다. 우리는 1장에서 셋업한 팀의 FastAPI 할 일 관리 저장소 `team-tasks-api` 에서 작업 중이고, 앞으로 테스트(`pytest`)는 수없이 돌릴 것이므로 매번 승인하기 번거롭습니다. 반면 파일 삭제(`rm`)는 절대 자동으로 실행되면 안 됩니다. 이 두 가지를 규칙으로 만들겠습니다.

저장소 루트에 `.claude/settings.json` 파일을 만들고 다음과 같이 작성합니다. 권한 규칙은 모두 `permissions` 객체 안에 둡니다.

```
{
  "permissions": {
    "allow": [
      "Bash(pytest:*)",
      "Bash(python -m pytest:*)"
    ],
    "ask": [
      "Bash(git push:*)"
    ],
    "deny": [
      "Bash(rm:*)",
      "Bash(git reset --hard:*)"
    ]
  }
}
```

- "permissions": 모든 권한 규칙을 담는 최상위 객체입니다. settings.json 은 권한 외 다른 설정도 담을 수 있어 이렇게 한 단계 안에 둡니다.
- "allow": 묻지 않고 자동 실행할 작업 목록입니다. Bash(pytest:*) 는 "pytest로 시작하는 모든 명령"을, Bash(python -m pytest:*) 는 모듈 형태 실행을 자동 허용합니다. 테스트를 마음껏 돌리되 승인 클릭이 사라집니다.
- "ask": 매번 확인할 작업입니다. Bash(git push:*) 는 원격에 코드를 올리는 작업이라, 자동화하지 않고 매번 사람이 한 번 더 확인하게 둡니다.
- "deny": 차단할 작업입니다. Bash(rm:*) 는 모든 rm 삭제 명령을, Bash(git reset --hard:*) 는 작업 내용을 날리는 강제 리셋을 막습니다. deny가 가장 먼저 검사되므로, 이 명령들은 어떤 경우에도 자동 실행되지 않습니다.

이 설정을 둔 상태에서 에이전트와 작업하면 다음과 같이 동작합니다. 터미널 세션의 모습은 이렇습니다.

> tasks 라우터에 테스트를 돌려서 통과하는지 확인해 줘

- Bash(pytest tests/test_tasks.py -q)
(allow 규칙에 맞아 자동 실행됨 - 승인 묻지 않음)

```
..... [100%]
7 passed in 0.34s
```

> 임시 파일들을 정리하려고 rm으로 지워 줘

- Bash(rm -rf tmp/)
차단됨: 이 명령은 deny 규칙(Bash(rm:*))에 막혀 실행할 수 없습니다.

실행 결과를 보면, pytest 는 묻지 않고 바로 실행되어 7 passed 가 나왔고, rm 은 deny 규칙에 걸려 아예 실행되지 않았습니다. 우리가 의도한 그대로입니다. 자주 쓰는 안전한 작업은 마찰 없

이, 위험한 작업은 확실히 차단했습니다.

왜 `git push` 는 `deny`가 아니라 `ask`인가

`git push` 는 위험하다기보다 **신중해야 하는** 작업입니다. 막아 버리면 정작 필요할 때 못 하니, `deny`가 아니라 `ask`에 둡니다. `ask`는 "해도 되지만, 할 때마다 사람이 한 번 확인한다"는 뜻입니다. 이렇게 `deny`(절대 금지)·`ask`(확인 후 허용)·`allow`(자유)를 작업의 위험도에 맞게 배분하는 것이 권한 설계의 감각입니다.

5.5 도구·명령 패턴 매칭(`Bash(pytest:)`·`Edit(routers/*)`)

권한 규칙의 힘은 **패턴 매칭(pattern matching)**에서 나옵니다. 도구 이름만 적으면 그 도구 전체가 대상이 되지만, 괄호 안에 패턴을 적으면 **특정 명령이나 특정 파일 경로**로 범위를 좁힐 수 있습니다. 신입에게는 이 "좁히기"가 안전의 핵심입니다.

Bash 명령 패턴

`Bash(...)` 안에는 실행 명령을 패턴으로 적습니다. `*` 는 임의의 문자열을 뜻하는 와일드카드입니다.

- `Bash(pytest:*)` — `pytest` 로 시작하는 모든 명령. `.*` 는 "뒤에 무엇이 와도 좋다"는 꼬리 와일드카드입니다. `pytest`, `pytest -q`, `pytest tests/` 가 모두 맞습니다.
- `Bash(git commit:*)` — `git commit` 으로 시작하는 명령만. `git push` 는 맞지 않습니다.
- `Bash(rm:*)` — 모든 `rm` 명령. 삭제를 통째로 막을 때 씁니다.

여기서 **띄어쓰기가 중요합니다**. `Bash(pytest:*)` 는 `Bash(pytest *)` 와 같은 의미이며, `pytest` 라는 명령 뒤에 인자가 붙는 형태를 가리킵니다. 만약 띄어쓰기 없이 `Bash(pytest*)` 처럼 적으면 `pytest-cov` 같은 다른 명령까지 걸릴 수 있으므로, 명령 단위로 정확히 좁히려면 `.*` 형태를 쓰는 것이 안전합니다.

Edit 파일 경로 패턴

`Edit(...)`·`Read(...)` 안에는 **파일 경로**를 패턴으로 적습니다. 경로 패턴은 `.gitignore` 와 비슷한 글롭(glob) 문법을 씁니다.

- `Edit(routers/**)` — `routers/` 폴더 아래의 모든 파일 편집을 허용. `**` 는 하위 폴더까지 전부 포함합니다. `routers/tasks.py`, `routers/__init__.py` 가 모두 맞습니다.

- `Edit(tests/**)` — 테스트 폴더만 편집 허용.
- `Read(.env)` — `.env` 파일 읽기를 (`deny`에 두면) 차단. 비밀 키가 든 파일을 에이전트가 못 읽게 할 때 씁니다.

예를 들어 "에이전트가 `routes/` 아래는 자유롭게 고치되, 그 밖의 설정 파일(`requirements.txt` 등)은 함부로 못 건드리게" 하고 싶다면 `Edit(routes/**)` 만 `allow`에 두고 나머지는 기본값(물어 보기)으로 둡니다. 이렇게 **경로로 작업 영역을 가두는** 것이 패턴 매칭의 진짜 가치입니다.

```
{
  "permissions": {
    "allow": [
      "Bash(pytest:*)",
      "Edit(routes/**)",
      "Edit(tests/**)"
    ],
    "deny": [
      "Read(.env)",
      "Read(secrets/**)",
      "Bash(rm:*)"
    ]
  }
}
```

- `Edit(routes/**)` · `Edit(tests/**)`: 라우터 코드와 테스트 코드 폴더는 자유롭게 편집하도록 허용해 작업 속도를 높입니다.
- `Read(.env)` · `Read(secrets/**)`: 환경 변수 파일과 비밀 폴더는 **읽기 자체를 차단**합니다. 읽기 전용 도구라도 비밀 노출은 막아야 하므로 `deny`에 둡니다. 이처럼 `deny`는 부수효과 도구뿐 아니라 민감 파일 읽기에도 씁니다.
- `Bash(rm:*)`: 삭제 명령 전체 차단은 유지합니다.

이름만 허용하면 너무 넓다

`Bash` 를 통째로 `allow`에 두면 **모든 셸 명령**이 자동 실행됩니다. `rm` 이든 `curl` 이든 가리지 않습니다. 이건 사실상 "전체 자동 승인"과 같아 매우 위험합니다. 반드시 `Bash(pytest:*)` 처럼 **명령을 좁혀** 허용하십시오. 도구 이름만 적는 넓은 허용은 신입이 가장 자주 저지르는 실수입니다.

5.6 권한 범위: 사용자·프로젝트 공유·로컬 개인

규칙을 **어디에 저장하느냐**에 따라 적용 범위가 달라집니다. 같은 `allow-deny` 규칙이라도 저장 위치가 다르면 "나에게만", "팀 전체에", "이 프로젝트의 나에게만"처럼 적용 대상이 바뀝니다.

범위는 세 가지입니다.

범위	파일 위치	적용 대상	Git 공유	답는 내용 예
사용자(개인)	<code>~/.claude/settings.json</code>	그 사용자의 모든 프로젝트	안 됨(개인 머신)	어디서나 쓰는 개인 기본 규칙
프로젝트 (팀 공유)	<code>.claude/settings.json</code>	그 저장소의 모든 팀원	됨(저장소에 커밋)	팀 공통 안전 규칙(테스트 허용·삭제 차단)
로컬(개인 프로젝트 한정)	<code>.claude/settings.local.json</code>	그 저장소의 나에게만	안 됨(보통 <code>.gitignore</code>)	내 머신에만 필요한 임시 허용

여기서 `~`는 **Windows 11에서 사용자 홈 폴더**(예: `C:\Users\내이름`)를 가리킵니다. Git Bash에서는 `~`로 그대로 쓸 수 있습니다.

세 범위를 나누는 이유는 명확합니다. **팀 전체가 지켜야 할 안전 규칙**(예: "삭제 명령은 차단한다")은 프로젝트 공유(`.claude/settings.json`)에 두고 저장소에 커밋해, 새로 합류하는 모든 신입에게 자동으로 적용되게 합니다. 반면 **나만의 사정**(예: "이 프로젝트에서 잠깐 특정 스크립트를 자동 허용하고 싶다")은 로컬(`.claude/settings.local.json`)에 뒀, 다른 사람에게 영향을 주지 않게 합니다. 로컬 파일은 보통 `.gitignore`에 들어가 공유되지 않습니다.

가장 중요한 실무 규칙은 이것입니다. **팀이 함께 지켜야 할 규칙은 로컬이 아니라 프로젝트 공유에 둔다**. 안전 규칙을 자기 로컬에만 넣으면, 같은 팀의 다른 신입에게는 그 보호막이 없습니다. 반대로 개인 취향이나 임시 허용을 프로젝트 공유에 넣으면 팀원 모두에게 강요가 됩니다.

또 하나 기억할 것은, **deny는 어느 범위에 있든 가장 강하게 작동**한다는 점입니다. 프로젝트 공유에 `Bash(rm:*)` 차단을 두면, 개인이 로컬에서 `rm`을 허용하려 해도 차단이 우선합니다. 안전의 바닥선은 팀 차원에서 deny로 깔아 두는 것이 좋습니다.

신입의 기본 배치

- 팀 공통 안전선(삭제·강제리셋 차단, 테스트·포맷 허용) → `.claude/settings.json` (프로젝트 공유, 커밋)
- 내 머신에서만 필요한 임시 허용 → `.claude/settings.local.json` (로컬, 커밋 안 함)
- 모든 프로젝트에서 쓰는 내 기본값 → `~/.claude/settings.json` (사용자)

이 배치만 지켜도 "팀 규칙과 개인 설정이 섞이는" 혼란을 피할 수 있습니다.

5.7 worked example: `/permissions`로 안전 명령 미리 허용·체크인

규칙을 손으로 `settings.json`에 적는 대신, Claude Code 안에서 `/permissions` 슬래시 커맨드로 권한 규칙을 보고 추가할 수 있습니다. 작업 중 "이 명령은 자주 쓰니 앞으로 자동 허용하자"는 판단이 서면, 그 자리에서 규칙을 더하는 방식입니다.

흐름을 따라가 보겠습니다. 테스트를 돌릴 때마다 승인을 묻는 게 번거로워, `pytest`를 자동 허용으로 바꾸려는 상황입니다.

```
> /permissions

권한 규칙 (Permissions)
-----
allow:
  (없음)
ask:
  (없음)
deny:
  Bash(rm:*)

[a] 규칙 추가  [d] 삭제  [q] 닫기
```

여기서 규칙 추가를 골라 `Bash(pytest:*)`를 `allow`에 더하면, 이후 테스트 실행은 묻지 않고 진행됩니다. 또 다른 방법으로, 에이전트가 권한을 물어보는 그 순간 "이번만 허용"이 아니라 **"항상 허용"**을 고르면, 같은 패턴이 자동으로 `allow` 규칙에 추가됩니다.

```
● Bash(pytest -q)
  이 명령을 실행할까요?
  > 1. 예 (이번만)
     2. 예, 그리고 앞으로 Bash(pytest:*)는 항상 허용
     3. 아니요

  (2번 선택 → settings.json의 allow에 Bash(pytest:*) 추가됨)
```

이렇게 추가된 규칙은 어느 범위에 저장할지 고를 수 있는데, **팀과 공유할 안전 규칙이라면 프로젝트 공유(`.claude/settings.json`)에 저장**하고, 그 파일을 git에 커밋(체크인)합니다. 그러면 다음에 합류하는 동료도 같은 허용·차단 규칙을 그대로 물려받습니다.

```
git add .claude/settings.json
git commit -m "chore: pytest 자동 허용, rm 차단 권한 규칙 추가"
```

실행 결과로, 이 커밋 이후 저장소를 받는 모든 팀원은 `pytest` 를 자동 허용하고 `rm` 을 차단하는 동일한 안전 설정에서 일하게 됩니다. 권한을 **개인이 매번 다시 만드는 것이 아니라, 팀의 자산으로 공유하는 것이 핵심**입니다.

"항상 허용"을 누르기 전에 한 번 생각하기

승인 창에서 "항상 허용"은 편하지만, 그 패턴이 정말 **항상 안전한지** 따져 보고 누르십시오.

`Bash(pytest:*)` 처럼 좁고 안전하면 좋지만, `Bash(git:*)` 처럼 넓게 항상 허용하면 `git push · git reset` 까지 자동화되어 위험합니다. 좁게 허용하는 습관이 사고를 막습니다.

5.8 전체 자동 승인의 위험과 최소 권한 원칙

Claude Code에는 권한 확인을 통째로 끄는 모드(흔히 "전체 자동 승인" 또는 `bypass`라 부르는 방식)가 있습니다. 모든 부수효과 도구를 묻지 않고 실행하므로 빠르지만, 신입에게는 **권하지 않습니다**. 에이전트가 잘못된 가정으로 위험한 명령을 실행해도 막을 사람이 없기 때문입니다. 격리된 일회용 환경(예: 컨테이너)이 아니라면, 자기 작업 머신에서 전체 자동 승인을 켜는 것은 안전벨트를 풀고 운전하는 것과 같습니다.

대신 따라야 할 원칙이 **최소 권한(least privilege)**입니다. 이는 보안의 오랜 기본 원칙으로, "작업에 **꼭 필요한 만큼만** 권한을 준다"는 뜻입니다. 권한 설계에 적용하면 이렇습니다.

- 자동 허용(allow)은 **자주 쓰고 확실히 안전한 것만** 좁게 추가합니다(예: `Bash(pytest:*)`, `Edit(routers/**)`).
- 위험하거나 되돌릴 수 없는 작업은 **deny로 막습니다**(예: `Bash(rm:*)`).
- 나머지는 굳이 미리 허용하지 말고 **기본값(물어보기)**로 둡니다. 필요할 때 그 자리에서 한 번 승인하면 됩니다.

이렇게 하면 "마찰은 줄이되 위험은 막는" 균형이 잡힙니다. 자주 하는 안전한 일은 빠르게, 가끔 하는 위험한 일은 신중하게 처리됩니다. 신입일수록 처음에는 좁게 시작해, 익숙해지면서 안전이 확인된 작업을 조금씩 allow에 더해 가는 것이 좋습니다.

전체 자동 승인을 켜기 전에

전체 자동 승인은 "에이전트가 무엇을 하든 다 실행해도 좋다"는 선언입니다. 신입이 낯선 코드베이스에서 이걸 켜면, 의도치 않은 삭제·덮어쓰기·외부 전송이 **확인 없이** 일어날 수 있습니다.

정말 필요하다면 인터넷이 차단된 일회용 컨테이너처럼 **피해가 격리되는 환경**에서만 쓰고, 평소 작업 머신에서는 최소 권한을 유지하십시오.

5.9 즉시 연습 (2종 1세트): team-tasks-api에 권한 규칙을 실제로 만들고 동작을 확인

배운 것을 종이로 채우는 대신, **team-tasks-api에 실제 `.claude/settings.json`을 만들고 허용·차단이 정말 동작하는지 화면으로 확인**합니다. allow가 자동 통과되는지, deny가 실제로 막는지는 직접 돌려 봐야 손에 잡힙니다.

모범: 실제로 만들고 동작을 확인한 예

team-tasks-api 루트에 `.claude/settings.json`을 아래처럼 만들고, 같은 폴더에서 `claude`를 실행해 동작을 확인합니다.

```
{
  "permissions": {
    "allow": [
      "Bash(python -m pytest:*)",
      "Edit(routers/**)"
    ],
    "ask": [
      "Bash(git push:*)"
    ],
    "deny": [
      "Bash(rm:*)",
      "Read(.env)"
    ]
  }
}
```

```
[allow 확인] > python -m pytest -q 를 돌려 줘.
→ 권한 확인 없이 바로 실행됨(allow에 있어 자동 통과). 출력: 8 passed
[deny 확인] > .env 파일을 읽어서 내용 보여 줘.
→ 차단됨. 에이전트가 권한 정책상 읽을 수 없다고 답함(deny가 먼저 검사됨)
[ask 확인] > 변경을 git push 해 줘.
→ 매번 승인 요청(ask)이 뜸
```

직접 해보기 (team-tasks-api에서 실제로 작성·확인)

연습 — team-tasks-api에 권한 규칙을 만들고 실제로 허용·차단을 확인하라.

team-tasks-api 루트에 `.claude/settings.json` 파일을 직접 만들고, 아래 상황에 맞게 작성하세요(정답은 하나가 아니며 **위험도에 맞는 배치**가 핵심). team-tasks-api에는 `secrets/` 나 `.env` 가 없을 수 있으니, deny 동작 확인용으로 루트에 `.env` 한 줄짜리 더미 파일을 만들어 두어도 됩니다.

상황:

- 단위 테스트(`python -m pytest`)는 수없이 돌리니 **자동 허용**한다.
- `routers/` 폴더 코드는 자유롭게 편집하게 **자동 허용**한다.
- 원격 푸시(`git push`)는 신중해야 하니 **매번 확인**한다.
- 삭제(`rm`)와 비밀 파일(`.env`) 읽기는 **차단**한다.

작성한 뒤 같은 폴더에서 `claude` 를 실행하고, **실제로 일어난 일**을 적으세요(추측이 아니라 화면).

```
[allow] "python -m pytest -q 돌려 줘"를 찾을 때
- 권한 확인이 났는가, 바로 실행됐는가:
- pytest 실제 출력:

[deny] ".env를 읽어 줘"를 찾을 때
- 실제로 차단됐는가, 에이전트가 뭐라고 했는가:

[ask] "git push 해 줘"를 찾을 때(실제 push까지 갈 필요 없이 승인 요청만 확인)
- 매번 승인 요청(ask)이 났는가:
```

추가: 이 규칙을 팀 전체가 공유하려면 어느 파일에 두어야 하는지, 그리고 본인만의 로컬 예외라면 어디에 두는지 한 줄로 적으세요.

팀 공유 위치 / 로컬 개인 위치:

자가체크: 손으로 만들고 확인했는지 점검하기

- [] team-tasks-api 루트에 `.claude/settings.json` 을 **실제로** 만들었다.
- [] `python -m pytest` 가 권한 확인 없이 자동 실행되는 것을 화면으로 봤다(allow 동작 확인).
- [] `.env` 읽기 시도가 실제로 차단되는 것을 화면으로 봤다(deny가 먼저 검사됨).
- [] `git push` 에 매번 승인 요청이 뜨는 것을 확인했다(ask 동작 확인).
- [] 와일드카드를 `Bash(python -m pytest:*)` 처럼 명령 단위로 좁혔다(Bash 만 통째로 허용하지 않았다).

- [] 팀 공유 규칙은 `.claude/settings.json` 에, 로컬 개인 예외는 `.claude/settings.local.json` 에 둔다는 것을 구분했다.

5.10 흔한 오류와 오개념: 전체 bypass·이름만 허용·로컬 오남용

마지막으로, 신입이 권한 설정에서 자주 저지르는 실수와 그 교정을 정리합니다.

오개념 1: "전체 자동 승인이 편하니 그냥 켜자"

전체 bypass(자동 승인)는 편하지만, 에이전트의 모든 부수효과 작업이 **확인 없이** 실행됩니다. 잘못된 가정 한 번이 삭제·덮어쓰기로 이어질 수 있습니다.

교정: 전체 자동 승인 대신 **최소 권한**으로 시작하십시오. 자주 쓰는 안전한 작업만 좁게 allow에 두고, 위험 작업은 deny, 나머지는 기본값(물어보기)으로 둡니다. 마찰은 생각보다 적고, 안전은 훨씬 커집니다.

오개념 2: "Bash를 통째로 허용하면 명령마다 안 물어봐서 좋다"

Bash 를 이름만으로 allow에 두면 **모든 셸 명령**이 자동 실행됩니다. `pytest` 뿐 아니라 `rm` · `curl` · `git push` 까지 전부입니다. 이름만 허용은 사실상 전체 자동 승인과 같습니다.

교정: 허용은 항상 **명령·경로 패턴까지 좁혀** 적습니다. `Bash(pytest:*)`, `Edit(routers/**)` 처럼 범위를 명시하면, 그 패턴에 맞는 것만 자동 허용되고 나머지는 여전히 확인을 거칩니다.

오개념 3: "안전 규칙은 내 로컬 파일에 넣으면 되겠지"

`.claude/settings.local.json` (로컬)은 **공유되지 않습니다**(보통 `.gitignore`). 여기에 팀 안전 규칙(예: `rm` 차단)을 넣으면, 같은 저장소를 받는 다른 신입에게는 그 보호막이 **없습니다**.

교정: 팀이 함께 지킬 규칙은 **프로젝트 공유**(`.claude/settings.json`)에 두고 **git에 커밋**하십시오. 로컬은 "나에게만, 이 프로젝트에서만" 필요한 임시 설정에만 씁니다.

자주 묻는 질문(FAQ)

Q. allow에도 deny에도 똑같은 명령이 있으면 어떻게 되나요? A. **deny가 이깁니다.** 검사 순서가 deny → ask → allow이고 먼저 맞는 규칙이 결과를 정하므로, deny에 걸리면 allow에 있어도 차단됩니다.

Q. 규칙을 아무것도 안 정하면 위험한가요? A. 위험하지 않습니다. 규칙이 없으면 부수효과 도구는 **매번 사용자에게 물어봅니다**(기본값). 규칙은 이 "매번 묻기"를 줄이거나(allow) 더 엄격히 막기(deny) 위한 선택입니다. 다만 `rm` 같은 위험 작업은 실수 방지를 위해 deny로 명시해 두는 편이 안전합니다.

Q. `~/.claude/settings.json` 과 `.claude/settings.json` 은 어떻게 다른가요? A. 앞은 **사용자**(모든 프로젝트), 뒤는 **이 프로젝트**(팀 공유, 커밋 대상)입니다. 팀 규칙은 뒤에, 어디서나 쓰는 내 기본값은 앞에 둡니다.

5.11 마무리: 안전장치를 갖쳤으니, 다음은 계획이다

이번 챕터에서 우리는 코드에 손대기 전에 갖춰야 할 첫 번째 안전장치인 **권한과 승인**을 배웠습니다. 핵심을 정리하면 다음과 같습니다.

- 권한 확인은 **부수효과 도구(Edit·Write·Bash)**를 호출할 때 발생하고, 읽기 전용 도구는 묻지 않습니다.
- 규칙은 `settings.json` 의 **allow·deny·ask**로 선언하며, 검사 순서는 **deny → ask → allow**, 먼저 맞는 규칙이 결과를 정합니다(deny가 가장 강함).
- **패턴 매칭**(`Bash(pytest:*) · Edit(routers/**)`)으로 명령·경로를 좁혀 안전하게 허용합니다.
- 규칙은 **사용자·프로젝트 공유·로컬** 범위로 나누고, 팀 안전 규칙은 프로젝트 공유에 커밋합니다.
- 전체 자동 승인 대신 **최소 권한**으로 시작합니다.

이제 "에이전트가 무엇을 해도 되는지"의 경계는 정해졌습니다. 하지만 안전에는 한 겹이 더 필요합니다. 권한은 "이 명령을 실행해도 되는가"를 통제하지만, **"애초에 무엇을 바꿀 계획인가"를 먼저 보는 것은 또 다른 안전장치**입니다. 다음 챕터(ch_06)에서는 변경을 실행하기 전에 계획부터 받아 검토하는 **플랜 모드**를 배웁니다. 권한(이번 챕터)과 플랜 모드(다음 챕터)를 함께 갖추면, 신입도 낯선 코드에 위험 없이 첫 변경을 시작할 수 있습니다.

안전하게 변경하기 (2): 플랜 모드로 먼저 계획

이 챕터에서 배우는 것

- 다중 파일을 건드리는 변경에서 플랜 모드를 켜(Shift+Tab) 읽기 전용으로 변경 계획을 받고, 그 단계 동안 파일이 바뀌지 않음을 확인할 수 있습니다.
- 제시된 계획에 담긴 잘못된 가정 1개 이상을 찾아, 단계 추가·삭제·재정렬로 방향을 바로 잡는 교정 지시를 작성할 수 있습니다.
- 검토·교정한 계획을 승인한 뒤 실행 모드로 전환해, 계획대로 변경이 적용되는 흐름을 운용할 수 있습니다.
- 주어진 작업이 복잡·고위험·다중 파일·첫 기능 추가에 해당하는지 따져 계획·실행 분리가 필요한지 판단할 수 있습니다.
- (단원 미니 프로젝트) pytest·편집만 허용하는 최소 권한 settings를 구성하고 플랜 모드로 작은 변경의 단계별 계획을 받아, 잘못된 가정을 교정한 뒤 확정할 수 있습니다.

이번 챕터에서 가르치는 개념은 **플랜 모드(plan mode)** 한 가지입니다. 선수 개념은 바로 앞 챕터(ch_05)에서 익힌 **권한과 승인(permissions and approval)**입니다. ch_05에서 우리는 "에이전트가 어떤 명령을 실행해도 되는가"를 allow·deny·ask로 통제했습니다. 권한은 개별 도구 호출 단위의 안전장치였습니다. 그런데 안전에는 한 겹이 더 있습니다. 권한이 "이 명령 하나를 실행해도 되나"를 본다면, 플랜 모드는 그보다 한 걸음 앞서 "**애초에 무엇을 바꿀 셈인가, 그 전체 계획부터 보자**"를 봅니다. ch_05가 실행의 안전이라면, 이번 챕터는 **계획의 안전**입니다. 둘을 함께 갖추면 신입도 낯선 코드에 위험 없이 첫 변경을 시작할 수 있습니다. 이 챕터는 단원 u5의 마지막 챕터이므로, 끝에 권한과 플랜 모드를 통합하는 **단원 미니 프로젝트**도 함께 다룹니다.

6.1 도입: 바꾸기 전에 계획을 먼저 본다

목수에게는 "두 번 재고 한 번 자른다"는 격언이 있습니다. 한 번 자른 나무는 되돌릴 수 없으니, 자르기 전에 충분히 재라는 뜻입니다. 코드 변경도 같습니다. 특히 신입이 낯선 팀 코드베이스에 첫 변경을 가할 때, 에이전트가 곧장 파일을 고치기 시작하면 위험합니다. 만약 에이전트가 **잘못**

된 파일을 골랐거나, 엉뚱한 방식으로 접근하고 있다면, 이미 여러 파일이 바뀐 뒤에야 그 사실을 알게 됩니다.

ch_02에서 배운 에이전트 루프를 떠올려 봅시다. 에이전트는 관찰 → 계획 → 도구 호출 → 결과 확인을 돕니다. 보통은 이 루프가 빠르게 돌면서 관찰하자마자 편집(도구 호출)으로 넘어갑니다. 빠른 작업에는 좋지만, **변경의 규모가 크거나 방향이 틀리면 손해가 큰 작업**에서는 "계획" 단계에서 한 번 멈춰 사람이 검토하고 싶습니다.

플랜 모드는 바로 그 "멈춤"을 만드는 장치입니다. 에이전트에게 **코드를 바꾸지 말고, 무엇을 어떻게 바꿀지 계획부터 세워 보여 달라**고 지시하는 모드입니다. 사용자는 그 계획을 읽고, 틀린 부분이 있으면 교정하고, 방향이 맞으면 그때 승인해 실행으로 넘깁니다. 자르기 전에 두 번 재는 것입니다. ch_05의 권한이 "위험한 실행을 막는" 안전장치였다면, 플랜 모드는 "잘못된 방향의 변경을 시작 전에 바로잡는" 안전장치입니다.

한눈에 보는 플랜 모드

플랜 모드(plan mode)는 에이전트가 코드를 바꾸기 전에 **무엇을 어떻게 바꿀지 계획을 먼저 제시하고 승인을 받는 읽기 전용 모드**입니다. 이 모드에서는 편집·쓰기 도구가 차단되어 **파일이 바뀌지 않으므로** 안전하게 탐색·분석하고 계획을 검토·교정할 수 있습니다. **Shift+Tab**으로 모드를 전환하며, 복잡·고위험·다중 파일·첫 기능 추가처럼 **방향이 틀리면 손해가 큰 작업**에서 계획·실행을 분리해 줍니다.

6.2 정의: 플랜 모드로 — 변경 전 읽기 전용 계획 단계

플랜 모드(plan mode)란, 에이전트가 코드베이스를 **읽기 전용**으로만 탐색·분석해 변경 계획을 세워 제시하고, 사용자가 그 계획을 승인하기 전까지는 **파일을 수정하지 않는** 작업 모드입니다.

핵심 성질을 정리하면 이렇습니다.

- **읽기 전용(read-only):** 플랜 모드에서 에이전트는 Read·Glob·Grep 같은 읽기 도구로 코드를 살펴볼 수 있지만, Edit·Write로 파일을 바꾸거나 부수효과 명령을 실행하지는 않습니다. 그래서 계획 단계에서는 **무엇도 망가지지 않습니다**.
- **계획을 산출물로 제시:** 에이전트는 분석을 마치면 "이 파일들을 이렇게 바꾸겠다"는 **단계별 계획**을 제시합니다. 이것이 플랜 모드의 결과물입니다.
- **승인 게이트:** 사용자가 계획을 검토해 승인하면, 그때 비로소 실행 모드로 넘어가 실제 변경이 시작됩니다. 승인 전에는 한 줄도 바뀌지 않습니다.
- **모드 전환은 Shift+Tab:** 터미널에서 Shift+Tab을 누르면 모드가 순환합니다(기본 → 편집 자동 수락 → 플랜). 화면 아래에 현재 모드가 표시됩니다.

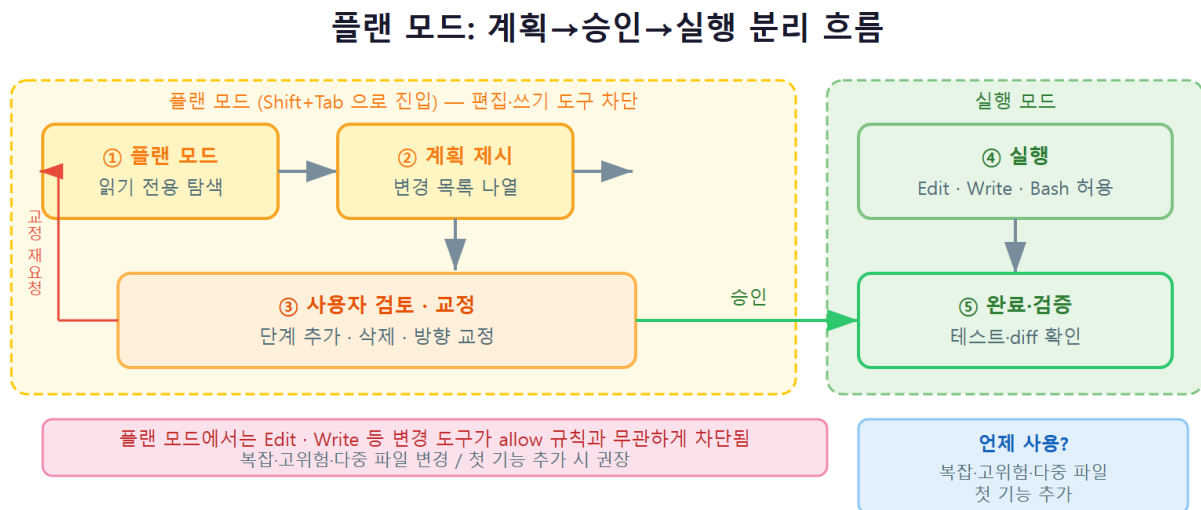
비유하자면, 플랜 모드는 **공사를 시작하기 전에 받는 설계 도면**입니다. 도면을 보고 "여기 벽은 빼고, 저 창문은 키워 달라"고 고친 뒤 착공을 승인하는 것과 같습니다. 도면 단계에서는 실제 건물이 손상되지 않으므로, 마음껏 검토하고 바꿀 수 있습니다.

권한과 플랜 모드는 어떻게 다른가

- **권한(ch_05)**: 도구 호출 **하나하나**를 검사해 실행 여부를 통제합니다(allow·deny·ask). "이 `rm` 명령을 실행해도 되나?"
- **플랜 모드(이번 챕터)**: 변경을 시작하기 전에 **전체 계획**을 먼저 보게 합니다. "무엇을 바꿀 계획인지부터 보자."

둘은 경쟁하지 않고 **겹쳐서** 작동합니다. 플랜 모드로 방향을 잡고, 실행 단계에서 개별 명령은 권한이 통제합니다. 안전의 두 겹입니다.

6.3 동작 원리: 계획→승인→실행 분리 흐름(Shift+Tab)



플랜 모드의 동작은 **계획 → 검토·교정 → 승인 → 실행**의 네 단계로 흐름입니다. 각 단계를 따라가 보겠습니다.

1. **플랜 모드 진입(Shift+Tab)**: 터미널에서 Shift+Tab을 눌러 플랜 모드로 전환합니다. 모드는 기본(Normal) → 편집 자동 수락(acceptEdits) → 플랜(Plan) 순으로 순환하며, 플랜 모드에 들어가면 화면 하단에 "plan mode on"처럼 표시됩니다. 이 상태에서 작업을 지시합니다.
2. **읽기 전용 분석·계획 제시**: 에이전트는 읽기 도구로 관련 코드를 탐색하고, "어떤 파일을 어떻게 바꿀지"를 단계별 계획으로 정리해 제시합니다. 이 동안 **파일은 바뀌지 않습니다**.

3. **검토·교정**: 사용자가 계획을 읽습니다. 틀린 가정(엉뚱한 파일·잘못된 접근)이 있으면 자연어로 교정 지시를 주고, 에이전트는 계획을 다시 다듬어 제시합니다. 이 검토·교정은 원하는 만큼 반복할 수 있습니다.
4. **승인 → 실행 전환**: 계획이 만족스러우면 승인합니다. 그러면 에이전트가 실행 모드로 넘어가 계획대로 실제 변경을 시작하고, 이때부터 개별 도구 호출은 ch_05의 권한 규칙으로 통제됩니다.

이 흐름의 핵심은 **계획과 실행을 분리**한다는 점입니다. 보통 모드에서는 관찰 직후 곧장 편집으로 넘어가지만, 플랜 모드는 그 사이에 "계획 제시 → 사람 승인"이라는 게이트를 끼웁니다. 덕분에 **잘못된 방향의 변경이 시작되기 전에** 사람이 개입할 기회가 생깁니다.

단계	사용자 행동	에이전트 동작	파일 변경
1. 진입	Shift+Tab으로 플랜 모드 켜기	플랜 모드로 전환	없음
2. 계획	작업 지시	읽기 전용 분석 → 계획 제시	없음
3. 교정	틀린 가정 교정 지시	계획 재작성	없음
4. 실행	계획 승인	실행 모드로 전환 → 변경 시작	발생(권한이 통제)

승인하지 않고 플랜 모드를 빠져나오고 싶다면 Shift+Tab을 다시 눌러 계획을 실행하지 않고 모드를 벗어날 수도 있습니다. 즉 **계획을 받았다고 반드시 실행할 의무는 없습니다**. 검토 후 "이 방향은 아니다" 싶으면 버리면 됩니다.

계획 단계는 "되돌릴 것이 없는" 안전 지대

플랜 모드에서는 파일이 바뀌지 않으므로, 계획이 아무리 엉뚱해도 **실제 손상은 없습니다**. 그래서 신입이 낯선 코드베이스에서 마음 편히 "일단 계획부터 받아 보는" 데 적합합니다. 손해 없이 에이전트의 의도를 미리 들여다볼 수 있는 것이 플랜 모드의 가장 큰 가치입니다.

6.4 플랜 모드에서 편집·쓰기 도구가 차단되는 이유

플랜 모드의 핵심 성질은 **편집·쓰기 도구가 차단된다**는 것입니다. 그런데 여기서 신입이 자주 헷갈리는 점이 있습니다. "ch_05에서 `Edit(routers/**)` 을 allow로 자동 허용했는데, 플랜 모드에서는 왜 편집이 안 되나?"

답은 이렇습니다. **플랜 모드의 읽기 전용 제약은 권한 규칙(allow)보다 우선합니다.** 즉 `Edit(routers/**)` 을 allow에 두었더라도, 플랜 모드에 있는 동안에는 그 편집이 **허용 규칙과 무관하게 차단**됩니다. 왜 이렇게 설계했을까요?

이유는 플랜 모드의 **목적이 "계획"이지 "실행"이 아니기 때문**입니다. 만약 플랜 모드에서도 allow된 편집이 그냥 실행돼 버린다면, "계획만 받고 검토하려던" 의도가 깨집니다. 사용자가 계획을 미처 검토하기도 전에 파일이 바뀌어 있을 테니까요. 그래서 플랜 모드는 "이 모드에 있는 동안에는 **무조건 읽기 전용**"이라는 강한 보장을 줍니다. 이 보장이 있어야 "계획 단계에서는 절대 파일이 안 바뀐다"고 신뢰할 수 있습니다.

정리하면 안전장치가 두 겹으로 쌓입니다.

- **플랜 모드(이번 챕터):** 모드 전체에 걸친 "읽기 전용" 보장. 계획 단계에서는 어떤 편집도 일어나지 않습니다.
- **권한 규칙(ch_05):** 실행 모드에서 개별 명령을 통제. 승인 후 실제 변경이 시작될 때 작동합니다.

이 둘의 관계를 한 문장으로 하면, "**플랜 모드는 계획 단계 전체를 잠그고, 권한은 실행 단계의 명령 하나하나를 통제한다**"입니다. 둘 다 있어야 "방향도 안전하고 실행도 안전한" 변경이 됩니다.

왜 읽기 도구는 되고 편집 도구는 안 되나

플랜 모드에서도 Read·Glob·Grep 같은 **읽기 도구는 동작**합니다. 계획을 세우려면 코드를 봐야 하니까요. 읽기는 아무것도 바꾸지 않으므로(부수효과 없음) 허용해도 안전합니다. 반면 Edit·Write·부수효과 Bash는 파일·시스템을 바꾸므로, 계획 단계에서는 막습니다. 이 구분은 ch_05에서 배운 "읽기 전용 도구 vs 부수효과 도구"와 정확히 같은 선입니다.

6.5 worked example: 엔드포인트 추가를 플랜 모드로 계획받아 실행

이제 실제로 플랜 모드를 써 보겠습니다. 상황은 이렇습니다. 우리는 1장에서 셋업한 팀의 FastAPI 할 일 관리 저장소 `team-tasks-api` 에서, 특정 할 일 하나를 삭제하는 엔드포인트 `DELETE`

/tasks/{task_id} 를 추가하려 합니다(이 저장소에는 아직 삭제 기능이 없습니다). 첫 기능 추가이니 곧장 편집하지 않고 **플랜 모드로 계획부터** 받겠습니다.

먼저 기존 코드의 일부를 봐 둡니다. 라우터 파일은 이런 모습입니다(데이터는 store 를 통해 다룹니다).

```
# routers/tasks.py
from fastapi import APIRouter, HTTPException
from models import Task, TaskCreate
import store

router = APIRouter(prefix="/tasks", tags=["tasks"])

@router.get("", response_model=list[Task])
def list_tasks() -> list[Task]:
    return store.get_all_tasks()
```

이제 터미널에서 Shift+Tab을 눌러 플랜 모드로 들어간 뒤 작업을 지시합니다.

(Shift+Tab → Shift+Tab 눌러 plan mode on 표시 확인)

> @routers/tasks.py 를 참고해서, task_id로 할 일 하나를 삭제하는 DELETE /tasks/{task_id} 엔드포인트를 추가하는 계획을 세워 줘. 데이터는 store를 통해 다루고, 없는 id면 404를 반환해야 해.

● (읽기 전용 분석 중...)

Read(routers/tasks.py)
Read(store.py)

계획(Plan):

1. store.py에 delete_task(task_id) 함수 추가 (없으면 False 반환)
2. routers/tasks.py에 delete_task(task_id: int) 엔드포인트 추가
3. store.delete_task가 False면 HTTPException(status_code=404) 발생
4. tests/test_tasks.py에 204·404 두 경우 테스트 추가

이 계획대로 진행할까요?

- > 1. 네, 실행하세요
2. 아니요, 수정할게요

- 에이전트는 플랜 모드에서 **Read·Grep 같은 읽기 도구만** 써서 기존 코드를 분석했습니다. 이 동안 파일은 전혀 바뀌지 않았습니다.
- 제시된 계획은 **단계별**입니다. 어떤 함수를 추가하고, 어떤 예외를 쓰고, 데이터 계층(store)을 어떻게 거치고, 테스트까지 어디에 추가할지 명시했습니다.
- 마지막에 **승인 게이트**가 있습니다. "네, 실행하세요"를 고르기 전까지는 실행 모드로 넘어가지 않습니다.

계획이 합리적이므로 승인합니다. 그러면 에이전트가 실행 모드로 전환해 계획대로 코드를 추가하고, 결과는 이렇게 됩니다.

```
# store.py (추가된 함수)
def delete_task(task_id: int) -> bool:
    return _db.pop(task_id, None) is not None
```

```
# routers/tasks.py (추가된 엔드포인트)
@router.delete("/{task_id}", status_code=204)
def delete_task(task_id: int) -> None:
    if not store.delete_task(task_id):
        raise HTTPException(status_code=404, detail="Task not found")
```

실행 후 추가된 테스트를 돌리면(ch_05에서 `Bash(pytest:*)` 를 allow에 둔 덕분에 자동 실행) 다음과 같은 실행 결과가 나옵니다.

```
● Bash(pytest tests/test_tasks.py -q)

..... [100%]
9 passed in 0.41s
```

실행 결과를 보면, 계획대로 엔드포인트가 추가되고 기존 7개에 204·404 두 경우 테스트가 더해져 모두 통과했습니다. 핵심은 **곧장 편집하지 않고 계획을 먼저 받아 검토한 뒤 실행했다**는 점입니다. 만약 계획이 엉뚱했다면, 파일이 바뀌기 전에 바로잡을 수 있었습니다.

6.6 계획 교정: 단계 추가·삭제·재정렬

플랜 모드의 진짜 가치는 **계획을 받는 것이 아니라 계획을 고치는 것**에 있습니다. 에이전트가 제시한 계획이 항상 완벽하지는 않습니다. 단계가 빠졌거나, 순서가 어색하거나, 불필요한 단계가 섞일 수 있습니다. 이때 사용자는 자연어로 계획을 교정합니다.

계획을 고치는 방식은 크게 세 가지입니다.

- **단계 추가:** 빠진 작업을 넣습니다. 예: "테스트를 추가하는 단계가 빠졌어. 204·404 테스트도 계획에 넣어 줘."
- **단계 삭제:** 불필요한 작업을 뺍니다. 예: "기존 `list_tasks` 는 건드리지 마. 그 수정 단계는 빼 줘."
- **단계 재정렬:** 순서를 바꿉니다. 예: "테스트를 먼저 작성하고 구현하는 순서로 바꿔 줘 (TDD)."

이렇게 교정 지시를 주면 에이전트는 **계획만 다시 그려** 제시합니다. 여전히 플랜 모드이므로 파일은 바뀌지 않습니다. 만족할 때까지 이 과정을 반복한 뒤 승인하면 됩니다. 계획 단계에서의 교정은 **공짜**입니다. 실제 코드가 바뀐 뒤 되돌리는 것보다 훨씬 싸고 안전합니다.

계획(Plan):

1. delete_task 함수 추가
2. HTTPException import 추가

이 계획대로 진행할까요?

> 2. 아니요, 수정할게요

> 테스트 단계가 빠졌어. 그리고 테스트를 먼저 작성한 뒤 구현하는 순서로 바꿔 줘.

● 계획(Plan, 수정됨):

1. tests/test_tasks.py에 204·404 테스트 먼저 작성
2. delete_task 함수 구현해 테스트 통과시키기
3. HTTPException import 추가
4. pytest로 통과 확인

이 계획대로 진행할까요?

좋은 교정 지시의 요령

ch_03에서 배운 "구체적인 지시"가 여기서도 통합니다. "계획이 별로야" 같은 모호한 말 대신, **무엇을(단계) 어떻게(추가·삭제·재정렬)** 바꿀지 꼭 집어 말하십시오. "4번 단계는 빼고, 테스트를 1번으로 올려 줘"처럼 구체적일수록 계획이 정확히 다듬어집니다.

6.7 worked example: 잘못된 가정을 교정 지시로 바로잡기

계획 교정이 가장 빛나는 순간은 에이전트가 **잘못된 가정**을 했을 때입니다. 낯선 코드베이스에서는 에이전트도 구조를 오해할 수 있습니다. 플랜 모드는 그 오해가 **실제 변경으로 번지기 전에** 드러나게 해 줍니다.

상황: 우리 `team-tasks-api` 는 데이터를 `store.py` 의 저장소 계층에서 관리하는데(ch_04에서 파악했습니다), 에이전트가 이를 잠시 잊고 라우터 파일에서 직접 데이터를 고치는 계획을 세웠습니다.

(plan mode on)

> 할 일을 완료 처리하는 PATCH /tasks/{task_id}/done 엔드포인트를 추가하는 계획을 세워 줘.

● 계획(Plan):

1. routers/tasks.py 안에서 할 일 디렉터리를 직접 수정해 status를 done으로 바꾸는 코드 추가
2. PATCH 엔드포인트 함수 추가

이 계획대로 진행할까요?

> 2. 아니요, 수정할게요

> 잘못된 가정이야. 데이터는 라우터가 아니라 store.py의 저장소 계층에서 관리해. @store.py 를 확인하고, store의 함수를 호출하는 방식으로 계획을 다시 세워 줘.

● Read(store.py)

계획(Plan, 수정됨):

1. store.py에 mark_done(task_id) 함수 추가 (없는 id면 None 반환)
2. routers/tasks.py에 PATCH 엔드포인트 추가, store.mark_done 호출 후 None이면 404 반환
3. tests/test_tasks.py에 완료 처리·404 테스트 추가

이 계획대로 진행할까요?

- 첫 계획은 **잘못된 가정**(라우터에서 데이터 직접 수정)을 담았습니다. 만약 플랜 모드가 아니었다면, 이 잘못된 구조가 이미 코드로 들어갔을 것입니다.
- 사용자는 `@store.py` 로 **실제 구조를 참조시키며**(ch_03의 파일 멘션) 교정 지시를 줬습니다.
- 에이전트는 저장소 계층을 읽고(Read) **계획을 통째로 다시** 세웠습니다. 이 모든 일이 **파일 변경 없이** 일어났습니다.

이렇게 교정한 계획대로 실행하면, 팀의 아키텍처(라우터-저장소 분리)를 지키는 올바른 코드가 만들어집니다. 실행 결과로 완료 처리 엔드포인트와 테스트가 추가되고 통과합니다.

● Bash(pytest tests/test_tasks.py -q)

```
..... [100%]
9 passed in 0.52s
```

핵심 교훈은 이것입니다. **에이전트의 잘못된 가정은 플랜 모드에서 가장 싸게 발견된다.** 실제 코드가 바뀐 뒤 디버깅으로 잡는 것보다, 계획 단계에서 한 줄 교정으로 바로잡는 것이 훨씬 빠르고 안전합니다.

6.8 언제 쓰는가: 복잡·고위험·다중 파일·첫 기능 추가

플랜 모드는 강력하지만 **모든 작업에 쓸 필요는 없습니다**. 오타 하나 고치는 데 계획부터 받으면 오히려 번거롭습니다. 그래서 "언제 쓰는가"를 판단하는 감각이 중요합니다. 다음과 같은 작업에 플랜 모드가 특히 가치 있습니다.

상황	플랜 모드가 필요한 이유
복잡한 작업	여러 결정이 얹혀 있어, 방향이 틀리면 손해가 큼
고위험 변경	핵심 로직·데이터처럼 잘못되면 영향이 큰 부분
다중 파일 변경	여러 파일에 걸쳐 일관되게 바뀌어야 해, 전체 그림이 먼저 필요
첫 기능 추가	낯선 코드베이스라 에이전트의 가정이 틀릴 가능성이 큼

반대로 다음과 같은 작업에는 플랜 모드가 **과합니다**.

- 오타·문구 수정처럼 **결과가 뻔하고 위험이 없는** 작업.
- 한 파일의 한 줄만 바꾸는 **사소한 변경**.
- 이미 구조를 잘 아는 곳의 **단순 반복 작업**.

신입에게 권하는 기본 습관은 이렇습니다. **낯선 코드베이스의 첫 기능 추가나, 여러 파일에 걸친 변경은 일단 플랜 모드로 시작한다**. 익숙해지고 위험이 작은 작업은 보통 모드로 빠르게 처리합니다. "이 변경이 틀리면 손해가 큰가?"를 스스로 물어, 답이 "그렇다"면 플랜 모드를 켜십시오.

판단 기준 한 줄: "틀리면 손해가 큰가?"

플랜 모드를 쓸지 말지는 작업의 '**방향이 틀렸을 때의 손해**'로 판단합니다. 손해가 크면(복잡·고위험·다중 파일·첫 기능) 계획을 먼저 받고, 손해가 작으면(오타·사소한 수정) 바로 진행합니다. 이 기준 하나면 대부분의 상황을 판단할 수 있습니다.

6.9 즉시 연습 (2종 1세트): team-tasks-api에서 플랜 모드를 켜 계획을 실제로 받아 교정

배운 것을 종이로 채우지 말고, **team-tasks-api**에서 **Shift+Tab**으로 플랜 모드를 켜고 작은 변경 계획을 실제로 받아 검토·교정합니다. 플랜 모드가 정말 파일을 안 바꾸는지, 받은 계획에 잘못

된 가정이 섞이는지는 직접 받아 봐야 보입니다.

모범: 플랜 모드로 계획을 받아 교정한 실제 흐름(예)

team-tasks-api에서 `claude` 를 실행한 뒤 `Shift+Tab`을 눌러 플랜 모드로 전환하고 작은 변경을 요청하면, 에이전트가 *파일을 바꾸지 않고* 계획만 제시합니다.

> (`Shift+Tab`으로 플랜 모드) `status`가 `done`인 항목만 거르는 필터를 추가할 계획을 세워 줘.

● 계획(Plan):

1. `routers/tasks.py`에서 데이터를 직접 수정해 상태 필터를 구현
2. `GET /tasks?status=done` 엔드포인트 추가

이 계획에는 잘못된 가정(데이터는 라우터가 아니라 `store.py`에서 관리)과 빠진 단계(테스트)가 있습니다. 그래서 승인하지 않고 교정 지시를 줍니다.

> 잘못된 가정이 있어. 데이터는 라우터가 아니라 `store.py`에서 관리하니 `store`의 조회 함수를 쓰도록 바꿔 줘. 테스트 단계가 빠졌으니 `status=done`·`status=todo` 두 경우의 테스트를 먼저 작성하는 단계도 계획에 넣어 줘.

→ 에이전트가 계획을 수정해 다시 제시(여전히 파일은 안 바꿈). 검토 후 승인하면 실행 모드로 전환.

직접 해보기 (team-tasks-api에서 실제로 플랜 모드 실행)

연습 — 플랜 모드를 실제로 켜 작은 변경 계획을 받아 교정하라.

team-tasks-api에서 `claude` 를 실행하고 `Shift+Tab`으로 플랜 모드로 전환한 뒤, 작은 변경(예: "할 일을 삭제하는 `DELETE /tasks/{task_id}` 엔드포인트 추가")의 계획을 세워 달라고 요청하세요. 에이전트가 실제로 제시한 계획을 그대로 옮겨 적고, 그 안의 문제를 찾아 교정 지시를 실제로 줘 보세요.

[받은 계획] 플랜 모드에서 에이전트가 실제로 제시한 계획(번호별로 옮겨 적기):

[검토] 계획에서 발견한 것

- 잘못된 가정(데이터를 어디서 관리해야 하나):
- 불필요/위험한 단계가 있다면:
- 빠진 단계(없는 `id`의 404 처리, 테스트 등):

[교정] 실제로 준 교정 지시 / 교정 뒤 다시 받은 계획이 어떻게 바뀌었나:

[읽기 전용 확인] 계획을 받는 동안 파일이 하나도 안 바뀌었는가 (에디터/탐색기로 `routers/tasks.py`가 그대로인지 확인):

확인: 교정된 계획이 만족스러우면 승인해 실행 모드로 전환하고, `python -m pytest -q` 까지 통과하는지 확인해도 좋습니다. 만족스럽지 않으면 승인하지 말고 버립니다.

자가체크: 손으로 플랜 모드를 돌렸는지 점검하기

- [] team-tasks-api에서 **Shift+Tab**으로 플랜 모드를 실제로 켜다.
- [] 작은 변경의 계획을 실제로 받아 화면 그대로 옮겨 적었다.
- [] 받은 계획에서 잘못된 가정·빠진 단계를 **구체적으로** 짚었다("별로다"가 아니라 무엇이 틀렸는지).
- [] 교정 지시를 **실제로** 줘 보고, 계획이 바뀌는 것을 확인했다.
- [] 계획을 받는 동안 파일이 한 줄도 바뀌지 않은 것을(읽기 전용) 직접 확인했다.
- [] 이 작업이 플랜 모드를 쓸 만한 작업인지(틀리면 손해가 큰지) 판단했다.

6.10 흔한 오류와 오개념: 단순작업 과용·무비판 실행·읽기 전용 오해

신입이 플랜 모드에서 자주 저지르는 실수와 그 교정을 정리합니다.

오개념 1: "안전하니까 모든 작업을 플랜 모드로 하자"

플랜 모드는 단순 작업에는 **과합니다**. 오타 하나 고치는 데 계획부터 받으면 오히려 느려지고 번거롭습니다.

교정: 플랜 모드는 **복잡·고위험·다중 파일·첫 기능** 추가처럼 방향이 틀리면 손해가 큰 작업에 씁니다. "틀리면 손해가 큰가?"를 기준으로, 손해가 작은 사소한 작업은 보통 모드로 빠르게 처리 하십시오.

오개념 2: "계획을 받았으니 그대로 승인하면 되겠지"

계획을 받았다고 **무비판적으로 실행하면** 플랜 모드의 의미가 없습니다. 핵심은 계획을 받는 것이 아니라 **검토·교정**하는 것입니다.

교정: 제시된 계획을 반드시 읽고, **잘못된 가정·빠진 단계·불필요한 단계**가 없는지 확인하십시오. 의심스러우면 교정 지시를 주거나, 방향이 아니면 버립니다. 검토 없는 승인은 보통 모드와 다를 바 없습니다.

오개념 3: "플랜 모드에서도 allow한 편집은 실행되겠지"

플랜 모드는 **읽기 전용**이라, `Edit(routers/**)` 을 allow에 두었더라도 **그 모드에 있는 동안에는 편집이 차단**됩니다. "왜 허용했는데 안 바뀌지?"라고 당황하기 쉽습니다.

교정: 플랜 모드의 읽기 전용 제약은 **권한 규칙보다 우선**합니다. 계획 단계에서는 파일이 절대 안 바뀐다는 보장이 플랜 모드의 핵심입니다. 실제 편집은 **계획을 승인해 실행 모드로 전환**한 뒤에야 시작됩니다.

자주 묻는 질문(FAQ)

Q. 플랜 모드는 어떻게 켜고 끄나요? A. 터미널에서 **Shift+Tab**으로 모드를 순환합니다(기본 → 편집 자동 수락 → 플랜). 화면 하단에 현재 모드가 표시됩니다. 계획을 승인하면 실행 모드로 넘어가고, 승인하지 않고 Shift+Tab을 다시 눌러 플랜 모드를 빠져나올 수도 있습니다.

Q. 계획이 마음에 안 들면 꼭 실행해야 하나요? A. 아닙니다. **계획을 받았다고 실행할 의무는 없습니다**. 방향이 아니면 교정 지시를 주거나 그냥 버리면 됩니다. 계획 단계에서는 파일이 바뀌지 않으므로 손해가 없습니다.

Q. 권한(ch_05)만 잘 설정하면 플랜 모드는 없어도 되지 않나요? A. 둘은 역할이 다릅니다. 권한은 **개별 명령의 실행**을 통제하고, 플랜 모드는 **변경의 전체 방향**을 먼저 보게 합니다. 권한은 "이 `rm` 을 실행해도 되나"를, 플랜 모드는 "애초에 무엇을 바꿀 계획인가"를 다룹니다. 안전은 두 겹일 때 가장 튼튼합니다.

6.11 단원 미니 프로젝트: 안전 장비 갖추고 변경 계획 받기 (2종 1세트)

이 챕터로 단원 u5(안전하게 변경하기)가 끝납니다. 마무리로, 이 단원에서 배운 **권한과 승인(c05)**과 **플랜 모드(c06)**를 하나로 통합하는 작은 프로젝트를 수행합니다. 목표는 "신입이 낯선 FastAPI 저장소에서 **안전 장비를 갖추고, 작은 변경의 계획을 받아 확정**하는" 전체 흐름을 한 번에 체험하는 것입니다.

프로젝트 목표

1. `team-tasks-api` 에서 **새 작업 브랜치**를 만들고, `pytest · routers/` 편집은 자동 허용하고, 삭제·강제리셋·비밀 파일 읽기는 차단하는 **권한 규칙**을 `settings.json` 에 작성한다(c05).
2. 작은 변경(엔드포인트 하나 추가)을 **플랜 모드로 계획받아 검토·교정**한 뒤 확정한다(c06).
3. 권한 규칙을 **프로젝트 공유 범위에 저장**해 팀이 공유할 수 있게 한다.

마일스톤(증분)

- **M0. 작업 브랜치:** `team-tasks-api` 에서 `git checkout -b feat/status-filter` 로 브랜치를 만든다.
- **M1. 안전 장비 설치(권한):** `.claude/settings.json` 에 `allow·deny·ask` 규칙을 작성한다.
- **M2. 계획 받기(플랜 모드):** `Shift+Tab`으로 플랜 모드에 들어가, 작은 엔드포인트 추가의 계획을 받는다.
- **M3. 검토·교정·확정·검증:** 계획의 가정·단계를 점검해 교정하고, 승인해 실행한 뒤 `python -m pytest -q` 로 검증한다.

모범 예시 (완성된 산출물)

M1의 권한 규칙(모범):

```
{
  "permissions": {
    "allow": [
      "Bash(pytest:*)",
      "Edit(routers/**)"
    ],
    "ask": [
      "Bash(git push:*)"
    ],
    "deny": [
      "Bash(rm:*)",
      "Bash(git reset --hard:*)",
      "Read(.env)"
    ]
  }
}
```

M2~M3의 계획 받기·확정 흐름(모범):

(Shift+Tab → plan mode on)

> @routers/tasks.py 를 참고해, status로 거른 할 일만 돌려주는
GET /tasks?status=done 필터 계획을 세워 줘. 데이터는
store.py에서 관리하니 store 함수를 써.

● 계획(Plan):

1. store.py에 list_by_status(status) 함수 추가
2. routers/tasks.py의 list_tasks에 status 쿼리 파라미터 추가
3. tests/test_tasks.py에 status=done·status=todo 테스트 추가
4. pytest로 통과 확인

이 계획대로 진행할까요?

> 1. 네, 실행하세요

승인 후 실행 결과(모범):

● Bash(pytest tests/test_tasks.py -q)

```
..... [100%]
9 passed in 0.55s
```

실행으로 검증: `python -m pytest -q` 가 통과(9 passed)해야 끝입니다. 통과를 확인한 뒤 브랜치를 커밋합니다.

직접 작성·실행: 나의 단원 미니 프로젝트 (team-tasks-api에서 실제로 실행)

빈칸에 추측을 적는 것이 아닙니다. team-tasks-api 에서 **실제로 권한 파일을 만들고 플랜 모드를 돌린 뒤**, 화면에 나타난 것을 그대로 기록하세요. 미리 정한 답이 아니라 **방금 화면에서 본 것**을 옮깁니다.

M1. 권한 규칙을 실제로 작성·적용하라. team-tasks-api 에 .claude/settings.json 을 직접 만들고, 5장에서 배운 allow / ask / deny 로 권한을 정해 저장하세요. 무엇을 어디에 둘지는 스스로 판단합니다.

- **허용(allow)** — 자주 쓰고 안전한 작업: 예) 테스트 실행 명령(pytest)과 routers/ 폴더 편집. 와일드카드는 Bash(pytest:*)·Edit(routers/**) 처럼 **명령·경로 단위로 좁혀** 씁니다.
- **차단(deny)** — 위험한 작업: 예) 파일 삭제(rm)·강제 리셋(git reset --hard)·비밀 파일(.env) 읽기.
- **저장 위치** — 팀이 공유할 규칙이므로 개인용(.local)이 아니라 **프로젝트 공유 .claude/settings.json** 에 둡니다.

내가 실제로 저장한 내용을 그대로 옮겨 적습니다.

[작성] 내가 `.claude/settings.json`에 실제로 저장한 `permissions(allow/ask/deny)`:

[확인] claude를 다시 실행해 권한이 적용됐는지 화면으로 본 것:

- `pytest`를 시켰을 때(`allow` 적용 — 권한 확인 없이 바로 실행됐는가?) 실제 화면:
- 삭제(`rm` 등)를 시켰을 때(`deny` 적용 — 차단·거부됐는가?) 실제 화면:
- 저장 위치(팀 공유 범위로 `.claude/settings.json`에 두었는가?):

M2. 플랜 모드를 실제로 돌려라. `Shift+Tab`으로 플랜 모드(읽기 전용)에 들어가, 작은 변경 하나를 계획만 받으세요(아직 실행하지 않습니다). 예를 들어 "`routes/tasks.py`에 할 일 개수를 세어 돌려주는 `GET /tasks/count` 엔드포인트를 추가하는 계획을 세워 줘"처럼, 자신이 정한 작은 작업을 지시하면 됩니다. 실제로 준 지시와 화면에 나타난 계획을 그대로 적습니다.

> 내가 실제로 준 지시:

[계획] 에이전트가 파일 변경 없이 제시한 계획(화면에 나타난 단계를 그대로):

M3. 계획을 실제로 점검·교정하고 실행하라. 위 계획을 보고 점검한 결과와, 교정·승인 후 실행한 테스트 출력을 적습니다.

- 가정 점검(잘못된 가정이 있었나? 어떻게 교정했나?):

- 빠진 단계 점검(빠진 단계가 있었나?):

- 위험 단계 점검(위험한 단계가 있었나?):

[결과 확인] 교정된 계획을 승인해 실행한 뒤 `python -m pytest -q` 실제 출력:

자가체크: 미니 프로젝트 완료 점검 (스스로 점검)

아래 체크리스트가 모두 "예"가 되면 단원 미니 프로젝트를 완료한 것입니다.

- [] (M0) `team-tasks-api`에서 새 작업 브랜치를 만들어 작업했는가?
- [] (M3) `python -m pytest -q`가 통과(9 passed)하는 것을 화면 출력으로 확인했는가?
- [] (M1·c05) `allow`에 둔 항목이 전부 자주 쓰고 안전한 작업인가?
- [] (M1·c05) 와일드카드를 `Bash(pytest:*)`처럼 명령 단위로 좁혔는가?
- [] (M1·c05) 삭제·강제리셋·비밀 읽기를 **deny**에 명시했는가?
- [] (M1·c05) 권한 규칙을 프로젝트 공유(`.claude/settings.json`)에 두었는가?
- [] (M2·c06) **Shift+Tab**으로 플랜 모드에 들어가 계획을 받았는가?
- [] (M2·c06) 계획이 파일 변경 없이 제시된다는 것을 확인했는가?
- [] (M3·c06) 계획의 잘못된 가정·빠진 단계·위험 단계를 점검·교정했는가?
- [] (M3·c06) 만족스러운 계획만 승인해 실행으로 넘겼는가?

6.12 마무리: 단원 u5를 마치며

이번 챕터에서 우리는 변경을 시작하기 전에 계획을 먼저 받는 **플랜 모드**를 배웠습니다. 핵심을 정리하면 다음과 같습니다.

- 플랜 모드는 **읽기 전용**으로 변경 계획을 먼저 제시하는 모드이며, **Shift+Tab**으로 전환합니다.
- 계획 단계에서는 **편집·쓰기 도구가 권한과 무관하게 차단**되어 파일이 바뀌지 않습니다.
- 흐름은 **계획 → 검토·교정 → 승인 → 실행**이며, 계획 교정(추가·삭제·재정렬)이 핵심 가치입니다.
- **틀리면 손해가 큰 작업**(복잡·고위험·다중 파일·첫 기능)에 쓰고, 사소한 작업에는 쓰지 않습니다.

이로써 단원 u5(안전하게 변경하기)가 끝났습니다. ch_05의 **권한과 승인**으로 "어떤 명령을 실행해도 되는가"를 통제하고, ch_06의 **플랜 모드**로 "무엇을 바꿀 계획인가"를 먼저 보는, 두 겹의 안전장치를 갖췄습니다. 이제 신입은 낯선 코드에 위험 없이 첫 변경을 시작할 준비가 되었습니다. 다음 단원(u6)에서는 그렇게 만든 변경이 **정말 동작하는지** 실행·테스트로 검증하는 법을 배웁니다. 안전하게 바꿨다면, 다음은 제대로 됐는지 증명할 차례입니다.

실행·테스트로 검증하기

학습 목표

- 에이전트가 만든 변경을 실제로 실행하게 지시해, 그 실행 결과를 근거로 의도대로 동작하는지 확인할 수 있습니다.
- 추가·수정한 엔드포인트에 대해 에이전트에게 pytest(파이테스트) 테스트를 작성·실행시켜, 통과 로그를 근거로 동작을 증명할 수 있습니다.
- 구현 전에 통과해야 할 테스트를 먼저 작성하게 해 완료 조건을 세우고(TDD, 테스트 주도 개발), 그 테스트를 통과하도록 구현을 유도할 수 있습니다.
- 실패한 테스트의 메시지를 근거로 원인을 짚어, 회귀 없이 통과시키는 최소 수정을 요청할 수 있습니다.

지금까지 우리는 신입으로서 안전하게 일하는 토대를 차곡차곡 쌓았습니다. 5단원에서 권한(c05)으로 무엇을 허용할지 통제했고, 같은 단원의 플랜 모드(c06)로 바꾸기 전에 계획을 먼저 받아 검토했습니다. 그렇게 계획을 승인하고 실행에 들어가면, 에이전트는 파일을 고치고 새 코드를 만들어 냅니다. 그리고 마지막으로 이렇게 말합니다. "엔드포인트를 추가했습니다. 완료했습니다."

여기서 신입이 가장 자주 빠지는 함정이 있습니다. 그 "완료했습니다"를 그대로 믿고 다음 일로 넘어가는 것입니다. 그러나 코드가 *고쳐졌다*는 사실과 코드가 *동작한다*는 사실은 전혀 다릅니다. 이 장에서 배우는 **실행·테스트로 검증하기(verifying with execution and tests)**는, 에이전트 루프(c02)의 마지막 단계인 "결과 확인"을 신입의 손에 쥐여 주는 기술입니다. 핵심 문장은 단 하나로 요약됩니다. **검증 전까지는 끝난 게 아닙니다.**

선수 개념 상기: 이 장은 2단원에서 배운 **에이전트 루프(c02)**의 네 단계(관찰 → 계획 → 도구 호출 → 결과 확인) 위에 세워집니다. 우리는 그 루프의 앞 세 단계는 자주 보지만, 마지막 "결과 확인"은 자칫 건너뛰기 쉽습니다. 이 장은 바로 그 마지막 고리를 실행과 테스트로 단단히 채웁니다. 또 6단원의 **플랜 모드(c06)**가 "바꾸기 전"의 안전장치였다면, 이 장의 검증은 "바꾼 후"의 안전장치입니다. 둘이 합쳐져야 변경의 앞뒤가 모두 통제됩니다.

이 장의 모든 예제는 신입이 첫 출근에서 받게 될 법한 **FastAPI(파이썬 웹 백엔드 프레임워크)** 저장소를 무대로 합니다. 작은 할 일 관리 API(`team-tasks-api`)에 엔드포인트를 더하고, 그것이

정말 동작하는지 pytest로 증명하는 과정을 처음부터 끝까지 따라가 봅니다.

도입: 검증 전까지는 끝난 게 아니다

"됐습니다"라는 말의 함정

상황을 그려 보겠습니다. 여러분은 팀 저장소에 "할 일 목록을 조회하는 `GET /tasks` 엔드포인트를 추가해 줘"라고 지시했습니다. 에이전트는 라우터 파일을 열어 코드를 더하고, 잠시 뒤 이렇게 답합니다. " `/tasks` 엔드포인트를 추가했습니다. 저장된 할 일 목록을 반환합니다." 코드를 열어 보니 그럴듯한 함수가 들어가 있습니다. 마음이 놓여 "고맙다"고 하고 PR을 올리려 합니다.

그런데 그 코드는 정말 동작할까요? 임포트(import) 한 줄이 빠졌을 수도 있고, 반환 타입이 FastAPI가 직렬화하지 못하는 객체일 수도 있고, 저장소가 비었을 때 예외를 던질 수도 있습니다. 에이전트의 "추가했습니다"는 **코드를 작성했다**는 뜻이지, **실행해서 통과를 확인했다**는 뜻이 아닐 수 있습니다. 둘 사이의 간극을 메우는 것이 바로 검증입니다.

왜 신입에게 검증이 특히 중요한가

경력자는 코드를 눈으로 훑기만 해도 "이건 안 돌아가겠다"를 어느 정도 압니다. 그러나 신입은 그 직관이 아직 없습니다. 그래서 신입일수록 **눈이 아니라 실행에 판단을 맡겨야** 합니다. 실행과 테스트는 경험이 부족해도 누구에게나 똑같이 객관적인 증거를 줍니다. "초록색 통과 막대를 보기 전까지는 믿지 않는다"는 태도는 신입의 약점을 그대로 강점으로 바꿔 줍니다.

검증을 에이전트에게 맡긴다는 것

좋은 소식은, 검증조차 에이전트와 함께 할 수 있다는 점입니다. 우리는 에이전트에게 "코드를 짜라"고만 시키는 것이 아니라, "짠 코드를 **실행해서 결과를 보여 달라**", "그 동작을 증명할 **테스트를 작성하고 돌려 달라**"고 시킬 수 있습니다. 에이전트는 Bash 도구로 `pytest` 나 `uvicorn` 을 직접 실행할 수 있으므로, 작성과 검증을 한 흐름 안에서 끝낼 수 있습니다. 이 장은 그 "작성 → 실행 → 테스트 → 통과 확인"의 한 바퀴를 신입이 능숙하게 돌리도록 만드는 데 목표가 있습니다.

정의: 결과 확인은 실행·테스트로 하는 증명이다

실행·테스트로 검증하기(verified with execution and tests)란, 에이전트가 만든 변경을 실제로 **실행** 하고 **테스트** 로 동작을 증명해, 에이전트 루프의 마지막 단계인 "결과 확인"을 완결하는 작업입니다. 여기서 **테스트(test)**란 코드가 기대대로 동작하는지 자동으로 검사하는 또 다른 코드이고, **pytest(파이테스트)**는 파이썬에서 가장 널리 쓰이는 테스트 실행 도구입니다.

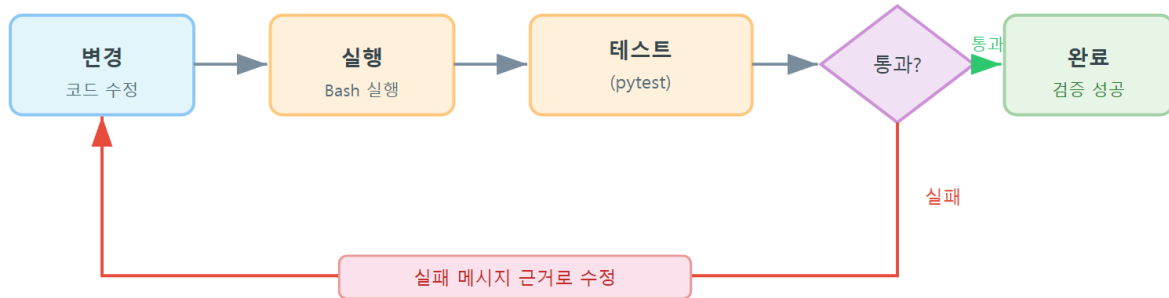
세 가지 핵심 용어를 먼저 정리하겠습니다.

- **실행(execution)**: 코드를 실제로 돌려 보는 것입니다. FastAPI라면 서버를 띄워 요청을 보내 보거나, 스크립트를 돌려 출력을 확인하는 것입니다.
- **단위 테스트(unit test)·통합 테스트(integration test)**: 함수 하나의 동작을 좁게 검사하면 단위 테스트, 여러 부품(라우터·모델·저장소)이 함께 도는 흐름을 검사하면 통합 테스트입니다. FastAPI 엔드포인트 검증은 보통 후자에 가깝습니다.
- **TestClient(테스트 클라이언트)**: FastAPI가 제공하는 도구로, 서버를 진짜 네트워크에 띄우지 않고도 메모리 안에서 엔드포인트에 요청을 보내 응답을 받을 수 있게 해 줍니다. 덕분에 테스트가 빠르고 안정적입니다.

이 정의의 핵심은 "증명"이라는 단어입니다. 검증은 주장이 아니라 증거를 만드는 일입니다. "동작합니다"는 주장이고, "이 테스트가 통과합니다"는 증거입니다. 신입은 주장 대신 증거로 말하는 법을 익혀야 합니다.

동작 원리: 변경 → 실행 → 테스트 → 통과 확인 루프

변경 → 실행 → 테스트 → 통과 확인 루프



검증 전까지는 끝난 것이 아니다

검증은 한 번 찍고 끝나는 점이 아니라, 통과할 때까지 도는 **루프(loop)**입니다. 이 루프의 각 단계를 따라가 보겠습니다.

1단계 — 변경(change). 에이전트가 Edit·Write 도구로 코드를 바꿉니다. 새 엔드포인트를 더하거나 버그를 고친 직후의 상태입니다. 이 시점의 코드는 아직 "추정"일 뿐입니다. 동작 여부는 아무도 모릅니다.

2단계 — 실행(run). 바꾼 코드를 실제로 돌립니다. 서버를 띄워 보거나(`uvicorn`), 모듈을 임포트해 봅니다. 여기서 임포트 예러나 문법 오류 같은 가장 기본적인 문제가 즉시 드러납니다. 코드가 "켜지기는 하는가"를 확인하는 단계입니다.

3단계 — 테스트(test). `pytest` 를 실행해, 미리 정해 둔 기대(예: "GET /tasks는 200과 리스트를 반환한다")가 실제로 충족되는지 검사합니다. 테스트는 사람의 눈보다 훨씬 꼼꼼하고, 한 번 써두면 다음에도 똑같이 검사해 줍니다.

4단계 — 통과 확인(verify). 테스트 결과를 읽습니다. 모두 통과(`passed`)면 이 변경은 **증명된** 것입니다. 루프를 빠져나가 다음 일로 넘어갑니다. 하나라도 실패(`failed`)하면 루프는 끝나지 않습니다.

실패 시 — 메시지 근거 수정. 실패한 테스트의 메시지(어떤 값이 기대됐고 실제로 무엇이 나왔는지)를 에이전트에게 그대로 전달해 수정을 요청합니다. 그리고 다시 2단계로 돌아가 실행·테스트를 반복합니다. 이 되먹임이 검증 루프의 심장입니다.

팁: "돌려서 통과를 보여 줘"를 한 문장으로 붙여라

지시 끝에 "그리고 `pytest`로 돌려서 통과하는 것까지 확인해 줘"를 습관처럼 덧붙이세요. 그러면 에이전트는 작성에서 멈추지 않고 위 루프를 스스로 한 바퀴 돌아, 통과한 테스트 출력까지 보

여 줍니다. 검증을 지시의 일부로 못 박는 것이 "완료" 정의를 분명히 하는 가장 간단한 방법입니다.

worked example: 엔드포인트를 pytest(TestClient)로 검증

이제 실제 세션을 처음부터 끝까지 따라가며, 엔드포인트 하나를 추가하고 그것을 테스트로 증명하는 흐름을 완전히 풀어 보겠습니다. 무대는 작은 할 일 관리 API입니다. 현재 저장소 구조는 다음과 같습니다.

```
team-tasks-api/
  main.py
  models.py
  store.py
  routers/
    __init__.py
    tasks.py
  tests/
    __init__.py
    test_tasks.py
  requirements.txt
```

store.py에는 메모리에 할 일을 보관하는 아주 단순한 저장소가 있고, routers/tasks.py에는 GET /tasks · POST /tasks · GET /tasks/{task_id}가 이미 있습니다. 이 절에서는 그중 **할 일 목록을 조회하는 GET /tasks**의 동작을, 우리 손으로 pytest 테스트를 작성해 증명해 봅니다(엔드포인트를 새로 만드는 게 아니라, 이 장의 주제인 "검증"을 익히는 것이 목표입니다).

1) 작업과 검증을 한 번에 지시한다

나: routers/tasks.py의 GET /tasks가 정말 기대대로 동작하는지 증명하는 pytest 테스트를 tests/test_tasks.py에 추가해 줘. 응답이 200이고 할 일 객체의 리스트인지, 그리고 POST로 만든 할 일이 목록에 보이는지 검사해야 해.
작성한 뒤 python -m pytest -q로 돌려서 통과하는 것까지 확인해 줘.

이 지시에는 3단원에서 배운 좋은 지시의 네 요소가 들어 있습니다. 목표(GET /tasks 검증 테스트 추가), 맥락(routers/tasks.py), 제약(200·리스트·목록 반영 검사), 완료 조건(테스트 작성·통과 확인)입니다. 특히 마지막 한 문장이 검증을 작업의 일부로 못 박습니다.

2) 검증 대상 엔드포인트 코드 (이미 저장소에 있음)

에이전트는 기존 라우터를 읽어(Read·Grep) 무엇을 검증할지 파악합니다.

```
# routers/tasks.py (발췌)
from fastapi import APIRouter, HTTPException
from models import Task, TaskCreate
import store

router = APIRouter(prefix="/tasks", tags=["tasks"])

@router.post("", response_model=Task, status_code=201)
def create_task(data: TaskCreate) -> Task:
    return store.create_task(data)

@router.get("", response_model=list[Task])
def list_tasks() -> list[Task]:
    return store.get_all_tasks()
```

- `import store`: 메모리 저장소 모듈을 가져옵니다. 데이터 보관 로직을 라우터에서 분리해 두는 구조입니다.
- `router = APIRouter(prefix="/tasks", ...)`: 이 라우터의 모든 경로 앞에 `/tasks` 가 붙습니다. 그래서 `@router.get("")` 은 실제로 `GET /tasks` 가 됩니다.
- `def list_tasks() -> list[Task]`: 반환 타입을 `list[Task]` 로 명시했습니다. FastAPI는 이 타입 힌트를 보고 응답을 자동으로 JSON 리스트로 직렬화합니다. 우리 테스트가 "응답은 리스트"를 검사할 근거가 되는 지점입니다.
- `return store.get_all_tasks()`: 저장소에 보관된 모든 할 일을 그대로 반환합니다.

3) 에이전트가 작성한 테스트 코드

이어서 에이전트는 이 엔드포인트의 동작을 증명할 테스트를 작성합니다. 각 테스트 전에 저장소를 비우는 `reset_store` 픽스처는 1장 저장소에 이미 들어 있습니다.

```
# tests/test_tasks.py (추가분)
from fastapi.testclient import TestClient

from main import app
import store

client = TestClient(app)

def test_list_tasks_returns_200_and_list():
    response = client.get("/tasks")
    assert response.status_code == 200
    assert isinstance(response.json(), list)

def test_created_task_appears_in_list():
    client.post("/tasks", json={"title": "온보딩 문서 읽기"})

    response = client.get("/tasks")
    titles = [task["title"] for task in response.json()]
    assert "온보딩 문서 읽기" in titles
```

- `from fastapi.testclient import TestClient`: FastAPI가 제공하는 테스트 클라이언트를 가져옵니다. 진짜 서버를 띄우지 않고도 요청을 보낼 수 있게 해 줍니다.
- `client = TestClient(app)`: 우리 앱(`main.py`의 `app`)을 감싼 클라이언트를 만듭니다. 이제 `client.get` · `client.post` 로 엔드포인트를 호출할 수 있습니다.
- `response = client.get("/tasks")`: 실제 HTTP 요청처럼 `GET /tasks` 를 보냅니다. 네트워크가 아니라 메모리 안에서 처리되므로 빠릅니다.
- `assert response.status_code == 200`: 정상 응답인지 확인합니다. `assert` 는 "이게 참이 아니면 테스트를 실패시켜라"는 뜻입니다.
- `assert isinstance(response.json(), list)`: 응답 본문이 리스트인지 확인합니다. 지시한 "응답은 리스트" 제약을 테스트가 직접 검사하는 부분입니다.
- 두 번째 테스트는 `POST` 로 할 일을 하나 넣은 뒤, `GET` 으로 그 제목이 목록에 보이는지 확인합니다. 두 엔드포인트가 같은 저장소를 공유하는 흐름을 통합적으로 검증합니다.

4) pytest 실행과 통과 확인

이제 에이전트가 Bash 도구로 테스트를 실행합니다. Windows 11의 터미널에서 실행되는 명령과 출력은 다음과 같습니다.

```
python -m pytest -q
```

```

===== test session starts =====
platform win32 -- Python 3.11.5, pytest-8.0.0, pluggy-1.4.0
rootdir: C:\dev\team-tasks-api
plugins: anyio-4.2.0
collected 9 items

tests\test_tasks.py ..... [100%]

===== 9 passed in 0.41s =====

```

`test_tasks.py`에는 1장 저장소의 기존 테스트 7개가 이미 있고, 우리가 쓴 2개가 더해져 모두 9개입니다. 점 아홉 개는 그 아홉 개가 각각 통과했다는 표시이고, 마지막 줄 `9 passed in 0.41s`가 우리가 기다리던 증거입니다. 이제 비로소 "GET /tasks의 동작을 테스트로 확인했습니다"라고 말할 수 있습니다. 테스트를 작성한 3)번 단계가 아니라, 이 통과를 확인한 4)번 단계가 작업의 진짜 끝입니다.

팁: `-q`와 `-v`를 골라 쓰기

`python -m pytest -q`는 결과를 한 줄로 간결히 보여 줍니다(빠른 확인용). 반대로 어떤 테스트가 도는지 이름까지 보고 싶으면 `python -m pytest -v`(verbose, 자세히)를 쓰면 각 테스트 이름과 통과/실패가 한 줄씩 나옵니다. 실패를 추적할 때는 `-v`가 편합니다.

FastAPI 검증: 서버 실행·요청으로 엔드포인트 확인

테스트가 검증의 자동화된 형태라면, 서버를 직접 띄워 요청을 보내 보는 것은 가장 원초적인 형태의 실행 검증입니다. 두 가지를 함께 쓸 줄 알아야 합니다. 때로는 "일단 켜지는지"부터 눈으로 보고 싶을 때가 있기 때문입니다.

에이전트에게 이렇게 지시할 수 있습니다.

나: 서버를 띄워서 GET /tasks가 실제로 응답하는지 확인해 줘.

에이전트는 개발 서버를 실행합니다. Windows 11에서는 `python -m`을 앞에 붙여 실행하는 것이 안전합니다.

```
python -m uvicorn main:app --reload
```

```
INFO: Will watch for changes in these directories: ['C:\\dev\\team-tasks-api']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [12345] using WatchFiles
INFO: Started server process [12346]
INFO: Application startup complete.
```

Application startup complete. 가 보이면 서버가 정상적으로 켜진 것입니다. 이 한 줄만으로도 "임포트 에러 없이 앱이 로딩된다"는 첫 번째 검증이 끝납니다. 이제 다른 터미널에서 요청을 보내 봅시다.

```
curl http://127.0.0.1:8000/tasks
```

```
[{"id":1,"title":"온보딩 문서 읽기","status":"todo"}]
```

응답으로 할 일 리스트가 JSON으로 돌아왔습니다. 사람이 눈으로 직접 "정말 목록이 나오는구나"를 확인하는 순간입니다. 브라우저로 `http://127.0.0.1:8000/docs` 에 접속하면, FastAPI가 자동 생성하는 대화형 문서에서 `GET /tasks` 를 클릭 한 번으로 실행해 볼 수도 있습니다.

주의: 서버 실행만으로는 검증이 끝나지 않는다

서버를 한 번 띄워 요청 한 번 보내 보는 것은 "지금 이 순간, 이 입력에서는" 동작한다는 확인일 뿐입니다. 빈 목록일 때, 잘못된 입력일 때, 동시에 여러 요청이 올 때는 어떻게 되는지 알 수 없습니다. 그래서 서버 실행은 **첫 확인** 이고, 그 확인을 **테스트로 박제해 반복 가능하게** 만드는 것이 진짜 검증입니다. "한 번 봤으니 됐다"에서 멈추지 마세요.

테스트 주도 개발(TDD): 테스트 먼저 → 구현

지금까지는 "코드를 만든 뒤 테스트로 검증"하는 순서였습니다. 이제 순서를 뒤집어 보겠습니다. **테스트를 먼저 쓰고, 그 테스트를 통과하도록 코드를 구현** 하는 방식입니다. 이것을 **테스트 주도 개발(TDD, Test-Driven Development)** 이라고 부릅니다.

왜 테스트를 먼저 쓰는가

언뜻 비효율적으로 보입니다. 아직 없는 코드의 테스트를 어떻게 쓴단 말일까요? 그러나 바로 그 점이 핵심입니다. 테스트를 먼저 쓴다는 것은 **"이 기능이 완성되면 무엇이 참이어야 하는가"를 코드로 못 박는** 일입니다. 즉, 완료 조건을 구현보다 먼저 정의하는 것입니다. 에이전트에

게 이 완료 조건을 미리 주면, 에이전트는 "통과해야 할 명확한 목표"를 갖고 구현합니다. 모호한 "잘 만들어 줘" 대신, "이 테스트를 통과시켜 줘"라는 검증 가능한 목표가 생깁니다.

TDD는 보통 세 박자로 돕니다.

단계	이름	하는 일
1	빨강(Red)	아직 구현이 없어 실패하는 테스트를 먼저 쓴다
2	초록(Green)	그 테스트를 통과시키는 최소 구현을 한다
3	정리(Refactor)	통과를 유지하며 코드를 다듬는다

worked example: 테스트 먼저 쓰고 구현시키기

이번에는 "할 일 하나를 완료 처리하는 POST /tasks/{task_id}/done 엔드포인트"를 TDD로 만들어 보겠습니다. 먼저 에이전트에게 테스트부터 시킵니다.

나: 아직 구현하지 말고, POST /tasks/{task_id}/done의 동작을 정의하는 테스트부터 tests/test_tasks.py에 작성해 줘. 기대 동작은 이거야.

- 존재하는 할 일을 완료 처리하면 200과 status가 done인 그 할 일을 반환한다.
- 없는 id면 404를 반환한다.

작성한 뒤 pytest로 돌려서, 구현이 없으니 실패하는 것을 먼저 보여 줘.

에이전트가 작성한 테스트입니다. (POST /tasks는 제목만 받아 id를 자동으로 매기므로, 저장소가 비워진 직후 만든 첫 할 일의 id는 1입니다.)

```
# tests/test_tasks.py (추가분)
def test_mark_task_done_returns_updated_task():
    client.post("/tasks", json={"title": "PR 올리기"}) # id=1

    response = client.post("/tasks/1/done")

    assert response.status_code == 200
    assert response.json()["status"] == "done"

def test_mark_done_unknown_id_returns_404():
    response = client.post("/tasks/9999/done")
    assert response.status_code == 404
```

이 테스트를 먼저 돌리면, 아직 엔드포인트가 없으므로 실패합니다(빨강 단계).


```

===== test session starts =====
collected 9 items

tests\test_tasks.py .....FF [100%]

===== FAILURES =====
_____ test_mark_task_done_returns_updated_task _____
    response = client.post("/tasks/1/done")
> assert response.status_code == 200
E   assert 404 == 200
E   + where 404 = <Response [404]>.status_code
===== 2 failed, 7 passed in 0.45s =====

```

지금은 완료 처리 엔드포인트가 없어서 FastAPI가 404 Not Found 를 돌려준 것입니다. 이 빨강이 우리가 의도한 출발점입니다. 이제 통과시키는 구현을 지시합니다.

나: 좋아. 이제 그 두 테스트가 통과하도록 POST /tasks/{task_id}/done을 구현해 줘.
구현 후 pytest로 돌려서 9개가 모두 통과하는 것을 확인해 줘.

에이전트가 추가한 구현입니다.

```

# routers/tasks.py (추가분)
from models import TaskStatus

@router.post("/{task_id}/done", response_model=Task)
def mark_task_done(task_id: int) -> Task:
    task = store.get_task(task_id)
    if task is None:
        raise HTTPException(status_code=404, detail="Task not found")
    task.status = TaskStatus.done
    return task

```

- `@router.post("/{task_id}/done", response_model=Task)`: 경로의 `{task_id}` 부분이 함수 인자 `task_id` 로 들어옵니다. prefix 와 합쳐져 실제 경로는 `POST /tasks/{task_id}/done` 입니다.
- `task = store.get_task(task_id)`: 저장소에서 해당 id의 할 일을 찾습니다. 없으면 `None` 을 돌려줍니다.
- `if task is None: raise HTTPException(status_code=404, ...)`: 없는 id면 404를 던집니다. 두 번째 테스트(`test_mark_done_unknown_id_returns_404`)가 검사하는 바로 그 동작입니다.
- `task.status = TaskStatus.done`: 찾은 할 일을 완료 상태로 바꿉니다. `store.get_task` 가 돌려준 객체는 저장소 안의 바로 그 객체라, 값을 바꾸면 저장소에도 반영됩니다. 첫 번째 테스트가 기대하는 `status == "done"` 을 만드는 부분입니다.

- `return task`: 갱신된 할 일을 반환합니다. FastAPI가 `Task`를 JSON으로 직렬화해 응답합니다.

구현 후 다시 테스트를 돌리면 초록으로 바뀝니다.

```
===== test session starts =====
collected 9 items

tests\test_tasks.py ..... [100%]

===== 9 passed in 0.47s =====
```

테스트를 먼저 썼기 때문에, 우리는 구현이 끝난 순간 그것이 *정확히 우리가 원하는 동작* 인지 곧바로 알 수 있었습니다. 완료 조건을 앞당겨 정의한 효과입니다.

worked example: 실패 메시지를 근거로 구현 교정

검증의 진짜 가치는 통과할 때가 아니라 **실패할 때** 드러납니다. 실패 메시지는 비난이 아니라 가장 정확한 수리 안내서입니다. 신입이 반드시 익혀야 할 기술은, 그 메시지를 잘라 내지 말고 **통째로** 에이전트에게 전달해 수정을 끌어내는 것입니다.

이번에는 일부러 어긋난 상황을 봅시다. 누군가 `POST /tasks/{task_id}/done`을 만들면서, 완료된 할 일을 반환할 때 상태 코드를 `201`로 잘못 지정했다고 합시다. 테스트는 `200`을 기대하므로 실패합니다.

```
===== test session starts =====
collected 9 items

tests\test_tasks.py .....F. [100%]

===== FAILURES =====
_____ test_mark_task_done_returns_updated_task _____

    def test_mark_task_done_returns_updated_task():
        client.post("/tasks", json={"title": "PR 올리기"}) # id=1
        response = client.post("/tasks/1/done")
>       assert response.status_code == 200
E       assert 201 == 200
E       + where 201 = <Response [201]>.status_code

tests\test_tasks.py:42: AssertionError
===== 1 failed, 8 passed in 0.46s =====
```

이 메시지를 **잘라 내지 않고** 그대로 에이전트에게 줍니다.

나: 다음 테스트가 실패했어. 메시지를 보고 원인을 진단해서 최소한으로 고쳐 줘.
(위 실패 메시지 전체를 붙여 넣음)

에이전트는 `assert 201 == 200` 이라는 핵심 줄을 읽고, "기대값 200, 실제값 201"이라는 사실에서 출발해 원인을 추적합니다. 그리고 응답 상태 코드를 잘못 지정한 부분을 찾아 고칩니다.

```
# 잘못된 코드
@router.post("/{task_id}/done", status_code=201)
def mark_task_done(task_id: int) -> Task:
    ...

# 고친 코드: 완료 처리는 새 자원 생성이 아니므로 200이 맞다
@router.post("/{task_id}/done")
def mark_task_done(task_id: int) -> Task:
    ...
```

팁: 실패 메시지에서 세 가지를 읽어라

pytest 실패 메시지에서 신입이 챙겨야 할 정보는 셋입니다. (1) 어느 테스트가 실패했나 (`test_mark_task_done...`), (2) 기대값과 실제값이 무엇인가(`assert 201 == 200`), (3) 어느 줄에서 났나(`test_tasks.py:42`). 이 셋을 그대로 에이전트에게 넘기면, 막연한 "안 돼" 대신 정확한 좌표를 준 셈이라 진단이 훨씬 빠릅니다. 이 습관은 11단원 디버깅에서 다시 본격적으로 다룹니다.

'검증 전까지는 끝난 게 아니다' 습관화

이 장의 모든 내용은 결국 하나의 습관으로 수렴합니다. **에이전트가 "완료했다"고 말해도, 실행과 테스트로 통과를 확인하기 전까지는 작업을 끝난 것으로 치지 않는다.** 이 습관을 몸에 붙이는 세 가지 작은 규칙을 제안합니다.

1. **지시에 검증을 붙인다.** "추가해 줘"가 아니라 "추가하고, 테스트를 써서 통과를 확인해 줘"라고 말합니다. 완료 정의를 지시에 못 박는 것입니다.
2. **초록 막대를 직접 본다.** 에이전트의 "통과했습니다"라는 문장 이 아니라, `N passed` 라는 출력을 눈으로 확인합니다. 가능하면 출력 전체를 보여 달라고 합니다.
3. **새 기능엔 새 테스트.** 동작을 바꾸거나 더했다면, 그 동작을 검사하는 테스트도 함께 남깁니다. 그래야 다음에 누가 코드를 건드려도 이 동작이 깨지지 않았는지 자동으로 확인됩니다.

이 습관은 다음 단원들의 토대가 됩니다. 8단원 git 커밋·PR에서 우리는 "테스트가 통과한 변경만 커밋·PR로 올린다"를 배우고, 9단원 코드 리뷰에서는 "테스트 결과를 PR 본문에 증거로 첨부한다"를 배웁니다. 검증은 이 책 후반부 전체를 떠받치는 기둥입니다.

즉시 연습 (2종 1세트): team-tasks-api에서 TDD를 직접 돌려 보기

이제 종이에 빈칸을 채우는 대신, 1장에서 준비한 실제 `team-tasks-api` 저장소를 **Claude Code**로 열고 TDD 루프를 직접 한 바퀴 돌립니다. 핵심은 "테스트 먼저 → 빨강(실패)을 눈으로 확인 → 구현 → 초록(통과)을 눈으로 확인"을 화면에서 실제로 겪는 것입니다. 미리 정한 답을 적는 것이 아니라, **화면에 실제로 나타난 pytest 출력을 그대로 옮겨 적습니다.**

1) 모범 예시 (실제 실행 예)

상황: 할 일을 삭제하는 `DELETE /tasks/{task_id}` 엔드포인트의 동작을, 구현보다 먼저 테스트로 정의합니다. 기대 동작은 "존재하는 할 일을 삭제하면 204(본문 없음)를 반환하고, 그 뒤 목록에서 사라진다"입니다. `team-tasks-api`에는 아직 삭제 엔드포인트가 없으므로, 이 테스트는 처음엔 빨강게 떨어집니다(TDD의 출발점).

```
# tests/test_tasks.py (테스트 먼저)
def test_delete_task_returns_204_and_removes_it():
    client.post("/tasks", json={"title": "삭제 대상"}) # id=1

    response = client.delete("/tasks/1")
    assert response.status_code == 204

    listed = client.get("/tasks").json()
    ids = [task["id"] for task in listed]
    assert 1 not in ids
```

구현 전 `python -m pytest -q`를 돌리면 엔드포인트가 없어 실패(빨강)합니다. 그다음 에이전트에게 구현을 시키고 다시 돌리면 통과(초록)로 바뀝니다.

```
> python -m pytest -q
.....F
1 failed, 7 passed
```

[100%]

```
> python -m pytest -q
.....
8 passed
```

[100%]

"삭제됐다"는 주장을 목록 재조회로 증명하고, 빨강 → 초록의 전환을 두 번의 pytest 출력으로 확인하는 것이 핵심입니다.

2) 직접 해보기 (team-tasks-api에서 TDD 한 바퀴)

연습 1 — 상태 필터를 TDD로 만들며 빨강 → 초록을 실제로 관찰하라.

team-tasks-api 폴더에서 `claude` 를 실행하고, 다음 지시를 그대로 주세요. (테스트를 먼저 추가하게 하는 것이 핵심입니다.)

```
> tests/test_tasks.py에 test_filter_by_status_done_only 테스트를 먼저 추가해 줘.
내용은: 할 일 둘을 만들고(POST /tasks) 하나만 완료 처리한 뒤,
GET /tasks?status=done으로 조회하면 완료된 할 일만 돌아오는지(status가 모두 "done")
검사하는 테스트다. 추가만 하고, 아직 GET /tasks 필터는 구현하지 마.
```

추가가 끝나면 직접 `python -m pytest -q` 를 돌려 **빨강(실패)**을 눈으로 확인하고, 화면에 실제로 나타난 출력을 그대로 옮겨 적으세요.

```
[테스트 먼저] 실패한 테스트 이름과 실패 단언(화면 그대로):
[빨강 확인] python -m pytest -q 실제 출력(예: 1 failed, 7 passed):
```

이어서 에이전트에게 구현을 시키세요.

```
> 이제 routers/tasks.py의 GET /tasks가 status 쿼리 파라미터를 받아
store의 할 일을 상태로 거르도록 구현하고, python -m pytest -q로 통과를 확인해 줘.
```

다시 `python -m pytest -q` 를 돌려 **초록(통과)**으로 바뀐 것을 확인하고 그대로 옮겨 적으세요.

```
[도구 호출] 에이전트가 바꾼 파일과 추가된 + 줄(diff 그대로):
[초록 확인] python -m pytest -q 실제 출력(예: 8 passed):
```

연습 2 — 서버 응답으로 한 번 더 눈으로 검증하라.

테스트가 통과했어도 실제 응답을 한 번 더 봅니다. 서버를 띄우고(`python -m uvicorn main:app -reload`) `http://127.0.0.1:8000/docs` 에서 `GET /tasks` 에 `status=done` 을 넣어 실행한 뒤, 완료된 할 일만 돌아오는지 응답 본문을 직접 확인하세요.

실제 응답 본문(완료 항목만 돌아왔는가?):

자가체크 — 머리로가 아니라 손으로 했는지 확인하세요.

- [] 구현보다 **테스트를 먼저** 추가했고, 그때 `python -m pytest -q` 가 **실패(빨강)** 하는 것을 화면 출력으로 봤다.
- [] [빨강 확인]·[초록 확인]에 적은 것이 추측이 아니라 화면에 실제로 나타난 pytest 출력이다.
- [] 구현 뒤 `python -m pytest -q` 가 **통과(초록, 8 passed)** 로 바뀐 것을 직접 확인했다.
- [] [도구 호출]에 적은 diff가 실제로 화면에 나타난 **+** 줄이다.
- [] Swagger UI 또는 테스트로 `status=done` 응답이 완료 항목만 돌려주는 것을 눈으로 확인했다.
- [] "테스트 먼저 → 빨강 → 구현 → 초록"의 순서를 직접 겪어 왜 검증 전까지는 끝난 게 아닌지 설명할 수 있다.

흔한 오류와 오개념

오개념 1: "에이전트가 '됐다'고 하면 된 것이다"

가장 위험한 착각입니다. 에이전트의 "완료했습니다"는 *코드를 작성했다* 는 뜻이지, *실행해서 통과를 확인했다* 는 뜻이 아닐 수 있습니다. 코드를 작성하는 일과 그 코드가 동작함을 증명하는 일은 별개입니다.

→ **교정:** 문장이 아니라 출력으로 확인하세요. "통과했습니다"라는 말 대신 `N passed` 라는 pytest 출력을 눈으로 봐야 비로소 끝난 것입니다. 지시 끝에 "pytest로 돌려서 통과를 보여 줘"를 붙이는 습관을 들이세요.

오개념 2: "테스트는 나중에, 일단 기능부터"

테스트를 "여유 있을 때 하는 뒷정리"로 미루면, 보통 영영 하지 않게 됩니다. 그리고 완료 조건이 모호한 채로 구현이 진행돼, "무엇이 되면 끝인지"를 아무도 모르는 상태가 됩니다.

→ **교정:** 테스트를 *먼저* 쓰면(TDD) 오히려 완료 조건이 분명해져 구현이 빨라집니다. 에이전트에게 "구현하기 전에 기대 동작을 테스트로 먼저 정의해 줘"라고 시키면, 목표가 코드로 못 박혀 작업이 흔들리지 않습니다.

오개념 3: "코드를 읽어 보니 맞으니까 동작한다"

눈으로 코드를 훑어 "맞는 것 같다"고 판단하는 것은 주장이지 증거가 아닙니다. 임포트 누락, 타입 불일치, 빈 입력에서의 예외처럼 눈으로 놓치기 쉬운 문제는 실행해야만 드러납니다. 통과한 테스트가 하나도 없다면 "동작한다"는 말에는 근거가 없습니다.

→ **교정**: "동작한다"고 말하고 싶을 때마다, 그것을 뒷받침하는 통과한 테스트나 실행 결과가 있는지 자문하세요. 증거가 없으면 그 말은 아직 추정일 뿐입니다. 신입은 주장 대신 증거로 말하는 법을 익혀야 합니다.

이 장에서 익힌 것

이 장에서 우리는 에이전트 루프의 마지막 단계인 "결과 확인"을 실행과 테스트로 완결하는 법을 배웠습니다. 핵심을 다시 정리합니다.

- 검증은 **변경 → 실행 → 테스트 → 통과 확인**의 루프이며, 실패하면 메시지를 근거로 수정해 루프를 다시 돕니다.
- FastAPI 엔드포인트는 **TestClient + pytest**로 빠르고 반복 가능하게 검증하고, 필요하면 `uvicorn`으로 서버를 띄워 눈으로도 확인합니다.
- **TDD(테스트 먼저 → 구현)**는 완료 조건을 앞당겨 정의해, 에이전트에게 통과해야 할 명확한 목표를 줍니다.
- 실패 메시지는 **통째로** 전달해야 정확한 진단을 끌어냅니다.
- 무엇보다, **검증 전까지는 끝난 게 아닙니다**.

다음 단원에서는 이렇게 검증한 변경을, 팀의 규약에 맞춰 다루는 법을 배웁니다. 세션마다 팀 규칙을 다시 설명하지 않도록 `CLAUDE.md`로 컨벤션을 명문화하는 8단원으로 이어집니다.

참고 출처

- [FastAPI 공식 문서 — Testing](#)
- [Claude Code 공식 문서 — How Claude Code works](#)

팀 규약 주입: CLAUDE.md로 컨벤션 명문화

학습 목표

- 주어진 FastAPI 저장소에 빌드·테스트 명령, 코딩 컨벤션, 금지사항을 항목으로 구조화한 CLAUDE.md를 작성할 수 있습니다.
- 팀 공유 규칙과 개인·하위 디렉터리 규칙을 구분해, 프로젝트·사용자·하위 디렉터리 세 계층의 올바른 위치에 배치할 수 있습니다.
- `/init` 로 CLAUDE.md 골격을 생성하고 `/memory` 와 `@` 참조로 규칙을 편집·모듈화할 수 있습니다.
- 간결·구체·최신 유지 원칙에 비추어 기존 CLAUDE.md에서 개선점 1개 이상을 찾아 고쳐 쓸 수 있습니다.

7단원까지 우리는 에이전트와 함께 코드를 탐색하고(4단원), 안전하게 바꾸고(5~6단원), 실행·테스트로 검증하는(7단원) 법을 익혔습니다. 그런데 매일 일하다 보면 같은 말을 반복하는 자신을 발견하게 됩니다. "테스트는 pytest로 돌려.", "들여쓰기는 4칸이야.", "main 브랜치에 직접 커밋하지 마.", "함수에는 타입 힌트를 붙여." 세션을 새로 열 때마다 에이전트는 이 규칙들을 모른 채 시작하므로, 신입은 매번 처음부터 팀의 약속을 설명해야 합니다.

이 장에서 배우는 **CLAUDE.md(클로드 점 엠디)** 는 바로 그 반복을 끝내는 장치입니다. 팀의 약속을 한 파일에 적어 두면, 에이전트가 세션을 시작할 때마다 그 파일을 *자동으로* 읽어 들입니다. 한 번 적어 두면 다시 설명할 필요가 없는, 저장소의 **헌법(constitution)** 같은 문서입니다. 이 장의 핵심 개념을 한마디로 하면 **CLAUDE.md로 팀 컨벤션 주입**입니다.

선수 개념 상기: 이 장은 4단원의 **코드베이스 탐색(c04)** 위에 세워집니다. 4단원에서 우리는 에이전트에게 "이 프로젝트는 무엇을 하나요?"를 물어 구조를 파악했습니다. CLAUDE.md는 그렇게 매번 묻고 답하던 맥락 중 *변하지 않는* 약속 들을 미리 적어 두어, 에이전트가 처음부터 알고 시작하게 만드는 것입니다. 또 5단원의 **권한(c05)**이 "무엇을 할 수 있는가"를 통제했다면, CLAUDE.md는 "무엇을 *어떻게* 해야 하는가"를 알려 준다는 점에서 짝을 이룹니다.

이 장의 모든 예제는 앞 장과 같은 할 일 관리 API(`team-tasks-api`) 저장소를 무대로 합니다. 팀의 규약을 이 저장소의 CLAUDE.md에 명문화해, 다음 작업부터 에이전트가 팀 표준대로 일하게 만들어 봅니다.

도입: 세션마다 팀 규칙을 다시 설명하지 않으려면

같은 말을 반복하는 신입의 하루

월요일 아침, 여러분은 세션을 열고 "할 일 삭제 엔드포인트를 추가해 줘"라고 지시합니다. 에이전트는 코드를 더하지만, 들여쓰기를 팀 규칙과 다르게 쓰고, 테스트를 빼먹고, 커밋 메시지를 영어 평문으로 적습니다. 여러분은 "들여쓰기는 4칸이야", "pytest 테스트도 같이 써", "커밋 메시지는 `feat:` 으로 시작해"라고 일일이 교정합니다.

화요일에 새 세션을 엽니다. 에이전트는 어제의 교정을 기억하지 못한 채 똑같은 실수를 반복합니다. 여러분은 다시 같은 말을 합니다. 이 비효율의 원인은 분명합니다. **에이전트는 세션마다 백지에서 시작** 하기 때문입니다. 사람이라면 한 번 들은 팀 규칙을 기억하지만, 새 세션의 에이전트에게 어제의 대화는 존재하지 않습니다.

왜 '문서에 적힌 규칙'이 '말로 한 규칙'보다 강한가

말로 한 규칙은 그 세션이 끝나면 사라집니다. 그러나 CLAUDE.md에 적힌 규칙은 *모든* 세션의 시작에 자동으로 다시 읽힙니다. 한 번 적는 수고로 무한히 반복되는 설명을 없애는 것입니다. 신입에게 이것은 단순한 편의가 아니라, 팀의 암묵지(말로만 전해지는 약속)를 명문화해 누구나 (에이전트 포함) 처음부터 표준대로 일하게 만드는 온보딩 장치입니다.

CLAUDE.md가 푸는 문제

CLAUDE.md는 이 "백지 시작" 문제를 정면으로 해결합니다. 저장소 루트에 `CLAUDE.md` 파일을 두면, 에이전트는 세션을 시작할 때마다 이 파일을 읽어 그 안의 지침을 맥락으로 삼습니다. "테스트는 pytest로", "타입 힌트 필수", "main에 직접 커밋 금지" 같은 약속을 한 번 적어 두면, 매 세션 그것을 알고 시작합니다. 신입은 반복 설명에서 해방되고, 팀 전체는 일관된 코드를 얻습니다.

정의: CLAUDE.md는 세션마다 자동으로 읽히는 헌법이다

CLAUDE.md 란, 저장소·사용자·조직 수준에 두는 마크다운 파일로, Claude Code가 세션을 시작할 때마다 자동으로 읽어 빌드·테스트 명령, 코딩 컨벤션, 아키텍처, 금지 사항 같은 영속 지침을 에이전트에 주입하는 **메모리 파일(memory file)**입니다. 사람이 평문으로 적고, 에이전트가 매 세션 그것을 맥락으로 삼는다는 점에서 저장소의 **헌법**에 비유됩니다.

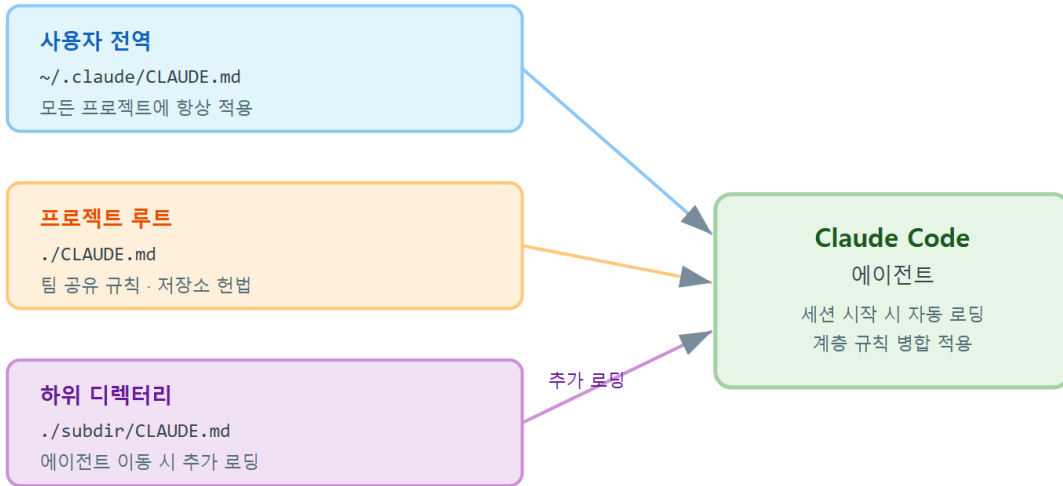
핵심 성질 세 가지를 정리합니다.

- **자동 로딩(automatic loading)**: 사람이 "이 파일을 읽어"라고 시키지 않아도, 세션이 시작되면 에이전트가 알아서 읽습니다. 매번 컨텍스트를 붙여 넣을 필요가 없습니다.
- **계층(hierarchy)**: CLAUDE.md는 한 곳에만 있는 게 아니라 여러 위치에 둘 수 있습니다. 조직 전체 규칙, 개인 작업 취향(~/.claude/CLAUDE.md), 프로젝트 약속(./CLAUDE.md), 특정 하위 폴더의 세부 규칙으로 나뉩니다.
- **추가 로딩(additive)**: 에이전트는 현재 작업 디렉터리에서 위쪽으로 디렉터리 트리를 거슬러 올라가며 만나는 CLAUDE.md들을 *더해서* 읽습니다. 상위의 큰 규칙 위에 하위의 세부 규칙이 얹히는 식입니다.

여기서 "메모리"라는 단어에 유의하세요. Claude Code의 메모리에는 두 종류가 있습니다. 하나는 우리가 직접 적는 **CLAUDE.md** 이고, 다른 하나는 에이전트가 사용자의 교정·선호를 보고 스스로 적어 두는 **자동 메모리(auto memory)**입니다. 이 장에서 우리가 설계하는 것은 전자, 즉 사람이 의도적으로 작성하는 팀 규약입니다.

동작 원리: 컨텍스트 계층과 추가 로딩

CLAUDE.md 컨텍스트 계층과 추가 로딩



각 계층은 순서대로 추가(additive)되며, 하위 경로 진입 시 해당 파일이 덧붙여 로딩됩니다
 사용자 전역(개인 취향) · 프로젝트 루트(팀 규칙) · 하위(모듈별 규칙) 로 관심사 분리

CLAUDE.md가 "어디에 두느냐"에 따라 적용 범위가 달라집니다. 에이전트가 세션을 시작할 때, 현재 작업 디렉터리에서 시작해 **위쪽으로 디렉터리 트리를 거슬러 올라가며** 만나는 CLAUDE.md 파일들을 차례로 읽어 합칩니다. 아래에서 위로 좁은 규칙부터 넓은 규칙까지 쌓이는 구조입니다.

위치	범위	무엇을 담나	공유 여부
조직(enterprise) 수준	회사 전체	보안·법무 같은 전사 정책	조직이 배포
~/ .claude/CLAUDE.md	내 모든 프로젝트 (사용자)	나의 개인 작업 취향	나만(공유 안 됨)
./CLAUDE.md (프로젝트 루트)	이 저장소	팀의 빌드·테스트·컨벤션 약속	git으로 팀 공유
하위 디렉터리의 CLAUDE.md	그 폴더 안 작업	그 모듈만의 세부 규칙	git으로 팀 공유

예를 들어 `team-tasks-api/routers/` 에서 작업하면, 에이전트는 `routers/CLAUDE.md` (있다면) → `team-tasks-api/CLAUDE.md` (프로젝트) → `~/ .claude/CLAUDE.md` (사용자)를 거슬러 올라가며 모두 읽어 합칩니다. 좁은 규칙(라우터 폴더)이 넓은 규칙(프로젝트·개인) 위에 얹혀, 그 위치에 맞는 맥락이 완성됩니다.

이 구조의 의미는 분명합니다. 모두가 지켜야 할 팀 약속은 프로젝트 루트 에, 나만의 취향은 사용자 수준 에, 특정 모듈에만 해당하는 세부는 하위 폴더 에 둡니다. 무엇을 어디에 두느냐가 곧 "누가 그 규칙의 영향을 받는가"를 결정합니다.

주의: 공유되는 곳과 안 되는 곳을 헷갈리지 말 것

`./CLAUDE.md` (프로젝트 루트)는 저장소에 포함되어 git으로 팀 전체와 공유됩니다. 반면 `~/.claude/CLAUDE.md` (사용자 수준)는 내 컴퓨터에만 있어 팀원에게 전달되지 않습니다. 그래서 "모두가 따라야 할 팀 규칙"을 사용자 수준에 적으면, 정작 동료의 에이전트는 그 규칙을 모른 채 일합니다. 팀 규칙은 반드시 프로젝트 `CLAUDE.md`에 두어야 합니다. 이 구분은 이 장 뒷부분에서 다시 강조합니다.

worked example: /init로 빌드.테스트.컨벤션 담은 CLAUDE.md 생성

CLAUDE.md를 처음부터 빈손으로 쓸 필요는 없습니다. Claude Code는 `/init` 커맨드로 저장소를 분석해 초기 골격을 만들어 줍니다. 받은 팀 저장소에서 처음 이 명령을 써 보겠습니다.

1) /init로 초기 골격 생성

세션을 열고 다음을 입력합니다.

```
> /init
```

에이전트는 저장소를 훑어(4단원의 에이전트 검색) 구조.의존성.실행 방식을 파악하고, 다음과 같은 초기 `CLAUDE.md`를 프로젝트 루트에 만들어 줍니다.

team-tasks-api

할 일(task)을 생성·조회·완료 처리하는 FastAPI 기반 백엔드 서비스입니다.

빌드·실행

- 의존성 설치: ``pip install -r requirements.txt``
- 개발 서버 실행: ``python -m uvicorn main:app --reload``

테스트

- 전체 테스트: ``python -m pytest -q``
- 동작을 바꾸거나 추가하면 그 동작을 검사하는 테스트를 함께 작성합니다.

구조

- ``main.py``: FastAPI 앱 생성과 라우터 등록 진입점
- ``routers/tasks.py``: 할 일 관련 엔드포인트
- ``models.py``: 요청·응답 데이터 모델
- ``store.py``: 메모리 기반 할 일 저장소

`/init`는 저장소를 자동 분석해 만든 초안입니다. 사실에 기반하지만 완벽하지는 않으므로, 이제 사람이 팀의 실제 약속을 더해 다듬어야 합니다.

2) 팀 컨벤션·금지사항을 더한다

초안에 팀의 코딩 컨벤션과 금지사항을 보강합니다. 다음 절(무엇을 담는가)을 참고해 다음과 같이 보강했다고 합시다.

코딩 컨벤션

- 모든 함수에 타입 힌트를 붙입니다.
- 들여쓰기는 공백 4칸, 한 줄은 79자를 넘기지 않습니다.
- 엔드포인트는 ``routers/``의 기존 스타일(`APIRouter.prefix`)을 따릅니다.

git 규칙

- 작업은 항상 새 브랜치에서 합니다. main에 직접 커밋하지 않습니다.
- 커밋 메시지는 Conventional Commits(``feat:``, ``fix:``, ``docs:``)를 따릅니다.

하지 말 것

- 테스트 없이 엔드포인트를 추가하지 않습니다.
- ``secrets/``·``.env`` 파일을 읽거나 수정하지 않습니다.

3) 효과 확인

이제 새 세션을 열어 같은 작업을 지시해 봅니다.

나: 할 일을 삭제하는 DELETE `/tasks/{task_id}` 엔드포인트를 추가해 줘.

이번에는 "들여쓰기 4칸", "테스트도 같이", "타입 힌트", "새 브랜치에서" 같은 말을 한마디도 하지 않았습니다. 그런데도 에이전트는 CLAUDE.md를 이미 읽었기 때문에, 4칸 들여쓰기와 타입 힌트를 지킨 코드를 만들고, pytest 테스트를 함께 작성하며, 작업 브랜치에서 진행합니다. **한 번 적어 둔 규약이 매 세션 자동으로 적용되는** 효과를 직접 확인하는 순간입니다.

팁: /init는 출발점이지 종착점이 아니다

/init가 만든 초안을 그대로 두지 마세요. 자동 분석은 "무엇이 있는지"는 잘 적지만, "팀이 어떻게 일하기로 약속했는지"(금지사항·리뷰 규칙·도메인 용어)는 사람만 압니다. /init로 뼈대를 얻은 뒤, 팀의 실제 약속을 더해 완성하는 흐름이 정석입니다.

무엇을 담는가: 명령·컨벤션·아키텍처·금지사항

좋은 CLAUDE.md에 무엇을 담아야 할지 막막할 수 있습니다. "에이전트가 매 세션 알고 있으면 시간을 아끼고 실수를 줄일 정보"를 담는다고 생각하면 쉽습니다. 다음 네 묶음이 핵심입니다.

묶음	답을 내용	예시
명령(commands)	빌드·실행·테스트·린트 명령	<code>python -m pytest -q</code> , <code>python -m uvicorn main:app --reload</code>
컨벤션(conventions)	코드 스타일·네이밍·구조 규칙	타입 힌트 필수, 4칸 들여쓰기, 엔드포인트는 라우터에
아키텍처(architecture)	핵심 모듈의 역할과 경계	저장소 로직은 <code>store.py</code> 에, 라우터는 검증만
금지사항(don'ts)	절대 하면 안 되는 것	테스트 없는 추가 금지, <code>secrets/</code> 접근 금지

특히 **명령** 과 **금지사항** 이 가장 효과가 큼니다. 명령은 에이전트가 매번 "테스트를 어떻게 돌리지?"를 추측하지 않게 해 주고, 금지사항은 위험한 행동을 사전에 차단합니다. 반대로 너무 일반적인 조언("좋은 코드를 쓰세요")은 적지 마세요. 에이전트가 실제 행동을 바꿀 수 있는 **구체적인** 규칙만 가치가 있습니다.

팁: "구체적이고 검증 가능한" 규칙을 적어라

"코드를 깔끔하게"는 행동을 바꾸지 못합니다. "한 줄은 79자를 넘기지 않는다"는 검사할 수 있는 구체 규칙이라 행동을 바꿉니다. CLAUDE.md의 각 줄을 적을 때 "이 규칙을 지켰는지 누군가 확인할 수 있는가?"를 자문하면, 모호한 조언과 효과적인 규칙을 가려낼 수 있습니다.

/memory 편집과 @참조로 모듈화

/memory로 메모리 열고 편집하기

CLAUDE.md를 손볼 때마다 에디터로 파일을 직접 여는 대신, 세션 안에서 `/memory` 커맨드를 쓰면 어떤 메모리 파일이 있는지 둘러보고 편집할 수 있습니다.

```
> /memory
```

이 명령은 현재 적용 중인 메모리 파일들(프로젝트 CLAUDE.md, 사용자 CLAUDE.md, 자동 메모리 폴더 등)을 보여 주고, 그중 하나를 골라 편집하도록 안내합니다. `/init`로 골격을 만들고 `/memory`로 다듬는 흐름이 자연스러운 한 쌍입니다.

또한 작업 중 "이건 앞으로도 늘 지켜야 할 규칙이다" 싶은 것을 발견하면, 세션에서 바로 메모리에 추가하도록 부탁할 수 있습니다.

나: 방금 정한 "엔드포인트 추가 시 테스트도 같이"를 프로젝트 CLAUDE.md의 규칙으로 추가해 줘.

@참조로 긴 규칙을 모듈화하기

CLAUDE.md가 길어지면 한 파일에 모든 것을 욕여넣기보다, 다른 파일을 @로 끌어와 모듈화하는 편이 깔끔합니다. CLAUDE.md 안 어디에서나 @경로를 쓰면, 세션 시작 시 그 파일의 내용도 함께 읽힙니다.

```
# team-tasks-api
```

핵심 약속은 아래 파일들을 참조합니다.

자세한 코딩 컨벤션: @docs/conventions.md

API 설계 가이드: @docs/api-guide.md

```
## 빠른 참조
```

- 테스트: ``python -m pytest -q``
- 작업은 새 브랜치에서, 커밋은 Conventional Commits.

이렇게 하면 CLAUDE.md 본문은 빠른 참조만 간결히 유지하고, 길고 상세한 컨벤션은 별도 문서로 분리해 관리할 수 있습니다. 컨벤션 문서를 고치면 CLAUDE.md를 건드리지 않아도 자동으로 반영됩니다.

주의: @참조는 정리를 돕지만 분량을 줄이지는 않는다

@로 분리한 파일도 세션 시작 시 함께 로딩됩니다. 즉, 파일을 나눈다고 에이전트가 읽는 총량이 줄어드는 것은 아닙니다. @참조 의 목적은 *조직화* (관리하기 좋게 나누기)이지 *경량화* 가 아닙니다. 정말 분량을 줄이려면, 참조하는 문서 자체를 간결하게 유지해야 합니다.

worked example: 팀 규칙(프로젝트)과 개인 취향 (~/.claude) 분리

같은 규칙이라도 "팀 전체의 약속"인지 "나만의 취향"인지에 따라 두는 곳이 달라야 합니다. 구체 상황으로 풀어 봅니다.

상황: 우리 팀은 "테스트는 pytest로, 커밋은 Conventional Commits로"를 약속했습니다. 한편 여러분 개인은 "에이전트가 답할 때 한국어로 설명하고, 코드 주석도 한국어로 달면 좋겠다"는 취향이 있습니다. 이 둘을 어디에 둘까요?

1) 팀 약속 → 프로젝트 CLAUDE.md

팀 전체가 따라야 하므로 git으로 공유되는 프로젝트 루트에 둡니다.

```
# team-tasks-api/CLAUDE.md (팀 공유)

## 테스트
- 모든 변경은 `python -m pytest -q`로 검증하고 통과를 확인합니다.

## git 규칙
- 작업은 새 브랜치에서, 커밋 메시지는 Conventional Commits를 따릅니다.
```

이 파일은 저장소에 커밋되므로, 동료가 저장소를 받아 세션을 열면 그의 에이전트도 같은 규칙을 읽습니다. 팀의 일관성이 자동으로 유지됩니다.

2) 개인 취향 → 사용자 수준 ~/.claude/CLAUDE.md

나에게만 해당하는 취향이므로 내 컴퓨터의 사용자 수준 파일에 둡니다.

```
# ~/.claude/CLAUDE.md (나의 모든 프로젝트에 적용, 공유 안 됨)
```

응답 취향

- 설명은 한국어로 합니다.
- 코드 주석은 한국어로 작성합니다.

이 파일은 git에 올라가지 않으므로 동료에게 영향을 주지 않습니다. 대신 내가 어떤 프로젝트에서 일하든 늘 적용됩니다. 두 파일은 충돌하지 않고 **함께** 읽혀, "팀 약속 + 나의 취향"이 합쳐진 맥락을 만듭니다.

왜 이 분리가 중요한가

만약 개인 취향("한국어로 설명")을 프로젝트 CLAUDE.md에 적으면, 영어권 동료의 에이전트에게도 강요됩니다. 반대로 팀 약속("pytest로 검증")을 사용자 수준에만 적으면, 정작 동료는 그 규칙을 모른 채 일합니다. **"누가 이 규칙의 영향을 받아야 하는가"** 를 기준으로 두는 곳을 정하는 것, 이것이 계층을 제대로 쓰는 핵심입니다.

한 걸음 더: 하위 디렉터리 CLAUDE.md로 모듈별 세부 규칙 두기

계층은 "프로젝트 대 개인"에서 끝나지 않습니다. 같은 저장소 안에서도 **특정 폴더에만** 해당하는 세부 규칙은 그 폴더에 CLAUDE.md를 따로 둘 수 있습니다. 예를 들어 라우터 코드에만 적용할 약속이 있다면 `routes/CLAUDE.md` 에 둡니다.

```
# routes/CLAUDE.md (라우터 폴더에만 적용)
```

라우터 작성 규칙

- 모든 엔드포인트는 APIRouter와 prefix를 써서 정의합니다.
- 비즈니스 로직은 라우터에 넣지 않고 store.py에 위임합니다.
- 입력 검증 실패는 HTTPException(status_code=422)로 응답합니다.

이제 에이전트가 `routes/tasks.py` 를 수정할 때, 디렉터리 트리를 거슬러 올라가며 `routes/CLAUDE.md` (라우터 세부)와 `team-tasks-api/CLAUDE.md` (프로젝트 전체)를 모두 읽어 합칩니다. 그래서 그 폴더 안에서는 라우터 전용 규칙이 추가로 적용되고, 다른 폴더에서는 그 규칙이 끼어들지 않습니다.

이 방식의 장점은 **규칙이 적용되는 범위를 폴더 위치로 표현** 한다는 점입니다. 프로젝트 루트 CLAUDE.md에 "라우터는 이렇게, 모델은 저렇게, 저장소는 그렇게"를 다 적으면 길고 잡음이 많아집니다. 대신 각 모듈의 세부 규칙을 그 폴더의 CLAUDE.md로 분산하면, 루트는 전체 약속만 간결히 유지하고 세부는 해당 위치에서만 적용됩니다. 다만 폴더마다 CLAUDE.md를 남발하면 관리가 어려워지므로, 모듈별로 **분명히 다른** 규칙이 있을 때만 쓰는 것이 좋습니다.

자주 하는 실수: 팀 규칙을 개인 파일에 두기

신입이 흔히 저지르는 실수는, 자기 컴퓨터의 `~/.claude/CLAUDE.md`에 팀 규칙까지 적어 두고 "잘 되네"라고 안심하는 것입니다. 내 화면에서는 잘 작동하지만, 그 규칙은 동료에게 공유되지 않습니다. 결과적으로 팀 안에서 "되는 사람과 안 되는 사람"이 갈리는 혼란이 생깁니다. 팀이 함께 지킬 규칙은 반드시 git으로 공유되는 프로젝트 `CLAUDE.md`에 두세요.

좋은 CLAUDE.md의 원칙: 간결·구체·최신 유지

CLAUDE.md를 처음 쓰면 "많이 적을수록 좋겠지"라는 유혹에 빠지기 쉽습니다. 그러나 좋은 CLAUDE.md를 가르는 것은 분량이 아니라 세 가지 원칙입니다.

- 1. 간결(concise).** 모든 것을 담으면 정작 중요한 규칙이 잡음에 묻힙니다. 에이전트가 매 세션 읽는 문서이므로, 핵심 규칙만 추려 짧게 유지해야 신호가 또렷합니다. "있으면 좋은" 정보보다 "없으면 실수하는" 정보를 우선하세요.
- 2. 구체(specific).** "좋은 코드를 쓰세요" 같은 일반론은 행동을 바꾸지 못합니다. "한 줄 79자 이내", "엔드포인트는 `app/routers/`에" 처럼 검사 가능한 구체 규칙만 효과가 있습니다.
- 3. 최신 유지(up to date).** CLAUDE.md는 한 번 쓰고 방치하면 낡습니다. 테스트 명령이 바뀌거나 폴더 구조가 달라졌는데 문서가 옛날 그대로면, 에이전트는 틀린 안내를 따라 잘못 일합니다. 관례가 바뀌면 CLAUDE.md도 같이 갱신해야 합니다. 낡은 규칙은 없는 것만 못합니다.

팁: 교정이 반복되면 그것을 규칙으로 승격하라

같은 지적을 두 번 이상 하고 있다면, 그것은 CLAUDE.md에 올라가야 할 신호입니다. "또 들어 쓰기를 지적하네" 싶은 순간, 그 교정을 `/memory`로 프로젝트 CLAUDE.md의 규칙으로 승격하세요. 반복되는 교정을 영속 규칙으로 바꾸는 것이 CLAUDE.md를 키워 가는 가장 자연스러운 방법입니다.

즉시 연습 (2종 1세트): team-tasks-api에 실제 CLAUDE.md를 만들고 효과 확인하기

이제 종이에 골격을 채우는 대신, 1장에서 준비한 실제 `team-tasks-api` 저장소를 Claude Code로 열고 `/init`로 CLAUDE.md를 직접 만든 뒤, 보강하고, 새 세션에서 효과를 실제로 확인합니

다. 핵심은 "CLAUDE.md가 있을 때와 없을 때 에이전트의 행동이 달라지는 것"을 화면에서 직접 겪는 것입니다. 미리 정한 답을 적는 것이 아니라, **화면에 실제로 나타난 파일 내용과 에이전트의 반응을 그대로 옮겨 적습니다.**

1) 모범 예시 (실제 실행 예)

상황: 작은 FastAPI 저장소에서 `/init` 를 실행하면, 에이전트가 저장소를 훑어 빌드·테스트·구조를 담은 CLAUDE.md 초안을 만들어 줍니다. 거기에 금지사항·컨벤션을 보강한 결과는 다음과 같습니다.

```
# orders-api

주문(order)을 생성·조회하는 FastAPI 백엔드입니다.

## 빌드·테스트
- 설치: `pip install -r requirements.txt`
- 실행: `python -m uvicorn main:app --reload`
- 테스트: `python -m pytest -q`

## 컨벤션
- 모든 함수에 타입 힌트를 붙입니다.
- 들여쓰기 4칸, 한 줄 79자 이내.

## git
- 새 브랜치에서 작업, 커밋은 Conventional Commits.

## 하지 말 것
- 테스트 없이 엔드포인트를 추가하지 않습니다.
- `.env`를 읽거나 수정하지 않습니다.
```

이 골격은 간결하면서도(짧음), 구체적이고(검사 가능한 규칙), 명령·컨벤션·금지사항을 모두 담았습니다. `/init` 가 만든 초안에 "하지 말 것"을 사람이 더해 완성한 것입니다.

2) 직접 해보기 (team-tasks-api에서 실제로 만들고 검증)

연습 1 — `/init` 로 CLAUDE.md를 실제로 만들고 보강하라.

team-tasks-api 폴더에서 `claude` 를 실행하고, 슬래시 명령 `/init` 를 입력하세요. 에이전트가 저장소를 훑어 CLAUDE.md 초안을 만듭니다. 만들어진 파일을 열어, **실제로 생성된 내용**을 그대로 옮겨 적으세요(추측이 아니라 파일에 적힌 그대로).

```
[/init 결과] CLAUDE.md에 실제로 들어간 빌드·테스트 명령:
[/init 결과] 자동으로 잡힌 프로젝트 구조 설명:
```

이어서 다음을 에이전트에게 지시해 CLAUDE.md를 보강하세요.

> CLAUDE.md에 "하지 말 것" 섹션을 추가해 줘. 테스트 없이 엔드포인트를 추가하지 말 것, .env를 읽거나 수정하지 말 것 두 가지를 넣어 줘.

[보강 결과] 추가된 "하지 말 것" 줄(파일에 적힌 그대로):

연습 2 — 새 세션에서 효과를 실제로 확인하라.

CLAUDE.md가 정말 자동으로 읽히는지 확인합니다. `claude` 를 한 번 종료했다가 같은 폴더에서 다시 열고(새 세션), 다음을 그대로 물어보세요.

> 이 프로젝트에서 테스트는 어떤 명령으로 돌려? 새 엔드포인트를 추가할 때 지켜야 할 규칙은?

에이전트가 **당신이 알려 주지 않았는데도** CLAUDE.md의 내용(예: `python -m pytest -q`, 테스트 없이 추가 금지)을 근거로 답하는지 화면에서 확인하고, 실제 답을 옮겨 적으세요.

[새 세션의 실제 답] 에이전트가 낸 테스트 명령과 규칙(CLAUDE.md를 참고했는가?):

연습 3 — 팀 규칙과 개인 취향을 실제로 나눠 배치하라.

"에이전트는 한국어로 설명한다"는 개인 취향이므로 프로젝트 CLAUDE.md가 아니라 사용자 `~/.claude/CLAUDE.md` 에 둡니다. 이 규칙을 사용자 메모리에 실제로 추가하고(직접 편집하거나 `/memory`), 새 세션에서 에이전트가 한국어로 설명하는지 확인하세요.

한국어 규칙을 둔 파일 경로(사용자 메모리가 맞는가?):
실제로 한국어로 설명했는가:

자가체크 — 머리로가 아니라 손으로 했는지 확인하세요.

- [] `team-tasks-api` 폴더에서 `/init` 를 실제로 실행해 CLAUDE.md 파일이 생성된 것을 확인했다.
- [] `/init` 결과]에 적은 것이 추측이 아니라 생성된 파일에 실제로 적힌 내용이다.
- [] "하지 말 것" 섹션을 보강하고, 파일에 그 줄이 실제로 들어간 것을 확인했다.
- [] 새 세션에서 에이전트가 CLAUDE.md를 자동으로 읽고 그 내용을 근거로 답하는 것을 직접 봤다.
- [] "테스트는 pytest로"는 팀 공유용 **프로젝트 CLAUDE.md**, "한국어로 설명"은 개인용 **사용자 ~/.claude/CLAUDE.md** 에 실제로 나눠 두었다.

- [] CLAUDE.md가 있을 때와 없을 때 에이전트의 행동이 어떻게 달라지는지 직접 겪어 설명할 수 있다.

단원 미니 프로젝트: team-tasks-api에 CLAUDE.md 심기

이 단원의 미니 프로젝트는 종이 위 빈칸이 아니라, 1장에서 준비한 **실제 team-tasks-api 저장소**에 CLAUDE.md를 직접 만들어 효과를 눈으로 확인하는 과제입니다. 1장 11절에서 이 저장소를 작업 폴더로 복사하고 `git init · pip install · python -m pytest -q (7 passed)`까지 끝냈다고 가정합니다. 그 폴더에서 그대로 이어 갑니다.

마일스톤(증분):

1. **M1 /init 로 초안 생성.** team-tasks-api 루트에서 `claude` 를 열고 `/init` 를 실행해, 저장소를 분석한 CLAUDE.md 초안을 만듭니다. 초안에 실제 파일 (`main.py · models.py · store.py · routers/tasks.py`)이 구조로 적혔는지 확인합니다. (적용 개념: `/init`, 자동 로딩)
2. **M2 빌드·테스트 명령 명문화.** 초안의 명령이 이 저장소의 실제 명령과 일치하도록 다듬습니다. 실행은 `python -m uvicorn main:app --reload`, 테스트는 `python -m pytest -q` 입니다. 잘못된 경로(`app.main:app` 등)가 있으면 바로잡습니다. (적용 개념: 명령, 최신 유지)
3. **M3 컨벤션·금지사항 보강.** 타입 힌트 필수·들여쓰기 4칸·"엔드포인트는 `routers/` 의 기존 스타일", "테스트 없이 엔드포인트 추가 금지", "`store.py` 에 저장소 로직 위임"을 적습니다. (적용 개념: 컨벤션·금지사항)
4. **M4 효과 검증.** 새 세션을 열어 "할 일을 삭제하는 `DELETE /tasks/{task_id}` 엔드포인트를 `routers/tasks.py` 에 추가하고 테스트도 함께 써 줘"라고만 지시합니다. CLAUDE.md를 한번 적어 둔 덕분에 타입 힌트·`store.py` 위임·테스트 작성이 자동으로 따라오는지 확인합니다.

실제 검증 단계. M4의 변경이 끝나면 반드시 저장소에서 테스트를 돌려 통과를 확인합니다("검증 전까지는 끝난 게 아니다", 7장).

```
> python -m pytest -q
.....
8 passed
```

[100%]

기존 7개에 삭제 엔드포인트 테스트 1개가 더해져 통과하면, CLAUDE.md가 실제로 에이전트의 작업 방식을 바꿨다는 증거입니다.

미니 프로젝트 자가체크

- [] `team-tasks-api` 루트에서 `/init` 로 `CLAUDE.md` 초안을 만들었는가
- [] 빌드·테스트 명령이 실제 명령(`python -m uvicorn main:app --reload` · `python -m pytest -q`)과 일치하는가
- [] 컨벤션·금지사항에 "테스트 없는 추가 금지" · "store.py 에 로직 위임"을 넣었는가
- [] 새 세션에서 추가 지시 없이도 에이전트가 규약대로(테스트 동반·타입 힌트) 작업했는가
- [] 변경 후 `python -m pytest -q` 로 통과를 직접 확인했는가
- [] `CLAUDE.md`를 git에 커밋해 팀과 공유되도록 했는가

흔한 오류와 오개념

오개념 1: "CLAUDE.md는 한 번 잘 써 두면 끝이다"

`CLAUDE.md`를 처음 정성껏 쓰고 나면 다 됐다고 여기기 쉽습니다. 그러나 프로젝트는 변합니다. 테스트 명령이 바뀌고, 폴더가 옮겨지고, 새 규칙이 생깁니다. 문서가 옛날 그대로면 에이전트는 틀린 안내(예: 더는 없는 폴더 경로)를 따라 잘못 일합니다. 낡은 규칙은 없는 것보다 해롭습니다.

→ **교정:** 관례가 바뀔 때 `CLAUDE.md`도 함께 고치는 것을 작업의 일부로 삼으세요. 같은 교정을 두 번 했다면, 그때마다 `/memory` 로 규칙을 갱신·추가하는 습관을 들이면 문서가 자연스럽게 최신으로 유지됩니다.

오개념 2: "많이 적을수록 에이전트가 똑똑해진다"

규칙을 빠짐없이 적으려다 `CLAUDE.md`가 수백 줄이 되면, 정작 중요한 핵심 규칙이 잡음에 묻혀 효과가 떨어집니다. 일반론("좋은 코드를 쓰자")과 사소한 취향까지 다 적으면 신호 대 잡음 비율이 나빠집니다.

→ **교정:** "없으면 실수하는" 핵심 규칙만 간결히 담으세요. 길고 상세한 컨벤션은 `@docs/conventions.md` 로 분리하고, `CLAUDE.md` 본문은 빠른 참조만 또렷하게 유지합니다. 각 줄에 대해 "이게 에이전트의 행동을 실제로 바꾸는가?"를 자문하세요.

오개념 3: "내 컴퓨터에서 되니까 팀도 될 것이다"

팀이 지켜야 할 규칙을 개인 파일(`~/claude/CLAUDE.md`)에만 적어 두는 실수입니다. 내 화면에서는 잘 작동하지만, 그 파일은 git으로 공유되지 않아 동료의 에이전트는 그 규칙을 전혀 모릅니

다. 팀 안에서 "되는 사람과 안 되는 사람"이 갈립니다.

→ **교정**: "누가 이 규칙의 영향을 받아야 하는가"를 기준으로 위치를 정하세요. 모두가 지킬 팀 약속은 git으로 공유되는 **프로젝트 CLAUDE.md** 에, 나만의 취향만 **사용자 ~/.claude/CLAUDE.md** 에 둡니다.

자주 묻는 질문

Q. CLAUDE.md를 만들었는데 에이전트가 규칙을 안 지키는 것 같습니다. 왜인가요?

가장 흔한 원인은 두 가지입니다. 첫째, 파일이 에이전트가 읽는 위치에 없는 경우입니다. 작업 중인 디렉터리에서 위로 거슬러 올라가는 경로 안(프로젝트 루트나 상위 폴더, `~/.claude`)에 있어야 읽힙니다. 둘째, 규칙이 모호한 경우입니다. "깔끔하게"처럼 검사 불가능한 조언은 행동을 바꾸지 못합니다. "한 줄 79자 이내"처럼 구체적으로 적었는지 확인하세요. 그래도 안 지켜진다면 `/memory` 로 현재 어떤 메모리가 적용 중인지 직접 확인할 수 있습니다.

Q. 규칙이 많아지는데 한 파일에 다 적어야 하나요?

아닙니다. 본문이 길어지면 `@docs/conventions.md` 처럼 `@참조` 로 분리해 모듈화하세요. 다만 분리한다고 에이전트가 읽는 총량이 줄지는 않으므로(세션 시작 시 함께 로딩), 분리의 목적은 *관리 편의* 라는 점을 기억하세요. 특정 폴더에만 해당하는 규칙은 그 폴더의 하위 CLAUDE.md로 분산하는 것도 방법입니다.

Q. CLAUDE.md와 자동 메모리는 무엇이 다른가요?

CLAUDE.md는 사람이 의도적으로 적는 영속 지침이고, 자동 메모리는 에이전트가 사용자의 교정·선호를 보고 스스로 적어 두는 노트입니다. 이 장에서 설계한 것은 전자입니다. 자동 메모리도 `/memory` 로 무엇이 저장됐는지 둘러볼 수 있습니다.

Q. CLAUDE.local.md는 무엇인가요?

CLAUDE.md와 같은 역할을 하되 공유하지 않으려는 개인용 메모리를 같은 저장소 안에 둘 때 쓰는 변형입니다. 보통 `.gitignore` 에 포함해 커밋되지 않게 합니다. "이 저장소에서 나만 쓰는 임시 메모"가 필요할 때 유용합니다.

이 장에서 익힌 것

이 장에서 우리는 세션마다 반복하던 팀 규칙 설명을, CLAUDE.md 한 파일로 자동화하는 법을 배웠습니다. 핵심을 정리합니다.

- CLAUDE.md는 **세션 시작마다 자동으로 읽히는** 저장소의 헌법으로, 빌드·테스트 명령·컨벤션·금지사항을 에이전트에 주입합니다.
- 위치에 따라 **계층**(조직·사용자·프로젝트·하위 디렉터리)이 나뉘며, 에이전트는 디렉터리 트리를 거슬러 올라가며 **추가로** 읽어 합칩니다.
- `/init` 로 골격을 만들고, `/memory` 로 다듬으며, `@참조` 로 긴 규칙을 모듈화합니다.
- **누가 영향을 받는가** 를 기준으로, 팀 약속은 프로젝트 CLAUDE.md에·개인 취향은 사용자 수준에 둡니다.
- 좋은 CLAUDE.md는 **간결·구체·최신 유지** 의 세 원칙을 따릅니다.

다음 단원에서는 이렇게 팀 표준대로 만들고 검증한 변경을, 작은 논리 단위로 커밋하고 신입의 통과의례인 첫 PR로 올리는 git 워크플로(9단원)로 이어집니다.

참고 출처

- [Claude Code 공식 문서 — How Claude remembers your project \(Memory\)](#).
- [Claude Code 공식 문서 — Explore the .claude directory](#).

첫 커밋과 첫 PR: git 워크플로

이 챕터에서 배우는 것

- 여러 변경을 작은 논리 단위로 나눠 스테이징하고, 관례(Conventional Commits) 형식 (feat/fix:)의 메시지로 커밋하게 할 수 있습니다.
- 작업용 브랜치를 만들고 `git status` · `git diff` 를 확인해 무엇이 어느 커밋에 들어갈지 커밋 단위를 직접 통제할 수 있습니다.
- 변경 요약·근거·테스트 결과 세 가지를 담아, 리뷰어가 빠르게 이해할 수 있는 PR 초안 1건을 작성할 수 있습니다.
- 커밋·PR 관련 명령을 allowlist(예: `Bash(git commit:*) · gh`)로 좁혀 안전하게 운용할 수 있습니다.

도입: 신입의 통과의례, 첫 PR

앞 단원(u6)에서 우리는 "검증 전까지는 끝난 게 아니다"를 배웠습니다. 변경을 실제로 실행하고 `pytest` 로 통과를 확인해야 비로소 작업이 끝났다고 말할 수 있다는 습관이었습니다. 그런데 한 가지 질문이 남습니다. 내 컴퓨터에서 테스트가 통과한 그 코드를, 어떻게 **팀의 코드로 합류시킬까요?** 혼자 만든 변경이 아무리 잘 동작해도, 팀의 메인 브랜치에 안전하게 들어가 동료들이 함께 쓰게 되기 전까지는 절반만 끝난 일입니다.

이 마지막 절반을 책임지는 절차가 **git 워크플로(git workflow)**, 그중에서도 신입의 통과의례로 불리는 **첫 PR(Pull Request, 풀 리퀘스트)**입니다. PR은 "내가 이런 변경을 만들었으니, 검토하고 메인 브랜치에 합쳐 주세요"라고 팀에 공식적으로 요청하는 단위입니다. 신입이 팀에서 처음으로 존재감을 드러내는 순간이자, 동료들이 여러분의 일하는 방식을 처음 보는 창문이기도 합니다. 이 장의 핵심 개념은 **git 워크플로: 커밋과 첫 PR**입니다.

문제는, git을 처음 다루는 신입에게 이 절차가 의외로 가파른 벽이라는 점입니다. "브랜치를 어떻게 만들지", "무엇을 한 커밋에 묶을지", "커밋 메시지를 어떻게 쓸지", "PR 본문에 뭘 적을지" 같은 결정이 한꺼번에 밀려옵니다. 여기서 Claude Code(클로드 코드)가 큰 도움이 됩니다.

Claude Code는 **버전 관리(version control)**를 **작업 흐름의 일부**로 다룹니다. 무엇이 어떻게 바뀌었는지 맥락을 알기에, 변경을 스테이징하고 관례에 맞는 커밋 메시지를 제안하며, 브랜치 생성·diff 검토·`gh` 를 이용한 PR 생성까지 도울 수 있습니다.

다만 한 가지를 분명히 해 둡니다. **무엇을·언제 커밋할지는 끝까지 사람이 통제합니다.** 에이전트는 손이 빠른 동료일 뿐이고, "이 변경들을 어떤 단위로 역사에 남길지"를 정하는 책임은 여러분에게 있습니다. 이 챕터는 그 통제력을 갖춘 채 에이전트와 함께 첫 커밋을 만들고 첫 PR을 올리는 전 과정을 다룹니다.

선수 개념 상기

이 챕터는 **실행·테스트로 검증하기(c07)**를 바로 앞 선수로 합니다. 좋은 PR의 핵심 재료 중 하나가 "이 변경이 동작함을 어떻게 증명했는가", 곧 **테스트 결과**입니다. ch_07에서 `pytest` 로 통과를 확인하는 습관을 들였다면, 그 통과 결과가 이번 챕터에서 PR 본문의 "어떻게 검증했는지" 항목으로 그대로 이어집니다. 또 멀게는 **권한과 승인(c05)**도 다시 등장합니다. 커밋·푸시·PR 생성은 파일을 바꾸고 외부(원격 저장소)에 영향을 주는 부수효과 작업이므로, 이번 챕터 끝에서 이 명령들을 `allowlist`로 어떻게 안전하게 허용할지 설계합니다.

git 상태·diff 확인과 변경 맥락 파악

커밋을 만들기 전에 가장 먼저 할 일은 **지금 무엇이 바뀌었는지 정확히 아는 것**입니다. 자기 변경을 모른 채 커밋하는 것은, 내용을 읽지 않고 서류에 서명하는 것과 같습니다. git에는 이 "현재 상태"를 보는 두 가지 핵심 명령이 있습니다.

- `git status`: 어떤 파일이 새로 생겼는지(untracked), 수정됐는지(modified), 스테이징됐는지(staged)를 **목록**으로 보여 줍니다. "무엇이 바뀌었나"의 개요입니다.
- `git diff`: 각 파일에서 **어떤 줄이 어떻게** 바뀌었는지 추가(+),삭제(-) 단위로 보여 줍니다. "어떻게 바뀌었나"의 세부입니다.

Claude Code와 함께 일할 때 좋은 점은, 이 두 명령을 직접 외워 입력하지 않아도 **변경의 의미를 사람 말로 요약**해 달라고 할 수 있다는 것입니다. 에이전트는 `git status` 와 `git diff` 를 읽기 전용 도구로 실행해 결과를 해석하고, "지금 `routers/tasks.py` 에 완료 처리 엔드포인트가 하나 추가됐고, 테스트 파일이 새로 생겼습니다" 같은 요약을 돌려줍니다.

다음은 받은 팀 저장소 `team-tasks-api` 에서 작은 변경을 만든 직후, 상태를 확인하는 대화입니다. 입력한 줄은 `>` 로 시작하는 자연어 지시이고, 그 아래는 에이전트가 실제로 실행한 명령과 요약입니다.

> 지금 워킹 트리에 어떤 변경이 있는지 git status와 git diff로 확인하고, 무엇이 바뀌었는지 한국어로 요약해 줘. 아직 커밋하지는 말고.

(Claude Code가 Bash 도구로 git status를 실행)

```
$ git status --short
M routers/tasks.py
?? tests/test_tasks_complete.py
```

(이어서 git diff를 실행해 내용을 읽고 요약)

요약하면 변경은 두 가지입니다.

1. routers/tasks.py: 기존 tasks 라우터에 PATCH /tasks/{task_id}/complete 엔드포인트가 추가되어, 특정 작업을 완료 상태로 바꿉니다.
2. tests/test_tasks_complete.py: 위 엔드포인트를 검증하는 새 테스트 파일입니다. 아직 어떤 변경도 스테이징되지 않았습니다.

- git status --short 의 M 은 수정됨(modified), ?? 은 git이 추적하지 않는 새 파일(untracked)을 뜻합니다. 한눈에 "수정 1개, 새 파일 1개"임을 읽을 수 있습니다.
- 에이전트가 git diff 까지 읽어 두 변경의 의미를 묶어 설명했습니다. 단순히 "두 파일이 바뀜"이 아니라 "엔드포인트 추가 + 그 테스트"라는 한 덩어리의 작업임을 파악한 점이 중요합니다. 이 파악이 다음 단계인 "커밋 단위 정하기"의 출발점입니다.
- "아직 커밋하지는 말고"라는 제약을 명시했습니다. 상태 파악 단계에서는 읽기만 하고 역사를 바꾸지 않습니다. 무엇을 커밋할지는 내가 본 다음에 정합니다.

git diff 는 변경을 스스로 다시 읽는 가장 좋은 습관입니다. 에이전트의 요약을 믿되, 핵심 변경은 diff를 직접 보거나 에이전트에게 "이 엔드포인트가 잘못된 상태 값을 받으면 어떻게 되나?" 같은 질문을 던져 한 번 더 확인하는 것이 좋습니다. 변경 맥락을 정확히 파악했다면, 이제 이 변경을 어떤 순서로 역사에 남길지 정할 차례입니다.

동작 원리: 브랜치 → 커밋 → 푸시 → PR 흐름

브랜치 → 커밋 → 푸시 → PR 흐름



변경 단위는 작게 · 커밋 메시지는 명확하게 · PR은 리뷰어가 이해하기 쉽게

첫 PR까지 가는 길은 정해진 네 단계의 흐름을 따릅니다. 각 단계가 "무엇을, 왜" 하는지 이해하면, 에이전트에게 단계별로 정확히 지시할 수 있습니다.

- 1. 작업 브랜치 생성(branch)** — 팀의 메인 브랜치(`main`)에서 직접 작업하지 않고, `feature/task-complete` 같은 **별도의 작업용 가지**를 만들어 그 위에서 변경합니다. 이렇게 하면 내 작업이 완성되어 검토를 통과하기 전까지 메인 브랜치는 안전하게 유지됩니다. 여러 사람이 동시에 일할 수 있는 것도 이 분리 덕분입니다.
- 2. 논리 단위 스테이징·커밋(stage-commit)** — 변경 중 **한 가지 의미를 이루는 묶음**만 골라 `git add` 로 스테이징하고, 그 묶음을 설명하는 메시지로 `git commit` 합니다. 커밋은 "되돌릴 수 있는 의미 있는 한 걸음"입니다. 한 커밋이 한 가지 일만 담아야 나중에 역사를 읽고 되돌리기가 쉽습니다.
- 3. 푸시(push)** — 내 컴퓨터(로컬)에만 있던 커밋들을 원격 저장소(GitHub 등)의 내 브랜치로 올립니다. 이 시점부터 동료들이 내 작업을 볼 수 있게 됩니다.
- 4. PR 초안 작성(pull request)** — "이 브랜치의 변경을 메인에 합쳐 주세요"라고 요청하는 PR을 만듭니다. 제목·설명·테스트 결과를 담아, 리뷰어가 **무엇을·왜·어떻게 검증했는지**를 빠르게 이해하게 합니다.

이 네 단계를 표로 정리하면 각 단계의 책임과 도구가 분명해집니다.

단계	핵심 명령	하는 일	무엇이 바뀌나
브랜치 생성	<code>git switch -c feature/...</code>	메인에서 작업 가지를 댐	현재 브랜치 위치
스테이징	<code>git add <경로></code>	커밋할 변경을 고름	스테이징 영역
커밋	<code>git commit -m "feat: ..."</code>	고른 변경을 역사에 기록	로컬 git 역사
푸시	<code>git push -u origin feature/...</code>	로컬 커밋을 원격에 올림	원격 브랜치
PR 초안	<code>gh pr create --draft ...</code>	합치기 요청을 만듦	원격에 PR 생성

왜 메인에서 바로 작업하지 않는가

신입이 가장 자주 하는 실수가 메인 브랜치에서 곧장 변경하는 것입니다. 메인은 "언제나 동작하는, 모두가 공유하는 기준선"입니다. 검토받지 않은 내 변경이 메인에 섞이면 다른 동료의 작업까지 영향을 받습니다. 작업 브랜치는 **내 실험실**입니다. 여기서 마음껏 만들고 부수고, 검토를 통과한 결과만 PR을 통해 메인으로 보냅니다. 이 분리가 협업의 안전장치입니다.

worked example: 작업 브랜치에서 feat: 메시지로 커밋

이제 앞에서 파악한 변경(완료 처리 엔드포인트 + 테스트)을 실제로 브랜치에 올려 커밋해 보겠습니다. 목표는 "작업 브랜치를 만들고, 변경을 하나의 논리 단위로 커밋하기"입니다.

먼저 작업 브랜치를 만들도록 지시합니다.

> 지금 main 브랜치에 있는지 확인하고, feature/task-complete 라는 작업 브랜치를 새로 만들어 그쪽으로 옮겨 줘.

(Claude Code가 현재 브랜치를 확인하고 새 브랜치를 생성)

```
$ git branch --show-current
main
$ git switch -c feature/task-complete
Switched to a new branch 'feature/task-complete'
```

작업 브랜치 feature/task-complete 로 옮겼습니다. 이제 이 브랜치에서 변경을 커밋해도 main 은 영향을 받지 않습니다.

- `git branch --show-current` 로 **지금 어디에 서 있는지** 먼저 확인했습니다. main에서 출발해 가지를 쳐야 깨끗한 브랜치가 됩니다.
- `git switch -c feature/task-complete` 의 `-c` 는 "create", 곧 새 브랜치를 만들면서 동시에 그쪽으로 이동(switch)한다는 뜻입니다. 브랜치 이름 `feature/task-complete` 는 "기능 추가(feature) 중 작업 완료 처리(task-complete)"라는 의미를 담아, 이름만 봐도 무슨 작업인지 알 수 있게 지었습니다.
- 옛 명령 `git checkout -b` 도 같은 일을 하지만, 최신 git에서는 브랜치 전환 전용인 `git switch` 가 더 명확해 권장됩니다.

다음으로 변경을 스테이징하고 커밋합니다. 이번 변경은 "완료 처리 엔드포인트와 그 테스트"라는 하나의 일관된 일이므로, 한 커밋으로 묶는 것이 자연스럽습니다.

> 변경된 `routes/tasks.py` 와 새 테스트 파일을 스테이징하고,
Conventional Commits 규칙에 맞는 커밋 메시지로 커밋해 줘.
메시지는 무엇을·왜 바꿨는지 한 줄 제목과 간단한 본문으로 적어 줘.

(Claude Code가 스테이징 후 커밋)

```
$ git add routes/tasks.py tests/test_tasks_complete.py
$ git commit -m "feat(tasks): 작업 완료 처리 엔드포인트 추가" -m "PATCH /tasks/{id}/complete 로
[feature/task-complete 3f9a1c2] feat(tasks): 작업 완료 처리 엔드포인트 추가
2 files changed, 41 insertions(+)
create mode 100644 tests/test_tasks_complete.py
```

- `git add` 에 **바꿀 파일 경로를 명시**했습니다. `git add .` 로 전부 쓸어 담는 대신 두 파일을 짚어 스테이징하면, 의도하지 않은 파일(예: 디버깅용 임시 파일)이 섞여 들어가는 사고를 막습니다. 무엇을 커밋할지 사람이 통제하는 지점입니다.
- `git commit -m "..."` -m `"..."` 처럼 `-m` 을 두 번 쓰면 첫 번째가 **제목**, 두 번째가 **본문**이 됩니다. 제목은 "무엇을 했나"(완료 처리 엔드포인트 추가), 본문은 "구체적으로 무엇이고 왜 인가"를 담았습니다.
- 제목 `feat(tasks): ...` 이 다음 절에서 다룰 Conventional Commits 형식입니다. `feat` 은 새 기능, `(tasks)` 는 영향 범위(scope), 콜론 뒤가 요약입니다.
- 출력의 `2 files changed, 41 insertions(+)` 는 이 커밋이 담은 변경 규모를, `[feature/task-complete 3f9a1c2]` 는 커밋이 만들어진 브랜치와 커밋 해시(앞 7자리)를 보여 줍니다. 커밋이 main이 아니라 작업 브랜치에 안전하게 쌓였음을 확인할 수 있습니다.

이렇게 한 커밋이 완성되었습니다. 핵심은 에이전트가 명령을 대신 실행했지만, **무엇을 한 커밋에 묶을지(완료 엔드포인트 + 테스트)와 메시지의 의미를 결정한 것은 사람**이라는 점입니다. 다음 절에서 이 "한 커밋에 무엇을 담을지"의 기준을 더 깊이 봅니다.

작은 논리 단위 커밋과 Conventional Commits

방금 우리는 자연스럽게 "완료 엔드포인트와 그 테스트"를 한 커밋으로 묶었습니다. 이 판단의 바탕에 있는 두 가지 원칙을 명시적으로 배웁니다.

정의: 작은 논리 단위 커밋

작은 논리 단위 커밋(atomic commit, 원자적 커밋)은 **하나의 완결된 의미**만 담아, 그 자체로 설명되고 단독으로 되돌릴 수 있는 가장 작은 변경 묶음입니다. 한 커밋은 "한 가지 일"만 합니다.

"엔드포인트를 추가하면서, 동시에 무관한 오타도 고치고, 설정값도 바꾼" 커밋은 나쁜 커밋입니다. 나중에 "엔드포인트 추가만 되돌리고 싶을 때" 오타 수정과 설정 변경까지 함께 팔려 와 되돌릴 수 없기 때문입니다. 반대로 좋은 커밋은 다음과 같이 나눕니다.

- 커밋 1: `feat(tasks)`: 작업 완료 처리 엔드포인트 추가
- 커밋 2: `docs(readme)`: 완료 엔드포인트 사용법 문서화
- 커밋 3: `fix(tasks)`: 잘못된 상태 값 입력 시 422 응답하도록 수정

각 커밋이 **하나의 의미**만 담으므로, 리뷰어는 한 번에 한 가지만 검토하면 되고, 문제가 생기면 그 커밋만 정확히 되돌릴 수 있습니다. "작게, 자주, 의미 단위로"가 좋은 커밋의 리듬입니다.

정의: Conventional Commits

Conventional Commits(컨벤셔널 커밋)는 커밋 메시지 제목을 `type(scope): 설명` 형식으로 통일하는 관례로, 메시지만 보고도 변경의 **종류**를 기계와 사람 모두 빠르게 파악하게 하는 약속입니다.

제목의 맨 앞 `type` (유형)이 변경의 성격을 한눈에 알려 줍니다. 신입이 가장 자주 쓰는 유형은 다음과 같습니다.

type	언제 쓰나	예시 제목
feat	새 기능 추가	feat(tasks): 상태 필터 쿼리 파라미터 추가
fix	버그 수정	fix(tasks): 빈 제목 작업 생성 시 500 오류 수정
docs	문서만 변경	docs(api): 완료 엔드포인트 예시 추가
test	테스트 추가·수정	test(tasks): 완료 처리 경계 케이스 테스트 추가
refactor	동작 변화 없는 구조 개선	refactor(tasks): 저장소 접근 함수 분리
chore	빌드·설정 등 잡무	chore: pytest 의존성 추가

(scope) 는 영향 범위로, 어느 모듈·기능이 바뀌었는지(tasks , api , auth 등)를 적습니다. 생략할 수도 있지만, 적으면 검색과 분류가 쉬워집니다. 콜론 뒤 설명은 **명령형 현재 시제로 간결하게** 적는 것이 관례입니다("추가한다"보다 "추가"처럼 짧게).

Claude Code는 변경 내용을 이미 읽었기에, "Conventional Commits로 커밋해 줘"라고만 해도 feat · fix 같은 유형을 변경 성격에 맞게 골라 제안합니다. 다만 제안된 유형이 맞는지(새 기능인데 fix 로 잘못 붙지 않았는지) 확인하는 것은 사람의 몫입니다.

한 작업이 여러 의미를 담았다면

실수로 엔드포인트 추가와 무관한 리팩터링을 한 번에 만들었다면, 에이전트에게 "엔드포인트 변경만 먼저 스테이징해서 feat 으로 커밋하고, 리팩터링은 따로 refactor 로 커밋해 줘"라고 **나뉘어 커밋**하도록 지시할 수 있습니다. git add 로 파일·변경을 선택하는 단위가 곧 커밋의 단위가 되므로, 무엇을 함께 묶을지를 사람이 정하면 됩니다.

PR 초안: 변경 요약·근거·테스트 결과

커밋이 쌓였다면, 이제 그 브랜치를 원격에 푸시하고 PR을 만들 차례입니다. 두 가지 도구가 등장합니다.

첫째, **gh (GitHub CLI, 깃허브 명령줄 도구)**입니다. GitHub를 터미널에서 다루는 공식 도구로, gh pr create 명령으로 PR을 만듭니다. Claude Code는 git을 직접 다루듯 gh 도 Bash 도구로 실행해 PR 생성을 도울 수 있습니다.

1장에서 이미 해 둔 것: 1장 11절에서 `gh`를 설치하고 `gh auth login`으로 로그인한 뒤, `gh repo create ... --remote=origin --push`로 `team-tasks-api`를 GitHub에 올려 원격 `origin`을 만들어 두었습니다. 그래서 이 장에서는 바로 푸시·PR로 넘어갈 수 있습니다. 막히면 두 가지를 먼저 확인하세요 — `git remote -v` (원격 `origin`이 보이는가)와 `gh auth status` (GitHub에 로그인돼 있는가). 둘 중 하나라도 없으면 1장 11절의 셋업을 다시 합니다.

이 장에서 쓰는 `git`·`GitHub` 명령을 한자리에 정리합니다. 로컬(내 컴퓨터)에서 한 일을 원격(GitHub)으로 올려 PR로 잇는 흐름입니다.

명령	무엇을 하나	언제 쓰나
<code>git remote -v</code>	연결된 원격 저장소(<code>origin</code>)를 확인	푸시 전 한 번 점검
<code>git push -u origin <브랜치></code>	로컬 브랜치를 원격 <code>origin</code> 에 처음 올리고 연결(<code>-u =upstream</code>)	그 브랜치를 처음 올릴 때
<code>git push</code>	<code>-u</code> 로 이미 연결된 브랜치에 이후 커밋을 올림	추가 커밋을 올릴 때
<code>gh auth status</code>	<code>gh</code> 가 GitHub에 로그인돼 있는지 확인	인증이 의심될 때
<code>gh pr create --draft</code>	푸시된 브랜치로 초안 PR 을 생성	푸시 직후
<code>gh pr view <번호></code>	만든 PR의 상태·주소를 확인	PR 생성 후

둘째, **커스텀 슬래시 커맨드**입니다. 커밋→푸시→PR 생성은 거의 항상 같은 순서로 반복되므로, 팀이 이 절차를 하나의 슬래시 커맨드로 묶어 두는 경우가 많습니다.

사실 확인: `/commit-push-pr`은 내장이 아니라 커스텀 커맨드입니다

`/commit-push-pr` 같은 이름의 커맨드는 Claude Code에 **기본 내장된 명령이 아닙니다**. 팀이 `.claude/commands/` 또는 스킬로 직접 정의해 두면 `/`이름으로 부를 수 있는 **커스텀 슬래시 커맨드**입니다. 즉 "커밋하고 푸시한 뒤 PR 초안을 만든다"는 절차를 한 파일에 적어 두고, 한 번의 호출로 실행하는 것입니다. Claude Code 자체는 별도 커맨드 없이도 자연어("이 변경을 커밋하고

PR 초안 만들어 줘")로 `git gh` 를 다룰 수 있으므로, 커스텀 커맨드는 그 반복을 줄이는 편의일 뿐입니다. 반면 다음 챕터에서 다룰 `/code-review` · `/security-review` 는 실제 **기본 내장(번들) 명령**입니다. 둘을 혼동하지 마십시오.

좋은 PR은 본문에 다음 네 가지를 담습니다. 리뷰어가 PR을 열었을 때 **읽는 순서대로** 구성하는 것이 좋습니다.

요소	답하는 질문	왜 필요한가
변경 요약(What)	무엇을 바꿨나	리뷰어가 전체 그림을 1분 안에 파악
변경 근거(Why)	왜 바꿨나	의도를 알아야 적절성을 판단
변경 내용(How)	어떻게 구현했나	핵심 설계·선택을 빠르게 따라감
테스트 결과(Verify)	어떻게 검증했나	동작한다는 주장의 근거 제시

특히 **테스트 결과**가 신입의 PR에서 신뢰를 좌우합니다. "잘 됩니다"가 아니라 "pytest 로 3개 테스트가 통과했고, 그 출력은 다음과 같습니다"라고 근거를 보이면, 리뷰어는 여러분을 "검증할 줄 아는 사람"으로 신뢰하기 시작합니다. ch_07에서 들인 검증 습관이 바로 여기서 빛을 발합니다.

다음은 브랜치를 푸시하고 PR 초안을 만드는 흐름입니다.

```
> feature/task-complete 브랜치를 원격에 푸시하고, gh로 PR을 초안(draft)
상태로 만들어 줘. 제목은 Conventional Commits 형식으로, 본문에는
변경 요약·근거·테스트 결과를 담아 줘.
```

(Claude Code가 푸시 후 gh로 PR 생성)

```
$ git push -u origin feature/task-complete
```

```
$ gh pr create --draft --title "feat(tasks): 작업 완료 처리 엔드포인트 추가" --body "(아래 본문)
https://github.com/team/team-tasks-api/pull/42
```

PR 초안을 만들었습니다. 위 주소에서 확인할 수 있습니다. 초안(draft)이므로 아직 리뷰 요청이 자동으로 가지 않습니다. 검토 후 'Ready for review'로 바꾸면 됩니다.

- `git push -u origin feature/task-complete` 의 `-u` 는 로컬 브랜치와 원격 브랜치를 연결(upstream 설정)해, 이후로는 `git push` 만 입력해도 같은 곳으로 올라가게 합니다. `origin` 은 원격 저장소의 기본 이름입니다.
- `gh pr create --draft` 의 `--draft` 는 **초안 PR**을 만듭니다. 초안은 "아직 리뷰 요청 전" 상태로, 본문을 다듬거나 추가 커밋을 올린 뒤 준비가 되면 정식 리뷰로 전환합니다. 신입의 첫 PR을 초안으로 시작하면 마음의 여유가 생깁니다.

- 출력의 마지막 줄 `https://.../pull/42` 가 만들어진 PR의 주소입니다. 이 번호(42)가 PR의 식별자가 됩니다.

worked example: 좋은 PR 본문 작성으로 리뷰어 돕기

PR 본문이 빈약하면("작업 완료 처리 추가함") 리뷰어는 코드를 처음부터 끝까지 혼자 해석해야 합니다. 잘 쓴 본문은 리뷰어의 시간을 아끼고, 그만큼 빠르고 정확한 리뷰를 끌어냅니다. 앞서 만든 PR의 본문을 직접 작성해 보겠습니다.

Claude Code에 변경과 테스트 결과를 근거로 본문 초안을 만들게 한 뒤, 사람이 다듬습니다.

> 이 PR의 본문을 마크다운으로 작성해 줘. 변경 요약·변경 근거·변경 내용·테스트 결과(방금 돌린 pytest 출력 포함)·범위 항목으로 나누고, 리뷰어가 1분 안에 파악할 수 있게 간결하게 써 줘.

에이전트가 돌려준 본문은 다음과 같습니다. 이 형식을 그대로 PR 본문 템플릿으로 익혀 두면 좋습니다.

변경 요약

작업을 완료 상태로 바꾸는 PATCH /tasks/{task_id}/complete 엔드포인트를 추가했습니다.

변경 근거

완료 여부를 표시할 방법이 없어 사용자가 끝낸 작업을 구분하지 못했습니다. (이슈 #37) 가장 작은 단위로 "완료 처리"만 먼저 추가합니다.

변경 내용

- routers/tasks.py: 완료 처리 엔드포인트 추가. 없는 task_id 는 404, 이미 완료된 작업은 변경 없이 현재 상태를 반환합니다.
- tests/test_tasks_complete.py: 정상 완료·없는 작업(404)·중복 완료의 3가지 경우를 검증하는 테스트를 추가했습니다.

테스트 결과

```
\\`\\`
$ pytest tests/test_tasks_complete.py -q
...
3 passed in 0.21s
\\`\\`
```

[100%]

범위

- 포함: 완료 처리 엔드포인트와 그 테스트
- 제외: 완료 취소(되돌리기) 기능, 완료 시각 기록 — 후속 PR에서 다룹니다.

- **변경 요약**은 한 문장입니다. 리뷰어가 제목과 이 한 줄만 읽어도 "무엇인지" 알게 합니다.

- **변경 근거**에 이슈 번호(#37)를 연결했습니다. "왜 이걸 지금 하는가"의 맥락을 PR 밖의 논의와 이어 줍니다.
- **변경 내용**에서 단순히 "엔드포인트 추가"가 아니라 **경계 동작**(없는 작업은 404, 중복 완료는 현재 상태 반환)까지 밝혔습니다. 리뷰어가 신경 쓸 엣지 케이스를 먼저 짚어 주는 것입니다.
- **테스트 결과**에 실제 `pytest` 출력을 붙였습니다. "3 passed"라는 근거가 "동작합니다"라는 주장을 뒷받침합니다. 출력은 꾸며 내지 말고 실제로 돌린 결과를 그대로 붙여야 합니다.
- **범위**에서 "이번에 하지 않은 것"(완료 취소·완료 시각)을 명시했습니다. 이렇게 하면 리뷰어가 "왜 이걸 빠졌지?"라고 묻기 전에 "의도적으로 후속으로 미뤘다"는 것을 알게 되어, 작은 PR이 오히려 신뢰를 얻습니다.

신입의 PR은 작을수록 좋다

"한 PR에 많은 것을 담아 인상을 남기고 싶다"는 유혹이 있지만, 신입의 첫 PR은 **작을수록 좋습니다**. 변경이 작으면 리뷰어가 빠르게 정확하게 봐 주고, 문제가 생겨도 영향 범위가 좁습니다. "완료 처리"와 "상태 필터"를 한 PR에 몰지 말고, 두 개의 작은 PR로 나누는 편이 거의 항상 낫습니다. 위 본문의 "범위 - 제외" 항목이 이 작은 단위를 정당화해 줍니다.

커밋·PR 권한(Bash(`git commit:*`)-gh) 설계

커밋·푸시·PR 생성은 모두 **부수효과 도구(Bash)**를 통해 일어납니다(c05). 기본 설정에서는 이런 명령마다 Claude Code가 승인을 묻습니다. 매번 묻는 것은 안전하지만, git 작업처럼 자주 반복하는 안전한 명령까지 매번 멈추면 흐름이 끊깁니다. 그래서 **자주 쓰는 안전한 git 명령은 미리 허용(allow)하고, 위험한 명령은 차단(deny)하는** 설정을 둡니다.

`.claude/settings.json` (프로젝트 공유 범위)에 다음과 같이 적습니다. 이렇게 체크인하면 팀 모두가 같은 안전 규칙을 공유합니다.

```
{
  "permissions": {
    "allow": [
      "Bash(git status)",
      "Bash(git diff:*)",
      "Bash(git switch:*)",
      "Bash(git add:*)",
      "Bash(git commit:*)",
      "Bash(git push:*)",
      "Bash(gh pr create:*)"
    ],
    "deny": [
      "Bash(git push --force:*)",
      "Bash(git reset --hard:*)",
      "Bash(git branch -D:*)"
    ]
  }
}
```

- "Bash(git commit:*)" 에서 끝의 `:*` 는 "git commit으로 시작하는 모든 명령"을 뜻하는 패턴입니다. `-m "..."` 같은 뒤 인자가 무엇이든 허용합니다. `git status` 처럼 인자가 거의 없는 명령은 패턴 없이 그대로 적었습니다.
- **allow** 목록에는 역사를 안전하게 쌓는 명령(상태 확인·diff·브랜치 전환·스테이징·커밋·푸시·PR 생성)만 넣었습니다. 이들은 "되돌릴 수 있거나 새로 더하는" 작업이라 비교적 안전합니다.
- **deny** 목록의 명령들은 역사를 파괴적으로 바꿉니다. `git push --force` 는 원격 역사를 덮어써 동료의 작업을 날릴 수 있고, `git reset --hard` 는 커밋하지 않은 변경을 복구 불가능하게 지우며, `git branch -D` 는 병합되지 않은 브랜치를 강제로 삭제합니다. deny에 두면 실수로라도 실행되지 않습니다.
- **deny가 allow보다 우선합니다.** `git push:*` 를 허용했어도 `git push --force:*` 가 deny에 있으면 강제 푸시는 차단됩니다. "넓게 허용하되 위험한 변형만 콕 집어 막는" 설계입니다.

이 설계의 핵심은 **최소 권한(least privilege)**입니다. "git 전부 허용"이나 "전체 자동 승인(bypass)"은 편하지만, 한 번의 실수가 팀 저장소를 망칠 수 있습니다. 자주 쓰는 안전한 명령만 골라 허용하고, 파괴적 명령은 명시적으로 막는 것이 신입에게 맞는 안전한 기본값입니다. PR 본문에 비밀 토큰을 넣지 않고, `gh auth login` 으로 인증을 분리하는 것도 같은 원칙의 연장입니다.

즉시 연습 (2종 1세트)

이제 종이에 빈칸을 채우는 대신, 1장에서 준비한 **실제 team-tasks-api 저장소에서** 작업 브랜치를 만들고, 작은 변경을 `feat:` 커밋으로 올린 뒤, 그 변경의 PR 본문에 **실제 pytest 출력을 붙여** 작성합니다. 핵심은 "추측한 커밋 메시지"가 아니라 화면에 실제로 찍힌 `git log · git status · python -m pytest -q` 출력을 그대로 옮기는 것입니다.

1) 모범 예시 (실제 실행 예)

작업 도중 "상태 필터 엔드포인트 추가"와 "관련 없는 README 오타 수정"을 한꺼번에 만들었다고 합시다. 이를 두 개의 논리 단위로 나눕니다.

```
> git status로 변경을 확인한 다음, routers/tasks.py 의 필터 추가는
    feat 커밋으로, README.md 오타 수정은 docs 커밋으로 따로 나눠 커밋해 줘.

$ git add routers/tasks.py
$ git commit -m "feat(tasks): 상태별 작업 조회 필터 추가"
$ git add README.md
$ git commit -m "docs(readme): 엔드포인트 설명 오타 수정"
```

두 변경이 서로 다른 의미이므로 두 커밋으로 분리했습니다. 나중에 둘 중 하나만 되돌리기 쉬워집니다.

2) 직접 해보기 (team-tasks-api에서 실제 브랜치·커밋·PR)

연습 1 — 작업 브랜치에서 작은 기능을 실제로 커밋하라.

team-tasks-api 폴더에서 새 작업 브랜치를 만들고(`git checkout -b feat/task-titles`), claude를 실행해 다음을 그대로 지시하세요.

```
> routers/tasks.py에 할 일 제목만 모아 반환하는 GET /tasks/titles 엔드포인트를 추가하고,
    tests/test_tasks.py에 그 테스트를 더한 뒤 python -m pytest -q로 통과를 확인해 줘.
    통과하면 feat: 커밋으로 올려 줘.
```

작업이 끝나면 직접 `git log --oneline -1` 과 `python -m pytest -q` 를 돌려, **화면에 실제로 나타난 출력을** 그대로 옮겨 적으세요(추측한 메시지가 아니라 실제 커밋 제목).

```
[실제 커밋] git log --oneline -1 출력(type(scope): 형식인가?):
[실제 테스트] python -m pytest -q 출력(예: 8 passed):
[브랜치 확인] git branch로 본 현재 브랜치(메인이 아니라 작업 브랜치인가?):
```

연습 2 — 두 변경을 두 커밋으로 실제로 나눠라.

같은 브랜치에서 README.md 에 한 줄 설명을 추가하는 무관한 변경을 더 만든 뒤, 에이전트에게 "기능 변경과 문서 변경을 서로 다른 커밋으로 나눠 줘"라고 지시하세요. `git log --oneline -2` 로 두 커밋이 실제로 나뉘었는지 확인하고 옮겨 적으세요.

[두 커밋 분리] `git log --oneline -2` 실제 출력(의미가 다른 두 커밋인가?):

연습 3 — PR 본문에 실제 pytest 출력을 붙여라.

연습 1의 변경을 PR로 올립니다(`gh pr create` 또는 GitHub 웹). 본문에는 **방금 직접 돌린** `python -m pytest -q` 의 실제 출력을 그대로 붙이세요(꾸며 쓰지 말 것).

변경 요약

(GET /tasks/titles 엔드포인트 추가 — 실제로 무엇을 했는지)

변경 내용

- routers/tasks.py: (실제 추가한 함수)
- tests/test_tasks.py: (실제 추가한 테스트)

테스트 결과

(여기에 방금 화면에 찍힌 `python -m pytest -q` 출력을 그대로 붙임)

범위

- 포함:
- 제외:

자가체크 — 머리로가 아니라 손으로 했는지 확인하세요.

- [] 메인이 아니라 **실제 작업 브랜치**에서 커밋한 것을 `git branch` 로 확인했다.
- [] [실제 커밋]에 적은 것이 추측이 아니라 `git log` 에 찍힌 실제 커밋 제목이며 `type(scope): 설명` 형식이다.
- [] `python -m pytest -q` 가 통과(8 passed)하는 것을 화면 출력으로 확인했다.
- [] 의미가 다른 두 변경을 **실제로 두 커밋**으로 나눈 것을 `git log` 로 확인했다.
- [] PR 본문의 테스트 결과가 "잘 됨"이 아니라 **방금 직접 돌려 화면에 찍힌 실제 출력**이다.
- [] PR을 실제로 올려 보고, 변경 요약·내용·테스트 결과·범위가 모두 들어간 것을 확인했다.

단원 미니 프로젝트: team-tasks-api에 첫 PR 올리기

종이 위 빈칸을 넘어, 1장에서 준비한 **실제 team-tasks-api 저장소**에 작은 변경을 만들고 첫 PR 까지 올리는 과제입니다. 저장소는 1장 11절에서 `git init` 첫 커밋이 되어 있고, `gh auth`

`login · gh repo create` 로 **GitHub 원격(origin)까지 연결**돼 있으며, `python -m pytest -q` 가 7 passed 상태라고 가정합니다. 시작 전에 `git remote -v` 로 origin 이 보이는지 한 번 확인하세요.

마일스톤(증분):

1. **M1 작업 브랜치.** `team-tasks-api` 루트에서 main에 있는지 확인하고 `git switch -c feature/health-detail` 같은 작업 브랜치를 만듭니다. (적용 개념: 작업 브랜치)
2. **M2 작은 변경 구현.** 에이전트에게 작은 변경 하나를 맡깁니다. 예: "`main.py` 의 `GET /health` 가 `{\"status\": \"ok\"}` 대신 버전도 함께 반환하도록 `{\"status\": \"ok\", \"version\": \"0.1.0\"}` 로 바꾸고, 그 동작을 검증하는 테스트를 `tests/` 에 추가해 줘." 변경 범위를 작게 유지합니다. (적용 개념: 작은 논리 단위)
3. **M3 논리 단위 커밋.** `git status · git diff` 로 무엇이 바뀌었는지 확인한 뒤, 변경된 파일만 골라 스테이징하고 `feat(health): /health 응답에 version 추가` 같은 Conventional Commits 메시지로 커밋합니다. (적용 개념: 스테이징 통제·Conventional Commits)
4. **M4 푸시·PR 본문.** 브랜치를 원격 `origin` 에 푸시하고(`git push -u origin feature/health-detail`), `gh pr create --draft` 로 변경 요약·근거·내용·테스트 결과·범위를 담은 초안 PR을 만듭니다(원격은 1장 11절에서 연결해 두었습니다). 만약 1장 셋업을 건너뛰어 `origin` 이 없다면, 1장 11절로 돌아가 `gh repo create` 로 원격을 만든 뒤 진행합니다. (적용 개념: PR 본문)

실제 검증 단계. 커밋 전과 PR 작성 전에 반드시 저장소에서 테스트를 돌려 통과를 확인하고, 그 출력을 PR 본문의 "테스트 결과"에 그대로 붙입니다.

```
> python -m pytest -q
.....
8 passed
```

[100%]

기존 7개에 `version` 동작 테스트 1개가 더해져 8 passed가 나오면, "잘 됩니다"가 아니라 **실제 출력**으로 변경을 증명한 것입니다.

미니 프로젝트 자가체크

- [] main이 아니라 `feature/...` 작업 브랜치에서 변경했는가(`git branch --show-current` 로 확인)
- [] 변경된 파일만 골라 스테이징했는가(`git add .` 로 쓸어 담지 않았는가)
- [] 커밋 제목이 `type(scope): 설명 형식( feat(health): ... )`인가
- [] PR 본문에 변경 요약·근거·내용·테스트 결과·범위가 모두 있는가
- [] `python -m pytest -q` 의 **실제 출력**(8 passed)을 테스트 결과로 붙였는가

- [] 파괴적 명령(`git push --force` 등)이 권한 설계로 차단돼 있는가

흔한 오류와 오개념

1. 무엇을 커밋할지 통제하지 않고 에이전트에 맡긴다

"알아서 커밋해 줘"라고만 하면 에이전트가 임의 단위로 묶거나 의도하지 않은 파일까지 담을 수 있습니다. **무엇을·어떤 단위로 커밋할지는 끝까지 사람이 정합니다.** 커밋 전 `git status`로 무엇이 스테이징됐는지 확인하고, "이 파일들만 이 메시지로 커밋해 줘"처럼 단위를 명시하세요. 커밋은 되돌리기 어려운 팀의 공식 기록이므로, 자동화의 편의보다 통제가 우선입니다.

2. 모든 변경을 한 커밋에 몰아 담는다

하루 작업을 "오늘 한 일"이라는 한 커밋에 전부 넣으면, 리뷰어가 무엇을 봐야 할지 길을 잃고, 나중에 일부만 되돌리는 것도 불가능합니다. **작은 논리 단위로 자주 커밋하세요.** "이 변경 하나를 한 문장으로 설명할 수 있는가?"가 기준입니다. 한 문장에 "그리고"가 여러 번 들어가면 커밋을 나눌 신호입니다.

3. PR 설명이 "무엇"만 있고 "왜"와 "검증"이 없다

"엔드포인트 추가함"으로 끝나는 PR은 리뷰어에게 코드 해석의 모든 부담을 떠넘깁니다. 좋은 PR은 **무엇(What)·왜(Why)·어떻게 검증했나(Verify)**를 함께 담습니다. 특히 "어떻게 검증했나"에 실제 테스트 출력을 붙이는 습관이 신입의 신뢰를 만듭니다. "잘 됩니다"는 근거가 아닙니다.

4. 메인 브랜치에서 바로 작업하고 커밋한다

작업 브랜치를 만들지 않고 `main`에서 곧장 변경·커밋하면, 검토받지 않은 코드가 팀의 기준선을 오염시킵니다. **항상 작업 브랜치를 만든 뒤 그 위에서 작업하세요**(`git switch -c feature/...`). 커밋 전에 `git branch --show-current`로 지금 위치를 확인하는 습관이 사고를 막습니다.

5. `/commit-push-pr` 이 내장 명령이라고 오해한다

`/commit-push-pr`은 Claude Code에 기본 내장된 명령이 아니라, 팀이 직접 정의하는 **커스텀 커맨드**입니다. 내 환경에서 동작한다면 누군가 만들어 둔 것입니다. 반대로 다음 챕터에서 `/code-review`·`/security-review`는 진짜 내장 명령입니다. "이 슬래시 커맨드가 어디서 왔는가"(내장인가 팀 커스텀인가)를 구분하면, 다른 환경에서 동작하지 않을 때 당황하지 않습니다.

이 챕터 한 줄 정리

첫 PR은 "작은 논리 단위 커밋(Conventional Commits) → 작업 브랜치 → 푸시 → 변경·근거·검증을 담은 PR 본문"의 흐름이며, 에이전트는 명령을 대신 실행하되 **무엇을 커밋할지와 메시지의 의미는 사람이 통제**하고, 위험한 git 명령은 권한 설계로 차단합니다. 다음 챕터(ch_10)에서는 이렇게 올린 PR에 따라오는 **코드 리뷰**를 받고 대응하는 법을 배웁니다.

코드 리뷰 받기·대응하기

이 챕터에서 배우는 것

- PR을 올리기 전 `/code-review` 로 변경(diff)을 점검해, 지적된 개선점 1개 이상을 미리 반영할 수 있습니다.
- 자동 코드 리뷰가 PR에 단 인라인 코멘트를 읽고, 그 지적이 버그·보안·엡지·회귀 중 무엇을 가리키는지 해석할 수 있습니다.
- PR 열림·매 푸시·요청('@claude review') 세 가지 리뷰 트리거를 구분해 필요한 시점에 리뷰를 실행할 수 있습니다.
- 리뷰 코멘트마다 타당한 지적은 변경으로 반영하고 부적절한 지적에는 근거 있는 반론을 달아 처리할 수 있습니다.

도입: PR을 올리면 리뷰가 따라온다

앞 단원(u8)에서 우리는 작은 논리 단위로 커밋하고, 변경 요약·근거·테스트 결과를 담은 첫 PR을 올렸습니다. 그런데 PR을 올리는 것은 끝이 아니라 **대화의 시작**입니다. PR은 "이 변경을 메인에 합쳐 주세요"라는 요청이고, 팀은 그 요청에 **코드 리뷰(code review)**로 응답합니다. 리뷰어가 변경을 읽고, 질문하고, 개선을 제안하고, 마침내 승인하면 비로소 메인에 합쳐집니다.

신입에게 코드 리뷰는 양날의 경험입니다. 한편으로는 **가장 많이 배우는 자리**입니다. 시니어가 "여기는 이렇게 하는 게 우리 팀 관례예요", "이 경우 입력이 비어 있으면 어떻게 되죠?"라고 짚어 줄 때, 책에 없는 실무 감각이 쌓입니다. 다른 한편으로는 **가장 긴장되는 자리**이기도 합니다. 내 코드에 빨간 줄이 그어지는 것을 비난으로 받아들이면, 방어적으로 굳어 버리기 쉽습니다.

이 챕터의 목표는 두 가지입니다. 첫째, **리뷰를 받기 전에 스스로 점검**해서 뻔한 지적을 미리 줄이는 법입니다. Claude Code(클로드 코드)의 `/code-review` 로 올리기 전에 셀프 점검을 하면, 리뷰어는 더 중요한 설계 논의에 집중할 수 있습니다. 둘째, **리뷰 피드백에 성숙하게 대응**하는 법입니다. 타당한 지적은 침착하게 반영하고, 동의하기 어려운 지적에는 근거를 들어 정중히 반론하는 것. 신입의 진짜 역량은 코드 그 자체보다 이 **대응의 태도**에서 드러납니다.

선수 개념 상기

이 챕터는 **git 워크플로: 커밋과 첫 PR(c09)**을 바로 앞 선수로 합니다. 리뷰는 "올린 PR"이 있어야 시작되므로, ch_09에서 배운 작업 브랜치·논리 단위 커밋·PR 본문 작성이 전제입니다. 또 리뷰 지적을 반영해 코드를 고칠 때는 **실행·테스트로 검증하기(c07)**가 다시 등장합니다. 지적을 반영한 뒤 "고쳤습니다"가 아니라 "고치고 테스트가 여전히 통과합니다"를 보여야 리뷰어가 안심합니다. 멀게는 **디버깅(c11, 다음 단원)**과도 이어지는데, 리뷰가 짚은 버그를 재현하고 최소 수정하는 과정이 곧 디버깅이기 때문입니다.

/code-review: 터미널에서 올리기 전 자가 점검

리뷰의 첫 단계는 사실 **남이 아니라 내가** 하는 것입니다. PR을 올리기 전에 내 변경을 한 번 더 점검하면, 사소한 실수로 리뷰 왕복을 낭비하지 않습니다.

정의: /code-review

/code-review (코드 리뷰 커맨드)는 Claude Code에 기본 내장된(번들) 슬래시 커맨드로, GitHub 앱을 설치하지 않고도 **터미널에서 현재 변경(diff)을 리뷰**해 정확성 버그와 재사용·단순화·효율 개선점을 짚어 주는 자가 점검 도구입니다.

/code-review의 동작을 정확히 알아 둡니다.

- **무엇을 보나:** 기본적으로 현재 브랜치가 upstream(보통 `main`)보다 앞선 커밋들 + 아직 커밋하지 않은 워킹 트리의 변경을 함께 봅니다. 곧 "이 PR이 될 변경 전체"를 점검합니다.
- **무엇을 짚나:** 정확성 버그(correctness bug)와 함께, 중복 제거(reuse)·단순화(simplification)·효율(efficiency) 같은 개선점을 보고합니다. 단순한 스타일 지적을 넘어 "이 로직이 빈 입력에서 깨질 수 있다" 같은 실질적 문제를 잡습니다.
- **고치기까지:** `--fix` 옵션을 붙이면 발견한 문제를 **바로 반영**하도록 할 수 있습니다. 다만 신입에게는 먼저 점검 결과를 읽고 **무엇을 받아들일지 판단한 뒤** 고치는 것을 권합니다.

비슷한 이름의 형제 명령도 함께 기억해 둡니다. **/review**는 같은 읽기 전용 리뷰를 **GitHub의 특정 PR**에 대해 수행하고, **/security-review**는 보안에 집중한 더 깊은 점검입니다(이 챕터 뒤에서 다룹니다). 이 셋은 모두 실제 내장 명령입니다.

/code-review (내장)와 자동 코드 리뷰(GitHub 연동)를 구분하기

두 가지를 헷갈리지 마십시오. **/code-review**는 **내 터미널에서, 올리기 전에, 내가 직접** 돌리는 자가 점검입니다. 반면 다음 절에서 다룰 **자동 코드 리뷰**는 조직이 GitHub에 **Code Review**를 활성화했을 때, PR이 올라오면 자동으로 돌아 **PR 화면에 인라인 코멘트**를 다는 별도 기능입니다.

전자는 사전 점검, 후자는 사후 자동 리뷰입니다. 좋은 신입은 둘을 함께 씁니다. 올리기 전 `/code-review` 로 거르고, 올린 뒤 자동 리뷰와 사람 리뷰로 한 번 더 거릅니다.

동작 원리: 자동 리뷰가 PR에 인라인 코멘트를 다는 흐름



조직이 Claude Code의 **Code Review(코드 리뷰 서비스)**를 활성화하면, PR이 올라올 때 전문 에이전트가 변경을 분석해 **문제를 발견한 바로 그 코드 줄에 인라인 코멘트(inline comment)**로 남깁니다. 이 자동 리뷰가 일하는 흐름을 단계로 풀면 다음과 같습니다.

1. **트리거(trigger)** — 설정된 조건이 충족되면 리뷰가 시작됩니다. 보통은 PR이 열리는 순간입니다(자세한 트리거 방식은 다음 절에서 다룹니다).
2. **분석(analyze)** — 에이전트가 PR의 변경(diff)과 주변 코드를 읽어, 논리 오류·보안 취약점·엣지 케이스·미묘한 회귀(regression, 기존 동작이 깨지는 것)를 찾습니다. 단순히 줄 단위로 보는 것이 아니라 변경의 맥락을 함께 봅니다.
3. **정리(dedupe·rank)** — 발견한 문제들을 **중복 제거(dedupe)**하고 **심각도순으로 정렬(rank)**합니다. 같은 문제를 여러 번 반복해 코멘트하지 않고, 중요한 것이 위로 오게 합니다. 신입이 "코멘트 폭탄"에 압도되지 않도록 걸러 주는 단계입니다.
4. **인라인 코멘트(comment)** — 각 문제를 해당 코드 줄에 직접 코멘트로 답니다. "이 줄에서 `task_id` 가 없으면 `KeyError`가 납니다" 같은 식으로, 어느 코드의 무엇이 문제인지가 한 눈에 연결됩니다.

5. **대응(respond)** — 신입은 각 코멘트를 읽고 대응합니다. 타당하면 코드를 고쳐 다시 푸시하고, 동의하기 어려우면 코멘트에 답글로 근거를 답니다. 이 왕복이 끝나고 사람 리뷰어가 승인하면 머지(merge)됩니다.

단계	무엇이 일어나나	신입이 할 일
트리거	PR 열림·푸시·요청으로 리뷰 시작	트리거 방식을 알고 기다림
분석	버그·보안·엣지·회귀 탐색	(기다림)
정리	중복 제거·심각도 정렬	(기다림)
인라인 코멘트	문제 줄에 코멘트 부착	코멘트를 읽고 이해
대응	반영·재요청·반론	판단해 대응, 테스트로 검증

자동 리뷰는 사람 리뷰를 대체하지 않는다

자동 코드 리뷰는 **뻔한 버그·보안 문제·엣지 케이스를 빠르게 걸러** 사람 리뷰어의 부담을 덜어줍니다. 하지만 "이 기능이 우리 제품 방향에 맞는가", "이 설계가 6개월 뒤에도 유지보수하기 좋은가" 같은 **판단**은 여전히 사람의 몫입니다. 자동 리뷰를 "1차 필터"로, 사람 리뷰를 "최종 판단"으로 보는 것이 건강한 관점입니다.

worked example: /code-review로 점검해 지적 미리 반영

ch_09에서 만든 "작업 완료 처리 엔드포인트" 브랜치를 PR로 올리기 전에 `/code-review`로 점검해 보겠습니다. 목표는 "리뷰어가 짚기 전에 내가 먼저 발견해 고치기"입니다.

리뷰 대상 코드는 다음과 같다고 합시다. 언뜻 동작하지만 숨은 문제가 있습니다.

```
# routers/tasks.py (점검 대상)
@router.patch("/tasks/{task_id}/complete")
def complete_task(task_id: int):
    task = TASKS[task_id]          # 없는 id 면 KeyError
    task["done"] = True
    return task
```

이제 터미널에서 자가 점검을 돌립니다.

```
> /code-review
```

(Claude Code가 현재 브랜치의 변경을 읽고 리뷰 결과를 보고)

코드 리뷰 결과 — feature/task-complete (2건)

[높음] 정확성 — routers/tasks.py:3

TASKS[task_id] 는 없는 task_id 에 대해 KeyError 를 던져 500 오류가 납니다. 존재하지 않는 작업에는 404 를 반환해야 합니다.

제안: task = TASKS.get(task_id) 후 None 이면 HTTPException(404).

[낮음] 견고성 — routers/tasks.py:4

이미 done=True 인 작업도 그대로 덮어씁니다. 동작엔 문제없지만, 중복 완료 요청을 멍등(idempotent)하게 처리한다는 점을 테스트로 명시해 두면 좋습니다.

- /code-review 는 인자 없이 부르면 **현재 브랜치의 변경 전체**를 점검합니다. PR을 올리기 전에 돌렸다는 점이 핵심입니다. 리뷰어가 아니라 내가 먼저 봅니다.
- 결과가 **심각도([높음] · [낮음])와 파일:줄**로 정리되어 있습니다. routers/tasks.py:3 처럼 위치가 명확해 어디를 고칠지 바로 압니다.
- [높음] 지적은 **실제 버그**입니다. 없는 task_id 로 요청하면 KeyError 가 나 500 오류가 됩니다. 이런 건 리뷰어가 짚기 전에 반드시 고쳐야 할 정확성 문제입니다.
- [낮음] 지적은 버그는 아니지만 **견고성** 제안입니다. 모든 지적을 같은 무게로 받지 말고, 심각도에 따라 우선순위를 두는 것이 좋습니다.

이제 높음 지적을 반영합니다. /code-review --fix 로 자동 반영할 수도 있지만, 여기서는 무엇을 고칠지 명시해 직접 고치게 합니다.

```
> [높음] 지적을 반영해 줘. 없는 task_id 면 404 를 반환하도록
HTTPException 으로 고치고, 그 경우를 검증하는 테스트도 추가해 줘.
```

반영 후 코드는 다음과 같이 바뀝니다.

```
# routers/tasks.py (반영 후)
from fastapi import HTTPException

@router.patch("/tasks/{task_id}/complete")
def complete_task(task_id: int):
    task = TASKS.get(task_id)
    if task is None:
        raise HTTPException(status_code=404, detail="작업을 찾을 수 없습니다")
    task["done"] = True
    return task
```

```
$ pytest tests/test_tasks_complete.py -q
...
3 passed in 0.20s
```

[100%]

- `TASKS[task_id]` 를 `TASKS.get(task_id)` 로 바꿔 없는 키에 `None` 을 받게 하고, `None` 이면 `HTTPException(404)` 로 명확한 응답을 줍니다. 500(서버 오류)이 아니라 404(찾을 수 없음)는 "클라이언트가 없는 자원을 요청했다"는 정확한 의미입니다.
- 고친 뒤 **반드시 테스트를 돌려** 통과를 확인했습니다(c07). "고쳤다"가 아니라 "고치고 3개 테스트가 통과한다"가 진짜 완료입니다.
- 이렇게 올리기 전에 정확성 버그를 스스로 잡으면, 실제 PR 리뷰는 더 가벼워지고 리뷰어는 더 중요한 부분에 집중합니다.

리뷰 트리거: PR 열림·매 푸시·요청('@claude review')

자동 코드 리뷰가 **언제** 돌지는 저장소별로 설정합니다. 조직이 Code Review를 활성화하면, 저장소마다 "리뷰 동작(Review Behavior)"을 골라 트리거 방식을 정합니다. 세 가지 방식이 있습니다.

트리거	언제 리뷰가 도나	어울리는 상황
PR 생성 후 1회	PR이 열리거나 'Ready for review'가 될 때 한 번	기본값. 완성된 변경을 한 번 점검
매 푸시마다	PR 브랜치에 푸시할 때마다	변경이 자주 바뀌는 PR을 계속 추적
수동(요청시)	코멘트로 <code>@claude review</code> 라고 적을 때만	필요할 때만 비용 없이 호출

- **PR 생성 후 1회**가 기본입니다. PR을 열면 리뷰가 한 번 돌아 초기 점검을 해 줍니다. 대부분의 작은 PR에는 이걸로 충분합니다.
- **매 푸시마다**는 PR이 진화하는 동안 새로 들어온 문제를 계속 잡아 줍니다. 다만 푸시가 잦으면 리뷰도 자주 돌아 비용·코멘트가 늘 수 있습니다.
- **수동**은 평소엔 돌지 않다가, 누군가 PR 코멘트에 `@claude review` 라고 적으면 그때 리뷰합니다. 비용을 아끼면서 "지금 봐 줘"라고 명시적으로 부를 때 유용합니다.

여기서 `@claude` 는 GitHub의 멘션(mention)입니다. PR이나 이슈 코멘트에 `@claude` 로 부르면 Claude가 요청을 받아 분석·리뷰·수정 같은 작업을 수행합니다(GitHub Actions 연동 기반). 예컨

대 자동 리뷰가 한 번 끝난 뒤 코드를 더 고쳐 올리고 나서, `@claude review` 로 다시 봐 달라고 명시적으로 요청할 수 있습니다.

트리거를 알면 리뷰를 능동적으로 쓴다

트리거 방식을 모르면 "왜 리뷰가 안 달리지?" 하고 막연히 기다리게 됩니다. 우리 저장소가 "수동" 설정이라면 PR을 올려도 자동으로 리뷰가 돌지 않으니, 직접 `@claude review` 로 불러야 합니다. 반대로 "매 푸시마다"라면 작은 수정을 푸시할 때마다 리뷰가 돌 수 있습니다. 내 저장소의 설정을 알아 두는 것이 리뷰를 능동적으로 활용하는 출발점입니다.

다음은 "수동" 트리거 저장소에서, 리뷰 지적을 반영해 다시 푸시한 뒤 명시적으로 재리뷰를 요청하는 흐름입니다. PR 화면의 코멘트 입력란에 멘션을 적습니다.

```
[PR #42 코멘트 입력]
@claude review
```

```
404 처리와 status 필터 기본값을 고쳐 다시 푸시했습니다. 변경된 두 줄을
다시 봐 주세요.
```

이렇게 `@claude` 를 멘션하면 Claude가 PR의 최신 상태를 다시 분석해 새 인라인 코멘트를 답니다. "수동" 트리거에서는 이 멘션이 있어야 리뷰가 돌므로, 고치고 푸시한 뒤 재요청하는 것을 잊지 마십시오. 같은 멘션 방식으로 `@claude` 에게 특정 작업(예: "이 함수의 테스트를 추가해 줘")을 PR 안에서 직접 부탁할 수도 있습니다.

worked example: 인라인 코멘트를 읽고 해당 줄 수정

PR을 올리자 자동 리뷰가 한 코드 줄에 인라인 코멘트를 달았다고 합시다. 이를 읽고 해당 줄을 고치는 흐름을 따라가 봅니다.

상태 필터를 추가한 다음 엔드포인트가 리뷰 대상입니다.

```
# routers/tasks.py (리뷰가 코멘트를 단 코드)
@router.get("/tasks")
def list_tasks(status: str = None):
    return [t for t in TASKS.values() if t["status"] == status]
```

자동 리뷰가 `return` 줄에 다음 인라인 코멘트를 남겼습니다.

[자동 리뷰 코멘트 — routers/tasks.py:4]

status 파라미터가 주어지지 않으면(None) 어떤 작업도 status == None 과 일치하지 않아 항상 빈 목록이 반환됩니다. 필터가 없을 때는 전체 목록을 반환해야 의도에 맞습니다.

이 코멘트는 **회귀에 가까운 논리 오류**를 짚었습니다. "필터를 안 주면 전체가 나와야 하는데, 빈 목록이 나온다"는 것입니다. 코멘트가 가리키는 줄을 에이전트와 함께 고칩니다.

> routers/tasks.py:4 에 달린 리뷰 코멘트를 반영해 줘. status 가 None 이면 전체를 반환하고, 값이 있으면 그 값으로 필터링하도록 고치고, 두 경우를 모두 검증하는 테스트를 추가해 줘.

```
# routers/tasks.py (반영 후)
@router.get("/tasks")
def list_tasks(status: str | None = None):
    tasks = list(TASKS.values())
    if status is None:
        return tasks
    return [t for t in tasks if t["status"] == status]
```

```
$ pytest tests/test_tasks_filter.py -q
....
4 passed in 0.23s [100%]
```

- 인라인 코멘트의 **위치(:4)**와 **내용**을 그대로 지시에 답아, 에이전트가 어느 줄의 무엇을 고칠지 정확히 알게 했습니다. "코멘트 반영해 줘"라고만 하지 않고 "어떻게 고칠지" (None이면 전체, 값이 있으면 필터)를 명시한 점이 중요합니다.
- 고친 로직은 `status is None` 을 먼저 분기해 **필터가 없으면 전체**를 반환합니다. 리뷰가 짚은 논리 오류를 정확히 해소했습니다.
- 한 코멘트를 반영할 때도 **그 경우를 검증하는 테스트(필터 없음·필터 있음 두 경우)**를 함께 추가하고 통과를 확인했습니다. 이렇게 하면 같은 문제가 다시 생기지 않습니다(회귀 방지).
- 고친 뒤에는 다시 푸시합니다. 트리거가 "매 푸시마다"라면 자동 리뷰가 다시 돌고, "수동"이라면 `@claude review` 로 다시 봐 달라고 요청합니다.

/security-review로 취약점 사전 점검

정확성 버그를 잡았다면, 한 가지 점검이 더 남았습니다. **보안**입니다. 보안 결함은 평소엔 잘 드러나지 않다가, 공격자가 악용할 때 큰 사고가 됩니다. 신입이 무심코 흘리기 쉬운 영역이라 별도의 도구가 있습니다.

정의: /security-review

/security-review (보안 리뷰 커맨드)는 Claude Code에 내장된 슬래시 커맨드로, 변경을 **보안 관점에서 더 깊이** 점검해 SQL 인젝션·XSS(크로스 사이트 스크립팅)·인증 결함·안전하지 않은 데이터 처리·의존성 취약점 같은 흔한 보안 문제를 찾아 주는 읽기 전용 점검입니다.

/security-review 는 터미널에서 바로 돌려 **올리기 전에** 보안을 점검할 수 있고, 발견한 문제를 고쳐 달라고 바로 이어서 요청할 수도 있습니다. GitHub Actions로 연동하면 PR마다 보안 리뷰를 자동으로 돌릴 수도 있습니다. 권장 습관은 **의미 있는 변경을 커밋하기 전에 /security-review** 를 한 번 돌리는 것입니다.

다음은 사용자 검색 엔드포인트를 보안 점검하는 예입니다.

```
> /security-review
```

(Claude Code가 변경을 보안 관점에서 분석)

보안 리뷰 결과 (1건)

[높음] SQL 인젝션 — routers/search.py:7

사용자 입력 keyword 를 f-string 으로 SQL 문에 직접 끼워 넣어, 공격자가 임의 SQL 을 실행할 수 있습니다.

제안: 파라미터 바인딩(플레이스홀더)을 사용해 입력을 값으로 전달하세요.

문제가 된 코드와 안전한 수정은 다음과 같습니다.

취약: 사용자 입력을 SQL 에 직접 끼워 넣음 (SQL 인젝션)

```
query = f"SELECT * FROM users WHERE name LIKE '{keyword}%"
db.execute(query)
```

안전: 파라미터 바인딩으로 입력을 값으로 전달

```
query = "SELECT * FROM users WHERE name LIKE ?"
db.execute(query, (f"%{keyword}%",))
```

- 취약한 코드는 keyword 를 **문자열로 직접** SQL에 붙였습니다. 공격자가 keyword 에 ' OR '1'='1 같은 값을 넣으면 의도하지 않은 쿼리가 실행됩니다(SQL 인젝션).
- 안전한 코드는 값이 들어갈 자리를 ? 플레이스홀더로 두고, 실제 값을 별도 인자로 전달합니다. 데이터베이스가 입력을 **코드가 아니라 값으로** 취급하므로 인젝션이 막힙니다.
- /security-review** 는 이런 결함을 **사람 리뷰어가 놓치기 쉬운 단계에서 미리** 잡아 줍니다. 다만 보안 리뷰가 모든 취약점을 잡는다고 맹신하지 말고, 입력 검증·인증 같은 기본기는 평소에 갖추는 것이 우선입니다.

피드백 대응: 반영·재요청·근거 있는 반론

리뷰 코멘트를 받았을 때의 대응은 세 가지로 나뉩니다. 신입의 성숙도는 이 셋을 **상황에 맞게** 고르는 데서 드러납니다.

1. **반영(accept)** — 지적이 타당하면 코드를 고치고, 테스트로 검증한 뒤, 무엇을 어떻게 고쳤는지 코멘트에 한 줄로 답합니다. 가장 흔하고 기본이 되는 대응입니다.
2. **재요청(re-review)** — 고쳐 푸시한 뒤 다시 봐 달라고 요청합니다. 트리거가 "매 푸시마다"면 자동으로 다시 돌고, "수동"이면 `@claude review` 나 리뷰어 멘션으로 명시적으로 부릅니다.
3. **근거 있는 반론(push back with reason)** — 모든 지적을 무조건 수용할 필요는 없습니다. 동의하기 어렵다면, **감정이 아니라 근거로** 정중히 반론합니다. "그건 이미 상위 미들웨어에서 검증하고 있어서 여기서 다시 검사하면 중복입니다. 관련 코드는 `middleware.py:20`입니다"처럼 사실과 위치를 들어 답합니다.

세 대응의 좋은 답글 예시를 비교합니다.

상황	나쁜 답글	좋은 답글
타당한 지적	(무시하고 그냥 머지)	"맞습니다. <code>get +404</code> 로 고치고 테스트 추가했습니다. 통과 확인했습니다."
고치고 재요청	"고쳤어요"	"404 처리와 테스트를 추가해 다시 푸시했습니다. <code>@claude review</code> 로 재점검 부탁드립니다."
동의 어려움	"이게 맞는데요"	"이 검증은 <code>middleware.py:20</code> 에서 이미 수행해 중복이라 판단합니다. 그래도 여기 두는 게 좋을까요?"

리뷰는 비난이 아니라 코드에 대한 대화다

리뷰 코멘트는 "당신이 틀렸다"가 아니라 "이 코드를 더 좋게 만들자"는 제안입니다. 지적의 대상은 **여러분이 아니라 코드**입니다. 이 관점을 가지면 방어적으로 굳지 않고, 지적을 학습 기회로 받아들일 수 있습니다. 반대로 모든 지적을 무비판적으로 수용하는 것도 좋지 않습니다. 근거를 이해하지 못한 채 고치면 같은 실수를 반복합니다. **"왜 이런 지적이 나왔는가"를 이해하는 것이** 반영이든 반론이든 모든 좋은 대응의 출발점입니다.

자주 묻는 질문

`/code-review` 와 `/review` 는 무엇이 다른가요?

`/code-review` 는 내 터미널의 현재 변경(diff)을 점검합니다. 아직 PR을 올리지 않았거나 커밋하지 않은 변경까지 봅니다. `/review` 는 같은 읽기 전용 리뷰를 이미 GitHub에 올라간 특정 PR에 대해 수행합니다. 곧 "올리기 전 내 변경 점검"은 `/code-review`, "올라간 PR 점검"은 `/review` 로 기억하면 됩니다.

자동 리뷰가 코멘트를 너무 많이 달면 어떻게 하나요?

Code Review 서비스는 발견한 문제를 중복 제거하고 심각도순으로 정렬해 답니다. 그래도 코멘트가 많다면 [높음] 부터 차례로 대응하세요. 모든 코멘트를 한 번에 완벽히 처리하려 하기보다, 심각도가 높은 것부터 반영하고 푸시해 가며 좁혀 가는 편이 현실적입니다.

리뷰 지적이 틀린 것 같으면 그냥 무시해도 되나요?

무시하지 말고 근거를 들어 답글로 반론하세요. 지적이 정말 부적절하다면 "이 검증은 `middleware.py:20` 에서 이미 수행해 중복입니다"처럼 사실과 위치로 답하면 됩니다. 답 없이 무시하면 리뷰어(사람이든 자동이든)는 여러분이 코멘트를 못 봤는지, 동의하는지 알 수 없습니다. 반론도 대응의 한 형태입니다.

`/security-review` 는 언제 돌리는 게 좋나요?

권장 습관은 의미 있는 변경을 커밋하기 전에 한 번 돌리는 것입니다. 특히 사용자 입력을 다루거나(검색·로그인), 데이터베이스 쿼리를 만들거나, 외부 요청을 보내는 코드를 바꿨다면 반드시 점검하세요. 프로덕션 코드 저장소라면 GitHub Actions로 PR마다 자동 보안 리뷰를 돌리도록 설정하는 것도 좋습니다.

즉시 연습 (2종 1세트)

이제 종이에 답글을 채우는 대신, 1장에서 준비한 실제 `team-tasks-api` 저장소에 변경을 만들고 `/code-review` 를 실제로 돌려 받은 지적을 기록하고, 그중 하나를 반영해 봅니다. 핵심은 "예상한 코멘트"가 아니라 화면에 실제로 나타난 리뷰 지적을 그대로 옮기고, 반영 뒤 `python -m pytest -q` 로 검증까지 직접 하는 것입니다.

1) 모범 예시 (실제 실행 예)

자동 리뷰가 다음 코멘트를 달았다고 합시다.

[자동 리뷰 코멘트 — routers/tasks.py:9]
삭제 엔드포인트가 없는 task_id 에도 200 을 반환합니다. 존재하지 않는
작업 삭제는 404 가 적절합니다.

좋은 대응은 다음과 같습니다.

> 이 코멘트를 반영해 줘. 없는 task_id 삭제는 404 를 반환하도록 고치고,
그 경우 테스트를 추가한 뒤 pytest 로 통과를 확인해 줘.

그리고 PR 코멘트에 답글을 답니다.

맞습니다. 없는 작업 삭제 시 404를 반환하도록 고치고, 해당 경우의 테스트를 추가
했습니다. pytest 로 통과를 확인해 다시 푸시했습니다.

2) 직접 해보기 (team-tasks-api에서 실제로 /code-review 돌리기)

연습 1 — 변경을 만들고 /code-review 를 실제로 돌려 지적을 기록하라.

team-tasks-api에서 작업 브랜치를 만들고, claude 로 작은 변경을 하나 만드세요(예:
"routers/tasks.py에 GET /tasks/titles 엔드포인트를 추가해 줘"). 변경을 만든 뒤 같은 세션에서
슬래시 명령 /code-review 를 실행하고, **화면에 실제로 나타난 지적**을 그대로 옮겨 적으세요(미
리 예상한 코멘트가 아니라 진짜 받은 지적).

[실제 지적 1] /code-review가 짚은 내용(파일·줄·문제):
[실제 지적 2] (있다면):
[심각도 판단] 위 지적 중 먼저 고쳐야 할 것과 그 이유:

연습 2 — 지적 하나를 실제로 반영하고 테스트로 검증하라.

받은 지적 중 타당한 것 하나를 골라 에이전트에게 반영을 지시하고("이 지적을 반영해 고치고,
그 경우 테스트를 더한 뒤 python -m pytest -q로 통과를 확인해 줘"), 반영 뒤 실제 출력을 옮
겨 적으세요.

[반영한 변경] 실제로 바뀐 부분(diff의 + 줄):
 [검증] `python -m pytest -q` 실제 출력(통과했는가?):

연습 3 — 보안 점검과 근거 있는 반론을 실제로 해보라.

같은 변경에 `/security-review` 를 실행해 보안 관점 지적이 있는지 확인하세요. 또, 받은 지적 중 **동의하기 어려운 것**이 있으면(예: 이미 FastAPI의 경로 파라미터 `int` 타입이 검증하는 부분) 감정이 아니라 근거·위치를 들어 정중히 반론하는 답글을 PR에 실제로 달아 보세요.

[`security-review` 결과] 보안 지적이 있었는가(없으면 "없음"):
 [근거 있는 반론] 동의하기 어려운 지적과 내가 든 근거:

자가체크 — 머리로가 아니라 손으로 했는지 확인하세요.

- [] `team-tasks-api`에 실제 변경을 만들고 `/code-review` 를 **실제로 실행**해 지적을 받았다.
- [] [실제 지적]에 적은 것이 예상이 아니라 화면에 실제로 나타난 리뷰 코멘트다.
- [] 받은 지적을 **심각도에 따라** 우선순위를 두고, 하나를 골라 실제로 반영했다.
- [] 지적을 반영한 뒤 `python -m pytest -q` 로 **통과를 직접 확인**했다(c07).
- [] `/security-review` 도 실제로 한 번 돌려 보안 관점 지적 유무를 확인했다.
- [] 동의하기 어려운 지적에는 감정이 아니라 **근거·위치**로 답했다.

단원 미니 프로젝트: `team-tasks-api` 변경을 올리기 전 자가 점검

이 미니 프로젝트는 9장에서 만든 `team-tasks-api` 의 작업 브랜치를, PR로 올리기 **전에** `/code-review` · `/security-review` 로 스스로 점검하고 지적을 반영하는 과제입니다. 9장 미니 프로젝트의 변경(예: `routers/tasks.py` 의 작은 기능 추가와 그 테스트)이 작업 브랜치에 있다고 가정합니다.

마일스톤(증분):

1. **M1 정확성 자가 점검.** `team-tasks-api` 루트에서 작업 브랜치에 선 채로 `/code-review` 를 실행해, 현재 브랜치의 변경 전체에 대한 정확성·개선 지적을 받습니다. (적용 개념: `/code-review` 자가 점검)
2. **M2 보안 자가 점검.** 이어서 `/security-review` 를 실행해 보안 관점의 지적이 없는지 확인합니다. 사용자 입력을 다루는 부분이 있다면 특히 주의해 봅니다. (적용 개념: `/security-review`)

3. **M3 심각도순 반영.** [높음] 지적부터 차례로 반영합니다. 없는 자원에 404를 주는지, 필터 없는 요청에 전체를 반환하는지 같은 경계 동작을 점검해 고칩니다. (적용 개념: 심각도순 대응)
4. **M4 검증·재요청.** 반영 후 테스트를 돌려 통과를 확인하고, 그 결과를 답글·PR 본문에 적습니다. "수동" 트리거 저장소라면 `@claude review` 로 재점검을 요청합니다. (적용 개념: 반영·재요청·검증)

실제 검증 단계. 지적을 반영한 뒤 반드시 저장소 전체 테스트를 돌려 통과를 확인합니다(c07). 한 군데를 고치다 다른 곳을 깨지 않았는지까지 봅니다.

```
> python -m pytest -q
.....
8 passed
```

[100%]

미니 프로젝트 자가체크

- ☐ `team-tasks-api` 에서 올리기 전 `/code-review` 를 직접 돌렸는가
- ☐ `/security-review` 로 보안 관점도 점검했는가
- ☐ 지적을 **심각도순**으로 우선순위를 두어 반영했는가
- ☐ 반영 후 `python -m pytest -q` 로 전체 통과를 확인했는가(부분 테스트만 보지 않았는가)
- ☐ 동의하기 어려운 지적에는 근거·위치(파일·줄)로 답했는가
- ☐ 우리 저장소의 리뷰 트리거 방식(1회·매 푸시·수동)을 확인하고 필요 시 `@claude review` 로 재요청했는가

참고: 위 worked example의 `routers/tasks.py` · `routers/search.py` 코드 조각은 리뷰가 잡아내는 "고치기 전 초안"을 보여 주기 위한 예시입니다. 실제 `team-tasks-api`의 라우터는 1장에서 본 대로 `store.py`에 로직을 위임하므로, 여러분의 점검 대상은 여러분이 그 저장소에 더한 실제 변경입니다.

흔한 오류와 오개념

1. 리뷰 지적을 방어적으로 받아들이다

빨간 줄을 비난으로 느껴 굳어 버리면 배울 기회를 잃습니다. 리뷰의 대상은 **여러분이 아니라 코드**입니다. "왜 이런 지적이 나왔는가"를 먼저 이해하면, 반영하든 반론하든 한 단계 성장합니다.

다. 신입의 평판은 완벽한 첫 코드가 아니라 **피드백에 대응하는 태도**에서 만들어집니다.

2. 자동 리뷰가 모든 걸 잡아 준다고 맹신한다

자동 코드 리뷰 `/code-review` `/security-review` 는 강력하지만 **만능이 아닙니다**. "이 기능이 제품 방향에 맞는가", "이 설계가 장기적으로 유지보수하기 좋은가" 같은 판단은 사람의 몫이고, 자동 리뷰가 놓치는 도메인 특유의 버그도 있습니다. 자동 리뷰는 "1차 필터"이지 "최종 보증"이 아닙니다. 통과했다고 검증을 생략하지 마십시오.

3. 모든 지적을 무비판적으로 반영한다

지적이 타당한지 따져 보지 않고 기계적으로 다 고치면, 근거 없는 변경이 쌓이고 때로는 멀쩡한 코드를 망칩니다. 이미 다른 곳에서 처리되는 검증을 중복으로 넣거나, 우리 맥락에 맞지 않는 일반론을 따르는 일이 생깁니다. **근거를 이해하고 판단해** 반영하거나, 합당한 이유가 있으면 정중히 반론하세요.

4. 고치기만 하고 검증하지 않는다

리뷰 지적을 반영한 뒤 "고쳤습니다"로 끝내면, 그 수정이 다른 곳을 깨뜨렸을 수 있습니다. 반영 후에는 **반드시 테스트를 돌려 통과를 확인**(c07)하고, 그 결과를 답글에 함께 적으세요. "고치고 테스트가 통과합니다"가 "고쳤습니다"보다 훨씬 신뢰를 줍니다.

5. `/code-review` 와 자동 코드 리뷰를 혼동한다

`/code-review` 는 **내 터미널에서 올리기 전에 내가** 돌리는 자가 점검이고, 자동 코드 리뷰는 조직이 GitHub에 활성화했을 때 **PR에 자동으로** 인라인 코멘트를 다는 기능입니다. 둘은 시점과 주체가 다릅니다. 또 `/code-review` `/security-review` 는 진짜 내장 명령이지만, 앞 챕터의 `/commit-push-pr` 은 커스텀 커맨드였다는 점도 함께 기억하면 "이 명령이 어디서 왔는가"를 정확히 구분할 수 있습니다.

이 챕터 한 줄 정리

좋은 리뷰 대응은 "올리기 전 `/code-review` `/security-review` 로 자가 점검 → 자동 리뷰의 인라인 코멘트를 심각도순으로 이해 → 타당하면 반영하고 테스트로 검증, 동의 어려우면 근거로 반론"의 흐름이며, 리뷰는 비난이 아니라 코드를 함께 다듬는 대화입니다. 다음 단원(u10)에서는 리뷰가 짚은 버그를 본격적으로 추적해 고치는 **디버깅**을 배웁니다.

11장 에이전트와 함께 디버깅하기

이 장은 단원 10 "에이전트와 함께 디버깅하기"의 단일 챕터입니다. 여기서는 개념 **에이전트와 함께 디버깅하기(debugging with the agent)** 하나를 깊이 있게 배웁니다. 신입 개발자는 자주 막힙니다. 그리고 그 막힘을 얼마나 빠르고 정확하게 푸느냐가 곧 생산성입니다. 이 장을 마치면 에러 메시지를 에이전트에게 제대로 건네 원인을 진단하고, 재현 → 가설 → 최소 수정 → 회귀 테스트의 루프로 버그를 잡고, 같은 버그가 다시 나지 않게 막는 방법을 갖게 됩니다.

이 장의 학습 목표

- 실패한 테스트나 예외의 전체 에러 메시지·스택 트레이스를 에이전트에게 그대로 주어 원인을 진단시킬 수 있습니다.
- 막연한 '고쳐 줘' 대신 증상·기대·실제 동작을 구체적으로 전달해 더 정확한 진단을 끌어낼 수 있습니다.
- 재현 → 가설 → 최소 수정 → 회귀 테스트의 디버깅 루프를 한 바퀴 돌려 버그를 해결할 수 있습니다.
- 수정 후 회귀 테스트를 실행해 다른 곳이 깨지지 않았는지 확인하고, 같은 버그의 재발을 막을 수 있습니다.

11.1 도입: 막힘을 푸는 능력이 곧 생산성이다

신입으로 팀에 합류해 첫 몇 주를 보내면, 코드를 새로 쓰는 시간보다 **막혀서 멈춰 있는 시간**이 더 길다는 사실을 곧 알게 됩니다. 테스트가 빨강게 떨어지고, 서버가 500을 뱉고, 분명 맞게 짰 것 같은 코드가 422를 돌려줍니다. 베테랑과 신입의 차이는 "버그를 안 만드느냐"가 아니라 "막혔을 때 얼마나 빨리 빠져나오느냐"에서 더 크게 벌어집니다. 누구나 버그를 만듭니다. 다만 어떤 사람은 30초 만에 원인을 찾고, 어떤 사람은 한나절을 헤맹니다.

이 장 직전까지 우리는 변경을 안전하게 만들고(5장 권한·6장 플랜 모드) 실행·테스트로 검증하는 법(7장)을 배웠습니다. 특히 7장에서 "검증 전까지는 끝난 게 아니다"라는 습관을 들였습니다. 그런데 검증을 하면 필연적으로 **실패하는 검증**을 만나게 됩니다. 테스트가 빨강게 뜨는 순간이 바로 디버깅의 출발점입니다.

선수 개념 한 줄 상기. 7장에서 배운 "변경 → 실행 → 테스트 → 통과 확인" 루프(개념 c07)에서, 테스트가 **실패**했을 때 무엇을 해야 하는가를 다루는 것이 이 장입니다. 실패한 테스트는 디버깅의 시작 신호이고, 회귀 테스트는 디버깅의 마무리 도구입니다.

좋은 소식은, 디버깅이야말로 에이전트와 함께할 때 가장 큰 도움을 받는 작업이라는 점입니다. 에이전트는 에러 메시지를 읽고, 스택 트레이스가 가리키는 파일을 직접 열어 보고, 관련 코드를 추적해 원인을 짚어 줍니다. 하지만 여기에는 함정이 있습니다. 신입이 가장 흔히 저지르는 실수는 막연하게 "에러 나는데 고쳐 줘"라고 말하는 것입니다. 정보가 부족하면 에이전트도 추측에 의존하게 되고, 추측은 엉뚱한 수정으로 이어집니다. 이 장의 핵심은 **에이전트가 정확히 진단할 수 있도록 정보를 제대로 건네는 기술과, 고친 뒤 그것이 진짜 고쳐졌는지 증명하는 절차**입니다.

11.2 에러 메시지·스택 트레이스를 통째로 전달하기

디버깅의 첫 단추는 정보 전달입니다. 에이전트는 여러분의 화면에 무엇이 났는지 직접 보지 못합니다. 여러분이 건네는 만큼만 압니다. 그래서 **에러 메시지와 스택 트레이스를 자르지 말고 통째로** 주는 것이 디버깅의 가장 기본적이면서도 가장 자주 빠뜨리는 원칙입니다.

스택 트레이스(stack trace, 호출 추적)는 에러가 발생한 순간까지 어떤 함수가 어떤 함수를 불렀는지의 경로를 보여 주는 기록입니다. 파이썬은 보통 가장 바깥쪽 호출부터 시작해 마지막 줄에 실제 예외 종류와 메시지를 보여 줍니다. 신입이 흔히 저지르는 실수는 "마지막 한 줄만" 복사해 주는 것입니다. 예를 들어 `KeyError: 'done'` 만 건네면, 에이전트는 그 키가 어느 파일 몇 번째 줄에서, 어떤 호출 경로를 거쳐 터졌는지 알 수 없어 다시 되물어야 합니다.

다음은 잘라서 주는 나쁜 예와, 통째로 주는 좋은 예의 비교입니다.

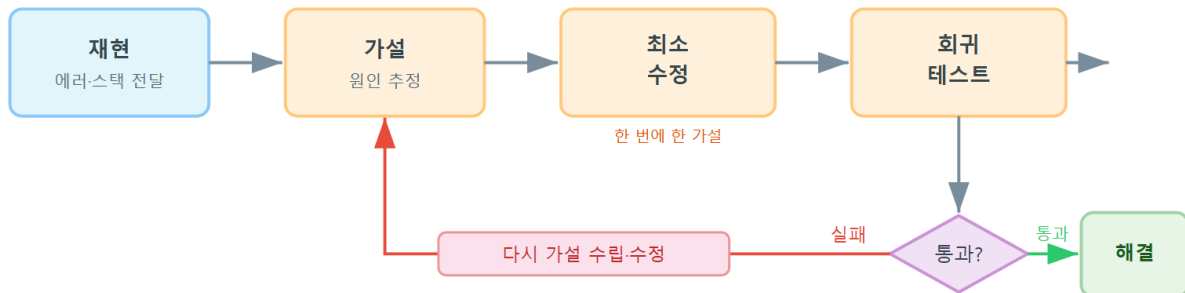
구분	나쁜 전달	좋은 전달
내용	" <code>KeyError</code> 나는데 고쳐 줘"	명령 + 전체 스택 트레이스 + 마지막 예외 줄 전부
위치 정보	없음	파일·줄 번호가 트레이스에 포함
호출 경로	없음	어떤 함수가 어디서 불렀는지 전부
에이전트 행동	어디를 볼지 되물음, 추측 시작	즉시 해당 파일을 열어 추적 시작

에이전트에게 건넬 때는 다음 세 가지를 함께 주는 것이 좋습니다. 첫째, **무엇을 실행했더니** 에러가 났는지(예: `python -m pytest tests/test_services.py`). 둘째, **전체 출력**(스택 트레이스의 처음부터 끝까지). 셋째, **무엇을 기대했는지**(다음 절에서 자세히 다룹니다). 셸에서 실행한 명령과 출력을 한 덩어리로 복사해 붙여 넣으면, 에이전트가 가장 빠르게 일을 시작할 수 있습니다.

실무 팁. 터미널에서 직접 명령을 실행해 에이전트가 출력을 그대로 보게 하면 가장 좋습니다. Claude Code에게 "python -m pytest 를 실행하고 실패하는 테스트의 원인을 진단해 줘"라고 지시하면, 에이전트가 직접 명령을 돌려 전체 출력을 읽고 스택 트레이스가 가리키는 파일을 열어 추적합니다. 사람이 손으로 복사·붙여넣기 하다가 일부를 빠뜨리는 위험도 없습니다.

11.3 동작 원리: 재현 → 가설 → 최소 수정 → 회귀 테스트 루프

재현 → 가설 → 최소 수정 → 회귀 테스트 루프



증상·기대·실제 동작을 구체적으로 전달할수록 정확한 가설을 얻는다

좋은 디버깅은 즉흥적인 시도의 반복이 아니라 **정해진 루프**를 도는 일입니다. 막무가내로 이것 저것 바꿔 보다 우연히 고쳐지는 것은 디버깅이 아니라 도박입니다. 그렇게 고치면 왜 고쳐졌는지 모르고, 같은 버그가 다시 나며, 그 과정에서 멀쩡한 코드를 망가뜨리기 쉽습니다. 프로다운 디버깅은 다음 네 단계를 순서대로 돕니다.

1. **재현(reproduce).** 버그를 **항상 똑같이 일으키는** 방법을 먼저 확보합니다. 가장 좋은 재현 수단은 실패하는 테스트입니다. "가끔 안 돼요"는 디버깅의 적입니다. 같은 입력으로 같은 실패가 반복돼야 원인을 좁히고, 고친 뒤 진짜 고쳐졌는지 확인할 수 있습니다. 실패하는 `pytest` 한 개가 있다면 이미 재현을 손에 쥐는 것입니다.

2. **가설(hypothesis)**. 에러 메시지·스택 트레이스·코드를 근거로 "원인은 아마 여기일 것"이라는 **한 가지** 추정을 세웁니다. 핵심은 한 번에 하나입니다. "키 이름이 틀렸거나, 조건이 반대거나, 타입이 안 맞거나..." 여러 개를 동시에 손대면 무엇이 진짜 원인이었는지 영영 알 수 없습니다.
3. **최소 수정(minimal fix)**. 세운 가설을 검증할 **가장 작은 변경**만 합니다. 한 줄이면 한 줄만 바꿉니다. "온 김에" 다른 곳을 정리하거나 리팩터링하지 않습니다. 수정 범위가 작아야 결과의 인과가 분명하고, 틀렸을 때 되돌리기도 쉽습니다.
4. **회귀 테스트(regression test)**. 재현용 테스트를 다시 돌려 **통과로 바뀌는지** 확인하고, 동시에 **나머지 테스트도 모두 통과하는지**를 확인합니다. 내 수정이 이 버그는 고쳤지만 다른 기능을 깨뜨렸을 수 있기 때문입니다. 통과하면 해결, 실패하면 가설이 틀린 것이니 2 단계로 돌아갑니다.

이 루프의 가치는 **각 단계가 다음 단계의 근거가 된다**는 데 있습니다. 재현이 있어야 가설을 시험할 수 있고, 최소 수정이라야 통과·실패의 원인이 명확하며, 회귀 테스트가 있어야 "정말 고쳤다"고 말할 자격이 생깁니다.

에이전트는 이 루프의 어디를 돕나요? 에이전트는 모든 단계를 함께 돕습니다. 재현 단계에서는 실패를 일으키는 테스트를 작성하고, 가설 단계에서는 스택 트레이스가 가리키는 코드를 읽어 원인을 설명하며, 최소 수정 단계에서는 작은 변경을 diff로 제안하고, 회귀 단계에서는 `pytest`를 다시 돌려 결과를 보여 줍니다. 여러분이 할 일은 **루프를 통제하는 것**입니다. "한 번에 한 가설만", "고쳤으면 전체 테스트를 돌려 확인"이라는 규율을 잡아 주는 것이 신입의 몫입니다.

11.4 worked example: 실패한 pytest 메시지로 원인 진단·수정

이제 루프를 실제로 한 바퀴 돌려 봅시다. 여러분이 팀에서 받은 FastAPI 기반 할 일(tasks) 서비스에 작은 요약 기능이 있다고 합시다. 할 일 목록을 받아 전체·완료·미완료 개수를 돌려주는 함수입니다.

다음은 `team-tasks-api`에 새로 추가한 요약 헬퍼 `services.py`의 코드입니다(할 일을 dict로 받는 작은 순수 함수). 어딘가에 버그가 숨어 있습니다.

```
# services.py
def summarize_tasks(tasks: list[dict]) -> dict:
    """할 일 목록을 받아 전체·완료·미완료 개수를 요약합니다."""
    done = [t for t in tasks if t["done"]]
    pending = len(tasks) - len(done)
    return {
        "total": len(tasks),
        "done": len(done),
        "pending": pending,
    }
```

그리고 팀이 함께 쓰는 테스트가 `tests/test_services.py` 에 있습니다.

```
# tests/test_services.py
from services import summarize_tasks

def test_summarize_counts():
    tasks = [
        {"id": 1, "title": "배포", "done": True},
        {"id": 2, "title": "리뷰", "done": False},
        {"id": 3, "title": "문서", "done": True},
    ]
    result = summarize_tasks(tasks)
    assert result["done"] == 2
    assert result["pending"] == 1
```

1단계 재현. 먼저 테스트를 돌려 실패를 확인합니다. 에이전트에게 "`tests/test_services.py` 를 pytest로 실행하고 결과를 보여 줘"라고 지시하면 다음과 같은 출력을 받습니다.

```
$ python -m pytest tests/test_services.py -q
F [100%]
===== FAILURES =====
_____ test_summarize_counts _____

    def test_summarize_counts():
        tasks = [
            {"id": 1, "title": "배포", "done": True},
            {"id": 2, "title": "리뷰", "done": False},
            {"id": 3, "title": "문서", "done": True},
        ]
        result = summarize_tasks(tasks)
>       assert result["done"] == 2
E       assert 1 == 2

tests/test_services.py:11: AssertionError
===== short test summary info =====
FAILED tests/test_services.py::test_summarize_counts - assert 1 == 2
1 failed in 0.04s
```

이 출력이 곧 재현입니다. 같은 명령으로 언제나 같은 실패가 납니다. 이제 이 **전체 출력을 통째로** 에이전트에게 줍니다(11.2의 원칙). 잘라서 `assert 1 == 2` 만 주지 않습니다.

2단계 가설. 출력을 읽어 봅니다. `result["done"]` 이 2가 아니라 1이 나왔습니다. 완료 처리한 할 일은 배포, 문서 두 개인데 1만 셧습니다. 거꾸로 미완료(리뷰)는 한 개입니다. 즉 함수가 **완료와 미완료를 거꾸로 세고 있다**는 가설이 섭니다. 코드를 보면 원인이 보입니다.

```
done = [t for t in tasks if not t["done"]]
```

`done` 이라는 변수에 담으려는 것은 **완료된** 할 일인데, 조건에 `not` 이 붙어 **미완료**를 담고 있습니다. 가설: "`not` 이 잘못 들어가 완료/미완료가 뒤바뀌었다."

3단계 최소 수정. 가설을 검증하는 가장 작은 변경은 `not` 한 단어를 지우는 것입니다. 에이전트에게 "`summarize_tasks` 에서 완료/미완료가 뒤바뀐 것 같다. `done` 리스트의 조건을 최소한으로 고쳐 줘"라고 지시합니다. 에이전트는 다음 diff를 제안합니다.

```
# services.py (수정 후)
def summarize_tasks(tasks: list[dict]) -> dict:
    """할 일 목록을 받아 전체·완료·미완료 개수를 요약합니다."""
    done = [t for t in tasks if t["done"]] # not 제거
    pending = len(tasks) - len(done)
    return {
        "total": len(tasks),
        "done": len(done),
        "pending": pending,
    }
```

- `if t["done"]`: 조건에서 `not` 만 제거했습니다. 이제 `done` 리스트에는 `done` 이 참인(완료된) 할 일만 담깁니다.
- **나머지는 손대지 않음**: `pending` 계산식·반환 구조는 그대로 둡니다. 가설은 "완료/미완료가 뒤바뀜" 하나뿐이므로 수정도 한 곳뿐입니다. 이것이 최소 수정 원칙입니다.

4단계 회귀 테스트. 같은 명령으로 테스트를 다시 돌립니다.

```
$ python -m pytest tests/test_services.py -q
.
1 passed in 0.03s
```

[100%]

통과로 바뀌었습니다. 재현용 테스트가 빨강에서 초록으로 바뀐 것이 "고쳤다"의 증거입니다. 여기에 더해 저장소 전체 테스트(`python -m pytest`)도 돌려, 이 수정이 다른 기능을 깨지 않았는지까지 확인하면 한 바퀴가 완성됩니다. 이것이 디버깅 루프의 모범적인 한 회전입니다.

11.5 증상·기대·실제 동작을 구체적으로 진술하기

에이전트에게 디버깅을 부탁할 때 가장 큰 차이를 만드는 것은 **진술의 구체성**입니다. "안 돼요", "고쳐 줘"는 정보가 거의 없습니다. 좋은 디버깅 요청은 다음 세 가지를 분명히 담습니다.

- **증상(symptom)**: 무엇이 어떻게 잘못됐나. "POST `/tasks` 에 요청하면 422가 돌아온다."
- **기대(expected)**: 원래 어떻게 동작해야 하나. "201과 함께 생성된 할 일이 돌아와야 한다."
- **실제(actual)**: 지금 실제로 무슨 일이 일어나나. "응답 본문에 `priority` 필드가 필수라는 검증 에러가 담겨 온다."

이 세 가지는 사실 의사에게 증상을 설명하는 것과 같습니다. "아파요"가 아니라 "어제부터 오른쪽 아랫배가, 누르면 더, 열은 없이"라고 말해야 정확한 진단이 나옵니다. 코드도 똑같습니다.

다음은 같은 버그를 두 가지 방식으로 전달한 비교입니다.

구분	막연한 요청	구체적 요청(증상·기대·실제)
말	"엔드포인트가 안 됩니다. 고쳐 주세요."	"POST <code>/tasks</code> 에 <code>{\"title\": \"배포\"}</code> 를 보내면(증상) 422가 난다. 201로 생성되길 기대(기대)했는데, 응답에 <code>priority</code> 필드가 필수라는 검증 에러가 온다(실제). 전체 응답 본문은 아래와 같다."
에이전트 행동	어디가 안 되는지 되묻거나, 엉뚱한 곳을 추측	모델 정의와 핸들러를 바로 열어 <code>priority</code> 필수 여부를 확인
결과	왕복이 늘어남	한 번에 원인 지점으로 직행

한 문장 틀. 디버깅 요청을 이 틀에 맞춰 적는 습관을 들이세요. "[무엇을 했더니] [실제 무슨 일] 이 일어난다. [기대한 동작]이어야 한다. 전체 메시지는 다음과 같다: [전체 출력]." 이 틀 하나면 신입의 디버깅 요청 품질이 단번에 올라갑니다.

11.6 worked example: FastAPI 422 검증 에러를 최소 수정

이번에는 FastAPI에서 신입이 가장 자주 만나는 422 검증 에러를 루프로 잡아 봅니다. 422 Unprocessable Entity는 요청은 도달했지만 본문이 모델이 요구하는 형식에 맞지 않을 때 FastAPI가 자동으로 돌려주는 응답입니다. 즉 **서버 버그가 아니라 데이터 계약 불일치** 신호인 경우가 많습니다.

team-tasks-api 에서 작업을 우선순위로 다루려고, models.py 의 TaskCreate 에 **우선순위 (priority) 필드를 새로 추가**했다고 합시다. 생성 엔드포인트는 1장에서 본 대로 routers/tasks.py 의 POST /tasks (store.create_task 에 위임)에 있습니다.

```
# models.py — TaskCreate 에 priority 필드를 새로 추가한 변경
class TaskCreate(BaseModel):
    """새 할 일 생성 요청 모델"""
    title: str
    priority: int # 새로 추가 — 기본값을 빠뜨렸다
```

그리고 신입이 새로 추가한 테스트가 있습니다. 제목만 주면 기본 우선순위로 할 일이 만들어지길 기대합니다(테스트는 1장 저장소의 관례대로 from main import app 으로 앱을 가져옵니다).

```
# tests/test_tasks_priority.py
import sys
import os

sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))

from fastapi.testclient import TestClient
from main import app

client = TestClient(app)

def test_create_task_minimal():
    response = client.post("/tasks", json={"title": "배포"})
    assert response.status_code == 201
```

1단계 재현. 테스트를 돌립니다.

```
$ python -m pytest tests/test_tasks_priority.py -q
F [100%]
===== FAILURES =====
_____ test_create_task_minimal _____

    def test_create_task_minimal():
        response = client.post("/tasks", json={"title": "배포"})
>       assert response.status_code == 201
E       assert 422 == 201
E       + where 422 = <Response [422]>.status_code

tests/test_tasks_priority.py:14: AssertionError
===== short test summary info =====
FAILED tests/test_tasks_priority.py::test_create_task_minimal - assert 422 == 201
1 failed in 0.12s
```

201을 기대했는데 422가 왔습니다. **증상·기대·실제**로 정리합니다. 증상: 제목만 보내면 422가 난다. 기대: 201로 생성된다. 실제: 검증 에러가 돌아온다. 그런데 422 응답 본문에는 더 정확한 단서가 들어 있습니다. 에이전트에게 "이 요청의 422 응답 본문(`response.json()`)을 보여 줘"라고 하면 다음을 확인합니다.

```
{
  "detail": [
    {
      "type": "missing",
      "loc": ["body", "priority"],
      "msg": "Field required",
      "input": {"title": "배포"}
    }
  ]
}
```

2단계 가설. 이 본문이 원인을 명확히 가리킵니다. `loc` 가 `["body", "priority"]`, `msg` 가 `Field required`, `type` 이 `missing` 입니다. 즉 **요청 본문에 priority가 없는데, 모델이 그것을 필수로 요구하고 있습니다.** `TaskCreate` 를 보면 `priority: int` 라서 기본값이 없어 필수 필드입니다. 가설: "priority에 기본값이 없어 제목만 보내면 검증에 걸린다. 우선순위를 선택 항목으로 만들면 된다."

422 응답 본문의 세 열쇠를 읽는 법을 정리해 둡니다.

필드	의미	이 예에서
type	검증 실패 종류	missing (필드 없음)
loc	문제가 난 위치 경로	["body", "priority"] (본문의 priority)
msg	사람이 읽을 설명	Field required (필수 필드)

3단계 최소 수정. 가설을 검증하는 최소 변경은 `priority`에 기본값을 주는 것입니다. 핸들러·반환 구조·다른 필드는 건드리지 않습니다.

```
# models.py (수정 후 — 모델만 변경)
class TaskCreate(BaseModel):
    """새 할 일 생성 요청 모델"""
    title: str
    priority: int = 3 # 기본값 부여 → 선택 필드가 됨
```

- `priority: int = 3`: 기본값 3을 주어 `priority`를 선택 필드로 바꿨습니다. 클라이언트가 `priority`를 보내지 않으면 3이 채워지고, 보내면 그 값이 쓰입니다.
- 타입은 그대로 `int`: 우선순위는 여전히 정수여야 하므로 타입 검증은 유지합니다. 기본값만 더했을 뿐, 검증 규칙을 느슨하게 풀지 않았습니다.
- 핸들러 무수정: `create_task` 함수와 응답 구조는 손대지 않았습니다. 가설이 "필수 여부" 하나였으므로 수정도 한 줄입니다.

4단계 회귀 테스트. 다시 돌립니다.

```
$ python -m pytest tests/test_tasks_priority.py -q
.
1 passed in 0.10s [100%]
```

통과했습니다. 여기서 한 걸음 더 나아가, 우선순위를 명시한 요청도 여전히 잘 동작하는지까지 확인하는 회귀 테스트를 추가하면(다음 절) 수정이 기존 동작을 깨지 않았음을 증명할 수 있습니다.

주의: 422를 무조건 모델 완화로 풀지 마세요. 이 예에서는 "제목만으로 생성"이 정당한 요구라 기본값이 맞는 해법이었습니다. 하지만 422가 진짜로 클라이언트가 필수 데이터를 빠뜨린 경우라면, 모델을 느슨하게 푸는 것은 버그를 가리는 잘못된 수정입니다. 그때는 요청 쪽을 고쳐야 합니다. 422를 만나면 "이 필드가 정말 선택이어도 되는가"를 먼저 판단하세요.

11.7 최소 수정 원칙: 한 번에 한 가설

앞의 두 예제에서 반복된 규율이 있습니다. 바로 **한 번에 한 가설, 한 번에 최소 변경**입니다. 이 원칙이 왜 그토록 중요한지 짚어 봅니다.

버그를 만나면 머릿속에 동시에 여러 의심이 떠오릅니다. "키 이름이 틀렸나? 조건이 반덴가? 타입이 안 맞나? 저 함수도 좀 이상한데 같이 고칠까?" 이때 욕심을 내 한꺼번에 여러 곳을 손대면 두 가지 문제가 생깁니다.

1. **인과가 흐려진다.** 세 곳을 고치고 테스트가 통과하면, 그중 무엇이 진짜 원인이었는지 알 수 없습니다. 다음에 비슷한 버그를 만나도 배운 것이 없습니다.
2. **새 버그를 심는다.** "온 김에" 손댄 곳이 멀쩡했다면, 그 변경이 오히려 새로운 문제를 만들 수 있습니다. 그러면 원래 버그와 새 버그가 뒤엉켜 디버깅이 훨씬 어려워집니다.

그래서 가설은 하나씩 세우고, 수정은 그 가설을 검증할 **가장 작은 변경**으로 제한합니다. 통과하면 그 가설이 맞았던 것이고, 실패하면 그 가설이 틀렸으니 변경을 되돌리고 다음 가설로 넘어 갑니다. 이렇게 하면 매 회전이 명확한 정보를 남깁니다.

에이전트에게 규율을 거는 지시. 에이전트는 때로 여러 개선을 한꺼번에 제안합니다. 이때 "지금 은 이 버그만 최소한으로 고치고, 다른 개선 제안은 별도로 정리만 해 줘"라고 명시하세요. 6장에서 배운 **플랜 모드**로 먼저 계획을 받아 "이 수정 한 줄만"으로 범위를 좁히는 것도 좋은 방법입니다. 리팩터링과 버그 수정은 섞지 않습니다.

11.8 회귀 테스트로 수정 검증·재발 방지

디버깅 루프의 마지막 단계이자, 신입이 가장 자주 빠뜨리는 단계가 **회귀 테스트**입니다. 회귀 (regression)란 한 곳을 고쳤더니 멀쩡하던 다른 곳이 망가지는 현상을 말합니다. 회귀 테스트는 이를 잡아내고 같은 버그의 재발을 막는 안전망입니다. 회귀 테스트는 두 가지 일을 합니다.

첫째, **이 수정이 정말 고쳤는지** 증명합니다. 재현용으로 쓴 실패 테스트가 통과로 바뀌면, "고친 것 같다"가 아니라 "고쳤다"가 됩니다. 둘째, **다른 곳을 깨지 않았는지** 확인합니다. 저장소 전체 테스트를 돌려 모두 통과하면, 내 수정이 의도한 곳만 바꿨다는 증거가 됩니다.

여기에 한 가지를 더합니다. 방금 잡은 버그가 **다시는 같은 모습으로 돌아오지 못하게** 못 박는 테스트를 추가하는 것입니다. 11.6의 422 버그를 예로 들면, "제목만으로도 생성된다"와 "우선순위를 줘도 정상 생성된다"를 각각 테스트로 남깁니다(응답 모델은 1장의 `Task` 라 `priority` 를 본문에 포함하지 않으므로, 여기서는 생성 성공 여부만 검증합니다).


```
# tests/test_tasks_priority.py (회귀 테스트 보강)
def test_create_task_minimal():
    # 제목만 보내도 (기본 우선순위로) 생성된다 — 이번에 고친 버그를 못 박음
    response = client.post("/tasks", json={"title": "배포"})
    assert response.status_code == 201

def test_create_task_with_priority():
    # 우선순위를 명시해도 정상 생성된다 — 기존 동작이 깨지지 않았는지 확인
    response = client.post("/tasks", json={"title": "리뷰", "priority": 1})
    assert response.status_code == 201
```

이렇게 해 두면, 나중에 누군가 `priority` 를 다시 필수로 바꾸는 변경을 하면 첫 번째 테스트가 빨갛게 떠 즉시 경고합니다. 버그를 한 번 잡은 경험이 영구적인 안전망으로 남는 것입니다. 이것이 디버깅을 단순한 "땀질"에서 "재발 방지"로 끌어올리는 핵심 습관입니다.

왜 버그를 잡을 때마다 테스트를 남기나요? 버그가 났다는 것은 그 경우를 검증하는 테스트가 없었다는 뜻이기도 합니다. 잡은 김에 그 빈틈을 테스트로 메우면, 코드베이스의 안전망이 버그를 만날 때마다 촘촘해집니다. 7장에서 배운 "테스트로 완료를 증명한다"가 디버깅에서는 "테스트로 재발을 막는다"로 이어집니다.

11.9 즉시 연습(2종 1세트)과 단원 미니 프로젝트

이 단원은 단일 챕터이므로, 즉시 연습과 단원 미니 프로젝트를 여기서 함께 진행합니다. 먼저 모범 예시를 보고, 그다음 빈 템플릿을 직접 채운 뒤 자가체크로 점검합니다.

연습 1 (모범): 디버깅 진술 작성

다음은 잘 작성된 디버깅 요청의 모범입니다. 증상·기대·실제·전체 출력이 모두 담겨 있습니다.

GET `/tasks?status=done` 에 요청하면(증상) 빈 목록 `[]` 이 돌아온다. 완료된 할 일 2건이 돌아와야(기대) 하는데, 실제로는 완료 항목이 하나도 안 잡힌다(실제). 아래는 실패한 pytest 전체 출력이다.

```
```text
```

```
assert len(response.json()) == 2
```

```
E assert 0 == 2 tests/test_filter.py:14: AssertionError ``
```

재현은 `python -m pytest tests/test_filter.py` 다. 한 번에 한 가설로, 필터 조건부터 최소 수정으로 확인해 줘.

## 연습 1 (직접 해보기): 실제 실패 메시지로 디버깅 진술 쓰기

종이에 빈칸을 채우는 대신, 1장에서 준비한 **실제** `team-tasks-api` 저장소에 작은 실패를 일부러 만들어 진짜 pytest 메시지를 뽑고, 그 **실제** 출력으로 디버깅 진술을 완성합니다. 추측한 메시지가 아니라 화면에 찍힌 그대로를 옮기는 것이 핵심입니다.

**과제 — 실패를 실제로 재현하고, 그 출력으로 진술을 채워라.**

`team-tasks-api`의 `routers/tasks.py`에서 `get_task`가 없는 할 일에 던지는 `HTTPException`의 상태 코드를 404에서 400으로 잠깐 바꿔, 일부러 실패를 만드세요(기존 테스트에 없는 작업 조회는 404를 기대하는 단언이 있다면 빨각게 떨어집니다). 그다음 `python -m pytest -q`를 돌려 **화면에 실제로 나타난 실패 메시지를 그대로 옮겨 적으세요.**

[실제 실패 출력] `python -m pytest -q`에 찍힌 실패 테스트와 단언(화면 그대로):

이제 위 **실제** 출력을 근거로 디버깅 진술을 채웁니다(빈칸에 추측이 아니라 방금 본 사실을 적습니다).

[증상] (어느 엔드포인트/명령에서 무엇이 일어났는가 — 실제로 본 것)  
 [기대] (원래 어떤 상태 코드/결과여야 하는가)  
 [실제] (지금 무엇이 일어나는가 — 위 출력 기준)  
 [재현] (항상 재현되는 명령: `python -m pytest -q` 등)  
 [전체 출력] (위에 옮긴 실제 pytest 출력을 그대로)  
 [요청] 한 번에 한 가설로, (어디부터) 최소 수정으로 확인해 줘.

작성한 진술을 에이전트에게 실제로 주고, 진단.최소 수정으로 다시 `python -m pytest -q`가 통과(7 passed)로 돌아오는지 확인하세요(바꿨던 상태 코드를 원래대로 되돌리는 것이 정답입니다).

[수정 후 검증] `python -m pytest -q` 실제 출력(초록으로 돌아왔는가?):

**자가체크 — 머리로가 아니라 손으로 했는지 확인하세요.**

- [ ] 일부러 만든 실패를 `python -m pytest -q`로 **실제로 재현**하고 출력을 화면에서 봤다.

- [ ] [전체 출력]에 적은 것이 추측이 아니라 화면에 찍힌 실제 pytest 메시지다.
- [ ] 증상·기대·실제를 각각 구체적으로, 위 실제 출력을 근거로 적었다.
- [ ] "한 번에 한 가설", "최소 수정" 규율을 요청에 명시했다.
- [ ] 수정 뒤 `python -m pytest -q`가 통과(7 passed)로 돌아온 것을 직접 확인했다.

## 단원 미니 프로젝트 (모범): 버그를 에이전트와 함께 잡기

이 단원에서 배운 개념만으로 완성하는 작은 산출물입니다. 이번에는 1장에서 준비한 **실제 team-tasks-api 저장소**에 일부러 버그를 심어, 그 저장소의 진짜 테스트(`python -m pytest -q`)로 재현하고 디버깅 루프로 잡습니다. 마일스톤을 증분으로 진행합니다.

- **M1 재현 확보.** 실패하는 테스트를 돌려 같은 실패가 반복됨을 확인하고, 전체 출력을 확보합니다. (적용 개념: 재현, 에러 통째 전달)
- **M2 진술·진단.** 증상·기대·실제를 한 문장 틀로 적어 에이전트에게 전체 출력과 함께 건네고, 원인 가설 하나를 받습니다. (적용 개념: 구체적 진술, 가설)
- **M3 최소 수정.** 가설을 검증할 가장 작은 변경 하나만 적용합니다. 리팩터링·다른 개선은 섞지 않습니다. (적용 개념: 최소 수정, 한 번에 한 가설)
- **M4 회귀 검증·재발 방지.** 재현 테스트가 통과로 바뀌는지 + 전체 테스트가 모두 통과하는지 확인하고, 이 버그를 못 박는 회귀 테스트를 추가합니다. (적용 개념: 회귀 테스트)

각 마일스톤 끝에서 다음 체크포인트(단위 테스트 통과)를 확인합니다. M1은 "실패가 재현됨", M3은 "재현 테스트가 통과로 바뀜", M4는 "전체 테스트 초록 + 새 회귀 테스트 추가됨"입니다.

## 단원 미니 프로젝트 (학생 작성형 빈 스타터)

**스타터.** team-tasks-api 저장소의 `store.py`에 아래처럼 **일부러 버그를 심어** 시작합니다. 새로 만든 할 일의 상태(`status`)를 `todo`가 아니라 `done`으로 잘못 설정하는 한 줄입니다. 이렇게 하면 1장에서 받은 기존 테스트(`tests/test_tasks.py`)가 빨강게 떨어지므로, **추가 테스트 없이도** 실패를 재현할 수 있습니다.

```
store.py — create_task 에 일부러 심은 버그
def create_task(data: TaskCreate) -> Task:
 """새 할 일을 저장하고 반환"""
 global _next_id
 task = Task(id=_next_id, title=data.title, status=TaskStatus.done) # 버그: todo 여야 한다
 _db[_next_id] = task
 _next_id += 1
 return task
```

기존 테스트 `tests/test_tasks.py`에는 "생성 직후 상태는 `todo`"를 검증하는 단언이 이미 들어 있습니다(예: `assert items[0]["status"] == "todo"`). 이 버그를 심으면 그 단언이 실패합니다.

**직접 실행·기록할 것** (추측을 적지 말고, `team-tasks-api`에서 실제로 돌린 뒤 화면 출력을 그대로 옮깁니다)

1. [M1 재현] 위 버그를 심은 뒤 `python -m pytest -q`를 실제로 돌려, 어떤 테스트가 어떤 단언에서 실패하는지 전체 출력을 그대로 붙입니다:
2. [M2 진술·진단] 증상·기대·실제를 한 문장 틀로 적고, 에이전트에게 전체 출력과 함께 건넬 때 받은 원인 가설을 그대로 적습니다:
3. [M3 최소 수정] 실제로 적용한 최소 수정 한 줄(힌트: `status=` 인자)과, 에이전트가 제시한 diff의 + 줄을 그대로 적습니다:

1. `python -m pytest -q`를 다시 돌려 **7 passed**(빨강 → 초록)로 바뀌는지 화면 출력으로 확인하고, 그 출력을 위 3번 아래에 적습니다.
2. 재발을 막기 위해, "생성 직후 상태가 `todo`"임을 명시적으로 검증하는 회귀 테스트가 이미 있는지 확인하고(없다면 보강), 전체 통과를 확인합니다.

## 자가체크

- [ ] 기대값을 코드 실행 전에 스스로 정했는가
- [ ] 한 번에 한 가설만 세우고 그 가설을 검증할 최소 변경만 했는가
- [ ] 재현 테스트가 빨강 → 초록으로 바뀐 것을 확인했는가
- [ ] 전체 테스트가 모두 통과하는지 확인했는가
- [ ] 잡은 버그를 못 박는 회귀 테스트를 추가했는가

## 11.10 흔한 오류와 오개념

**오개념 1: "그냥 고쳐 줘"라고만 해도 에이전트가 알아서 잘 고친다.** 막연한 요청은 에이전트를 추측에 의존하게 만들고, 추측은 엉뚱한 수정으로 이어집니다. 증상·기대·실제를 구체적으로 주지 않으면, 에이전트는 무엇이 "올바른 동작"인지조차 모릅니다. 11.5의 한 문장 틀("무엇을 했더니 무슨 일이 일어난다, 무엇을 기대한다, 전체 출력은 이렇다")로 적는 습관을 들이세요. 정보를 주는 만큼 진단이 정확해집니다.

**오개념 2: 에러 메시지는 핵심 한 줄만 주면 된다.** 스택 트레이스를 마지막 줄(`KeyError: 'done'`)만 잘라 주면, 에이전트는 그 에러가 어느 파일 몇 번째 줄에서, 어떤 호출 경로로 터졌는지 알 수 없습니다. 결국 다시 되문거나 추측합니다. 스택 트레이스는 처음부터 끝까지 통째로 주세요. 가장 좋은 방법은 에이전트가 직접 명령을 실행해 전체 출력을 읽게 하는 것입니다.

**오개념 3: 테스트가 초록이면 디버깅은 끝이다(회귀는 안 봐도 된다).** 방금 고친 그 테스트만 통과하는 것으로는 부족합니다. 내 수정이 다른 기능을 깨뜨렸을 수 있기 때문입니다. 반드시 저장소 **전체 테스트**를 돌려 모두 통과하는지 확인하고, 잡은 버그를 못 박는 회귀 테스트를 추가하세요. 회귀 확인 없이 "고쳤다"고 말하는 것은 절반만 한 것입니다.

**오개념 4: 의심되는 곳을 한꺼번에 다 고치면 빨리 끝난다.** 여러 곳을 동시에 손대면 무엇이 진짜 원인이었는지 알 수 없고, "온 김에" 고친 곳에서 새 버그가 생기기 쉽습니다. 한 번에 한 가설, 그 가설을 검증할 최소 변경만 적용하세요. 통과하면 그 가설이 맞았던 것이고, 실패하면 되돌리고 다음 가설로 갑니다. 느려 보이지만 결국 가장 빠르고 안전한 길입니다.

---

## 11.11 정리

---

이 장에서 우리는 에이전트와 함께 디버깅하는 법을 배웠습니다. 핵심은 세 가지입니다. 첫째, **정보를 제대로 건넨다** — 에러 메시지·스택 트레이스를 통째로, 증상·기대·실제를 구체적으로. 둘째, **루프를 통제한다** — 재현 → 가설(한 번에 하나) → 최소 수정 → 회귀 테스트. 셋째, **재발을 막는다** — 잡은 버그를 회귀 테스트로 못 박는다.

다음 장에서는 온보딩 이후 더 나아갈 길로, 서브에이전트와 MCP를 맞춥니다. 디버깅 같은 경계가 분명한 작업을 별도 컨텍스트의 서브에이전트에 위임하고, 외부 도구·데이터를 표준 방식으로 연결하는 방법을 가볍게 살펴봅니다.



# 12장 더 나아가기(맛보기): 서브에이전트와 MCP

이 장은 단원 11 "더 나아가기(맛보기): 서브에이전트와 MCP"의 단일 챕터입니다. 여기서는 개념 **자동화 맛보기: 서브에이전트와 MCP(a taste of automation: subagents and MCP)** 하나를 다룹니다. 다만 이 개념 안에는 성격이 다른 두 도구, **서브에이전트(subagent)**와 **MCP(Model Context Protocol)**가 들어 있으므로, 각각을 정의 → 동작 → 예제 → 오개념의 순서로 충실히 가르칩니다. 제목에 "맛보기"라고 적은 이유는, 깊은 설계와 운영은 온보딩 다음 단계의 몫으로 남기고 여기서는 "여기서 더 갈 수 있다"를 정확히 알게 하는 데 초점을 두기 때문입니다.

## 이 장의 학습 목표

- 서브에이전트가 컨텍스트 격리·병렬 처리에 왜 유리한지 위임에 적합한 작업 예를 들어 설명할 수 있습니다.
- `.claude/agents`에 역할·도구를 지정한 서브에이전트 정의 파일 1개를 작성해 작업을 위임할 수 있습니다.
- MCP가 외부 도구·데이터를 어떻게 표준 프로토콜로 연결하는지 설명할 수 있습니다.
- `.mcp.json`에 서버 1개를 등록하되 권한은 동일 체계로 통제하고 토큰은 환경으로 주입하는 비밀 관리 원칙에 맞게 작성할 수 있습니다.

## 12.1 도입: 온보딩 이후, 어디로 더 갈 수 있나

여기까지 오느라 수고했습니다. 1장에서 Claude Code를 설치한 뒤, 에이전트 루프를 이해하고(2장), 명확히 지시하고(3장), 낯선 코드를 파악하고(4장), 안전하게 변경하고(5·6장), 실행·테스트로 검증하고(7장), 팀 규약을 주입하고(8장), 첫 커밋과 첫 PR을 올리고(9장), 리뷰에 대응하고(10장), 막힘을 디버깅으로 풀었습니다(11장). 이제 여러분은 한 사람의 팀원으로서 첫 기능을 끝까지 끌고 갈 수 있습니다.

그런데 익숙해지면 새로운 답답함이 보이기 시작합니다. 긴 작업을 하다 보면 메인 대화에 테스트 로그·리뷰 코멘트·조사 메모가 뒤섞여 맥락이 어지러워집니다. 또 팀이 쓰는 GitHub 이슈나

사내 문서를 매번 손으로 복사해 에이전트에게 붙여 넣는 일이 반복됩니다. 이 두 가지 답답함을 푸는 다음 단계가 바로 이 장의 두 도구입니다.

**선수 개념 한 줄 상기.** 이 장은 9장의 git 워크플로(개념 c09)를 선수로 둡니다. 서브에이전트에 "테스트 실행"이나 "리뷰" 같은 작업을 위임한다는 발상은, 여러분이 이미 그 작업들(검증·커밋·PR·리뷰 대응)을 손으로 해 봤기에 이해됩니다. 해 본 일을 자동화로 넘기는 것이지, 모르는 일을 떠넘기는 것이 아닙니다.

- **서브에이전트**는 "이 경계가 분명한 작업은 별도의 일꾼에게 맡기자"는 발상입니다. 테스트 실행, 코드 리뷰, 조사처럼 독립적으로 떼어 낼 수 있는 일을 자체 컨텍스트와 도구를 가진 별도 세션에 위임합니다.
- **MCP**는 "외부 서비스를 표준 방식으로 연결하자"는 발상입니다. GitHub·데이터베이스·사내 문서 같은 외부 도구·데이터를 매번 복사해 붙이지 않고, 정해진 프로토콜로 에이전트에 이어 줍니다.

이 장은 맛보기입니다. 두 도구의 정의·동작·기본 사용법과, 신입이 흔히 빠지는 오해를 정확히 짚는 데까지만 갑니다. 본격적인 다중 에이전트 오케스트레이션이나 MCP 서버 직접 개발은 이 책의 범위를 넘어서며, 다음 학습 단계에서 다룰 주제입니다.

## 12.2 서브에이전트란: 독립 컨텍스트·프롬프트·도구

**정의.** 서브에이전트(subagent)란 메인 세션과 분리된, **자체 컨텍스트·시스템 프롬프트·도구 권한을 가진 별도의 에이전트**입니다. `.claude/agents/` 폴더에 마크다운 파일(YAML 프런트매터 + 본문)로 정의하며, 코드 리뷰·테스트 실행·조사처럼 경계가 분명한 작업을 위임받아 독립적으로 수행한 뒤 결과 요약만 메인 세션에 돌려줍니다.

핵심 용어를 한글(영문)으로 정리합니다.

- **컨텍스트(context, 맥락):** 에이전트가 현재 들고 있는 대화·파일·도구 호출 기록의 묶음입니다. 길어질수록 무거워지고 잡음이 섞입니다.
- **컨텍스트 격리(context isolation):** 서브에이전트가 자기만의 별도 컨텍스트를 가지므로, 그 안에서 무슨 일이 오가든 메인 세션의 맥락을 어지럽히지 않는 성질입니다.
- **위임(delegation):** 메인 에이전트가 특정 작업을 서브에이전트에 넘겨 처리하게 하는 것입니다.
- **병렬 처리(parallelism):** 서로 독립적인 작업을 여러 서브에이전트가 동시에 처리하는 것입니다.

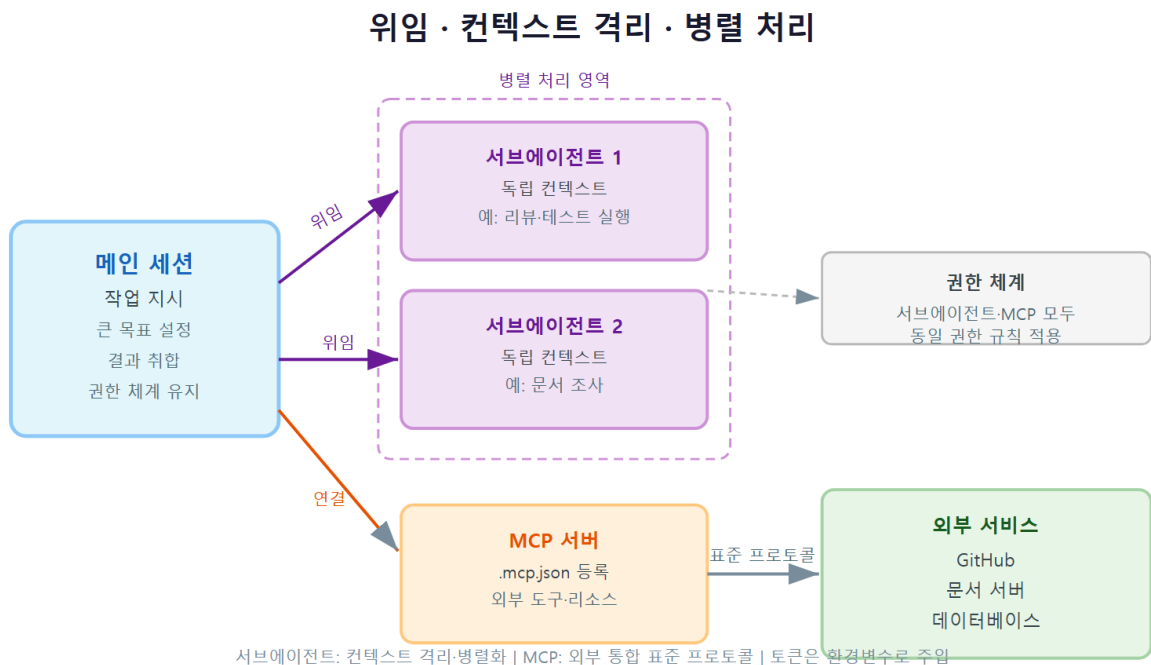
서브에이전트가 일반적인 "프롬프트로 부탁하기"와 다른 점을 표로 못 박습니다.



구분	메인 세션에 그냥 부탁	서브에이전트에 위임
컨텍스트	모든 대화·로그가 한곳에 쌓임	작업별 별도 컨텍스트(격리)
시스템 프롬프트	하나(공통)	작업에 특화된 별도 프롬프트
도구 권한	세션 전체 권한	그 작업에 필요한 도구만 좁혀 부여 가능
결과	과정·로그가 메인에 다 남음	결과 요약만 메인에 돌려줌
동시성	순차 처리	독립 작업은 병렬 가능

쉽게 비유하면, 메인 에이전트는 여러분 자신이고 서브에이전트는 **특정 일을 맡긴 전담 동료**입니다. "테스트만 돌려 결과를 알려 줘", "이 PR만 리뷰해 줘"처럼 명확한 임무를 주면, 그 동료는 자기 책상(별도 컨텍스트)에서 일하고 결과만 보고합니다. 여러분의 책상(메인 세션)은 그동안 깔끔하게 유지됩니다.

## 12.3 동작 원리: 위임·컨텍스트 격리·병렬 처리



서브에이전트가 어떻게 동작하는지 단계로 따라가 봅니다.

1. **정의.** `.claude/agents/이름.md` 파일에 서브에이전트를 정의합니다. YAML 프론트매터에 이름(`name`), 설명(`description`), 사용할 도구(`tools`)를 적고, 본문에 그 에이전트의 역할을 지시하는 시스템 프롬프트를 씁니다.
2. **위임 판단.** 메인 에이전트는 작업이 어떤 서브에이전트의 `description`에 맞는지 보고 위임할지 결정합니다. 여러분이 명시적으로 "test-runner 서브에이전트로 테스트를 돌려줘"라고 호출할 수도 있습니다.
3. **격리된 실행.** 서브에이전트는 **자기만의 빈 컨텍스트**에서 시작합니다. 메인 세션의 긴 대화를 물려받지 않으므로, 메인은 필요한 맥락(예: "어떤 테스트 파일을 돌릴지")을 명시해 넘겨야 합니다. 서브에이전트는 부여받은 도구만 써서 작업합니다.
4. **결과 반환.** 서브에이전트는 작업을 마치면 **결과 요약**만 메인 세션에 돌려줍니다. 중간 과정의 수많은 도구 호출·로그는 서브에이전트의 컨텍스트에 남고 메인을 더럽히지 않습니다.
5. **병렬화(선택).** 서로 독립적인 작업이라면 여러 서브에이전트를 동시에 돌려 시간을 절약할 수 있습니다.

이 흐름에서 서브에이전트의 두 가지 이점이 나옵니다.

- **컨텍스트 격리의 이점.** 메인 세션이 깨끗하게 유지됩니다. 테스트 로그 수백 줄이 메인에 쌓이면 이후 작업의 품질이 떨어지지만, 서브에이전트가 그 로그를 자기 안에서 소화하고 "3건 실패, 원인은 X"라는 요약만 돌려주면 메인은 가볍게 유지됩니다.
- **병렬 처리의 이점.** "테스트 실행"과 "보안 점검"처럼 서로 무관한 두 작업을 동시에 진행해 대기 시간을 줄일 수 있습니다.

**언제 서브에이전트가 어울리나요?** 경계가 분명하고, 메인 대화의 세세한 맥락이 없어도 수행할 수 있으며, 결과를 요약으로 받으면 충분한 작업이 좋은 후보입니다. 테스트 실행, 코드 리뷰, 특정 주제 조사가 대표적입니다. 반대로, 메인의 흐름과 긴밀히 얽혀 한 줄씩 협의해야 하는 작업은 굳이 위임하지 않는 편이 낫습니다.

## 12.4 worked example: 테스트 실행 서브에이전트 정의·호출

말로만 들으면 막연하니, 실제로 테스트 실행을 맡는 서브에이전트를 하나 만들어 봅니다. 목표는 "긴 pytest 로그를 메인에 쌓지 않고, 별도 컨텍스트에서 테스트를 돌려 결과 요약만 받기"입니다.

서브에이전트는 프로젝트 루트의 `.claude/agents/` 폴더에 마크다운 파일로 정의합니다. 파일 위쪽의 YAML 프런트매터에 메타데이터를 적고, 아래 본문에 역할을 지시합니다. 다음은 `test-runner` 서브에이전트의 정의 파일입니다.

```

name: test-runner
description: pytest를 실행하고 실패한 테스트와 원인을 간결히 요약합니다. 변경 후 테스트 검증이 필요할 때 사용됩니다.
tools: Read, Grep, Bash
model: sonnet

```

당신은 테스트 실행 전담 에이전트입니다. 다음 절차를 따르세요.

1. 지시받은 테스트 대상(파일 또는 전체)을 `python -m pytest` 로 실행합니다.
2. 전체 통과면 "모두 통과(N건)"로 한 줄 보고합니다.
3. 실패가 있으면 실패한 테스트 이름, 핵심 에러 메시지, 추정 원인 위치(파일·줄)만 간결히 정리합니다.
4. 코드를 직접 수정하지 않습니다. 진단과 보고까지만 합니다.

- **name: test-runner** : 서브에이전트의 식별자입니다. 메인 세션에서 이 이름으로 호출합니다.
- **description** : 이 에이전트가 무엇을 하고 **언제 쓰는지**를 적습니다. 메인 에이전트는 이 설명을 보고 위임 여부를 판단하므로, 용도를 구체적으로 적는 것이 중요합니다.
- **tools: Read, Grep, Bash** : 이 작업에 필요한 도구만 부여했습니다. 테스트를 읽고(Read), 검색하고(Grep), 실행(Bash)하면 충분합니다. `Edit·Write` 를 주지 않았으므로 이 에이전트는 코드를 바꿀 수 없습니다(역할을 진단·보고로 좁힘).
- **model: sonnet** : 사용할 모델을 지정합니다. 생각하면 기본 모델을 따릅니다.
- **본문**: YAML 아래 본문은 이 서브에이전트의 시스템 프롬프트입니다. 절차와 금지사항("코드를 직접 수정하지 않습니다")을 명시해 역할을 분명히 했습니다.

이 파일을 저장하면 서브에이전트가 등록됩니다. 메인 세션에서는 다음처럼 호출합니다.

```
> test-runner 서브에이전트로 tests/ 전체를 돌려 결과를 요약해 줘
```

서브에이전트는 자기 컨텍스트에서 `python -m pytest` 를 돌리고, 수백 줄의 로그를 자기 안에서 소화한 뒤, 메인에는 다음과 같은 요약만 돌려줍니다.

```
모두 통과(7건). 실패 없음. 소요 0.3s.
```

만약 실패가 있었다면(예: 11장 미니 프로젝트에서 `store.py` 에 심은 상태 버그가 남아 있다면) 다음처럼 핵심만 보고합니다.

7건 중 2건 실패.

- 실패: tests/test\_tasks.py::TestListTasks::test\_list\_after\_create 외 1건
- 에러: assert 'done' == 'todo'
- 추정 원인: store.py 의 create\_task 가 새 할 일 상태를 todo 가 아닌 done 으로 설정

메인 세션에는 이 요약만 남고, 장황한 pytest 출력은 서브에이전트의 컨텍스트에 머뭅니다. 이것이 컨텍스트 격리의 실제 효과입니다. 메인은 가벼운 상태로 다음 작업(예: 수정)을 이어 갈 수 있습니다.

**맥락은 물려받지 않습니다.** 서브에이전트는 빈 컨텍스트에서 시작하므로, 메인 대화에서 방금 논의한 내용을 알지 못합니다. "방금 그 버그"처럼 메인 맥락에 의존하는 지시는 통하지 않습니다. "tests/test\_tasks\_api.py를 돌려 줘"처럼 **필요한 정보를 명시**해 위임해야 합니다.

## 12.5 정의 파일: YAML 프런트매터 (name·description·tools)

서브에이전트 정의 파일의 형식을 한 번 더 정리합니다. 파일은 두 부분으로 나뉩니다. 위쪽의 **YAML 프런트매터**(세 개의 하이픈 `---` 사이)는 메타데이터이고, 아래쪽 **본문**은 시스템 프롬프트입니다.

필드	필수 여부	역할
name	필수	서브에이전트 식별자(호출에 사용)
description	필수	무엇을 하고 언제 쓰는지 — 위임 판단 근거
tools	선택	부여할 도구 목록(생략 시 기본 도구 상속)
model	선택	사용할 모델(생략 시 기본값)

저장 위치도 두 가지로 나뉩니다. 권한 범위(5장)와 같은 발상입니다.

- **프로젝트 범위** `.claude/agents/` : 그 저장소에 한정되며 git으로 팀과 공유됩니다. 팀 공통 작업(테스트·리뷰)은 여기에 둡니다.
- **사용자 범위** `~/.claude/agents/` : 모든 프로젝트에서 쓸 수 있는 개인 서브에이전트입니다.

**description**을 구체적으로 쓰세요. 메인 에이전트가 "이 작업을 어느 서브에이전트에 맡길까"를 판단하는 근거가 바로 **description**입니다. "테스트 관련"처럼 막연하면 엉뚱하게 위임되거나 위임되지 않습니다. "pytest를 실행하고 실패 원인을 요약합니다. 변경 후 검증이 필요할 때 사용"처럼 **무엇을·언제**를 함께 적으세요. 3장에서 배운 "명확한 지시"의 원칙이 여기에도 그대로 적용됩니다.

## 12.6 MCP란: 외부 통합을 위한 표준 프로토콜

이제 두 번째 도구로 넘어갑니다.

**정의.** MCP(Model Context Protocol, 모델 컨텍스트 프로토콜)란 Claude Code 같은 AI 도구를 GitHub·데이터베이스·문서 같은 **외부 서비스에 있는 표준 프로토콜**입니다. `.mcp.json`에 MCP 서버를 등록하면, 그 서버가 노출한 도구·리소스를 Claude Code가 **기존과 동일한 권한 체계 안에서** 사용할 수 있고, 인증 토큰 같은 비밀은 코드가 아니라 환경 변수로 안전하게 주입합니다.

핵심 용어를 한글(영문)으로 정리합니다.

- **프로토콜(protocol, 표준 규약):** 서로 다른 프로그램이 같은 방식으로 대화하기로 정한 약속입니다. USB 단자처럼, 한 번 표준을 따르면 여러 서비스를 같은 방식으로 꽂을 수 있습니다.
- **MCP 서버(MCP server):** 외부 서비스(예: GitHub) 앞에 서서, 그 기능을 MCP 표준으로 노출하는 중개 프로그램입니다.
- **도구·리소스(tools and resources):** MCP 서버가 노출하는 기능(예: "이슈 목록 가져오기")과 데이터(예: "문서 본문")입니다.

MCP의 가치는 **표준화**에 있습니다. MCP가 없다면 GitHub용 연동, 사내 문서용 연동, 데이터베이스용 연동을 각각 다른 방식으로 만들어야 합니다. MCP는 이 모두를 하나의 규약으로 통일합니다. 그래서 새로운 외부 서비스를 붙이고 싶을 때, 그 서비스의 MCP 서버만 `.mcp.json`에 등록하면 Claude Code가 같은 방식으로 그 도구들을 쓸 수 있습니다.

구분	MCP 없이	MCP로
외부 자료 주입	매번 손으로 복사·붙여넣기	서버 등록 후 에이전트가 직접 조회
연동 방식	서비스마다 제각각	하나의 표준 프로토콜
권한	별도 관리 필요	Claude Code의 동일 권한 체계
확장	통합마다 새로 개발	서버만 추가 등록

## 12.7 worked example: 문서 MCP 서버를 .mcp.json에 등록·참조

팀이 사내 문서를 제공하는 MCP 서버를 운영한다고 합시다. 매번 문서를 복사해 붙이는 대신, 이 서버를 등록해 에이전트가 직접 문서를 참조하게 만들어 봅니다.

MCP 서버는 두 가지 방법으로 등록할 수 있습니다. 하나는 `claude mcp add` 명령으로 등록하는 것이고, 다른 하나는 `.mcp.json` 파일을 직접 작성하는 것입니다. 먼저 명령으로 등록하는 방법입니다.

```
claude mcp add docs --scope project -- npx -y @example/docs-mcp-server
```

`--scope project` 는 이 서버 등록을 프로젝트 범위의 `.mcp.json` 에 기록해 팀과 공유한다는 뜻입니다. 범위는 서브에이전트·권한과 같은 세 종류입니다. 로컬(나만, 현재 프로젝트), 프로젝트(팀 공유, `.mcp.json`), 사용자(내 모든 프로젝트)입니다. 이 명령은 결국 다음과 같은 `.mcp.json` 을 만듭니다(직접 작성해도 동일합니다).

```
{
 "mcpServers": {
 "docs": {
 "command": "npx",
 "args": ["-y", "@example/docs-mcp-server"],
 "env": {
 "DOCS_API_TOKEN": "${DOCS_API_TOKEN}"
 }
 }
 }
}
```

- **"mcpServers"**: 등록할 MCP 서버들을 담는 최상위 키입니다. 여러 서버를 나란히 등록할 수 있습니다.
- **"docs"**: 이 서버에 붙인 이름입니다. 에이전트가 이 서버의 도구를 쓸 때 이 이름을 씁니다.
- **"command" · "args"**: 서버를 실행하는 명령과 인자입니다. 여기서는 `npx` 로 문서 MCP 서버 패키지를 실행합니다.
- **"env"**: 서버에 전달할 환경 변수입니다. `${DOCS_API_TOKEN}` 은 토큰 값을 파일에 직접 쓰지 않고, 실행 시점에 환경 변수에서 값을 가져오라는 표기입니다. 토큰 자체는 `.mcp.json` 에 절대 적지 않습니다.

토큰은 셸 환경 변수로 따로 설정합니다. 예를 들어 PowerShell에서는 세션 환경 변수로 두거나, 운영체제의 환경 변수에 등록합니다. 핵심은 **비밀 값이 .mcp.json (그리고 git)에 들어가지 않는**다는 것입니다.

등록이 끝나면 메인 세션에서 그 서버의 도구를 쓸 수 있습니다. 예를 들어 다음처럼 지시합니다.

```
> docs 서버에서 "배포 절차" 문서를 찾아 핵심 단계를 요약해 줘
```

이제 에이전트는 문서를 손으로 붙여 넣지 않아도 `docs` 서버를 통해 직접 조회해 답합니다. `/mcp` 명령으로 현재 연결된 MCP 서버와 그 상태를 확인할 수 있습니다.

**아무 MCP 서버나 붙이지 마세요.** MCP 서버는 외부 코드를 실행하고 외부 서비스에 접근합니다. 신뢰할 수 있는 제공자의 서버만 등록하고, 팀이 공유하는 프로젝트 범위(`.mcp.json`)에 넣을 때는 동료와 합의하세요. 출처가 불분명한 서버는 권한·데이터 위험이 있습니다.

## 12.8 권한·비밀 관리: 동일 권한 체계·토큰은 환경으로

서브에이전트와 MCP를 맞본 신입이 가장 먼저 챙겨야 할 것은 **안전**입니다. 두 도구 모두 5장에 서 배운 권한 체계의 연장선에 있습니다. 새로운 예외가 아닙니다.

- **MCP 도구도 같은 권한 체계 안에 있습니다.** MCP 서버가 더한 도구라고 해서 권한 밖에서 멋대로 동작하지 않습니다. Claude Code는 그 도구들도 기존 `allow·deny·ask` 판정으로 통제합니다. 외부 서비스에 쓰기·삭제 같은 부수효과를 일으키는 도구라면, 5장의 최소 권한 원칙대로 신중히 허용하세요.
- **토큰·비밀은 코드가 아니라 환경으로.** 인증 토큰을 `.mcp.json` 에 직접 적으면, 그것이 git 에 올라가 유출됩니다. 반드시 `${TOKEN_NAME}` 표기로 환경 변수를 참조하고, 실제 값은 셸

환경 변수나 운영체제 환경 변수로 둡니다. 이것은 9장에서 배운 "비밀은 커밋하지 않는다"의 연장입니다.

- **서브에이전트의 도구도 좁힙니다.** 12.4에서 `test-runner`에 `Edit·Write`를 주지 않은 것처럼, 서브에이전트에는 그 역할에 필요한 도구만 부여합니다. 진단만 하는 에이전트에 편집 권한을 줄 이유가 없습니다.

**한 줄 원칙.** "새 도구가 늘어도 안전의 기본은 그대로다 — 최소 권한, 비밀은 환경으로." 서브에이전트와 MCP는 능력을 넓혀 주지만, 5장에서 배운 안전 원칙을 면제해 주지 않습니다. 오히려 외부와 닿는 만큼 더 엄격히 적용해야 합니다.

## 12.9 즉시 연습(2종 1세트)과 단원 미니 프로젝트

이 단원은 단일 챕터이므로, 즉시 연습과 단원 미니 프로젝트를 여기서 함께 진행합니다.

### 연습 (모범): 리뷰 서브에이전트 정의

다음은 코드 리뷰를 맡는 서브에이전트의 모범 정의입니다. 역할에 맞게 도구를 좁히고, `description`에 용도를 구체적으로 적었습니다.

```

name: pr-reviewer
description: 변경(diff)을 읽고 버그·중복·단순화 지점을 짚어 인라인 형식으로 보고합니다. PR을 올릴 때
tools: Read, Grep, Bash
model: sonnet

```

당신은 코드 리뷰 전담 에이전트입니다. `git diff`를 읽고 버그 가능성, 중복, 단순화 여지를 항목별로 짚되, 코드를 직접 수정하지 않고 보고만 합니다.

### 연습 (직접 해보기): team-tasks-api에 서브에이전트·MCP를 실제로 구성하기

종이에 빈칸을 채우는 대신, 1장에서 준비한 실제 `team-tasks-api` 저장소에서 서브에이전트를 직접 정의해 호출하고, MCP 서버를 등록해 연결을 확인합니다. 핵심은 추측한 YAML이 아니라 화면에 실제로 나타난 호출 결과, `/mcp` 출력을 그대로 옮기는 것입니다.

**과제 A — 서브에이전트를 실제로 정의하고 호출하라.** `team-tasks-api` 루트에

`.claude/agents/dep-checker.md` (또는 경계가 분명한 다른 작업) 파일을 직접 만드세요. `tools`는



역할에 필요한 것만으로 좁힙니다(예: 의존성 점검에는 Read, Grep, Bash 만, Edit·Write 는 빼기).

```

name: dep-checker
description: requirements.txt와 import를 대조해 빠진/불필요한 의존성을 보고합니다. 의존성 점검이
tools: Read, Grep, Bash

당신은 의존성 점검 전담 에이전트입니다. requirements.txt와 코드의 import를
대조해 불일치를 항목별로 보고하되, 코드를 직접 수정하지 않고 보고만 합니다.

```

파일을 만든 뒤 `claude` 세션에서 "dep-checker 서브에이전트로 이 프로젝트의 의존성을 점검해 줘"라고 **실제로 호출**하고, 메인 세션에 무엇이 남는지 확인해 옮겨 적으세요.

[실제 호출 결과] 서브에이전트가 보고한 내용(화면 그대로):  
[컨텍스트 격리] 메인에 긴 로그가 쌓였는가, 요약만 남았는가:

**과제 B — MCP 서버를 실제로 등록하고 연결을 확인하라.** 같은 저장소에서 외부 서비스 하나(문서·이슈 트래커 등)를 `.mcp.json`에 등록하세요. 토큰은 직접 적지 말고 환경 변수 표기(`${...}`)를 씁니다.

```

{
 "mcpServers": {
 "docs": {
 "command": "npx",
 "args": ["-y", "@modelcontextprotocol/server-everything"],
 "env": {
 "API_TOKEN": "${DOCS_API_TOKEN}"
 }
 }
 }
}

```

등록한 뒤 `/mcp`를 실행해 연결 상태를 확인하고, 화면에 실제로 나타난 결과를 옮겨 적으세요.

[/mcp 실제 출력] 서버가 연결됐는가(상태):  
[비밀 점검] `.mcp.json`에 실제 토큰 값이 없고 `${...}` 참조만 있는가:

**자가체크 — 머리로가 아니라 손으로 했는지 확인하세요.**

- [ ] `.claude/agents/`에 서브에이전트 정의 파일을 **실제로 만들고** 호출까지 했다.
- [ ] [실제 호출 결과]에 적은 것이 추측이 아니라 화면에 나타난 서브에이전트 보고다.
- [ ] 메인 세션에 긴 로그가 아니라 **요약만 남는** 컨텍스트 격리를 직접 확인했다.

- [ ] 서버에이전트 `tools` 를 역할에 필요한 것만으로 좁혔다(불필요한 `Edit·Write` 배제).
- [ ] `.mcp.json` 을 실제로 작성하고 `/mcp` 로 연결 상태를 확인했다.
- [ ] `.mcp.json` 에 토큰 값을 직접 적지 않고 `${...}` 환경 변수 표기만 썼다.

## 단원 미니 프로젝트 (모범): 위임과 외부 연동 맛보기

이 단원 개념만으로 완성하는 작은 산출물입니다. 이번 과제는 1장에서 준비한 **실제 `team-tasks-api` 저장소 안에서** 서버에이전트와 MCP를 직접 구성해 봅니다. 마일스톤을 증분으로 진행합니다.

- **M1 테스트 서버에이전트 정의·호출.** `team-tasks-api` 루트에 `.claude/agents/test-runner.md` 를 작성하고(앞 12.4의 정의 그대로 써도 됩니다), 메인 세션에서 "test-runner 서버에이전트로 tests/ 전체를 돌려 결과를 요약해 줘"라고 호출합니다. 메인에는 긴 pytest 로그가 아니라 결과 요약만 남는 것을 확인합니다. (적용 개념: 서버에이전트, 컨텍스트 격리, 도구 좁히기)
- **M2 MCP 서버 등록.** 같은 저장소에서 문서 또는 이슈 MCP 서버를 `.mcp.json` 에 등록하고(`claude mcp add ... --scope project`), 토큰을 환경 변수로 분리합니다. `/mcp` 로 연결을 확인합니다. (적용 개념: MCP 등록, 비밀은 환경으로)
- **M3 안전 점검.** 서버에이전트의 도구가 역할에 맞게 좁혀졌는지(`test-runner` 에 `Edit·Write` 미부여), `.mcp.json` 에 비밀이 직접 들어 있지 않은지 점검합니다. (적용 개념: 동일 권한 체계, 최소 권한)

**실제 검증 단계.** M1에서 서버에이전트가 돌린 결과가 실제 저장소 상태와 일치하는지, 직접 `python -m pytest -q` 를 돌려 대조합니다. 기본 상태라면 서버에이전트 요약과 직접 실행이 모두 같은 결과여야 합니다.

```
> python -m pytest -q
..... [100%]
7 passed
```

체크포인트: M1은 "메인에 요약만 남고 긴 로그가 안 쌓임 + 서버에이전트 요약이 직접 실행한 `7 passed` 와 일치", M2는 "`/mcp` 에 서버가 연결됨 + 토큰이 파일에 없음", M3은 "불필요한 권한·노출된 비밀 0건"입니다.

## 단원 미니 프로젝트 (학생 작성형 빈 스타터)

**직접 작성·실행.** 추측을 적지 말고, `team-tasks-api` 에서 위임 + 외부 연동을 **실제로 구성·호출**한 뒤 화면에 나타난 것을 그대로 옮겨 적으세요.

1. [위임 작업 정하기] 내가 정한 위임 작업(경계가 분명한가? 메인 맥락 없이 가능한가?):
2. [서브에이전트 실제 작성] 내가 만든 `.claude/agents/` 파일 경로 / name / 좁힌 tools:  
[실제 호출 결과] 메인 세션에서 호출했을 때 화면에 남은 서브에이전트 요약(긴 로그가 아니라 요약)
3. [MCP 실제 등록] 등록한 외부 서비스 / `.mcp.json`의 서버 이름 / 토큰 환경 변수 이름(`${...}`):  
[/mcp 실제 출력] 서버 연결 상태:
4. [호출 지시문] 내가 실제로 준 호출 지시 한 줄과 그 결과:

## 자가체크

- ☐ 위임 작업이 경계가 분명하고 메인 맥락 없이도 수행 가능한가
- ☐ 서브에이전트에게 필요한 맥락(어떤 파일·범위)을 지시에 명시했는가
- ☐ 서브에이전트 도구를 최소로 좁혔는가
- ☐ `.mcp.json`에 실제 토큰 값이 들어 있지 않은가(환경 변수 참조만)
- ☐ `/mcp`로 서버 연결 상태를 확인했는가

## 12.10 혼한 오류와 오개념

**오개념 1: 서브에이전트는 메인 세션의 맥락을 그대로 물려받는다.** 서브에이전트는 빈 컨텍스트에서 시작합니다. 메인 대화에서 방금 논의한 "그 버그", "아까 그 파일"을 알지 못합니다. 그래서 "방금 그거 고쳐 줘" 같은 지시는 통하지 않습니다. 위임할 때는 "tests/test\_tasks\_api.py를 돌려 줘"처럼 **필요한 맥락을 명시**해야 합니다. 컨텍스트 격리는 장점(메인이 깨끗함)인 동시에, 맥락을 직접 넘겨야 하는 책임이기도 합니다.

**오개념 2: MCP는 잡다한 플러그인을 마구 꽂는 난립이다.** MCP는 난립이 아니라 그 반대입니다. 서비스마다 제각각이던 연동을 **하나의 표준 프로토콜**로 통일해 통합을 캡슐화합니다. USB 단자처럼, 표준을 따르므로 여러 서비스를 같은 방식으로 꽂고 같은 권한 체계로 통제합니다. 다만 표준이라 해도 신뢰할 수 있는 서버만 등록해야 한다는 점은 변하지 않습니다.

**오개념 3: MCP가 더한 도구는 권한 밖에서 자유롭게 동작한다.** MCP 서버가 노출한 도구도 Claude Code의 **동일한 권한 체계**(5장의 allow·deny·ask) 안에서 통제됩니다. 외부 서비스에 쓰기·삭제를 일으키는 도구라면 최소 권한 원칙대로 신중히 허용하세요. 그리고 인증 토큰은 절대 `.mcp.json`에 직접 적지 말고 `${TOKEN}` 환경 변수로 주입해, git에 비밀이 올라가지 않게 합니다.

**오개념 4: 모든 작업을 서브에이전트로 위임하면 더 빠르다.** 서브에이전트는 경계가 분명하고 결과를 요약으로 받으면 충분한 작업에 어울립니다. 메인의 흐름과 긴밀히 얽혀 한 줄씩 협의해야 하는 작업까지 위임하면, 맥락을 넘기는 비용이 오히려 더 큼니다. 위임은 도구이지 만능이 아닙니다. "이 일이 독립적으로 떼어 낼 수 있는가"를 먼저 판단하세요.

## 12.11 자주 묻는 질문(FAQ)

**Q. 서브에이전트와 그냥 새 세션을 여는 것은 무엇이 다른가요?** 새 세션을 직접 열면 여러분이 그 창을 오가며 관리해야 하고, 결과를 손으로 메인에 옮겨야 합니다. 서브에이전트는 메인 에이전트가 **자동으로 위임하고 결과 요약만 돌려받습니다**. 또 `description · tools`로 역할과 권한을 미리 못 박아 둘 수 있어, 매번 같은 작업을 일관되게 맡길 수 있습니다.

**Q. 서브에이전트는 메인보다 똑똑한가요?** 아닙니다. 똑똑함의 문제가 아니라 **분업과 격리**의 문제입니다. 서브에이전트의 이점은 별도 컨텍스트로 메인을 깨끗이 유지하고, 독립 작업을 병렬로 처리하며, 역할에 맞게 도구를 좁힐 수 있다는 데 있습니다. `model` 필드로 어떤 모델을 쓸지 정할 수는 있지만, 핵심 가치는 능력이 아니라 구조입니다.

**Q. MCP 서버는 직접 만들어야 하나요?** 이 단원(맛보기) 수준에서는 만들 필요가 없습니다. 많은 서비스가 이미 MCP 서버를 제공하므로, `claude mcp add` 나 `.mcp.json` 등록으로 **가져다 쓰기**만 하면 됩니다. 서버를 직접 개발하는 것은 다음 학습 단계의 주제입니다.

**Q. `.mcp.json` 을 git에 올려도 되나요?** 프로젝트 범위의 `.mcp.json` 은 팀과 공유하려고 올리는 파일입니다. 단, **토큰 같은 비밀이 그 안에 없을 때만** 안전합니다. 비밀은 항상 `${TOKEN}` 환경 변수 표기로 두고 실제 값은 각자의 환경에 둡니다. 이렇게 하면 설정은 공유하되 비밀은 유출되지 않습니다.

**Q. 여기서 배운 것으로 충분한가요?** 이 장은 의도적으로 "맛보기"입니다. 두 도구가 무엇이고 어떻게 시작하는지, 그리고 안전의 기본을 정확히 아는 것까지가 목표입니다. 다중 에이전트 오케스트레이션, MCP 서버 개발, 혹은 통한 자동화 강제 같은 심화 주제는 온보딩을 마친 뒤 별도로 익히면 됩니다.

## 12.12 정리: 온보딩을 마치며

이 장에서 우리는 온보딩 이후로 나아갈 두 도구를 맛봤습니다. **서브에이전트**는 경계가 분명한 작업을 별도 컨텍스트·도구를 가진 일꾼에게 위임해, 메인 세션을 깨끗이 유지하고 병렬 처리까지 가능하게 합니다. **MCP**는 외부 도구·데이터를 표준 프로토콜로 연결해, 매번 복사·붙여넣기 하던 외부 자료를 에이전트가 직접 다루게 합니다. 두 도구 모두 5장의 안전 원칙(동일 권한 체계·최소 권한·비밀은 환경으로)을 그대로 따릅니다.

이로써 12개 개념을 모두 배웠습니다. 첫날 설치부터 코드베이스 파악, 안전한 변경, 검증, 팀 규약, 첫 PR, 리뷰 대응, 디버깅, 그리고 자동화의 맛보기까지. 다음 장부터는 이 모든 개념을 하나

로 모아, 기존 팀 FastAPI 저장소에 첫 기능을 추가하고 첫 PR을 올리는 **종합 프로젝트**를 분석  
→ 설계 → 구현 → 발표·성찰의 단계로 진행합니다. 여기서 배운 것을 진짜 첫 업무에서 통합해  
적용할 차례입니다.



# 종합 프로젝트: 첫 기능·첫 PR — 분석(요구사항·범위)

## 이 챕터에서 배우는 것

- 드라이빙 퀘스천(핵심 질문)을 기능 요구사항(FR)과 비기능 요구사항(NFR) 항목으로 각각 구체화해 표로 작성할 수 있습니다.
- 처음 받은 FastAPI 저장소를 에이전트 탐색으로 파악해, 첫 기능을 사용자 스토리와 유스 케이스 1건 이상으로 정의할 수 있습니다.
- 범위를 In/Out으로 나눠, 신입이 감당할 작은 첫 기능으로 한정되도록 통제할 수 있습니다.
- 각 요구사항이 앞서 배운 어느 단원 개념으로 구현되는지 요구사항↔개념 매핑 표로 작성할 수 있습니다.

## 종합 프로젝트를 시작하며: 지금까지 배운 것을 하나의 업무로 엮습니다

여기까지 우리는 11개 단원에 걸쳐 12개의 개념을 하나씩 배웠습니다. 설치·인증(c01)으로 시작해, 에이전트 루프(c02)와 효과적인 작업 지시(c03)를 익히고, 낯선 코드를 탐색하는 법(c04), 권한(c05)과 플랜 모드(c06)로 안전하게 변경하는 법, 실행·테스트로 검증하기(c07), CLAUDE.md로 팀 규약을 주입하기(c08), 첫 커밋과 첫 PR(c09), 코드 리뷰 대응(c10), 디버깅(c11), 그리고 서버 에이전트·MCP 맛보기(c12)까지 왔습니다.

각 단원의 미니 프로젝트로 개념을 따로따로 적용해 보았지만, 실제 신입의 첫 업무는 이 개념들이 **하나의 흐름으로 모일 때** 비로소 완성됩니다. 이번 종합 프로젝트(capstone, 캡스톤)는 그 흐름을 처음부터 끝까지 한 번에 통과하는 통과예입니다. 이 챕터부터 네 챕터(분석 → 설계 → 구현 → 발표·성찰)에 걸쳐, 다음 한 가지 실제 업무를 끝까지 수행합니다.

**종합 프로젝트 주제: 기존 팀 FastAPI 저장소에 첫 기능 추가하고 첫 PR 올리기** 받은 팀의 team-tasks-api (할 일 관리 API) 저장소에 **완료 처리(PATCH**

`/tasks/{id}/complete`)와 상태 필터(`GET /tasks?status=`) 두 엔드포인트를 테스트와 함께 추가하고, 작은 논리 단위로 커밋해 첫 PR을 올려 리뷰까지 통과시킵니다.

## 이 챕터는 새 개념을 배우지 않습니다

종합 프로젝트의 네 챕터는 **새로운 도구나 개념을 가르치지 않습니다**. 대신 앞 12개 개념을 실제 업무에 통합 적용합니다. 그래서 이 챕터의 모든 요구사항과 결정에는 "이건 어느 단원에서 배운 무엇인가"를 함께 표시합니다. 만약 어떤 부분이 막힌다면, 매핑된 단원으로 돌아가 복습하면 됩니다. 종합 프로젝트는 **복습이자 전이(transfer)**의 장입니다.

## 드라이빙 퀘스천과 POV 다시 보기

프로젝트 기반 학습(PBL)은 하나의 **드라이빙 퀘스천(driving question, 핵심 질문)**을 끝까지 붙들고 답을 만들어 가는 방식입니다. 우리의 드라이빙 퀘스천은 다음과 같습니다.

### 드라이빙 퀘스천

신입 개발자가 처음 받은 팀의 FastAPI 저장소를 Claude Code로 안전하게 파악하고, 작은 기능 하나를 테스트와 함께 추가해 리뷰어가 신뢰할 수 있는 첫 PR로 올리려면 어떻게 해야 할까?

이 질문은 "정답이 하나로 정해진" 질문이 아니라, **여러 단계의 판단과 검증을 요구하는 열린 질문**입니다. 분석 → 설계 → 구현 → 발표의 네 단계가 바로 이 질문에 답해 가는 과정입니다.

이 질문의 주인공이 누구인지를 분명히 해 두면 모든 결정의 기준이 뚜렷해집니다. 이것이 **POV(Point of View, 관점)**입니다. POV는 "누가(user) · 무엇이 필요하며(need) · 왜 그것이 핵심인가(insight)"를 한 문장으로 모은 것입니다.



POV 요소	내용
사용자 (user)	팀에 막 합류해 첫 업무로 기존 FastAPI 저장소에 작은 기능 추가를 맡은 주니어 신입 개발자
필요 (need)	낯선 팀 코드를 안전하게 파악해 작은 기능 하나를 테스트와 함께 추가하고, 리뷰어가 신뢰할 수 있는 첫 PR을 올려 리뷰까지 통과한다
통찰 (insight)	기능을 따로 배우는 것과 한 번의 실제 업무로 엮어 쓰는 것은 다르다. 첫 기능을 직접 추가해 PR까지 올려 보면 설치·탐색·권한·플랜·검증·CLAUDE.md·커밋·리뷰·디버깅이 어떻게 하나의 온보딩 흐름으로 모이는지를 비로소 체득한다

POV가 있으면 "기능을 더 화려하게 만들까?" 같은 유혹 앞에서 멈출 수 있습니다. 우리의 사용자는 **신입**이고, 그가 필요한 것은 화려함이 아니라 **안전하게·검증 가능하게·리뷰어가 신뢰할 수 있게** 작은 기능 하나를 끝까지 해내는 것이기 때문입니다. 이 기준이 다음 절부터의 모든 요구 사항과 범위 결정을 이끍니다.

## 기존 FastAPI 저장소 파악 (에이전트 탐색)

분석의 첫걸음은 "무엇을 만들지" 정하기 전에 **무엇이 이미 있는지** 아는 것입니다. 신입이 받은 것은 빈 폴더가 아니라 **이미 동작하는 팀의 코드**입니다. 여기서 ch\_04에서 배운 **코드베이스 탐색(c04)**을 그대로 씁니다. 변경 없이 읽기만 하면서, 시니어에게 묻듯 에이전트에게 질문해 구조를 파악합니다.

먼저 저장소가 무엇을 하는지, 엔드포인트가 무엇인지 요약하게 합니다.

- > 이 저장소가 무엇을 하는 API인지, 어떤 엔드포인트가 있는지, 핵심 파일 구조가 어떻게 되는지 한국어로 요약해 줘. 아직 아무것도 바꾸지 말고 읽기만 해.

#### (Claude Code가 Glob·Read·Grep으로 저장소를 훑은 뒤 요약)

이 저장소(team-tasks-api)는 할 일(task)을 관리하는 FastAPI 기반 REST API입니다. 데이터는 데이터베이스 없이 인메모리(메모리 안 리스트)에 저장합니다. 현재 엔드포인트는 세 개입니다.

- GET /tasks                    전체 할 일 목록 조회
- POST /tasks                새 할 일 생성(title을 받음)
- GET /tasks/{id}          단일 할 일 조회(없으면 404)

핵심 파일 구조는 다음과 같습니다.

main.py	FastAPI 앱 생성, 라우터 등록
routers/tasks.py	tasks 관련 엔드포인트 정의
models.py	Task 등 pydantic 모델
store.py	인메모리 저장소(목록·조회·추가 함수)
tests/test_tasks.py	기존 엔드포인트 테스트

- 에이전트는 Glob (파일 찾기)·Read (파일 읽기)·Grep (코드 검색) 같은 읽기 전용 도구만 써서 구조를 파악했습니다. "아직 아무것도 바꾸지 말고"라는 제약(c03의 작업 지시 4요소 중 하나)을 명시해, 탐색 단계에서 코드가 변하지 않게 했습니다.
- 핵심 산출물은 "엔드포인트 목록"과 "파일 구조"입니다. 이 두 가지를 알면 "내 기능을 어디에 끼워 넣을지"의 출발점이 잡힙니다. 완료 처리·상태 필터는 모두 tasks에 관한 일이므로 routers/tasks.py 가 주된 작업 무대가 될 것임을 짐작할 수 있습니다.
- 데이터가 인메모리라는 점도 중요합니다. 데이터베이스 연동이 없으므로, 우리 작업의 범위가 "저장소(store.py)의 리스트를 다루는 일"로 좁혀집니다(범위 결정의 근거).

다음으로, 우리가 바꿀 핵심인 데이터(할 일 한 건)가 어떻게 생겼는지와 검증·에러가 어떻게 처리되는지를 더 구체적으로 묻습니다.

> Task 데이터 한 건이 어떤 필드로 이루어져 있는지, 그리고 없는 id를 조회할 때 404를 어떻게 내는지 해당 코드를 인용해서 설명해 줘.

(Claude Code가 models.py와 routers/tasks.py의 관련 부분을 읽어 설명)

Task 모델(models.py)은 다음 필드를 가집니다.

- id: 정수, 생성 시 자동 부여
- title: 문자열, 필수
- status: 문자열, 기본값 "todo" (이미 status 필드가 존재합니다)

없는 id 처리는 routers/tasks.py의 GET /tasks/{id}에서 store.get\_task(id)가 None을 반환하면 다음과 같이 404를 냅니다.

```
task = store.get_task(task_id)
if task is None:
 raise HTTPException(status_code=404, detail="task를 찾을 수 없습니다")
return task
```

- 가장 값진 발견은 **status 필드가 이미 존재한다**는 사실입니다. 기본값이 "todo" 이므로, 우리는 새 필드를 만드는 게 아니라 **기존 필드의 값을 "done" 으로 바꾸는** 일을 하면 됩니다. 탐색이 작업의 난이도를 크게 낮춰 준 것입니다. 추측으로 시작했다면 불필요한 필드를 새로 추가했을지도 모릅니다.
- 기존 코드가 없는 id에 `HTTPException(status_code=404, ...)` 로 응답한다는 **기존 패턴도** 확인했습니다. 우리가 추가할 완료 처리도 "없는 id면 404"라는 같은 패턴을 따르면, 코드가 팀의 스타일과 일관됩니다(ch\_03에서 배운 "기존 스타일 따르기").
- 이처럼 탐색은 "무엇을 만들지"가 아니라 **"무엇이 이미 있어서 재사용·일관되게 따라갈 수 있는지"**를 알려 줍니다. 이 정보가 다음 절의 요구사항을 현실적으로 만듭니다.

탐색으로 파악한 핵심을 한 문단으로 정리하면 이렇습니다. team-tasks-api 는 인메모리 tasks API이고, 할 일은 id·title·status 를 가지며 status 는 이미 존재해 "todo" 가 기본값입니다. 엔드포인트는 목록·생성·단건 조회 세 개이고, 없는 id에는 404를 내는 패턴이 자리잡혀 있습니다. **변경 없이** 이만큼 파악했으니, 이제 "무엇을 더할지"를 요구사항으로 적을 수 있습니다.

## 기능 요구사항(FR): 완료 처리·상태 필터 엔드포인트

**요구사항(requirement)**은 "이 기능이 무엇을 해야 하는가"를 검증 가능한 문장으로 적은 것입니다. 그중 **기능 요구사항(FR, Functional Requirement)**은 시스템이 **무엇을 하는지**(동작·기능)를 정의합니다. 모호한 "완료 기능을 추가한다" 대신, 확인할 수 있는 구체적 문장으로 적습니다.

탐색 결과를 바탕으로 이번 첫 기능의 기능 요구사항을 다음 일곱 가지로 정리했습니다. 각 요구사항 끝에는 **그것을 구현할 때 쓰는 단위·개념**을 함께 적었습니다(추적성).

ID	기능 요구사항	적용 단원	적용 개념
FR1	기존 tasks API의 구조·엔드포인트·데이터 흐름을 파악해 변경 지점을 찾는다	u4	c04 탐색
FR2	완료 처리 엔드포인트(PATCH /tasks/{id}/complete)로 해당 작업의 status를 done으로 변경한다	u3, u5	c03 지시, c06 플랜
FR3	상태 필터 엔드포인트(GET /tasks?status=done todo)로 목 록을 상태별로 거른다	u3, u5	c03 지시, c06 플랜
FR4	없는 id는 404, 잘못된 status 값은 422로 적절한 상태 코 드와 에러 응답을 낸다	u10	c11 디버깅
FR5	추가한 두 엔드포인트를 pytest(TestClient)로 검증한다	u6	c07 검증
FR6	변경을 작은 논리 단위로 커밋하고 첫 PR로 올린다	u8	c09 커밋 ·PR
FR7	코드 리뷰를 받아 지적을 반영하거나 근거로 대응한다	u9	c10 리뷰

각 요구사항이 왜 이렇게 적혔는지 핵심만 짚겠습니다.

- **FR2(완료 처리)**는 "어떻게(PATCH)"와 "무엇이 바뀌는가(status가 done으로)"를 모두 담았습니다. "complete 엔드포인트 추가"보다 훨씬 검증하기 쉽습니다. 나중에 "PATCH /tasks/1/complete를 호출하면 1번 작업의 status가 done이 된다"를 테스트로 그대로 옮길 수 있습니다.
- **FR3(상태 필터)**는 입력(쿼리 파라미터 `status`)과 출력(거른 목록)을 명시했습니다. `todo|done` 두 값만 받는다고 못박아, FR4의 "잘못된 status는 422"와 자연스럽게 연결됩니다.
- **FR4(에러 응답)**는 "잘 안 되면 어떻게 되는가"라는 **불행 경로(unhappy path)**를 미리 정의합니다. 신입이 가장 자주 빠뜨리는 부분이지만, 실제 업무에서는 "정상일 때"만큼 "비정상일 때"가 중요합니다. 기존 코드가 이미 404를 쓰므로 같은 패턴을 따릅니다.

## 좋은 요구사항은 "테스트로 옮길 수 있는" 문장입니다

요구사항이 잘 적혔는지 판단하는 가장 쉬운 기준은 "이 문장을 그대로 테스트(pytest)로 옮길 수 있는가?"입니다. "완료 기능을 잘 만든다"는 테스트로 옮길 수 없지만, "PATCH /tasks/{존재하

는 id)/complete를 호출하면 200과 status=done이 반환된다"는 그대로 한 줄짜리 테스트가 됩니다. 요구사항을 적을 때부터 ch\_07의 검증(c07)·TDD를 염두에 두면, 분석 단계가 곧 테스트 설계가 됩니다.

## 비기능 요구사항(NFR): 테스트·안전·팀 규약

**비기능 요구사항(NFR, Non-Functional Requirement)**은 "무엇을 하는가"가 아니라 "**어떻게·얼마나 잘 하는가**"(품질·방식·제약)를 정의합니다. 같은 기능이라도 안전하게·검증 가능하게·일관되게 만드는 것이 NFR입니다. 신입의 첫 기능에서는 결과물의 기능만큼 **일하는 방식**이 평가받으므로 NFR이 특히 중요합니다.

ID	비기능 요구사항	적용 개념
NFR1	에이전트 루프를 이해하고 목표·맥락·제약·완료 조건의 4요소 지시로 작업한다	c02 루프, c03 지시
NFR2	부수효과 도구(편집·실행)를 최소 권한 settings.json으로 통제한다	c05 권한
NFR3	변경 전에 플랜 모드로 계획을 받아 검토한다(안전)	c06 플랜
NFR4	테스트 우선(TDD)으로 완료 조건을 먼저 세운다	c07 검증
NFR5	팀 규약을 CLAUDE.md로 영속화해 세션 간 일관성을 유지한다	c08 CLAUDE.md
NFR6	막힘은 디버깅 루프(재현→가설→최소 수정→회귀 테스트)로 해소한다	c11 디버깅
NFR7	반복 작업은 서브에이전트로, 외부 자료는 MCP로 가볍게 활용한다(맛보기)	c12 자동화

기능 요구사항(FR)과 비기능 요구사항(NFR)의 차이를 한 문장으로 구분하면 이렇습니다. **FR**은 "**무엇을 만드는가**"(완료 처리·상태 필터), **NFR**은 "**어떻게 만드는가**"(안전하게·검증하며·일관되게)입니다. 둘 다 충족해야 신입의 첫 업무가 "동작하긴 하는데 위험하게 만든 코드"가 아니라 "안전하게·검증 가능하게 만든 신뢰할 수 있는 코드"가 됩니다.

## NFR을 빠뜨리면 "동작은 하지만 못 믿을" 결과가 됩니다

기능(FR)만 채우고 비기능(NFR)을 무시하면, 엔드포인트는 동작하지만 테스트가 없고(NFR4 위반), 권한 통제 없이 위험하게 만들어졌으며(NFR2 위반), 팀 규약과 어긋나(NFR5 위반) 리뷰에서 막힙니다. 평가 루브릭(ch\_16)이 기능 완성도뿐 아니라 안전·절차·협업을 함께 보는 이유가 여기 있습니다. 신입의 첫 PR이 신뢰를 얻는지는 NFR에서 갑니다.

## 사용자 스토리와 유스케이스

요구사항을 **사용자의 입장에서** 다시 적으면, "왜 이 기능이 필요한가"가 또렷해집니다. 이를 위한 두 가지 도구가 사용자 스토리와 유스케이스입니다.

**사용자 스토리(user story)**는 "[누구]로서, [무엇을] 하고 싶다, [왜]"의 형식으로 요구를 사용자 관점에서 적은 짧은 문장입니다. 우리 프로젝트의 사용자(신입 개발자)가 가진 네 가지 스토리는 다음과 같습니다.

1. 신입으로서, 받은 저장소가 무엇을 하는지 빠르게 파악해 어디를 바꿔야 할지 알고 싶다. (→ FR1 탐색)
2. 신입으로서, 기존 코드를 망가뜨릴까 봐 두렵지 않게 안전장치(권한·플랜)를 갖추고 손대고 싶다. (→ NFR2·NFR3)
3. 신입으로서, 추가한 기능이 정말 동작하는지 테스트로 증명하고 싶다. (→ FR5·NFR4)
4. 신입으로서, 리뷰어가 빠르게 이해하는 좋은 첫 PR을 올리고 리뷰를 통과하고 싶다. (→ FR6·FR7)

**유스케이스(use case)**는 한 걸음 더 들어가, 사용자가 시스템과 **어떻게 상호작용해 목표를 이루는지**를 입력→처리→결과의 흐름으로 적은 시나리오입니다. 스토리가 "무엇을 원하는가"라면, 유스케이스는 "그래서 실제로 어떤 요청을 보내고 어떤 응답을 받는가"입니다.

ID	유스케이스	흐름(입력 → 결과)
UC1	완료 처리	<code>PATCH /tasks/1/complete</code> → 1번 task의 status가 done으로 바뀌고 200 응답
UC2	상태 필터	<code>GET /tasks?status=done</code> → status가 done인 task만 목록으로 반환
UC3	첫 PR	작업 브랜치 커밋 → PR 생성 → <code>/code-review</code> 자동 리뷰 → 반영 → 통과

- UC1·UC2의 흐름이 곧 **테스트 시나리오**입니다. 예컨대 UC1은 "1번 작업을 만들고 → PATCH로 완료 처리하고 → 응답이 200이고 status가 done인지 확인"이라는 pytest 한 케이스로 그대로 옮겨집니다(ch\_07·ch\_15에서 구현).
- UC3은 코드가 아니라 **협업 절차**의 유스케이스입니다. 커밋·PR·리뷰·반영이라는 사람과 사람(그리고 자동 리뷰)의 상호작용을 담아, 종합 프로젝트가 "코드 작성"으로 끝나지 않고 "팀에 합류시키기"까지 포함함을 보여 줍니다.
- 사용자 스토리·유스케이스를 분석 단계에서 적어 두면, 구현(ch\_15)에서 "무엇을 만들고 무엇을 테스트할지"를 다시 고민하지 않고 이 목록을 그대로 따라가면 됩니다. 분석이 충실하면 구현이 단순해집니다.

## 범위 In/Out 확정 (작은 첫 기능으로)

신입이 첫 업무에서 저지르는 흔한 실수는 **범위를 키우는 것**입니다. "기왕 하는 김에 인증도 붙이고, 데이터베이스도 연동하고..."라는 생각은 첫 PR을 끝없이 미루게 만듭니다. POV가 알려 주듯 우리의 사용자는 신입이고, 필요한 것은 **작은 기능 하나를 끝까지** 해내는 것입니다. 그래서 무엇을 **포함(In)**하고 무엇을 **제외(Out)**할지 명시적으로 못박습니다.

범위 In (이번에 한다)	범위 Out (이번에 하지 않는다)
기존 저장소 탐색·구조 지도 작성	데이터베이스 연동·마이그레이션(기존 인메모리 유지)
완료 처리 엔드포인트(PATCH /tasks/{id}/complete) 추가	인증·권한 같은 큰 신규 기능(범위 초과)
상태 필터 엔드포인트(GET /tasks?status=done todo) 추가	프론트엔드·UI
입력 검증과 적절한 상태 코드·에러 응답(404·422)	배포·인프라·CI 무인 자동화(본서는 맛보기까지만)
각 변경에 대한 pytest 테스트(TestClient)	서브에이전트·MCP 심화 설계(맛보기 단원까지만)
권한 설정·플랜 모드·CLAUDE.md 적용	
작은 논리 단위 커밋과 첫 PR 작성·리뷰 대응	

범위를 좁히면 무엇이 좋을까요? 첫째, **끝낼 수 있습니다**. 작은 범위는 한 번의 작업 세션으로 완결되어 첫 PR을 실제로 올릴 수 있습니다. 둘째, **리뷰가 쉽습니다**. 변경이 작으면 리뷰어가 빠르고 정확하게 봐 주고, 문제가 생겨도 영향 범위가 좁습니다(ch\_09에서 배운 "신입의 PR은 작을수록 좋다"). 셋째, **다음을 위한 여지가 생깁니다**. "완료 취소"나 "완료 시각 기록" 같은 확장은 후속 PR로 미루면 되므로, 이번 PR이 깔끔하게 유지됩니다.

## MVP: 가장 작은 첫 버전부터

범위 안에서도 **MVP(Minimum Viable Product, 최소 기능 제품)**를 먼저 정합니다. 우리의 MVP는 "완료 처리 엔드포인트(PATCH /tasks/{id}/complete) 하나를 테스트와 함께 추가해 통과시키고 첫 커밋·PR을 올리는 최소 버전"입니다. 상태 필터·에러 응답 보강·리뷰 대응은 MVP 이후의 **증분 확장**입니다. 한꺼번에 다 만들려다 아무것도 끝내지 못하는 대신, 가장 작은 동작하는 버전을 먼저 완성하고 거기서 키워 갑니다. 이 증분 전략이 구현 챕터(ch\_15)의 마일스톤 순서가 됩니다.

## 추적성: 요구사항 ↔ 단원 개념 매핑

종합 프로젝트의 핵심은 **추적성(traceability)**입니다. 모든 요구사항이 "허공에서 나온 것"이 아니라 **앞서 배운 어느 개념으로 구현되는지** 한눈에 이어지게 만드는 것입니다. 추적성 표가 있으면 두 가지가 가능합니다. (1) 구현 중 막히면 해당 개념의 단원으로 정확히 돌아가 복습할 수 있고, (2) 이 프로젝트가 정말 12개 개념을 모두 통합하는지 빠짐없이 확인할 수 있습니다.

아래는 이번 프로젝트가 다루는 요구사항(FR·NFR·UC)을 단원·개념별로 모은 전체 추적성 표입니다.



단 원	개념	이 프로젝트에서의 쓰임	연결된 요구사항
u1	c01 설치·인증	작업 환경(Claude Code) 준비, 세션 운 용	전반 환경
u2	c02 에이전트 루 프	관찰→계획→도구→확인 흐름으로 작업 진행	NFR1
u3	c03 작업 지시	4요소 지시로 엔드포인트 추가 요청	FR2, FR3, NFR1
u4	c04 코드 탐색	기존 저장소 파악·구조 지도	FR1, UC 전반
u5	c05 권한	최소 권한 settings.json	NFR2
u5	c06 플랜 모드	변경 전 계획 검토	FR2, FR3, NFR3
u6	c07 검증	pytest·TDD로 동작 증명	FR5, NFR4, UC1, UC2
u7	c08 CLAUDE.md	팀 규약 영속화	NFR5
u8	c09 커밋·PR	논리 단위 커밋·첫 PR	FR6, UC3
u9	c10 코드 리뷰	리뷰 받기·대응	FR7, UC3
u10	c11 디버깅	404·422 에러·실패 진단	FR4, NFR6
u11	c12 자동화	서브에이전트·MCP 맛보기	NFR7

표를 보면 **12개 개념이 빠짐없이 이 프로젝트의 어딘가로 이어진다는 것**을 확인할 수 있습니다. 이것이 종합 프로젝트가 "그냥 던져 주는 과제"가 아니라 **커리큘럼 전체를 통합 적용하도록 역설계된 업무**라는 증거입니다. 다음 챕터(ch\_14)에서는 이 요구사항들을 실제 **설계**(데이터 구조·모듈 변경·ADR)로 옮깁니다.

## 작성형 연습 (2종 1세트)

분석을 직접 해 봅니다. 먼저 ① 모범 예시를 확인하고, ② 빈 템플릿을 스스로 채운 뒤, ③ 자가 체크로 점검합니다.

## 모범 예시: 요구사항 한 줄을 검증 가능하게 다듬기

모호한 요구사항 "완료된 작업만 볼 수 있게 한다"를 검증 가능한 기능 요구사항으로 다듬으면 다음과 같습니다.

항목	내용
요구사항 ID	FR3
검증 가능한 문장	GET /tasks?status=done 요청 시 status가 done인 task만 목록으로 반환한다
입력	쿼리 파라미터 status=done (허용 값: todo, done)
정상 결과	200, done인 task만 담긴 목록
비정상 결과	status=foo 같은 허용 외 값이면 422
적용 개념	c03 작업 지시, c06 플랜 모드, c07 검증, c11 디버깅(422)

핵심은 "완료된 작업만 본다"는 막연한 말을 **입력·정상 결과·비정상 결과**로 쪼개, 그대로 테스트로 옮길 수 있게 만든 점입니다.

## 연습 1: 내 team-tasks-api의 완료 처리 요구사항을 직접 작성하라

받은 `team-tasks-api`를 탐색한 내용을 바탕으로, "완료 처리" 기능(FR2)의 요구사항을 **직접 작성**하세요. 위 모범 예시(FR3)를 본떠, 추측을 빈칸에 적는 것이 아니라 **자신이 탐색으로 확인한 실제 코드(status 필드·404 패턴)에 근거해** 각 항목을 검증 가능한 문장으로 적습니다.

아래 표를 그대로 옮겨, 오른쪽 칸에 자신의 분석 결과를 직접 적으세요(예시를 베끼지 말고, 내 저장소에서 확인한 것으로 적습니다).

항목	내가 작성한 내용
요구사항 ID	FR2
검증 가능한 문장	(여기에 직접 적는다: PATCH 경로와 status가 어떻게 바뀌는지)
입력	(여기에 직접 적는다: 어떤 경로 파라미터로 어느 작업을 가리키나)
정상 결과	(여기에 직접 적는다: 상태 코드와 바뀐 status 값)
비정상 결과(없는 id)	(여기에 직접 적는다: 내 저장소의 실제 404 패턴을 확인해 적는다)
적용 개념	(여기에 직접 적는다: 본문 추적성 표에서 FR2에 연결된 단위·개념)

작성 안내: 입력은 경로 파라미터(어느 작업?)입니다. 정상 결과의 상태 코드와 바뀐 status 값을 적으세요. 비정상은 "없는 id를 완료 처리하려 하면?"을 생각해, **추측이 아니라 ch\_13에서 탐색한 기존 저장소의 404 패턴**을 옮겨 적습니다.

## 연습 2: 추적성을 직접 이어 작성하라

아래 요구사항이 어느 단위·개념으로 구현되는지, 본문의 추적성 표를 참고해 **직접** 작성하세요. 단위·개념은 본문 표에서 찾아 적고, "한 줄 근거"는 베끼지 말고 **자신의 말로** 왜 그 개념이 그 요구사항을 구현하는지 적습니다.

아래 표를 그대로 옮겨, 빈 칸에 자신이 이어 낸 단위·개념·근거를 직접 적으세요.

요구사항	적용 단위(직접 적기)	적용 개념(직접 적기)	한 줄 근거(왜 그 개념인가 — 내 말로)
FR4 없는 id 404·잘못된 status 422	(직접 적는다)	(직접 적는다)	(직접 적는다)
FR5 두 엔드포인트를 pytest로 검증	(직접 적는다)	(직접 적는다)	(직접 적는다)
NFR3 변경 전 플랜 모드로 계획 검토	(직접 적는다)	(직접 적는다)	(직접 적는다)

## 자가체크

- [ ] 모든 기능 요구사항(FR)을 "테스트로 옮길 수 있는" 검증 가능한 문장으로 적었는가?
- [ ] 기능 요구사항(FR)과 비기능 요구사항(NFR)을 "무엇을/어떻게"로 구분했는가?
- [ ] 정상 경로뿐 아니라 **비정상 경로**(없는 id 404·잘못된 status 422)를 요구사항에 포함했는가?
- [ ] 범위 Out에 "이번에 하지 않을 것"을 명시해 첫 기능을 작게 통제했는가?
- [ ] 각 요구사항이 어느 단위·개념으로 구현되는지 추적성으로 이어 두었는가?
- [ ] 기존 저장소를 **변경 없이 읽기만** 하며 탐색했는가(탐색 중 코드를 바꾸지 않았는가)?

## 흔한 오류와 오개념

### 1. 기존 코드를 안 보고 바로 만들기 시작한다

신입이 가장 자주 하는 실수는 받은 저장소를 파악하지 않고 곧장 코드를 쓰는 것입니다. 그러면 이미 있는 `status` 필드를 모른 채 새 필드를 만들거나, 팀의 404 패턴과 다른 방식으로 에러를 내 일관성이 깨집니다. **분석은 만들기 전에 무엇이 이미 있는지 아는 단계**입니다. ch\_04의 탐색(c04)으로 먼저 읽고, 그다음에 요구사항을 적으세요.

### 2. 요구사항을 "잘 만든다"처럼 막연하게 적는다

"완료 기능을 잘 만든다"는 요구사항이 아닙니다. 무엇이 "잘"인지 검증할 수 없기 때문입니다. 요구사항은 **입력·정상 결과·비정상 결과**를 담아 그대로 테스트로 옮길 수 있어야 합니다. "테스트로 옮길 수 있는가?"를 매 요구사항마다 자문하세요.

### 3. 범위를 키워 첫 PR을 끝내지 못한다

"기왕 하는 김에" 인증·데이터베이스·UI까지 붙이려다 첫 PR을 영원히 올리지 못하는 함정입니다. 범위 Out을 명시하고 MVP(완료 처리 하나)부터 완성하세요. **작게 끝내고 다음 PR로 키우는** 것이 크게 벌이고 못 끝내는 것보다 항상 낫습니다.

### 4. 비기능 요구사항(NFR)을 무시한다

"동작만 하면 됐지"라며 테스트·권한·플랜·CLAUDE.md를 건너뛰면, 기능은 되지만 리뷰에서 막히고 팀의 신뢰를 잃습니다. 신입의 첫 업무는 **결과만큼 일하는 방식**으로 평가받습니다. FR과 NFR을 함께 채우세요.

## 이 챕터 한 줄 정리

분석은 드라이빙 퀘스천·POV로 방향을 잡고, 기존 저장소를 **변경 없이 탐색**해 파악한 뒤, 무엇을 만드는지(FR)와 어떻게 만드는지(NFR)를 검증 가능한 요구사항으로 적고, 범위(In/Out)로 작게 통제하며, 각 요구사항을 **단원 개념으로 추적**하는 단계입니다. 다음 챕터(ch\_14)에서는 이 요구사항을 **설계**(데이터 구조·모듈 변경·ADR·순서도)로 옮깁니다.



# 종합 프로젝트: 첫 기능·첫 PR — 설계(데이터·모듈·ADR)

## 이 챕터에서 배우는 것

- 추가할 두 엔드포인트의 요청·응답 데이터 구조(필드·타입·상태 값)를 모델로 설계해 명시할 수 있습니다.
- 기존 모듈(라우터·모델·저장소·테스트)에 변경을 어디에 어떻게 끼울지, 모듈·함수 단위로 분해해 목록화할 수 있습니다.
- 플랜 모드로 도출한 설계 결정을 ADR 5필드(결정·맥락·선택지·근거·결과)로 기록할 수 있습니다.
- 입력→검증→상태 변경→응답의 요청 처리 순서를 다이어그램 1장으로 표현할 수 있습니다.

## 설계 개요: 기존 구조에 최소 변경으로 끼우기

앞 챕터(ch\_13)에서 우리는 분석을 마쳤습니다. 무엇을 만드는지(FR: 완료 처리·상태 필터·에러 응답)와 어떻게 만드는지(NFR: 테스트·안전·팀 규약)를 검증 가능한 요구사항으로 적고, 범위를 작게 통제했으며, 기존 저장소가 `id·title·status`를 가진 인메모리 tasks API임을 탐색으로 파악했습니다. 이제 그 요구사항을 **실제 코드 구조**로 옮기는 **설계(design)** 단계입니다.

설계의 첫 원칙은 이미 정해져 있습니다. **기존 구조에 최소 변경으로 끼웁니다.** 우리는 빈 폴더에서 새 앱을 만드는 것이 아니라, **이미 동작하는 팀의 코드**에 작은 기능을 더하는 신입입니다. 그러므로 새 구조를 발명하기보다, 탐색으로 파악한 기존 패턴(라우터에 엔드포인트 정의, 없는 id는 404, 저장소 함수로 데이터 접근)을 **그대로 따라** 최소한만 더합니다.

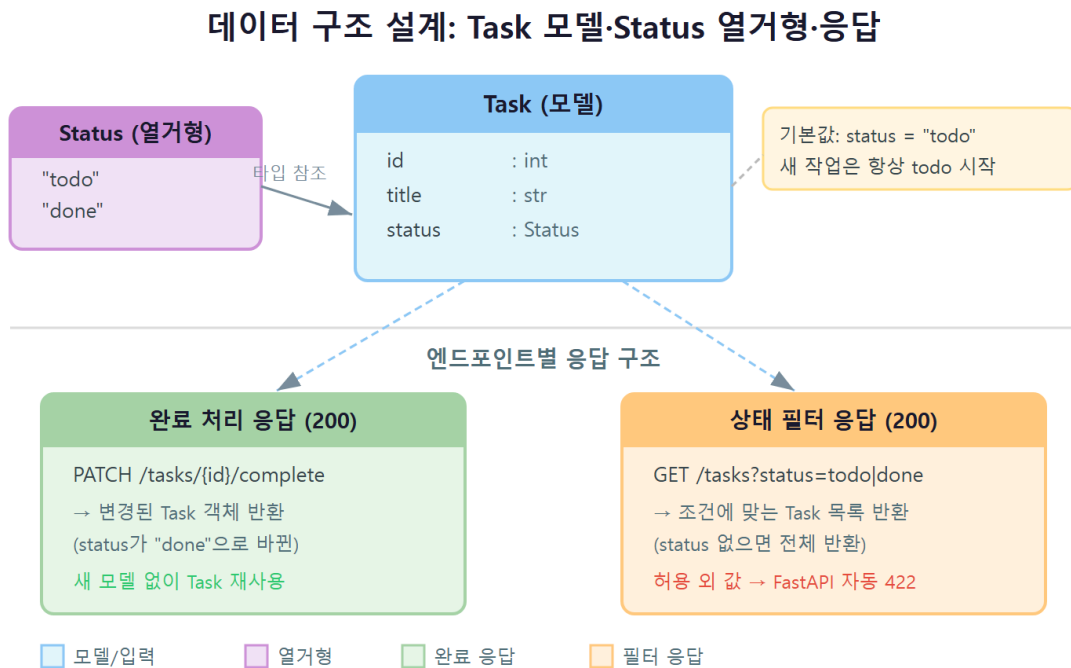
## 설계 단계도 새 개념을 배우지 않습니다

이 챕터의 설계는 앞 단원 개념을 통합 적용합니다. 데이터 구조·모듈 분해는 ch\_04 탐색(c04)으로 파악한 기존 구조 위에 얹고, ADR(설계 결정 기록)은 ch\_06 플랜 모드(c06)로 계획을 받아 검토하며 도출하고, 에러 처리 설계(404·422)는 ch\_11 디버깅(c11)의 시각으로 봅니다. 설계는 "코

드를 쓰기 전에 머릿속 그림을 종이에 옮기는" 단계이며, 그 그림은 구현(ch\_15)의 설계도가 됩니다.

이 챕터의 설계는 세 가지를 산출합니다. (1) **데이터 구조**(요청·응답 모델과 상태 필드), (2) **모듈 변경 분해**(어느 파일을 어떻게 바꾸는가), (3) **ADR**(왜 그렇게 결정했는가). 그리고 이 셋을 묶는 **요청 처리 순서**로 마무리합니다.

## 데이터 구조 설계: 요청·응답 모델과 상태 필드



데이터 구조 설계란 "이 기능이 주고받는 데이터가 어떤 모양인가"를 정하는 일입니다. 우리 기능이 다루는 데이터는 세 가지입니다.

**첫째, 할 일 한 건(Task)의 모양.** 탐색에서 확인했듯 `status` 필드는 이미 존재합니다. 우리는 이 필드의 값을 명확히 두 가지( `todo` · `done` )로 제한하기 위해 **문자열 열거형(enum, 이넘)**으로 표현합니다. 열거형은 "허용된 값이 정해진 목록"을 코드로 못박는 방법입니다.



```

from enum import Enum
from pydantic import BaseModel

class Status(str, Enum):
 todo = "todo"
 done = "done"

class Task(BaseModel):
 id: int
 title: str
 status: Status = Status.todo

```

- `class Status(str, Enum)` 는 문자열이면서 열거형인 타입을 만듭니다. `str` 을 함께 상속해, JSON 응답에서 `"todo"` · `"done"` 같은 **문자열로 그대로** 나가게 합니다. 허용된 값이 이 둘로 고정되므로, 다른 값(`"foo"` 등)은 들어올 수 없습니다.
- `Task` 는 기존 모델을 그대로 따릅니다. `status: Status = Status.todo` 는 "status는 Status 타입이고 기본값은 todo"라는 뜻입니다. 기존 저장소의 동작(새 작업은 todo로 시작)을 바꾸지 않으면서, 타입만 문자열에서 열거형으로 좁혔습니다.
- 왜 불리언(`done: bool`)이 아니라 열거형일까요? 지금은 두 상태뿐이지만, 나중에 `in_progress` (진행 중) 같은 상태가 생길 수 있습니다. 열거형은 값을 한 줄 추가하는 것만으로 확장되지만, 불리언은 구조를 갈아엎어야 합니다(이 결정의 근거는 뒤의 ADR2에서 정식으로 기록합니다).

**둘째, 완료 처리의 응답(complete response).** 완료 처리는 새 데이터를 만들지 않고 **기존 task의 status만 바꿉니다**. 그러므로 응답은 별도 모델이 아니라, **변경된 Task 객체 그 자체**(status가 `done`이 된)입니다. 클라이언트(요청한 쪽)는 응답을 보고 "정말 done이 됐구나"를 바로 확인할 수 있습니다.

**셋째, 상태 필터의 입력(filter query).** 상태 필터는 목록 엔드포인트에 **쿼리 파라미터(query parameter)** `status` 를 더해 거릅니다. 쿼리 파라미터는 URL의 `?` 뒤에 붙는 **이름=값** 형식의 입력입니다(`/tasks?status=done`). 받을 수 있는 값을 위에서 정의한 `Status` 열거형으로 제한하면, 허용 외 값은 FastAPI가 자동으로 막아 줍니다(뒤에서 다룰 422).

세 데이터 구조를 표로 정리하면 이렇습니다.

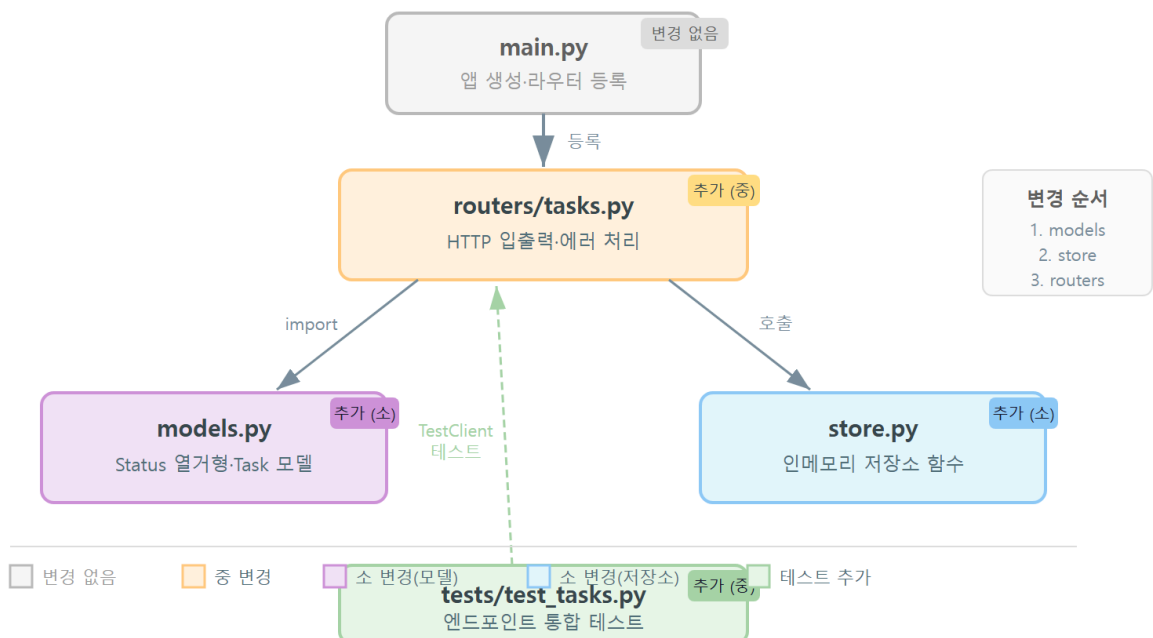
데이터	모양	쓰이는 곳
Task	<code>{id: int, title: str, status: Status}</code>	모든 응답의 기본 단위
status 열거형	<code>Status = todo \  done</code> (문자열)	완료 처리의 결과 값, 필터의 입력 값
완료 응답	변경된 Task 객체(status=done)	PATCH /tasks/{id}/complete의 200 응답
필터 입력	쿼리 파라미터 <code>status (todo done)</code>	GET /tasks?status= 의 입력

## 새 데이터를 만들기보다 기존을 좁히고 재사용합니다

우리 설계의 데이터 구조는 거의 전부 **기존 Task의 재사용**입니다. 완료 응답은 Task 그대로, 필터 입력은 기존 status 값 그대로입니다. 새로 더한 것은 "status를 열거형으로 좁힌 것" 하나뿐입니다. 신입의 좋은 설계는 화려한 새 구조가 아니라, **기존 것을 최대한 재사용하면서 필요한 만큼만 좁히거나 더하는** 절제에서 나옵니다.

## 모듈·함수 변경 분해 (라우터·모델·저장소·테스트)

### 모듈 변경 분해: 의존 관계와 변경 범위



이제 "어느 파일을 어떻게 바꾸는가"를 정합니다. 이를 **모듈 변경 분해(module decomposition)**라고 합니다. 큰 변경을 한 덩어리로 보지 않고, **파일·함수 단위의 작은 변경 묶음**으로 쪼개면, 각 변경이 무엇을 책임지는지 분명해지고 구현·커밋 단위(ch\_09)로도 자연스럽게 이어집니다.

탐색으로 파악한 기존 구조(main·routers·models·store·tests)에 우리 변경을 어떻게 끼울지 정리하면 다음과 같습니다.

모듈(파일)	기존 책임	이번 변경	변경 크기
main.py	FastAPI 앱 생성·라우터 등록	변경 없음(기존 라우터에 더하므로)	없음
routers/tasks.py	tasks 엔드포인트 정의	완료 처리·상태 필터 엔드포인트 <b>추가</b>	중
models.py	pydantic 모델	status를 Status 열거형으로 정의	소
store.py	인메모리 저장소 함수	상태 변경·필터 조회 함수 <b>추가</b>	소
tests/test_tasks.py	기존 엔드포인트 테스트	추가 엔드포인트의 테스트 <b>추가</b> (또는 새 파일)	중

각 모듈의 변경을 한 걸음 더 들여다봅니다.

- **models.py**: 위에서 설계한 Status 열거형을 정의하고, Task.status의 타입을 그 열거형으로 둡니다. 가장 작은 변경이지만, 다른 모듈이 이 타입을 함께 쓰므로 **가장 먼저** 자리잡혀야 합니다.
- **store.py**: 저장소에 두 함수를 더합니다. 하나는 "특정 id의 작업 상태를 done으로 바꾸는" 함수, 다른 하나는 "주어진 status로 목록을 거르는" 함수입니다. 엔드포인트가 직접 리스트를 만지지 않고 이 함수를 통하게 해, 데이터 접근을 한곳에 모읍니다(기존 패턴 유지).
- **routers/tasks.py**: 실제 엔드포인트 두 개를 더합니다. 라우터는 "요청을 받아 → 검증하고 → store 함수를 호출하고 → 응답을 돌려주는" 얇은 층입니다. 데이터 처리는 store에 맡기고, 라우터는 HTTP의 입출력·에러(404·422)만 책임집니다.

- `tests/test_tasks.py`: 분석의 유스케이스(UC1·UC2)와 에러 요구사항(FR4)을 테스트로 옮깁니다. 정상 경로(완료되면 200·done, 필터하면 거른 목록)와 비정상 경로(없는 id 404, 잘못된 status 422)를 모두 검증합니다.
- `main.py`가 **변경 없음**이라는 점이 좋은 신호입니다. 기존 라우터에 엔드포인트를 더하는 방식이라, 앱 진입점은 건드리지 않아도 됩니다. 변경이 닿는 파일이 적을수록 위험과 리뷰 부담이 줄입니다.
- 변경 순서는 **의존성 방향**을 따릅니다. models(타입) → store(데이터 함수) → routers(엔드포인트) → tests(검증). 다만 NFR4(TDD)에 따라 **실제 구현에서는 테스트를 먼저** 쓰고 그다음 구현합니다(ch\_15에서 다룸). 분해도는 "무엇이 무엇에 의존하는가"의 그림이고, 구현 순서는 TDD가 정합니다.
- 각 모듈 변경이 **커밋 단위 후보**입니다(ch\_09). 예컨대 "models·store 추가"와 "라우터 엔드포인트 추가"를 별도 논리 단위로 나눌 수 있습니다. 설계 분해가 곧 커밋 계획으로 이어 집니다.

## 설계 결정 기록: ADR 5건 (플랜 모드로 도출)

설계에는 **선택**이 따릅니다. 완료 처리를 PATCH로 할까 POST로 할까? 상태를 불리언으로 둘까 열거형으로 둘까? 이런 선택을 그냥 코드에만 남기면, 나중에 "왜 이렇게 했지?"라는 질문에 아무도 답하지 못합니다. 그래서 **ADR(Architecture Decision Record, 아키텍처 결정 기록)**로 결정의 **이유**를 남깁니다.

ADR은 다음 **5필드**로 한 결정을 기록합니다.

필드	무엇을 적나
결정(Decision)	무엇으로 정했는가
맥락(Context)	어떤 상황·문제에서 결정하는가
선택지(Options)	고려한 대안들은 무엇인가
근거(Rationale)	왜 그 선택지를 골랐는가
결과(Consequences)	이 결정이 가져올 영향(좋은·나쁜)

이 ADR들은 **플랜 모드(c06)**로 도출합니다. 변경을 시작하기 전에 에이전트에게 "어떤 설계 선택지가 있고 무엇을 추천하는지" 읽기 전용으로 계획을 받아, 그 계획을 검토하며 결정을 확정

합니다. 플랜 모드는 "바꾸기 전에 먼저 생각하고 합의하는" 단계이고, ADR은 그 합의의 기록입니다.

다음은 이번 프로젝트의 핵심 설계 결정 다섯 건입니다.

## ADR1. 완료 처리 메서드 선택

필드	내용
결정	<code>PATCH /tasks/{id}/complete</code>
맥락	기존 task의 일부 상태(status)만 바꾼다
선택지	PATCH 전용 경로 · PUT 전체 교체 · POST 액션
근거	부분 상태 변경에는 PATCH가 의미상 맞고 <code>/complete</code> 경로가 의도를 명확히 드러낸다
결과	나중에 전체 교체가 필요해지면 별도 PUT을 추가하면 된다

PATCH는 "자원의 일부만 수정"을 뜻하는 HTTP 메서드입니다. 완료 처리는 title은 그대로 두고 status만 바꾸므로, 전체를 교체하는 PUT보다 PATCH가 정확합니다.

## ADR2. 상태 표현 방식

필드	내용
결정	status 필드에 문자열 열거형(todo   done)
맥락	완료 여부를 표현해야 한다
선택지	불리언 done · 문자열 열거형 · 별도 상태 테이블
근거	향후 상태 확장(in_progress 등) 여지를 위해 열거형을 택한다
결과	불리언보다 검증 코드가 약간 늘지만 확장에 유연하다

## ADR3. 필터 인터페이스

필드	내용
결정	GET /tasks에 <code>status</code> 쿼리 파라미터
맥락	기존 목록 엔드포인트를 거른다
선택지	쿼리 파라미터 · 별도 경로(/tasks/done) · 본문 필터
근거	REST 관례상 목록 필터는 쿼리 파라미터( <code>?status=done</code> )가 표준이다
결과	필터가 늘면 파라미터가 많아질 수 있어 문서화가 필요하다

## ADR4. 에러 처리 정책

필드	내용
결정	없는 id는 404, 잘못된 status 값은 422
맥락	신입도 일관된 에러를 내야 한다
선택지	일괄 400 · 표준 상태 코드 · 커스텀 코드
근거	FastAPI·HTTP 관례를 따르는 표준 상태 코드가 클라이언트에 명확하다
결과	프론트가 상태 코드로 분기할 수 있고, 메시지 일관성을 유지해야 한다

## 사실 확인: 404와 422는 의미가 다릅니다

- **404 Not Found:** 요청한 자원이 **없을** 때입니다. 예: 존재하지 않는 `PATCH /tasks/999/complete`. 우리가 직접 `HTTPException(status_code=404, ...)` 로 냅니다(기존 저장소 패턴과 동일).
- **422 Unprocessable Entity:** 요청 형식은 맞지만 **값이 검증을 통과하지 못할** 때입니다. 예: `GET /tasks?status=foo` 처럼 허용 외 값. `status`를 `Status` 열거형으로 받게 하면, FastAPI가 **자동으로 422**를 냅니다(우리가 코드를 따로 쓰지 않아도 됩니다). 둘을 섞어 일괄 400으로 내면 클라이언트가 "없는 것"과 "잘못 보낸 것"을 구분하지 못합니다.

## ADR5. 작업 방식(Claude Code 워크플로)

필드	내용
결정	탐색 → 플랜 모드 계획 → TDD 구현 → 커밋·PR → 리뷰 대응
맥락	신입이 안전하게 첫 기능을 추가해야 한다
선택지	바로 편집 · 탐색·플랜·TDD·리뷰 흐름
근거	안전(권한·플랜)·검증(TDD)·협업(커밋·PR·리뷰)을 결합해 신뢰성을 확보한다
결과	절차가 늘지만 첫 업무를 안전하고 검증 가능하게 수행한다

ADR5는 코드 구조가 아니라 **일하는 방식 자체**에 대한 결정입니다. 이 결정이 곧 구현 챕터(ch\_15)의 마일스톤 순서(셋업·계획 → 테스트 먼저 → 구현 → 커밋·PR → 리뷰)가 됩니다. 종합 프로젝트의 12개 개념이 어떻게 하나의 흐름으로 묶이는지를 한 줄로 요약한 결정이기도 합니다.

## 요청 처리 순서도: 입력 → 검증 → 상태 변경 → 응답



데이터 구조와 모듈 분해, ADR을 모두 정했다면, 마지막으로 **요청이 들어와 응답이 나가기까지의 처리 순서**를 그립니다. 이 순서도는 구현(ch\_15)에서 그대로 코드의 골격이 됩니다. 두 엔드

포인트의 처리 순서를 차례로 봅니다.

**완료 처리(PATCH /tasks/{id}/complete)의 순서**는 다음과 같습니다.

1. 입력: 경로에서 `id` (완료할 작업 번호)를 받는다.
2. 검증: 저장소에서 그 `id`의 작업을 찾는다. **없으면 404**로 응답하고 끝낸다.
3. 상태 변경: 찾은 작업의 `status`를 `done`으로 바꾼다.
4. 응답: 변경된 작업 객체를 **200**으로 돌려준다.

**상태 필터(GET /tasks?status=)의 순서**는 다음과 같습니다.

1. 입력: 쿼리 파라미터에서 `status`를 받는다.
2. 검증: `status`가 허용 값(todo-done)인지 확인한다. **허용 외면 422**(FastAPI가 자동 처리).
3. 조회: 저장소에서 그 `status`인 작업만 거른다. `status`가 없으면 전체를 돌려준다.
4. 응답: 거른 목록을 **200**으로 돌려준다.

두 순서를 표로 비교하면 "정상 흐름"과 "에러 분기"가 한눈에 들어옵니다.

단계	완료 처리(PATCH)	상태 필터(GET)
입력	경로 파라미터 <code>id</code>	쿼리 파라미터 <code>status</code>
검증	작업 존재? 없으면 404	허용 값? 아니면 422(자동)
처리	<code>status</code> 를 <code>done</code> 으로 변경	<code>status</code> 로 목록 필터
정상 응답	200 + 변경된 작업	200 + 거른 목록

설계대로 완료 처리 엔드포인트의 골격을 코드로 옮기면 다음과 같습니다. 아직 구현 챕터가 아니므로 **설계가 어떻게 코드 모양으로 이어지는지** 확인하는 용도로 봅니다.



```

from fastapi import APIRouter, HTTPException

import store
from models import Task, Status

router = APIRouter()

@router.patch("/tasks/{task_id}/complete", response_model=Task)
def complete_task(task_id: int) -> Task:
 task = store.get_task(task_id)
 if task is None:
 raise HTTPException(status_code=404, detail="task를 찾을 수 없습니다")
 task.status = Status.done
 store.save_task(task)
 return task

```

- `@router.patch(...)` 이 ADR1의 결정(PATCH 메서드·`/complete` 경로)을 코드로 옮긴 것입니다. 순서도의 1번(입력)이 함수 매개변수 `task_id`로 들어옵니다.
- `if task is None: raise HTTPException(status_code=404, ...)` 가 순서도 2번(검증·404)이며, **기존 저장소의 404 패턴을 그대로** 따랐습니다(ch\_04 탐색에서 확인한 패턴 재사용).
- `task.status = Status.done` 이 순서도 3번(상태 변경), `return task` 가 4번(200 응답)입니다. `response_model=Task`로 응답이 Task 모양임을 명시해, 완료 응답이 곧 변경된 Task라는 ADR/데이터 설계와 일치시켰습니다.
- 데이터 접근은 `store.get_task`·`store.save_task` 함수를 통합합니다. 라우터가 리스트를 직접 만지지 않고 store에 맡겨, 모듈 분해(라우터=HTTP, store=데이터)를 지킵니다.

이렇게 분석(ch\_13)의 요구사항이 설계(ch\_14)의 데이터 구조·모듈 분해·ADR·순서도를 거쳐, 다음 챕터(ch\_15)의 구현으로 매끄럽게 이어집니다. 설계가 충실하면 구현은 "설계도를 코드로 옮기는" 비교적 단순한 작업이 됩니다.

## 작성형 연습 (2종 1세트)

설계를 직접 해 봅니다. 먼저 ① 모범 예시를 확인하고, ② 빈 템플릿을 스스로 채운 뒤, ③ 자가 체크로 점검합니다.

### 모범 예시: ADR 한 건 작성

"상태 필터 결과를 어떻게 정렬할까"라는 작은 설계 결정을 ADR 5필드로 적으면 다음과 같습니다.

필드	내용
결정	필터 결과를 id 오름차순으로 반환한다
맥락	필터한 목록의 순서가 정해지지 않으면 응답이 들쭉날쭉해 테스트가 불안정하다
선택지	정렬 안 함(저장 순서) · id 오름차순 · 생성 시각 내림차순
근거	id 오름차순은 단순하고 예측 가능하며, 인메모리 저장소에서 추가 비용이 거의 없다
결과	순서가 고정돼 테스트가 안정적이고, 나중에 정렬 옵션이 필요하면 쿼리 파라미터로 확장한다

핵심은 "정렬을 한다/안 한다"의 한 줄 결정에도 **맥락·선택지·근거·결과**를 함께 남겨, 나중에 누군가 "왜 이렇게 했는지" 알 수 있게 한 점입니다.

## 연습 1: 내 설계 결정을 ADR로 직접 작성하라

"완료된 작업을 다시 완료 처리하면(중복 완료) 어떻게 할까"라는 설계 결정을 내려, 위 모범 예시를 본떠 ADR 5필드로 **직접 작성**하세요. 정답을 빈칸에 맞추는 것이 아니라, **자신이 실제로 어떻게 설계하기로 정했는지를** 적는 것이 핵심입니다.

아래 표를 그대로 옮겨, 오른쪽 칸에 내가 내린 결정을 직접 적으세요.

필드	내가 작성한 ADR
결정	(여기에 직접 적는다: 내가 택한 한 줄 결정)
맥락	(여기에 직접 적는다: 왜 이 결정이 필요한 상황인가)
선택지	(여기에 직접 적는다: 내가 비교한 대안들)
근거	(여기에 직접 적는다: 왜 그 선택지를 골랐나)
결과	(여기에 직접 적는다: 이 결정으로 무엇이 달라지나)

작성 안내: 선택지로 "이미 done이면 에러(409)를 낸다 / 그냥 200으로 현재 상태를 돌려준다 / 아무 변화 없이 멍등하게 둔다"를 떠올려 보세요. 신입의 첫 기능이라는 맥락에서 무엇이 더 단

순하고 안전한지 **자신의 판단과 근거**를 직접 적습니다.

## 연습 2: 상태 필터의 처리 순서와 호출을 직접 설계해 작성하라

상태 필터(GET /tasks?status=)를 어떻게 설계할지, 처리 순서와 코드를 **직접 작성**하세요. 빈칸을 맞추는 것이 아니라, 입력·검증·조회·응답을 자신의 설계로 한 단계씩 적습니다.

처리 순서를 자신의 말로 직접 적으세요(각 단계가 무엇을 받아 무엇을 하는지):

1. 입력: (직접 적는다: 어떤 쿼리 파라미터를 받나)
2. 검증: (직접 적는다: 허용 외 값이면 어떤 상태 코드로 응답하나)
3. 조회: (직접 적는다: 저장소에서 무엇을 거르나)
4. 응답: (직접 적는다: 무엇으로 거른 목록을 돌려주나)

아래 코드를 옮겨, 마지막 줄의 `store` 호출을 **자신의 설계대로 직접 완성**해 적으세요(무엇을 넘길지 정해 직접 작성합니다).

```
from fastapi import APIRouter, Query
from typing import Optional

import store
from models import Task, Status

router = APIRouter()

@router.get("/tasks", response_model=list[Task])
def list_tasks(status: Optional[Status] = Query(default=None)) -> list[Task]:
 # status가 None이면 전체, 값이 있으면 그 status만 거른다
 # 아래 한 줄을 내 설계대로 직접 완성해 적는다(store에 무엇을 넘길지)
 return store.list_tasks(...)
```

작성 안내: 매개변수 타입을 `Optional[Status]` 로 두면 허용 외 값에 FastAPI가 자동으로 422를 냅니다. `store` 함수에는 받은 `status` 를 그대로 넘기도록 설계하면 됩니다. 다음 챕터(ch\_15)에서 이 설계를 실제 저장소에 구현합니다.

## 자가체크

- [ ] `status`를 불리언이 아니라 **열거형(todo·done)**으로 설계해 확장 여지를 남겼는가?
- [ ] 완료 응답을 새 모델이 아니라 **변경된 Task 재사용**으로 설계했는가?
- [ ] 모듈 변경 분해에서 각 파일의 책임(라우터=HTTP, store=데이터)을 분리했는가?
- [ ] 각 ADR이 **5필드(결정·맥락·선택지·근거·결과)**를 모두 갖췄는가?
- [ ] 에러 설계에서 **없는 id=404, 잘못된 status=422**를 구분했는가?

- [ ] 요청 처리 순서도에 정상 경로와 **에러 분기**를 함께 그렸는가?
- [ ] 설계를 ch\_06 **플랜 모드**로 받아 검토하며 도출했는가(바로 편집하지 않았는가)?

## 흔한 오류와 오개념

### 1. 기존 구조를 무시하고 새 구조를 발명한다

신입이 의욕이 앞서 "더 좋은 구조"를 새로 만들려다, 팀의 기존 패턴(라우터·store 분리, 404 패턴)과 어긋난 코드를 내놓는 일이 흔합니다. 설계의 기본은 **기존 구조에 최소 변경으로 끼우기**입니다. 탐색(c04)으로 파악한 패턴을 그대로 따르고, 정말 필요한 만큼만 더하세요. 새 구조는 리뷰에서 "왜 기존과 다르냐"는 질문을 부릅니다.

### 2. 결정만 적고 이유(맥락·근거·결과)를 빠뜨린다

"PATCH로 한다"만 적으면 ADR이 아닙니다. 나중에 "왜 POST가 아니라 PATCH지?"라는 질문에 답할 수 없기 때문입니다. ADR의 가치는 **결정이 아니라 이유**에 있습니다. 맥락·선택지·근거·결과를 함께 남겨야, 미래의 동료(그리고 미래의 나)가 결정을 이해하고 안전하게 바꿀 수 있습니다.

### 3. 404와 422를 일괄 400으로 뭉갠다

"에러면 다 400"으로 처리하면, 클라이언트가 "없는 것(404)"과 "잘못 보낸 것(422)"을 구분하지 못합니다. **표준 상태 코드**를 의미에 맞게 쓰세요. 없는 자원은 404, 검증 실패는 422입니다. FastAPI는 열거형 쿼리 파라미터의 422를 자동으로 해주므로, 우리는 404만 직접 처리하면 됩니다.

### 4. 플랜 없이 바로 편집하며 설계한다

설계를 머릿속으로만 하고 곧장 코드를 쓰면, 잘못된 방향으로 여러 파일을 바꾼 뒤에야 문제를 발견합니다. **플랜 모드(c06)로 먼저 계획을 받아** 데이터 구조·모듈 분해·결정을 검토하고, 합의된 설계를 ADR로 남긴 다음 구현에 들어가세요. "바꾸기 전에 생각한다"가 안전한 신입의 기본기입니다.

## 이 챕터 한 줄 정리

설계는 기존 구조에 **최소 변경**으로 끼우는 원칙 아래, 데이터 구조(status 열거형·Task 재사용)와 모듈 변경 분해(라우터·모델·저장소·테스트)를 정하고, 결정의 이유를 **ADR 5필드**로 남기며, 요청

처리 순서를 순서도로 그려 구현의 설계도를 완성하는 단계입니다. 다음 챕터(ch\_15)에서는 이 설계를 **TDD·마일스톤 증분**으로 실제 구현하고 첫 PR을 올립니다.



# 종합 프로젝트: 첫 기능·첫 PR — 구현(마일스톤 증분)

## 이 챕터에서 배우는 것

- CLAUDE.md·권한·플랜 모드부터 테스트·구현·커밋·PR·리뷰까지를 5개 마일스톤(M1~M5)의 증분으로 구현할 수 있습니다.
- 각 마일스톤이 어느 단원 개념(c01~c12)을 적용했는지 마일스톤↔개념으로 짝지어 설명할 수 있습니다.
- 각 마일스톤 끝에서 pytest 테스트와 실행 결과로 동작을 확인해, 체크포인트 통과 여부로 검증할 수 있습니다.
- 5개 마일스톤을 통합해 테스트가 통과하는 두 엔드포인트와, 변경 요약·근거·테스트 결과를 담은 첫 PR을 완성할 수 있습니다.

이 챕터는 종합 프로젝트의 **구현(implementation)** 단계입니다. 새 개념을 배우지 않습니다. 앞의 12개 개념 챕터(c01~c12)와 분석(ch\_13)·설계(ch\_14)에서 정한 요구 사항·ADR을 실제 코드로 옮기고, 그 과정을 안전·검증·협업의 흐름으로 엮습니다.

## 구현 전략과 마일스톤 개요

분석(ch\_13)에서 우리는 기존 팀 저장소 `team-tasks-api` 에 두 가지 기능을 더하기로 정했습니다.

- **완료 처리** — `PATCH /tasks/{task_id}/complete` : 특정 할 일의 상태를 `done` 으로 바꿉니다(FR2).
- **상태 필터** — `GET /tasks?status=done|todo` : 목록을 상태로 거릅니다(FR3).

그리고 없는 `id` 는 404, 잘못된 `status` 는 422로 응답하기로 했습니다(FR4). 설계(ch\_14)에서는 상태를 불리언이 아니라 `todo|done` 문자열 enum으로 표현하기로(ADR2), 완료 처리는 부분 변경이므로 PATCH로(ADR1), 필터는 쿼리 파라미터로(ADR3) 결정했습니다.

이제 이것을 **한 번에 다 만들지 않습니다**. 신입이 첫 기능을 안전하게 추가하는 가장 좋은 방법은, 검증 가능한 작은 단계(마일스톤)로 나누어 한 걸음마다 통과를 확인하며 나아가는 것입니다. 이 챕터는 다음 여섯 마일스톤을 증분으로 진행합니다. 각 마일스톤은 앞 단원에서 배운 개념을 적용하며, 끝에는 반드시 **체크포인트**로 다음 단계로 넘어가도 되는지 확인합니다.

마일스톤	하는 일	적용 단위·개념	체크포인트
M1 셋업·계획	CLAUDE.md·권한·플랜 모드로 환경과 계획을 갖추	u5·u7 / c05·c06·c08	계획이 검증 가능한 단계로 승인되고 CLAUDE.md·권한이 준비됨
M2 테스트 먼저	두 엔드포인트의 기대 동작을 pytest로 먼저 작성(TDD)	u3·u6 / c03·c07	테스트가 기대 동작을 표현하고 아직 실패함
M3 구현·검증	두 엔드포인트를 구현하고 pytest를 통과시킴	u2·u6 / c02·c07	두 엔드포인트의 테스트가 모두 통과
M4 커밋·PR	작은 논리 단위로 커밋하고 첫 PR 초안을 올림	u8 / c09	작업 브랜치 커밋과 PR 본문이 리뷰 가능하게 작성됨
M5 리뷰·디버깅	<code>/code-review</code> 로 점검하고 지적을 디버깅·반영	u9·u10 / c10·c11	타당한 지적 반영, 부적절한 지적에 근거 답변, 회귀 테스트 통과
M6 확장(선택)	테스트 실행을 서브에이전트에, 외부 자료를 MCP로	u11 / c12	서브에이전트 호출 또는 MCP 참조가 동작

## MVP를 먼저, 그다음 증분 확장

분석에서 정한 MVP(최소 기능)는 "완료 처리 엔드포인트 하나를 테스트와 함께 추가해 통과시키고 첫 커밋·PR을 올리는 것"이었습니다. 그래서 M2~M4에서는 두 엔드포인트를 함께 다루되, 막히면 **완료 처리 하나만** 먼저 통과시키고 상태 필터를 뒤에 붙여도 됩니다. "작게 만들어 통과시키고, 통과한 것 위에 더한다"가 증분 구현의 리듬입니다.

## M1: CLAUDE.md·권한·플랜 모드로 셋업·계획



코드를 한 줄도 바꾸기 전에, **안전하게 일할 환경**부터 갖추세요. 이 마일스톤은 단원 5(권한 c05·플랜 c06)와 단원 7(CLAUDE.md c08)의 적용입니다.

## 1) CLAUDE.md로 팀 규약을 영속화한다 (c08)

세션마다 "이 프로젝트는 어떻게 빌드하고 테스트하나요"를 다시 설명하지 않도록, 빌드·테스트 명령과 컨벤션을 CLAUDE.md에 적어 둡니다. /init로 골격을 만든 뒤 핵심만 간결하게 채웁니다.

```
team-tasks-api

할 일(tasks)을 관리하는 작은 FastAPI 서비스입니다.

빌드·실행·테스트
- 의존성 설치: `python -m pip install -r requirements.txt`
- 개발 서버: `python -m uvicorn app.main:app --reload`
- 테스트: `python -m pytest -q`

코딩 컨벤션
- 들여쓰기 4칸, 타입 힌트 필수
- 엔드포인트는 `app/routers/`에, 데이터 보관은 `app/store.py`에 둔다
- 상태(status)는 `todo|done` 문자열 enum으로 표현한다

금지사항
- main 브랜치에 직접 커밋하지 않는다(작업 브랜치 + PR)
- 테스트가 통과하지 않은 변경은 커밋하지 않는다
```

- **빌드·실행·테스트 명령**을 가장 먼저 적었습니다. 에이전트가 "검증해 줘"라는 지시를 받으면 여기 적힌 `python -m pytest -q`를 그대로 쓰므로, 팀이 정한 검증 방식이 자동으로 통일됩니다.
- **컨벤션**에 "상태는 todo|done 문자열 enum"을 박아 두었습니다. 설계(ADR2)의 결정을 메모리에 영속화해, 이번 구현은 물론 다음 작업도 같은 규칙을 따르게 합니다.
- **금지사항**은 짧고 구체적입니다. "main에 직접 커밋 금지"가 곧 M4의 작업 브랜치 규칙으로 이어집니다. 길고 추상적인 규칙보다 검증 가능한 짧은 규칙이 잘 지켜집니다.

## 2) 최소 권한 settings.json을 둔다 (c05)

이번 작업에 필요한 안전한 명령(테스트·서버 실행, app/\*\* 편집)만 허용하고, 파괴적 명령은 차단합니다. 프로젝트 공유 범위인 `.claude/settings.json`에 적어 체크인하면 팀 모두가 같은 안전 규칙을 공유합니다.

```
{
 "permissions": {
 "allow": [
 "Bash(python -m pytest:*)",
 "Bash(python -m uvicorn:*)",
 "Edit(app/**)",
 "Edit(tests/**)",
 "Bash(git status)",
 "Bash(git diff:*)",
 "Bash(git switch:*)",
 "Bash(git add:*)",
 "Bash(git commit:*)",
 "Bash(git push:*)",
 "Bash(gh pr create:*)"
],
 "deny": [
 "Bash(git push --force:*)",
 "Bash(git reset --hard:*)",
 "Bash(rm -rf:*)"
]
 }
}
```

- `Bash(python -m pytest:*)` 는 검증에 쓸 테스트 실행만, `Edit(app/**)` · `Edit(tests/**)` 는 코드와 테스트 디렉터리 편집만 허용합니다. 이번 작업에 **필요한 만큼만** 여는 최소 권한 원칙입니다.
- `deny` 의 명령들은 역사를 파괴적으로 바꾸거나 파일을 지웁니다. `deny` 가 `allow` 보다 우선하므로, 실수로라도 강제 푸시·하드 리셋이 실행되지 않습니다.
- 커밋·푸시·PR 명령을 미리 허용해 두면 M4에서 흐름이 끊기지 않습니다. 안전한 반복 작업은 허용하고, 위험한 변형만 꼭 집어 막는 설계입니다.

### 3) 플랜 모드로 단계별 계획을 받아 검토·확정한다 (c06)

이제 `Shift+Tab` 으로 플랜 모드(읽기 전용 계획 단계)에 들어가, 변경 계획을 먼저 받습니다. 코드를 바꾸지 않고 "어떻게 바꿀지"만 제안받아 검토하는 안전장치입니다.

나: (플랜 모드) `@app/routers/tasks.py` 와 `@app/store.py` 를 참고해서, `PATCH /tasks/{task_id}/complete`(완료 처리)와 `GET /tasks?status=` 필터를 추가하려고 해. 기존 스타일을 따르고, 없는 id는 404·잘못된 status는 422로. 바로 고치지 말고, 검증 가능한 단계로 나눈 구현 계획부터 세워 줘.

에이전트가 돌려준 계획을 검토합니다.

구현 계획(플랜):

1. app/models.py 에 Status(todo|done) enum을 추가하고 Task.status 필드에 적용
2. app/store.py 에 get\_task·save\_task 가 있는지 확인(없으면 추가)
3. tests/test\_tasks.py 에 완료 처리·필터의 기대 동작 테스트를 먼저 작성(실패 확인)
4. app/routers/tasks.py 에 complete\_task·list\_tasks(status) 구현
5. python -m pytest -q 로 전체 통과 확인

이 계획에서 한 가지를 교정합니다. 설계에서 status 는 처음부터 enum이었으므로 1번은 "추가"가 아니라 "확인·정리"가 맞습니다. 또 3번을 4번보다 **앞에** 두어 테스트 우선(TDD)을 분명히 합니다.

나: 좋아. 다만 status enum은 이미 설계에서 todo|done 으로 정했으니  
1번은 그 정의를 확인·정리하는 것으로 하고, 테스트(3번)를 구현(4번)보다  
먼저 진행하는 순서를 지켜 줘. 이 계획으로 확정하자.

## M1 체크포인트

다음은 모두 만족하면 M2로 넘어갑니다.

- [ ] CLAUDE.md 에 빌드·테스트 명령·컨벤션·금지사항이 들어 있는가
- [ ] .claude/settings.json 이 필요한 명령만 allow하고 파괴적 명령을 deny하는가
- [ ] 플랜 모드로 받은 계획이 **검증 가능한 단계**로 나뉘고, 교정해 확정했는가

## M2: 테스트 먼저(TDD) — 완료 처리·필터 테스트 작성

확정한 계획대로, **구현 전에 기대 동작을 테스트로 못 박습니다**(c07 TDD). 이때 단원 3의 좋은 지시 네 요소(목표·맥락·제약·완료 조건, c03)를 그대로 씁니다.

나: 아직 엔드포인트는 구현하지 말고, tests/test\_tasks.py 에 다음 기대 동작을  
검증하는 pytest 테스트부터 작성해 줘. fastapi TestClient 를 쓰고,  
각 테스트는 store 를 초기화한 상태에서 시작해야 해.

- 완료 처리: PATCH /tasks/{id}/complete 하면 200 과 status=done 이 온다
- 없는 id: PATCH /tasks/999/complete 는 404
- 상태 필터: GET /tasks?status=done 은 done 인 항목만 돌려준다
- 잘못된 status: GET /tasks?status=working 은 422

작성한 뒤 pytest 로 돌려서, 구현이 없으니 실패하는 것을 먼저 보여 줘.

에이전트가 작성한 테스트 파일입니다.

```
tests/test_tasks.py
import pytest
from fastapi.testclient import TestClient

from app import store
from app.main import app

client = TestClient(app)

@pytest.fixture(autouse=True)
def clear_store():
 # 테스트마다 저장소를 비워 서로 영향을 주지 않게 한다
 store.reset()
 yield
 store.reset()

def _seed_two_tasks():
 client.post("/tasks", json={"id": 1, "title": "온보딩 문서 읽기", "status": "todo"})
 client.post("/tasks", json={"id": 2, "title": "개발 환경 설정", "status": "done"})

def test_complete_task_sets_status_done():
 _seed_two_tasks()
 response = client.patch("/tasks/1/complete")
 assert response.status_code == 200
 assert response.json()["status"] == "done"

def test_complete_unknown_task_returns_404():
 _seed_two_tasks()
 response = client.patch("/tasks/999/complete")
 assert response.status_code == 404

def test_filter_done_returns_only_done():
 _seed_two_tasks()
 client.patch("/tasks/1/complete") # 1번도 done 으로 만든다
 response = client.get("/tasks", params={"status": "done"})
 assert response.status_code == 200
 assert all(task["status"] == "done" for task in response.json())

def test_filter_invalid_status_returns_422():
 response = client.get("/tasks", params={"status": "working"})
 assert response.status_code == 422
```

- `@pytest.fixture(autouse=True)` 로 매 테스트 전후에 `store.reset()` 을 호출합니다. 인메모리 저장소는 테스트 간에 데이터가 남을 수 있으므로, 격리해야 결과가 안정적입니다(설계의 인메모리 저장소 유지 결정에 맞춘 처리).

- `_seed_two_tasks()` 는 todo 1건·done 1건을 미리 넣는 보조 함수입니다. 같은 준비를 여러 테스트가 재사용해 중복을 줄입니다.
- 네 테스트는 분석의 유스케이스·요구사항과 1:1로 대응합니다. 완료 처리(UC1·FR2), 없는 id(FR4의 404), 필터(UC2·FR3), 잘못된 status(FR4의 422). **요구사항이 곧 테스트가 되는 것**이 TDD의 핵심입니다.
- 아직 `PATCH .../complete` 라우트도, `status` 필터도 없으므로 이 테스트들은 지금 실패해야 정상입니다.

테스트를 먼저 돌려 "빨강"을 확인합니다.

```
$ python -m pytest -q
F.FF [100%]
===== FAILURES =====
----- test_complete_task_sets_status_done -----
> assert response.status_code == 200
E assert 404 == 200
===== short test summary info =====
FAILED tests/test_tasks.py::test_complete_task_sets_status_done - assert 404 == 200
FAILED tests/test_tasks.py::test_filter_done_returns_only_done
FAILED tests/test_tasks.py::test_filter_invalid_status_returns_422 - assert 200 == 422
3 failed, 1 passed in 0.42s
```

## 왜 하나는 통과했을까 — 의미 있는 빨강과 우연한 초록

`test_complete_unknown_task_returns_404` 가 통과(1 passed)한 점을 눈여겨보십시오. 아직 완료 처리 라우트가 없어 FastAPI가 **경로를 못 찾아 404**를 돌려주는데, 이 테스트가 기대하는 값도 마침 404라 우연히 통과한 것입니다. 즉 "없는 id라서 404"가 아니라 "라우트 자체가 없어서 404"입니다. 이것은 아직 의미 있는 검증이 아닙니다. M3에서 라우트를 구현한 뒤 이 테스트가 여전히 통과하는지 다시 봐야 비로소 "없는 id를 제대로 404로 처리한다"가 증명됩니다. 빨강·초록의 **이유**까지 읽는 것이 TDD를 제대로 쓰는 법입니다.

## M2 체크포인트

- [ ] 네 가지 기대 동작이 모두 테스트로 표현되었는가(요구사항 ↔ 테스트 매핑)
- [ ] 테스트가 저장소를 초기화한 상태에서 시작하는가(격리)
- [ ] 핵심 동작 테스트가 아직 **실패(빨강)**하는 것을 눈으로 확인했는가

## M3: 엔드포인트 구현과 실행·테스트 검증

이제 빨강을 초록으로 바꿉니다. 에이전트 루프(c02)를 의식하며, "구현하고 pytest로 통과까지 확인해 줘"라고 검증을 지시의 일부로 못 박습니다(c07).

나: 이제 그 테스트들이 통과하도록 구현해 줘.  
 1) app/models.py 에 Status(todo|done) enum 과 Task.status 를 확인·정리  
 2) app/routers/tasks.py 에 PATCH /tasks/{id}/complete 와  
 GET /tasks 의 status 필터를 추가. 없는 id 는 404.  
 구현 후 python -m pytest -q 로 4개가 모두 통과하는 것을 확인해 줘.

에이전트는 기존 모듈을 읽고(Read·Grep) 같은 스타일로 변경합니다. 설계(ch\_14)의 모듈 분해대로 models.py → store.py → routers/tasks.py 순으로 손댁니다.

## 1) 데이터 모델: 상태 enum (models.py)

```
app/models.py
from enum import Enum

from pydantic import BaseModel

class Status(str, Enum):
 todo = "todo"
 done = "done"

class Task(BaseModel):
 id: int
 title: str
 status: Status = Status.todo
```

- class Status(str, Enum): 상태를 todo|done 문자열 enum으로 정의했습니다(ADR2). str 을 함께 상속해, JSON으로 직렬화하면 "done" 같은 문자열로 나옵니다.
- status: Status = Status.todo: 새 할 일의 기본 상태를 todo 로 두었습니다. 불리언 done 대신 enum을 쓰면 나중에 in\_progress 같은 상태를 확장하기 쉽습니다.
- enum이라서 FastAPI가 잘못된 status 값(예: working)을 **자동으로 422**로 막아 줍니다. FR4의 422 요구를 모델 정의만으로 상당 부분 충족하는 지점입니다.

## 2) 저장소: 조회·저장 함수 (store.py)

```
app/store.py
from app.models import Task

_tasks: dict[int, Task] = {}

def reset() -> None:
 """테스트 격리를 위해 저장소를 비웁니다."""
 _tasks.clear()

def add_task(task: Task) -> Task:
 _tasks[task.id] = task
 return task

def all_tasks() -> list[Task]:
 return list(_tasks.values())

def get_task(task_id: int) -> Task | None:
 return _tasks.get(task_id)

def save_task(task: Task) -> Task:
 _tasks[task.id] = task
 return task
```

- `get_task` 는 해당 id가 없으면 `None` 을 돌려줍니다. 라우터는 이 `None` 을 보고 404를 던집니다. "찾기"와 "에러 응답"의 책임을 분리한 구조입니다.
- `save_task` 는 변경된 할 일을 다시 저장합니다. 완료 처리는 상태만 바꾸므로 `add_task` 와 동작이 같지만, 의도를 드러내려 이름을 나눴습니다.
- 데이터베이스 없이 `dict` 만 쓰는 인메모리 저장소를 유지했습니다(설계의 `scope_out`: DB 연동 제외). 범위를 작게 지키는 결정이 코드에도 그대로 반영됩니다.

### 3) 엔드포인트: 완료 처리·상태 필터 (routers/tasks.py)

```
app/routers/tasks.py
from fastapi import APIRouter, HTTPException

from app import store
from app.models import Status, Task

router = APIRouter(prefix="/tasks", tags=["tasks"])

@router.post("", status_code=201)
def create_task(task: Task) -> Task:
 return store.add_task(task)

@router.get("")
def list_tasks(status: Status | None = None) -> list[Task]:
 tasks = store.all_tasks()
 if status is None:
 return tasks
 return [task for task in tasks if task.status == status]

@router.patch("/{task_id}/complete")
def complete_task(task_id: int) -> Task:
 task = store.get_task(task_id)
 if task is None:
 raise HTTPException(status_code=404, detail="task not found")
 task.status = Status.done
 return store.save_task(task)
```

- `def list_tasks(status: Status | None = None)`: 기존 목록 엔드포인트에 **선택적** 쿼리 파라미터 `status` 를 더했습니다(ADR3). 기본값이 `None` 이라 `status` 를 안 주면 전체를 반환해, 기존 동작과 호환됩니다. enum 타입이라 `working` 같은 잘못된 값은 FastAPI가 422로 막습니다.
- `if status is None: return tasks`: 필터가 없으면 전체를, 있으면 그 상태인 항목만 거릅니다. 한 엔드포인트가 "전체 조회"와 "필터 조회"를 모두 처리합니다.
- `@router.patch("/{task_id}/complete")`: 부분 상태 변경이므로 PATCH를 썼습니다(ADR1). `prefix="/tasks"` 와 합쳐져 실제 경로는 `PATCH /tasks/{task_id}/complete` 입니다.
- `if task is None: raise HTTPException(status_code=404, ...)`: 없는 id를 404로 처리합니다. M2에서 "우연히" 통과하던 404 테스트가, 이제 **라우트가 있는데도 id가 없어서** 404를 내는 진짜 검증으로 바뀝니다.
- `task.status = Status.done`: 완료 처리의 핵심 한 줄입니다. 상태만 바꾸고 다른 필드는 건드리지 않습니다.

#### 4) 실행·테스트로 통과 확인



```
$ python -m pytest -q
.... [100%]
4 passed in 0.45s
```

....의 점 네 개가 네 테스트의 통과를 뜻하고, 4 passed가 우리가 기다린 증거입니다. 코드를 작성한 순간이 아니라, 이 통과를 확인한 지금이 구현의 진짜 끝입니다. 서버를 직접 띄워 눈으로도 확인할 수 있습니다.

```
$ python -m uvicorn app.main:app --reload
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Application startup complete.
```

다른 터미널에서 완료 처리와 필터를 요청해 봅니다.

```
$ curl -X PATCH http://127.0.0.1:8000/tasks/1/complete
{"id":1,"title":"온보딩 문서 읽기","status":"done"}

$ curl "http://127.0.0.1:8000/tasks?status=done"
[{"id":1,"title":"온보딩 문서 읽기","status":"done"}]
```

## M3 체크포인트

- [ ] `python -m pytest -q`에서 네 테스트가 모두 통과(4 passed)하는가
- [ ] 완료 처리 응답의 `status`가 `done`으로 바뀌는가
- [ ] 없는 id가 404, 잘못된 status가 422로 응답하는가(에러 경로 검증)

## M4: 논리 단위 커밋과 첫 PR 초안 작성

기능이 검증됐으니 이제 팀의 코드로 합류시킬 차례입니다. 단원 8(커밋·PR c09)의 적용입니다. 핵심 원칙은 둘입니다. **작업 브랜치에서, 작은 논리 단위로 커밋합니다.**

```
나: main 인지 확인하고 feature/task-complete-filter 작업 브랜치를 만들어 줘.
 그리고 변경을 두 논리 단위로 나눠 커밋해 줘.
 1) 완료 처리·상태 필터 구현(models·store·routers)
 2) 그 동작을 검증하는 테스트
 Conventional Commits 형식으로, 메시지에 무엇을·왜를 담아 줘.
```

```
$ git switch -c feature/task-complete-filter
Switched to a new branch 'feature/task-complete-filter'

$ git add app/models.py app/store.py app/routers/tasks.py
$ git commit -m "feat(tasks): 완료 처리·상태 필터 엔드포인트 추가"
[feature/task-complete-filter 8c1d2e4] feat(tasks): 완료 처리·상태 필터 엔드포인트 추가
3 files changed, 24 insertions(+), 3 deletions(-)

$ git add tests/test_tasks.py
$ git commit -m "test(tasks): 완료 처리·필터·에러 경로 테스트 추가"
[feature/task-complete-filter 5b7a9f1] test(tasks): 완료 처리·필터·에러 경로 테스트 추가
1 file changed, 38 insertions(+)
```

- 구현( feat )과 테스트( test )를 두 커밋으로 나눴습니다. 성격이 다른 변경이라, 나중에 한 쪽만 살펴보거나 되돌리기 쉽습니다. git add 에 파일을 명시해 무엇을 묶을지 사람이 통제했습니다.
- 제목이 type(scope): 설명 형식입니다. feat 은 새 기능, test 는 테스트 추가, (tasks) 는 영향 범위입니다.
- 두 커밋 모두 feature/task-complete-filter 작업 브랜치에 쌓였습니다. main 은 영향을 받지 않습니다.

이제 브랜치를 푸시하고 PR 초안을 만듭니다. 변경 요약·근거·테스트 결과를 담아 리뷰어를 돕습니다.

```
$ git push -u origin feature/task-complete-filter
$ gh pr create --draft --title "feat(tasks): 완료 처리·상태 필터 추가" --body "(아래 본문)"
https://github.com/team/team-tasks-api/pull/57
```

PR 본문은 다음과 같이 작성합니다.

**## 변경 요약**

할 일을 완료 처리하는 PATCH /tasks/{id}/complete 와, 상태로 거르는 GET /tasks?status=done|todo 두 엔드포인트를 추가했습니다.

**## 변경 근거**

완료 여부를 바꾸거나 상태별로 볼 방법이 없었습니다(이슈 #51). 가장 작은 단위로 "완료 처리 + 상태 필터"만 먼저 추가합니다.

**## 변경 내용**

- app/models.py: 상태를 todo|done 문자열 enum 으로 정의(ADR2)
- app/routers/tasks.py: 완료 처리(PATCH)·상태 필터(GET ?status=) 추가
- 없는 id 는 404, 잘못된 status 는 422 로 응답
- tests/test\_tasks.py: 완료·404·필터·422 네 경로를 검증

**## 테스트 결과**

```

\\`\\`
$ python -m pytest -q
....
4 passed in 0.45s
\\`\\`
[100%]
```

**## 범위**

- 포함: 완료 처리·상태 필터와 그 테스트
- 제외: 완료 취소(되돌리기), 완료 시각 기록 — 후속 PR에서 다룹니다.

## M4 체크포인트

- [ ] main 이 아니라 작업 브랜치에서 커밋했는가
- [ ] 변경이 작은 논리 단위( feat · test )로 나뉘고 메시지가 관례를 따르는가
- [ ] PR 본문에 변경 요약·근거·내용·**실제 테스트 출력**이 모두 있는가

## M5: /code-review로 리뷰 받고 디버깅·반영

PR을 올렸으니 리뷰가 따라옵니다. 단원 9(리뷰 c10)·단원 10(디버깅 c11)의 적용입니다. 먼저 올리기 전 자가 점검으로 /code-review 를 씁니다.

### 사실 확인: /code-review 는 내장 명령입니다

/code-review · /security-review 는 Claude Code에 **기본 내장(번들)된** 슬래시 커맨드입니다(앞 챕터에서 다른 /commit-push-pr 같은 팀 커스텀 커맨드와 다릅니다). 터미널에서 /code-review 를 실행하면 현재 변경을 검토해 개선점을 짚어 줍니다.

나: /code-review

(Claude Code가 변경 diff 를 검토)

리뷰 결과:

1. [개선] complete\_task 가 이미 done 인 작업도 다시 done 으로 저장합니다.  
동작엔 문제없지만, 의도를 주석이나 테스트로 드러내면 좋겠습니다.
2. [질문] status 필터에 빈 문자열(?status=)이 오면 어떻게 되나요?
3. [칭찬] 에러 경로(404·422)까지 테스트한 점이 좋습니다.

세 지적을 모두 같은 방식으로 받지 않습니다. 타당한 지적은 반영하고, 확인이 필요한 지적은 검증하며, 부적절하면 근거로 답합니다.

2번 지적("빈 status")은 검증이 필요합니다. 디버깅 루프(c11)의 첫 단계인 **재현**으로 직접 확인합니다.

나: GET /tasks?status= (빈 값)이 오면 어떻게 응답하는지 테스트로 재현해 줘.

```
$ curl -i "http://127.0.0.1:8000/tasks?status="
HTTP/1.1 422 Unprocessable Entity
```

빈 문자열은 enum 값이 아니므로 FastAPI가 이미 422로 막고 있었습니다. 즉 추가 수정 없이 안전합니다. 이 사실을 **회귀 테스트로 못 박아** 재발을 막고, 리뷰어에게 근거로 답합니다.

```
tests/test_tasks.py (회귀 테스트 보강)
def test_filter_empty_status_returns_422():
 response = client.get("/tasks", params={"status": ""})
 assert response.status_code == 422
```

1번 지적("이미 done 인 작업")은 타당하므로 의도를 드러내는 테스트를 추가해 반영합니다.

```
tests/test_tasks.py (회귀 테스트 보강)
def test_complete_already_done_is_idempotent():
 client.post("/tasks", json={"id": 3, "title": "이미 끝낸 일", "status": "done"})
 response = client.patch("/tasks/3/complete")
 assert response.status_code == 200
 assert response.json()["status"] == "done"
```

테스트를 다시 돌려 회귀 확인을 합니다.

```
$ python -m pytest -q
.....
6 passed in 0.48s [100%]
```

여섯 테스트가 모두 통과합니다. 리뷰 지적을 반영(1번)·검증(2번)하고도 기존 동작이 깨지지 않았음이 증명됐습니다. 반영 내용을 커밋해 PR에 올립니다.

```
$ git add tests/test_tasks.py
$ git commit -m "test(tasks): 맥등 완료·빈 status 회귀 테스트 추가"
$ git push
```

## M5 체크포인트

- [ ] 리뷰 코멘트의 의미를 이해하고, 반영·검증·반론을 구분해 대응했는가
- [ ] 확인이 필요한 지적을 재현→검증의 디버깅 루프로 확인했는가
- [ ] 잡은/확인한 동작을 회귀 테스트로 못 박고 전체 테스트가 통과하는가(6 passed)

## 통합 실행과 결과 확인

다섯 마일스톤을 통합하면, `team-tasks-api`에 완료 처리·상태 필터가 실제로 동작하고, 모든 pytest가 통과하며, 첫 PR이 리뷰를 거쳐 머지 가능한 상태가 됩니다. 최종 모듈 구조와 검증 결과를 한 번에 확인합니다.

```
team-tasks-api/
 app/
 __init__.py
 main.py # FastAPI 앱, tasks 라우터 등록
 models.py # Task 모델, Status(todo|done) enum ← 변경
 store.py # 인메모리 저장소(get_task·save_task) ← 변경
 routers/
 __init__.py
 tasks.py # 완료 처리·상태 필터 엔드포인트 ← 변경
 tests/
 __init__.py
 test_tasks.py # 6개 테스트(완료·404·필터·422·맥등·빈값) ← 변경
 CLAUDE.md # 빌드·테스트·컨벤션 ← 신규
 .claude/settings.json # 최소 권한 ← 신규
 requirements.txt
```

`app/main.py`는 라우터를 등록만 하므로 변경이 없습니다.

```
app/main.py
from fastapi import FastAPI

from app.routers import tasks

app = FastAPI(title="team-tasks-api")
app.include_router(tasks.router)
```

마지막으로 전체 테스트를 한 번 더 돌려, 통합된 상태가 온전히 통과하는지 확인합니다.

```
$ python -m pytest -q
..... [100%]
6 passed in 0.48s
```

이로써 분석(요구사항) → 설계(ADR) → 구현(마일스톤)의 흐름이 동작하는 코드와 통과한 테스트, 그리고 리뷰를 거친 PR로 마무리했습니다. PR을 초안(draft)에서 정식 리뷰(Ready for review)로 전환하면 머지를 받을 준비가 끝납니다.

## 선택: M6 — 서버에이전트·MCP 맞보기 (c12)

여기서 한 걸음 더 나아가고 싶다면, 반복되는 "테스트 실행·요약"을 서버에이전트에 위임해 볼 수 있습니다. `.claude/agents/tester.md`에 역할과 도구를 줌에 준 서버에이전트를 정의하면, 메인 대화의 맥락을 어지럽히지 않고 테스트 실행을 맡길 수 있습니다. 또 팀의 API 컨벤션 문서를 MCP 서버로 `.mcp.json`에 등록하면, 다음 작업에서 외부 자료를 참조할 수 있습니다. 이는 **선택 과제**이며, 본격적인 설계는 이 책 다음 단계의 학습으로 남겨 둡니다(ch\_16에서 다음 단계로 안내합니다).

## 작성형 연습 (2종 1세트)

배운 구현 흐름을 직접 적용해 봅니다. ① 모범 예시를 보고, ② 빈칸을 스스로 채운 뒤, ③ 자가 체크로 점검합니다.

### 모범 예시: 완료 취소(되돌리기) 엔드포인트를 TDD로 추가

후속 작업으로 "완료를 취소해 다시 `todo`로 되돌리는" `PATCH /tasks/{id}/reopen`을 TDD로 추가하는 모범입니다. 테스트를 먼저 쓰고(빨강), 구현해(초록) 통과시킵니다.

```
tests/test_tasks.py (모범 — 테스트 먼저)
def test_reopen_task_sets_status_todo():
 client.post("/tasks", json={"id": 5, "title": "되돌릴 일", "status": "done"})
 response = client.patch("/tasks/5/reopen")
 assert response.status_code == 200
 assert response.json()["status"] == "todo"
```

```
app/routers/tasks.py (모범 — 통과시키는 구현)
@router.patch("/{task_id}/reopen")
def reopen_task(task_id: int) -> Task:
 task = store.get_task(task_id)
 if task is None:
 raise HTTPException(status_code=404, detail="task not found")
 task.status = Status.todo
 return store.save_task(task)
```

완료 처리(`complete_task`)와 대칭 구조입니다. 상태를 `done` 대신 `todo` 로 바꾸는 한 줄만 다릅니다.

## 연습 1: team-tasks-api에 직접 구현하고 실제 pytest 출력을 기록하라

이번 연습은 종이에 채우지 말고 **team-tasks-api**에서 실제로 구현·실행합니다. 상태별 개수를 돌려주는 `GET /tasks/stats` 엔드포인트를 TDD로 직접 추가하세요. 기대 동작은 "전체 ·todo·done 개수를 담은 객체를 200으로 반환"입니다. 미리 정한 답을 적는 것이 아니라, **자신이 작성한 코드와 화면에 나타난 pytest 실제 출력을 그대로 옮기는 것**이 핵심입니다.

**1단계 — 테스트를 먼저 직접 작성한다(빨강).** 위 모범 예시(`reopen`)의 테스트 형식을 본떠, `tests/test_tasks.py` 에 `test_stats_counts_by_status` 를 직접 작성하세요. 상태 코드와 `total·done·todo` 개수가 무엇이어서 하는지 자신이 정해 단언(`assert`)을 적습니다. 작성한 테스트를 아래에 그대로 옮기세요.

[내가 작성한 테스트 코드]  
(여기에 직접 적는다)

[python -m pytest -q 실제 출력 — 아직 구현 전이라 실패해야 정상]  
(화면에 나타난 출력을 그대로 옮긴다)

**2단계 — 통과시키는 구현을 직접 작성한다(초록).** `app/routers/tasks.py` 에 `GET /tasks/stats` 를 직접 구현하세요 (`/{task_id}` 보다 위에 두는 점에 주의). 그런 뒤 `python -m pytest -q` 를 실제로 돌려 통과를 확인하고, 화면 출력을 그대로 옮기세요.

[내가 작성한 구현 코드]  
(여기에 직접 적는다)

[python -m pytest -q 실제 출력 — 구현 후 통과]  
(화면에 나타난 출력을 그대로 옮긴다)

작성 안내: 상태 비교에는 `Status.done` 을 씁니다. `todo` 개수는 전체에서 `done`을 뺀 값으로 둘 수 있습니다. 추측을 적지 말고, **실제로 구현해 실행한 결과**를 기록하세요.

## 연습 2: 내가 실제로 진행한 마일스톤과 적용 개념을 직접 작성하라

team-tasks-api에서 **자신이 실제로 진행한** 마일스톤마다 무엇을 했는지, 어떤 단위 개념을 적용했는지 한 줄로 **직접 작성**하고, 체크포인트 통과 여부를 표시하세요(추측이 아니라 실제로 한 일을 적습니다).

아래 표를 그대로 옮겨, 빈 칸에 내가 한 일을 직접 적으세요.

마일스톤	내가 실제로 한 일(직접 적기)	적용 개념(예: c05)	체크포인트 통과
M1 셋업·계획	(직접 적는다)	(직접 적는다)	[ ]
M2 테스트 먼저	(직접 적는다)	(직접 적는다)	[ ]
M3 구현·검증	(직접 적는다)	(직접 적는다)	[ ]
M4 커밋·PR	(직접 적는다)	(직접 적는다)	[ ]
M5 리뷰·디버깅	(직접 적는다)	(직접 적는다)	[ ]

## 자가체크 — 머리로가 아니라 손으로 했는지 확인하세요.

- [ ] 연습 1의 `GET /tasks/stats` 를 team-tasks-api에 **실제로 구현**했고, `python -m pytest -q` 가 통과하는 것을 화면 출력으로 봤다
- [ ] 기록란에 적은 것이 추측이 아니라 **화면에 실제로 나타난 코드·pytest 출력**이다
- [ ] 구현 전에 테스트가 먼저 실패(빨강)하는 것을 화면으로 직접 확인했다
- [ ] 각 마일스톤을 끝낼 때 체크포인트(테스트 통과 등)로 검증했다
- [ ] 변경을 작은 논리 단위로 커밋하고, 작업 브랜치에서 작업했다
- [ ] PR 본문의 테스트 결과에 "잘 됨"이 아니라 **실제 pytest 출력**을 넣었다



막혔다면: 상태 비교에는 `Status.done` 을, todo 개수에는 `len(tasks) - len(done)` 을 떠올려 보세요. 다만 정답을 적는 것이 목적이 아니라, **실제로 구현해** `python -m pytest -q` 를 통과시키는 것이 이 연습의 목표입니다.

## 흔한 함정과 점검

### 1. 마일스톤을 건너뛰고 한 번에 다 만든다

"빨리 끝내자"며 셋업·테스트 없이 곧장 구현·커밋·PR을 한 번에 진행하면, 어디서 잘못됐는지 좁히기 어렵고 리뷰어도 큰 덩어리를 한꺼번에 떠안습니다. **작게 나눠 한 걸음마다 통과를 확인**하세요. 각 마일스톤의 체크포인트가 그 걸음을 검증합니다.

### 2. 테스트 없이 "동작하는 것 같다"로 PR을 올린다

서버에서 한 번 눌러 보고 PR을 올리면, 빈 목록·잘못된 입력·중복 호출 같은 경우는 검증되지 않은 채 남습니다. **요구사항을 테스트로 못 박고**(M2), 그 통과 출력을 PR 본문의 근거로 붙이세요. "잘 됩니다"는 근거가 아닙니다.

### 3. 리뷰 지적을 무조건 반영하거나 무조건 방어한다

타당성을 따지지 않고 다 반영하면 불필요한 변경이 늘고, 무조건 방어하면 배울 기회를 잃습니다. 지적마다 **반영·검증·근거 있는 반론**을 구분하세요. 확인이 필요한 지적은 디버깅 루프(재현→검증)로 사실을 확인한 뒤, 회귀 테스트로 못 박아 답하면 됩니다.

## 이 챕터 한 줄 정리

첫 기능 구현은 "셋업·계획(M1) → 테스트 먼저(M2) → 구현·검증(M3) → 커밋·PR(M4) → 리뷰·디버깅(M5)"의 증분 마일스톤이며, 각 단계는 앞 단원 개념을 적용하고 체크포인트로 검증합니다. 다음 챕터(ch\_16)에서는 이 산출물을 시연하고 루브릭으로 평가하며, 12개 개념을 어떻게 적용했는지 성찰합니다.



# 종합 프로젝트: 첫 기능·첫 PR — 발표·평가·성찰

## 이 챕터에서 배우는 것

- 완성한 첫 기능·통과한 테스트·올린 PR을 공개 산출물로 시연할 수 있습니다.
- 주어진 평가 루브릭의 항목별 기준에 맞춰 자신과 동료의 작업을 점수·근거로 평가할 수 있습니다.
- 12개 개념(c01~c12) 각각을 종합 프로젝트의 어느 단계에서 어떻게 적용했는지 성찰 회고로 작성할 수 있습니다.
- 현재 결과물의 한계 1개 이상과 다음 학습(서브에이전트·MCP 심화) 방향을 제시할 수 있습니다.

이 챕터는 종합 프로젝트의 마지막 단계인 **발표·평가·성찰(present & reflect)**입니다. 새 개념을 배우지 않습니다. 구현(ch\_15)한 산출물을 남에게 보이고, 기준으로 평가하며, 온보딩 여정 전체를 되짚어 다음 단계로 연결합니다.

## 산출물 시연: 동작하는 기능·통과한 테스트·올린 PR

발표의 핵심은 "잘 만들었다"는 말이 아니라 **보이는 증거**입니다. 신입의 첫 기능 발표는 세 가지를 순서대로 보여 주면 충분합니다. 동작하는 기능, 통과한 테스트, 올린 PR입니다.

### 1) 동작하는 기능 시연

서버를 띄우고, 추가한 두 엔드포인트가 실제로 응답하는 모습을 보여 줍니다. 말이 아니라 요청·응답으로 증명합니다.

```
$ python -m uvicorn app.main:app --reload
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Application startup complete.

$ curl -X PATCH http://127.0.0.1:8000/tasks/1/complete
{"id":1,"title":"온보딩 문서 읽기","status":"done"}

$ curl "http://127.0.0.1:8000/tasks?status=done"
[{"id":1,"title":"온보딩 문서 읽기","status":"done"}]

$ curl -i "http://127.0.0.1:8000/tasks?status=working"
HTTP/1.1 422 Unprocessable Entity
```

완료 처리가 상태를 `done` 으로 바꾸고(UC1), 필터가 `done` 인 항목만 거르며(UC2), 잘못된 상태는 422로 막힙니다(FR4). 정상 경로뿐 아니라 **에러 경로까지** 보여 주는 것이 신뢰를 줍니다.

## 2) 통과한 테스트 시연

기능이 "지금 이 순간"만이 아니라 반복적으로 동작함을 테스트로 보입니다.

```
$ python -m pytest -q
.....
6 passed in 0.48s
```

[100%]

완료·404·필터·422·먹등·빈값의 여섯 경로가 모두 통과합니다. 이 출력이 "동작합니다"라는 주장의 증거입니다.

## 3) 올린 PR 시연

마지막으로 PR을 보여 줍니다. 변경 요약·근거·내용·테스트 결과가 담겨 리뷰어가 1분 안에 파악할 수 있는 형태입니다.

```
$ gh pr view 57
title: feat(tasks): 완료 처리·상태 필터 추가
state: OPEN
url: https://github.com/team/team-tasks-api/pull/57

완료 처리(PATCH /tasks/{id}/complete)와 상태 필터(GET /tasks?status=)를
추가했습니다. 없는 id 는 404, 잘못된 status 는 422. 테스트 6개 통과.
```

## 발표 한 장 요약 틀

발표는 다음 한 문장 틀에 맞춰 말하면 군더더기 없이 전달됩니다. "[무엇을] 추가했고, [어떤 테스트]로 검증했으며, [어떤 PR]로 올렸습니다. 설계에서는 [핵심 ADR]을 택했고, 남은 범위는 [제외 항목]입니다." 예: "완료 처리·상태 필터를 추가했고, 여섯 개 테스트로 검증했으며, #57 PR로 올렸습니다. 상태는 확장을 위해 enum으로 택했고(ADR2), 완료 취소는 후속으로 남겼습니다."

## 평가 루브릭

좋은 평가는 "느낌"이 아니라 **공유된 기준**으로 합니다. 아래 루브릭은 이 종합 프로젝트의 여섯 가지 기준과 가중치, 그리고 미흡·보통·우수의 수준을 정의합니다. 자신을 먼저 평가하고, 동료의 작업도 같은 기준으로 봅니다.

기준	가중치	미흡	보통	우수
기능 완성도(완료 처리·상태 필터·에러 응답)	0.20	한 엔드포인트만 부분 동작	두 엔드포인트 동작	두 엔드포인트 + 404·422 에러 응답까지 동작
검증(테스트)	0.20	테스트 없음	행복 경로 테스트만	TDD로 작성한 행복·에러 경로 테스트가 모두 통과
안전·절차(권한·플랜·CLAUDE.md)	0.15	바로 편집	권한 또는 플랜 중 하나만	최소 권한·플랜 모드·CLAUDE.md를 모두 적용
협업(커밋·PR 품질)	0.20	한 덩어리 커밋·빈약한 PR	관례 커밋과 기본 PR	작은 논리 커밋 + 요약·근거·테스트 결과를 담은 PR
리뷰 대응·디버깅	0.15	대응 없음	지적을 반영만	근거를 이해해 반영·반론하고 회귀 테스트로 확인
발표·성찰	0.10	동작만 보여 줌	변경 의도 설명	ADR 근거·개념 적용 회고·다음 학습 방향까지 제시

## 점수를 매기는 법

각 기준의 수준(미흡 1점·보통 2점·우수 3점)에 가중치를 곱해 더하면 가중 평균이 나옵니다. 예를 들어 모든 기준이 "보통(2점)"이면 2.00, 모두 "우수(3점)"이면 3.00입니다. 점수 자체보다 **어느 기준이 약한지**를 찾아 다음에 보완하는 데 의미가 있습니다.

## worked example: 이 프로젝트를 루브릭으로 자가 평가하기

ch\_15에서 완성한 산출물을 루브릭으로 직접 채점해 보겠습니다. 각 기준마다 수준을 고르고, 그 판단의 **근거**를 한 줄로 답니다. 근거 없이 점수만 매기면 다음에 무엇을 고칠지 알 수 없습니다.

기준	가중치	내 수준	점수	근거
기능 완성도	0.20	우수	3	두 엔드포인트 + 404·422 에러 응답까지 동작
검증(테스트)	0.20	우수	3	완료·404·필터·422를 TDD로 작성, 6개 통과
안전·절차	0.15	우수	3	최소 권한·플랜·CLAUDE.md를 M1에서 모두 적용
협업(커밋·PR)	0.20	우수	3	feat·test 분리 커밋 + 근거·테스트 담은 PR
리뷰 대응·디버깅	0.15	우수	3	지적을 재현·검증하고 회귀 테스트로 못 박음
발표·성찰	0.10	보통	2	변경 의도는 설명했으나 ADR 근거 연결이 얇음

가중 평균은  $3 \times 0.20 + 3 \times 0.20 + 3 \times 0.15 + 3 \times 0.20 + 3 \times 0.15 + 2 \times 0.10 = 2.90$  입니다. 점수가 높아도 의미는 마지막 줄에 있습니다. **발표·성찰 기준이 가장 약했고**, "ADR 근거를 발표에 더 분명히 연결한다"가 다음에 보완할 한 가지입니다. 이렇게 루브릭은 잘한 점을 확인하는 동시에 **다음 목표 하나**를 짚어 줍니다.

## 연습: 나의 실제 산출물을 루브릭으로 자가 평가하라

자신이 **실제로 만든** 첫 기능(team-tasks-api의 완료 처리·상태 필터)을 위 모범 표처럼 직접 평가해 작성하세요. 점수보다 **근거와 가장 약한 기준**을 찾는 데 집중합니다. 각 칸은 추측이 아니라,

내가 실제로 한 작업(커밋·테스트·리뷰 대응)에 비추어 적습니다.

아래 표를 그대로 옮겨, 각 칸을 자신의 산출물로 직접 채우세요.

기준	내 수준	점수	근거(내가 실제로 한 것)
기능 완성도	(직접 적기)	(직접)	(내 산출물 근거를 직접 적는다)
검증(테스트)	(직접 적기)	(직접)	(내 산출물 근거를 직접 적는다)
안전·절차	(직접 적기)	(직접)	(내 산출물 근거를 직접 적는다)
협업(커밋·PR)	(직접 적기)	(직접)	(내 산출물 근거를 직접 적는다)
리뷰 대응·디버깅	(직접 적기)	(직접)	(내 산출물 근거를 직접 적는다)
발표·성찰	(직접 적기)	(직접)	(내 산출물 근거를 직접 적는다)

가중 평균: (직접 계산해 적기)    가장 약한 기준: (직접 적기)    다음에 보완할 한 가지: (직접 적기)

## 루브릭은 채점표가 아니라 길잡이

루브릭의 진짜 쓸모는 점수를 매기는 데 있지 않고, "우수"의 모습을 미리 보여 주는 데 있습니다. 예컨대 검증 기준의 "우수"는 "행복 경로만이 아니라 **에러 경로(404·422)까지** TDD로 검증"입니다. 작업을 시작하기 **전에** 이 기준을 읽으면, 무엇을 목표로 삼아야 할지가 분명해집니다. 끝나고 채점하기보다, 시작 전에 목표로 읽으십시오.

## 동료 피드백 (모의 리뷰)

발표 뒤에는 동료의 눈으로 한 번 더 점검합니다. 실제 팀에서는 동료가 PR에 코멘트를 달지만, 여기서는 **모의 동료 리뷰**로 연습합니다. 좋은 동료 리뷰는 비난이 아니라, 같은 기준(루브릭)을 근거로 한 구체적 제안입니다.

다음은 이 프로젝트에 대한 모의 동료 리뷰의 예입니다.

**동료 A (검증 기준):** 에러 경로(404·422)까지 테스트한 점이 좋습니다. 다만 완료 처리를 **연속 두 번** 호출했을 때도 안전한지 궁금합니다. 멍등성 테스트가 이미 있다면 그 의도를 PR 본문에도 한 줄 적어 주면 리뷰어가 더 빨리 이해할 것 같습니다.

**동료 B (협업 기준):** 구현과 테스트를 `feat · test` 로 나눈 커밋이 깔끔합니다. PR 제목도 관례를 잘 따랐습니다. 한 가지, "범위 - 제외"에 적힌 완료 취소 기능은 후속 이슈로 등록해 두면 추적이 쉬울 것 같습니다.

**동료 C (설계 기준):** 상태를 불리언이 아니라 enum으로 택한 결정(ADR2)이 확장에 유리해 보입니다. 다만 enum 값이 늘어날 때를 대비해, 필터가 여러 상태를 받는 경우(예: `?status=todo,done`)는 어떻게 할지 후속에서 한 번 정리하면 좋겠습니다.

이 리뷰들에 대응하는 방식은 ch\_15에서 배운 그대로입니다. **타당한 지적은 반영하고, 확인이 필요하면 재현·검증하며, 부적절하면 근거로 답합니다.** 예컨대 동료 A의 제안은 PR 본문에 한 줄 추가하는 것으로 바로 반영할 수 있고, 동료 C의 제안은 이번 범위 밖이므로 "후속 이슈로 남기겠습니다"라고 근거와 함께 답하면 됩니다.

## 모의 동료 리뷰 작성 연습

짝의 산출물(또는 자신의 실제 산출물)에 대해, 루브릭 기준 하나를 골라 동료 리뷰 한 건을 **직접** 작성하세요. 비난이 아니라 **근거 있는 구체적 제안**으로 적습니다. 위 동료 A·B·C 예시를 본떠, 실제로 본 코드·PR에 근거해 적습니다.

아래 형식을 그대로 옮겨, 각 항목을 직접 작성하세요.

[고른 기준] (직접 적는다 — 예: 검증)  
 [좋은 점] (직접 적는다)  
 [제안] (직접 적는다)  
 [근거] 루브릭의 "우수" 수준은 (직접 적는다) 인데,  
 현재는 (직접 적는다) 이므로 한 걸음 더 가면 좋겠습니다.

## 성찰 회고: 12개 개념의 적용

이 온보딩 여정의 핵심은, 따로 배운 12개 개념이 **한 번의 실제 업무에서 어떻게 하나로 모이는**가를 체득하는 것이었습니다. 첫 기능 추가의 각 단계가 어느 개념에서 왔는지 되짚어 봅니다.



개념	단 원	첫 기능 추가의 어느 단계로 이어졌나
c01 설치·인증·세션	u1	작업 시작 전 환경을 갖추고 세션을 열어 첫 작업을 지시
c02 에이전트 루프	u2	구현(M3)에서 관찰→계획→도구→확인의 흐름으로 변경을 진행
c03 작업 지시 4요소	u3	테스트·구현 지시(M2·M3)를 목표·맥락·제약·완료 조건으로 작성
c04 코드베이스 탐색	u4	분석(ch_13)에서 기존 저장소 구조·데이터 흐름을 파악
c05 권한·승인	u5	M1에서 최소 권한 settings.json으로 안전한 명령만 허용
c06 플랜 모드	u5	M1에서 변경 전 계획을 받아 검토·교정해 확정
c07 실행·테스트 검증	u6	M2·M3에서 TDD로 완료 조건을 먼저 세우고 통과를 확인
c08 CLAUDE.md	u7	M1에서 빌드·테스트 명령·컨벤션을 메모리에 영속화
c09 커밋·PR	u8	M4에서 작은 논리 단위 커밋과 첫 PR 초안 작성
c10 코드 리뷰	u9	M5에서 /code-review 로 점검하고 지적에 대응
c11 디버깅	u10	M5에서 리뷰 지적을 재현→검증→회귀 테스트로 처리
c12 서브에이전트·MCP	u11	M6(선택)에서 테스트 실행 위임·외부 자료 참조를 맛봄

이렇게 보면, 12개 개념이 **분석·설계·구현·발표의 한 흐름**에 빠짐없이 녹아 있습니다. 따로 배울 때는 조각이었지만, 한 번의 첫 기능 추가로 엮으니 비로소 "온보딩이 이렇게 하나로 이어지는구나"를 체득하게 됩니다. 이것이 POV에서 말한 통찰 — "기능을 따로 배우는 것과 한 번의 실제 업무로 엮어 쓰는 것은 다르다" — 의 실제 모습입니다.

## 성찰 질문

다음 다섯 질문에 스스로 답해 보며 여정을 정리합니다.

1. 12개 개념이 첫 기능 추가의 어느 단계로 이어졌는지, 위 표를 바탕으로 자신의 말로 다시 적어 보세요.

2. 플랜 모드(M1)로 받은 초기 계획과 실제 구현(M3)이 달라진 부분이 있다면, 그 이유는 무엇이었습니까?
3. 코드 리뷰(M5)에서 가장 배운 점은 무엇이고, 지적을 방어적으로 받지 않으려면 어떤 태도가 필요합니까?
4. 테스트를 먼저 쓰는 것(TDD, M2)이 결과 품질에 어떤 차이를 만들었습니까?
5. 다음 단계로 서브에이전트·MCP를 더 깊이 쓴다면, 어떤 반복 작업부터 자동화하고 싶습니까?

## 깊이 있는 답을 위한 예시 (질문 3·4)

성찰 질문은 한 문장으로 끝내기보다, 구체적 사건과 연결할 때 값이 생깁니다. 두 질문의 모범 답을 봅니다.

**질문 3(리뷰에서 배운 점·방어적이지 않으려면)의 예시 답:** "가장 배운 점은, 리뷰 지적이 '내 코드가 틀렸다'는 비난이 아니라 '여기 확인이 필요하다'는 신호라는 것이었습니다. `/code-review`가 던진 '빈 status는 어떻게 되나'라는 질문에 처음엔 방어하고 싶었지만, 직접 재현해 보니 이미 422로 막히고 있었습니다. 만약 확인 없이 '괜찮습니다'라고만 답했다면 근거 없는 방어였을 것입니다. 방어적으로 받지 않으려면, 지적을 **검증할 사실**로 바꿔 재현해 보는 습관이 필요합니다."

**질문 4(TDD가 품질에 만든 차이)의 예시 답:** "테스트를 먼저 쓰니(M2) '무엇이 되면 끝인지'가 구현 전에 분명해졌습니다. 특히 404·422 같은 에러 경로를 먼저 테스트로 적어 두니, 구현할 때 그 경우를 빠뜨리지 않았습니다. 만약 구현부터 했다면 정상 경로만 만들고 에러 처리는 잊었을 가능성이 큼니다. TDD는 '완료 조건을 앞당겨 정의'해 구현이 흔들리지 않게 잡아 주었습니다."

## 성찰은 "다음에 무엇을 바꿀까"로 끝낸다

성찰의 목적은 잘잘못을 가리는 것이 아니라, **다음 작업에서 바꿀 한 가지**를 건지는 것입니다. 예컨대 질문 2에서 "계획에 없던 회귀 테스트를 나중에야 추가했다"를 깨달았다면, 다음에는 "리뷰 대응 단계를 처음부터 계획에 넣자"가 건진 교훈입니다. 막연한 반성보다 구체적인 다음 행동 하나가 더 값집니다.

## 다음 단계: 서브에이전트·MCP 심화로 가는 길

이 책은 온보딩, 곧 "첫 기능부터 첫 PR까지"를 다뤘습니다. 여기서부터 더 나아갈 길의 출발점이 단원 11에서 맞본 **서브에이전트와 MCP**입니다. 본서는 맛보기까지만 다뤘으므로(scope\_out), 다음 학습 방향을 안내합니다.

- **반복 작업의 위임(서브에이전트):** 이번 프로젝트에서 가장 자주 반복한 일은 "테스트를 돌려 결과를 확인"하는 것이었습니다. 이런 경계가 분명한 작업을 별도 컨텍스트의 서브에이전트( `.claude/agents/tester.md` )에 맡기면, 메인 대화를 어지럽히지 않고 위임할 수 있습니다. 다음 단계에서는 역할·도구를 더 정교하게 설계하는 법을 익힙니다.
- **외부 자료의 표준 연결(MCP):** 팀의 API 컨벤션 문서나 이슈 트래커처럼 코드 밖의 자료를 MCP 서버로 연결하면, 에이전트가 외부 맥락을 참조하며 일할 수 있습니다. 다음 단계에서는 `.mcp.json` 등록과 권한·비밀 관리를 더 깊이 다룹니다.
- **CI·자동화로의 확장:** 본서는 무인 자동화를 범위 밖으로 두었지만(scope\_out), 통과한 테스트와 좋은 PR 습관이 갖춰지면 자연스러운 다음 걸음은 PR마다 자동으로 테스트·리뷰를 도는 흐름입니다.

## 온보딩 이후의 한 걸음

가장 좋은 다음 단계는 거창한 자동화가 아니라, **이번에 가장 자주 반복한 일 하나를 골라 위임해 보는 것**입니다. 성찰 질문 5에 적은 "자동화하고 싶은 반복 작업"이 바로 그 출발점입니다. 작게 시작해 통과를 확인하며 넓혀 가는 리듬은, 온보딩에서 배운 그대로입니다.

## 작성형 연습 (2종 1세트)

여정을 마무리하는 성찰을 직접 정리합니다. ① 모범 예시를 보고, ② 빈 템플릿을 채운 뒤, ③ 자가체크로 점검합니다.

### 모범 예시: 성찰 회고 한 편

다음은 한 신입이 작성한 성찰 회고의 모범입니다.

**무엇을 했나:** 기존 `team-tasks-api` 에 완료 처리·상태 필터를 추가하고, 여섯 개 테스트로 검증해 #57 PR로 올렸습니다.

**개념 적용:** 가장 크게 와닿은 것은 c07(검증)과 c06(플랜)이었습니다. 테스트를 먼저 쓰니(M2) "무엇이 되면 끝인지"가 분명해져, 구현이 흔들리지 않았습니다. 플랜 모드(M1)로 계획을 먼저 받아 보니, status를 새로 추가할 뻔한 가정을 시작 전에 교정할 수 있었습니다.

**달라진 점:** 초기 계획에는 리뷰 대응 단계가 없었습니다. 막상 `/code-review` (M5)에서 "빈 status는 어떻게 되나"라는 지적을 받고 나서야, 재현·검증·회귀 테스트가 필요함을 알았습니다.

**다음에 바꿀 것:** 다음에는 리뷰 대응과 회귀 테스트를 처음부터 계획(M1)에 넣겠습니다. 또 반복된 "테스트 실행"을 서브에이전트에 위임해 보겠습니다.

## 연습: 나의 성찰 회고 채우기

위 모범을 참고해, 자신이 **실제로 거친** 첫 기능 추가 여정을 성찰 회고로 **직접** 작성하세요. 다섯 성찰 질문을 녹여, 추측이 아니라 자신이 실제로 한 것(시연·평가·12개념 적용)을 서술합니다.

아래 형식을 그대로 옮겨, 각 항목을 자신의 경험으로 직접 채우세요.

[무엇을 했나] (직접 서술한다: 내가 실제로 추가한 기능과 올린 PR)

[개념 적용] 가장 크게 와닿은 개념과 그 이유를 직접 서술한다.  
(예: c07 검증, c06 플랜 — 왜 그랬는지 내 경험으로)

[달라진 점] 플랜 모드 초기 계획과 실제 구현이 달라진 부분과 그 이유:  
(직접 서술한다)

[리뷰에서 배운 점] 내가 받은 리뷰에서 배운 것과, 방어적으로 받지 않으려면:  
(직접 서술한다)

[다음에 바꿀 것] 다음 작업에서 바꿀 한 가지와 자동화하고 싶은 반복 작업:  
(직접 서술한다)

## 자가체크

- [ ] 산출물 시연에 동작하는 기능·통과한 테스트·올린 PR의 **세 증거**를 모두 담았는가
- [ ] 루브릭 여섯 기준으로 자신의 작업을 평가하고, 약한 기준을 찾았는가
- [ ] 12개 개념이 각각 어느 단계로 이어졌는지 자신의 말로 적었는가
- [ ] 성찰을 막연한 반성이 아니라 "다음에 바꿀 한 가지"로 끝맺었는가
- [ ] 다음 학습(서브에이전트·MCP 심화)에서 먼저 시도할 작업을 골랐는가

## 자주 묻는 질문

### Q. 발표 때 코드를 한 줄씩 다 설명해야 하나요?

아닙니다. 리뷰어·청중이 알고 싶은 것은 줄 단위 구현이 아니라 **무엇을·왜·어떻게 검증했는가**입니다. "완료 처리·상태 필터를 추가했고(무엇), 상태 확장을 위해 enum을 택했으며(왜·ADR2), 여섯 개 테스트로 검증했습니다(검증)"처럼 핵심 결정과 증거 위주로 말하세요. 세부 구현은 질문이 나올 때 PR의 해당 줄을 열어 보이면 충분합니다.

## Q. 테스트가 모두 통과했는데도 "보통"보다 낮게 평가될 수 있나요?

있습니다. 검증 기준의 "우수"는 단순 통과가 아니라 **"행복 경로와 에러 경로(404-422)를 TDD로 검증"**입니다. 정상 동작만 테스트하고 에러 경로를 빠뜨렸다면, 모두 통과했더라도 "보통"에 머물립니다. 루브릭은 통과 여부가 아니라 **무엇을 검증했는지의 폭**을 봅니다.

## Q. 리뷰 지적에 반론해도 괜찮나요? 신입이 건방져 보일까 걱정됩니다.

근거가 있다면 괜찮습니다. 중요한 것은 태도가 아니라 **근거의 유무**입니다. "빈 status는 이미 422로 막힙니다. 다음 회귀 테스트로 확인했습니다"처럼 재현·검증 결과를 들어 정중히 답하면, 오히려 "확인하고 말하는 사람"이라는 신뢰를 줍니다. 반대로 확인 없이 "제 코드가 맞아요"라고만 하면 방어로 보입니다. 반론은 감정이 아니라 증거로 합니다.

## Q. 성찰은 형식적인 글쓰기 아닌가요?

성찰이 형식이 되는 것은 "잘 배웠다" 같은 막연한 문장으로 끝낼 때입니다. 성찰의 값은 **다음에 바꿀 구체적 한 가지**를 건지는 데 있습니다. "다음에는 리뷰 대응을 처음부터 계획에 넣겠다"처럼 행동으로 적으면, 그 한 줄이 다음 작업의 품질을 실제로 바꿉니다.

## 이 책 한 줄 정리

설치·탐색·권한·플랜·검증·CLAUDE.md·커밋·PR·리뷰·디버깅 — 따로 배운 12개 개념은, 기존 팀 저장소에 작은 기능 하나를 테스트와 함께 추가해 첫 PR로 올리는 **한 번의 실제 업무**로 엮일 때 비로소 하나가 됩니다. 첫 기능을 끝까지 해 본 지금, 여러분은 이미 신입의 첫 통과의례를 지났습니다. 다음 기능은 더 자신 있게, 더 안전하게, 더 빠르게 해낼 수 있습니다.